

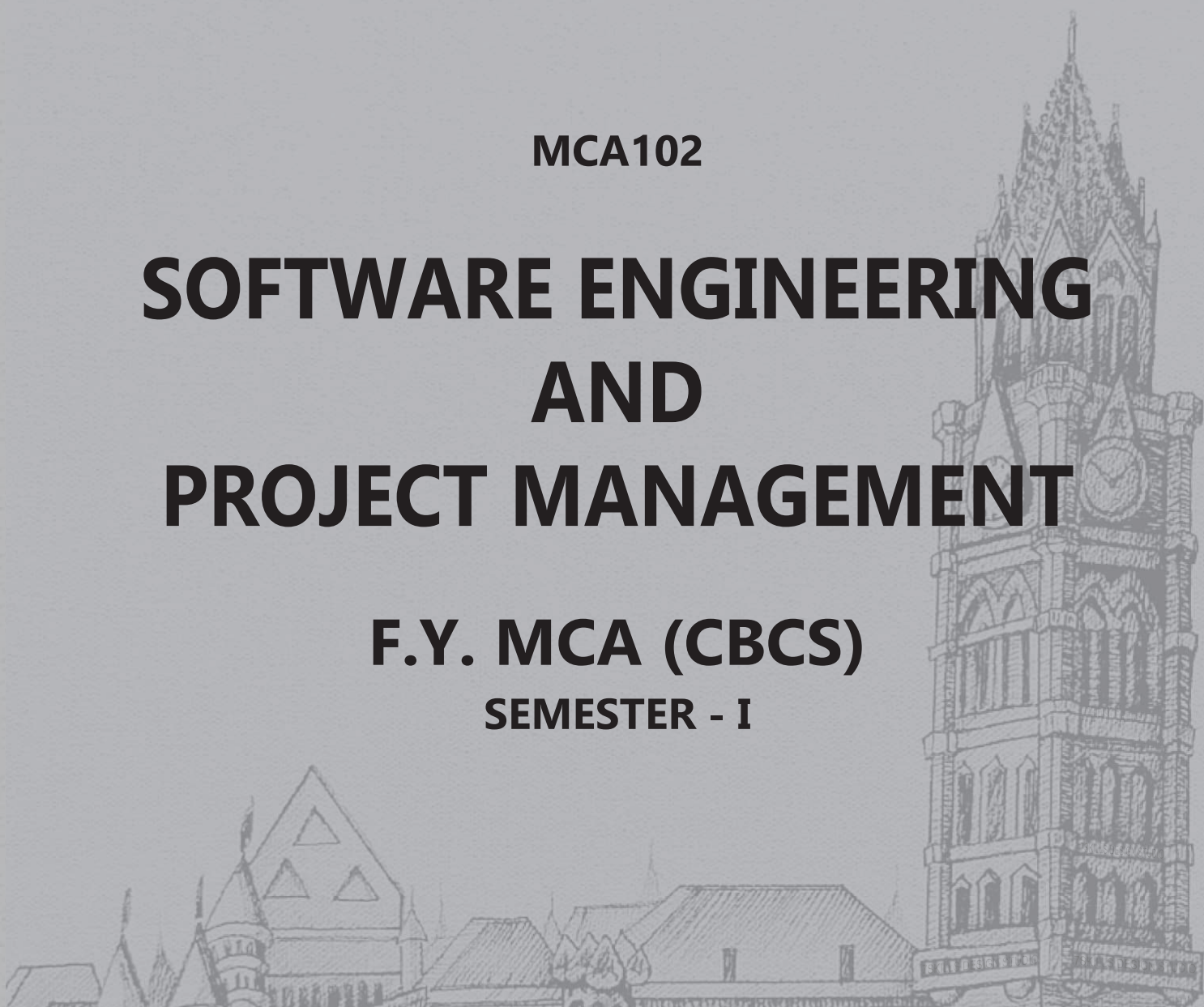


University of Mumbai
INSTITUTE OF DISTANCE AND OPEN LEARNING

MCA102

SOFTWARE ENGINEERING AND PROJECT MANAGEMENT

F.Y. MCA (CBCS)
SEMESTER - I



Software Engineering and Project Management

F.Y. MCA (CBCS)

Semester - I

Prof. Suhas Pednekar

Vice Chancellor

University of Mumbai, Mumbai

Prof. Prakash Mahanwar

Director Incharge

Institute of Distance & Open Learning,

University of Mumbai, Mumbai

Prof. Madhura Kulkarni

Asst. Director & Incharge

Study Material Section,

University of Mumbai, Mumbai

Programme Co-ordinator, I/c Faculty of Science & Technology

Prof. Mandar Bhanushe

Department of Mathematics, University of Mumbai, IDOL

Course Co-ordinator

Asst. Prof. Reshma Kurkute

Department - MCA, University of Mumbai, IDOL

Asst. Prof. Shardul Gavande

Department - MCA, University of Mumbai, IDOL

Course Writer

Asst. Prof. Amit Kukreja

K. J. Somaiya Institute of Engineering & Information Technology, Sion. Mumbai.

Asst. Prof. Vishal Khandekar

K. J. Somaiya Institute of Engineering & Information Technology, Sion. Mumbai.

Asst. Prof. Pallavi Chavda

K. J. Somaiya Institute of Engineering & Information Technology, Sion. Mumbai.

Asst. Prof. Prachi Surve

Department - I.T., Ramniranjan Jhunjunwala College, Ghatkopar (W), Mumbai.

Published by

Dr. Prakash Mahanwar, Director Incharge

on behalf of Institute of Distance & Open Learning, University of Mumbai, Mumbai

and Printed at Printers Name

Printers address

DTP Composed by M/s 7SKILLS

CONTENTS

Chapter No.	Title	Page No.
1.	Introduction to Software Engineering and Project Management	01
2.	Overview of IT Project Management.....	09
3.	Software Process Models.....	19
4.	Software Requirement Analysis and Specification	47
5.	Software Requirement Specification	73
6.	Software Project Planning	101
7.	Project Scheduling and Procurement Management	119
8.	Project Procurement Management	140
9.	Out Sourcing	161
10.	Software and System Quality Management.....	169
11.	Modern Quality Management	187
12.	Human Resource Management.....	195
13.	Managing Change, Resistance and Conflict	215
14.	Software Risk Management	232
15.	Software Reliability Issues	245

Software Engineering and Project Management

F.Y. MCA (CBCS)

Semester - I

SYLLABUS

Sr. No	Module	Detailed Contents	Hours
1	Introduction to software engineering and project management	Introduction to Software Engineering: Software, Evolving role of software, Three “R”-Reuse, Reengineering and Retooling, An Overview of IT Project Management: Define project, project management framework, The role of project Manager, Systems View of Project Management, Stakeholder management, Project phases and the project life cycle.	6
2	Software Process Models	Waterfall Model, Evolutionary Process Model: Prototype and Spiral Model, Incremental Process model: Iterative approach, RAD, JAD model, Concurrent Development Model, Agile Development: Extreme programming, Scrum.	6
3	Software Requirement Analysis and Specification	Types of Requirement, Feasibility Study, Requirement Analysis and Design: DFD, Data Dictionary, HIPO Chart, Warnier Orr Diagram, Requirement Elicitation: Interviews, Questionnaire, Brainstorming, Facilitated Application Specification Technique (FAST), Use Case Approach. SRS Case study, Software Estimation: Size Estimation: Function Point (Numericals). Cost Estimation: COCOMO(Numericals), COCOMO-II (Numericals). Earned Value Management.	11
4	Software Project Planning	Business Case, Project selection and Approval, Project charter, Project Scope management: Scope definition and Project Scope management, Creating the Work Breakdown Structures, Scope Verification, Scope Control.	8

5	Project Scheduling and Procurement management	Relationship between people and Effort: Staffing Level Estimation, Effect of schedule Change on Cost, Degree of Rigor & Task set selector, Project Schedule, Schedule Control, CPM (Numericals), Basic Planning Purchases and Acquisitions, Planning Contracting, Requesting Seller Responses, Selecting Sellers, Out Sourcing: The Beginning of the outsourcing phenomenon, Types of outsourcing relationship, The realities of outsourcing, Managing the outsourcing relationship.	6
6	Software Quality	Software and System Quality Management: Overview of ISO 9001, SEI Capability Maturity Model, McCall's Quality Model, Six Sigma, Formal Technical Reviews, Tools and Techniques for Quality Control, Pareto Analysis, Statistical Sampling, Quality Control Charts and the seven Run Rule. Modern Quality Management, Juran and the importance of Top management, Commitment to Quality, Crosby and Striving for Zero defects, Ishikawa and the Fishbone Diagram.	7 Hrs
7	Human Resource Management	Human Resource Planning, Acquiring the Project Team: Resource Assignment, Loading, Leveling, Developing the Project Team: Team Structures, Managing the Project Team, Change management: Dealing with Conflict & Resistance Leadership & Ethics.	4 Hrs
8	Software Risk Management and Reliability issues	Risk Management: Identify IT Project Risk, Risk Analysis and Assessment, Risk Strategies, Risk Monitoring and Control, Risk Response and Evaluation. Software Reliability: Reliability Metrics, Reliability Growth Modeling.	4 Hrs

INTRODUCTION TO SOFTWARE ENGINEERING AND PROJECT MANAGEMENT

Unit Structure

- 1.0 Objectives
- 1.1 Introduction
 - 1.1.1 Software Engineering
 - 1.1.2 Software
 - 1.1.3 Evolving Role of Software
 - 1.1.4 Three “R” Reuse
- 1.2 Summary

Objectives

After going through this unit, you will be able to learn:

- What is Software Engineering?
- Evolution of software.
- Software Development and Reuse
- Project Management and role of Project Managers
- Project Lifecycle

1.1 Introduction

The world can't operate without software. Industries are controlled by software systems, as the financial systems, scientific labs, infrastructures and utilities, games, film, television, and the list goes on.

Good software should deliver the main required functionality.

Other set of attributes — called quality or non-functional — should be also delivered. Examples of these attributes are, the software is written in a way that can be adapted to changes, response time, performance (less use of resources such as memory and processor time), usable; acceptable for the type of the user it's built for, reliable, secure, safe, ...etc.

1.1.1 Software Engineering:

Software engineering is an engineering discipline that's applied to the development of software in a systematic approach (called a software process). It's the application of theories, methods, and tools to design build a software that meets the specifications efficiently, cost-effectively, and ensuring quality. It's not only concerned with the technical process of building software, it also includes activities to manage the project, develop tools, methods and theories that support the software production. Not applying software engineering methods results in more expensive, less reliable software, and it can be vital on the long term, as the changes come in, the costs will dramatically increase. Different methods and techniques of software engineering are appropriate for different types of systems. For example, games should be developed using series of prototypes, while critical control systems require a complete analyzable specification to be developed.

Computer Science Vs Software Engineering

Computer science focuses on the theory and fundamentals, like algorithms, programming languages, theories of computing, artificial intelligence, and hardware design, while software engineering is concerned with the activities of developing and managing software.

Software Engineers

The job of a software engineer is difficult. It has to balance between different people involved, such as

Dealing with users: User doesn't know what to expect exactly from the software. The concern is always about the ease of use and response time.

Dealing with technical people: Developers are more technically inclined people so they think more of database terms, functionality, etc.

Dealing with management: They are concerned with return on their investment, and meeting the schedule. For success in large software developments, it is important to follow an engineering approach, consisting of a well defined process. That's what we're going to explore next in the "Software Processes".

1.1.2 Software

Software: "So, what's software anyway?" ... software is a computer programs along with the associated documents and the configuration data that make these programs operate correctly.

A program is a set of instructions (written in form of human-readable code) that performs a specific task.

Types of Software Systems:

There are many different types of software systems from simple to complex systems. These systems may be developed for a particular customer, like systems to support a particular business process, or developed for a general purpose, like any software for our computers such as word processors.

Many systems are now being built with a generic product as base, which is then adapted to suit the requirements of a customer such as SAP system.

1.1.3 Evolving Role of Software

Evolving Role of Software:

Software takes dual role. It is both a product and a vehicle for delivering a product.

As a product: It delivers the computing potential embodied by computer Hardware or by a network of computers. As a vehicle: It is information transformer-producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as single bit or as complex as a multimedia presentation. Software delivers the most important product of our time-information. It transforms personal data. It manages business information to enhance competitiveness. It provides a gateway to worldwide information networks. It provides the means for acquiring information.

The role of computer software has undergone significant change over a span of little more than 50 years. Dramatic Improvements in hardware performance Vast increases in memory and storage capacity, A wide variety of exotic input and output options.

1970s and 1980s:

Osborne characterized a new industrial revolution Toffler called the advent of microelectronics part of the third wave of change in human history. Naisbitt predicted the transformation from an industrial society to an information society. Feigenbaum and McCorduck suggested that information and knowledge would be the focal point for power in the twenty-first century. Stoll argued that the electronic community created by networks and software was the key to knowledge interchange throughout the world.

1990s began:

Toffler described a power shift in which old power structures disintegrate as computers and software lead to a democratization of knowledge. Yourdon worried that U.S companies might lose their competitive edge in software related business and predicted the decline and fall of the American programmer. Hammer and Champy argued that information technologies were to play a pivotal role in the reengineering of the corporation.

Mid-1990s:

The pervasiveness of computers and software spawned a rash of books by neo-luddites.

Later 1990s:

Yourdon re-evaluated the prospects of the software professional and suggested the rise and resurrection of the American programmer. The impact of the Y2K time bomb was at the end of 20th century.

2000s progressed:

Johnson discussed the power of emergence a phenomenon that explains what happens when interconnections among relatively simple entities result in a system that self-organizes to form more intelligent, more adaptive behaviour. Yourdon revisited the tragic events of 9/11 to discuss the continuing impact of global terrorism on the IT community. Wolfram presented a treatise on a new kind of science that posits a unifying theory based primarily on sophisticated software simulations. Daconta and his colleagues discussed the evolution of the semantic web. Today a huge software industry has become a dominant factor in the economies of the industrialized world.

1.1.4 Three “R” Reuse

1. Software Reuse:

A definition of software reuse is the process of creating software systems from predefined software components.

The advantage of software reuse:

- The systematic development of reusable components.
- The systematic reuse of these components as building blocks to create new systems.

A reusable component may be code, but the bigger benefits of reuse come from a broader and higher-level view of what can be reused. Software specifications, designs, tests cases, data, prototypes, plans, documentation, frameworks, and templates are all candidates for reuse.

Software reuse can cut software development time and costs. The major advantages for software reuse are to:

- Increase software productivity.
- Shorten software development time.
- Improve software system interoperability.
- Develop software with fewer people.
- Move personnel more easily from project to project.
- Reduce software development and maintenance costs.
- Produce more standardized software.
- Produce better quality software and provide a powerful competitive advantage.

Select Business Solutions has been helping companies achieve software reuse through both technology and Component Based Development (CBD) methodology for over 10 years.

Solution Breakdown

- Design component and service oriented systems with Select Solution Factory.
- Implement Select Perspective, the leading Software Reuse and Component Based Development life-cycle with Select Process Director.
- Use proven project and mentoring skills to help teams to adopt Component Based Development and Service based architectures.
- Visit our Services area to see what courses and consultancy are available.

2. Reengineering :

Software Re-engineering is a process of software development which is done to improve the maintainability of a software system. Re-engineering is the examination and alteration of a system to reconstitute it in a new form. This process encompasses a combination of sub-processes like reverse engineering, forward engineering, reconstructing etc.

Re-engineering is the reorganizing and modifying existing software systems to make them more maintainable.

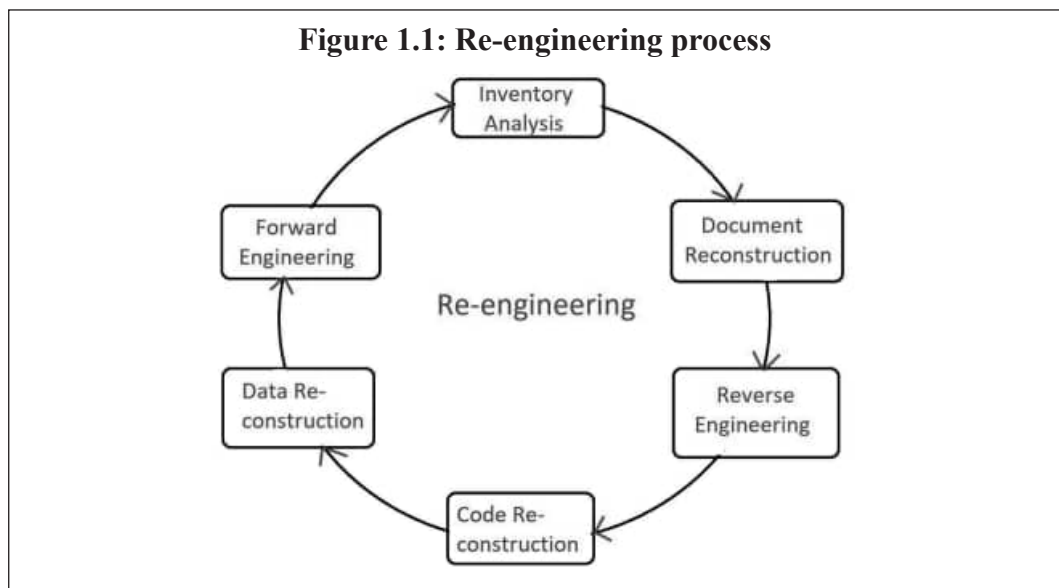
Objectives of Re-engineering:

- To describe a cost-effective option for system evolution.
- To describe the activities involved in the software maintenance process.
- To distinguish between software and data re-engineering and to explain the problems of data re-engineering.

Steps involved in Re-engineering:

1. Inventory Analysis
2. Document Reconstruction
3. Reverse Engineering
4. Code Reconstruction
5. Data Reconstruction
6. Forward Engineering

Diagrammatic Representation (Figure 1.1):



Re-engineering Cost Factors:

- The quality of the software to be re-engineered
- The tool support available for re-engineering
- The extent of the required data conversion
- The availability of expert staff for re-engineering

Advantages of Re-engineering:

- **Reduced Risk:**

As the software already exists, the risk is less as compared to new software development. Development problems, staffing problems and specification problems are the lots of problems which may arise in new software development.

- **Reduced Cost:**

The cost of re-engineering is less than the costs of developing new software.

- **Reverse Engineering:**

It is the de-compilation of any application, regardless of the programming language that was used to create it, so that one can acquire its source code or any part of it. Design for change for instance, focuses on the aspect of developing software due to the fact that each software system will age. Reverse engineering techniques provide the means for recovering lost information and developing alternative representations of a system, such as generation of structure charts, data flow diagrams, entity-relationship diagrams, etc. Software reverse engineering involves reversing a program's machine code back into the source code that it was written in, using program language statements. Software reverse engineering is done in order to retrieve the source code of a program because the source code is lost. It is used to study how the program performs certain operations. Software Reverse Engineering is done to improve the performance of a program and fix a bug. It is adapted to identify malicious content in a program such as a virus. It is used to adapt a program written for use with one microprocessor for use with another and it is done to understand how a product works more comprehensively than by merely observing it. It is used to Investigate and correct errors and limitations in existing programs. The purpose of Software Reverse Engineering is to study the design principles of a product as part of an education in engineering

and make products and systems compatible so they can work together or share data. It is done to evaluate one's own product to understand its limitations and determine whether someone else has literally copied elements of one's own technology. The common use of it is to transform obsolete products into useful ones by adapting them to new systems and platforms.

The reverse engineer can reuse code in his own programs or modify an existing program to perform in other ways. He can use the knowledge gained from RE to correct application programs, also known as bugs. But the most important is that one can get extremely useful ideas by observing how other programmers work and think, thus improving his skills and knowledge.

3. Retooling:

To achieve the first two R's, we need a third R—a new generation of software tools. In retooling the

Software engineering process, we must remember the mistakes of the 1980s and early 1990s. At that time, CASE tools were inserted into a process vacuum, and failed to meet expectations. Tools for the next ten years will address all aspects of the methods landscape. But they should emphasize reuse and re-engineering.

1.2 Summary

- Software Engineering is a systematic approach of software development.
- Software is a program which is a set of instructions that performs a specific task.
- Reuse, Reengineering and Retooling are three R's of Software.



OVERVIEW OF IT PROJECT MANAGEMENT

Unit Structure

- 2.0 Objectives
- 2.1 Introduction to An Overview of IT Project Management
 - 2.1.1 What Is IT Project Management?
 - 2.1.2 Project Management Framework
 - 2.1.3 Responsibilities of an IT Project Manager
 - 2.1.4 System view of a project
 - 2.1.5 Stakeholder Management
 - 2.1.6 Project Phases and the project life cycle
- 2.2 Summary

2.0 Objective

In this unit you are going to learn:

- IT Project Management
- Project Management Framework
- Role of IT Project Manager
- Project System View
- How to manage stakeholder?
- Project phases and lifecycle

2.1 Introduction to an Overview of IT Project Management

In IT, projects have become more complex as technologies rapidly change and end-users demand greater ease-of-use and flexibility. For an IT project manager to achieve their objectives, it is imperative that these initiatives are completed on time and on budget.

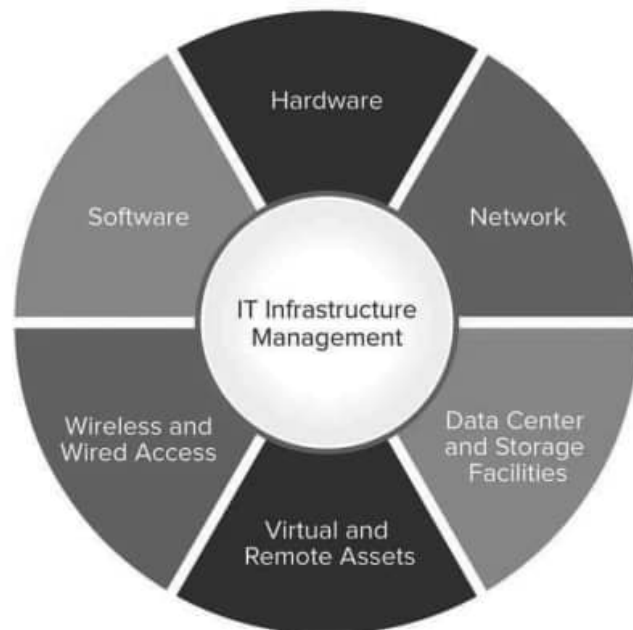
Discover what it means to manage IT projects, common challenges faced by IT project managers, and tips to make your next IT project a success. You'll also find helpful resources, like guides and free templates.

2.1.1 What Is IT Project Management?

IT project management (ITPM) is the process of managing the plan, organization, and accountability to achieve information technology goals. Since the reach of IT spans across most of a business or enterprise, the scope of these projects can be large and complex.

The magnitude of IT project management often means that it's more than just applying knowledge, aligning skills, and using regular tools and techniques to drive a project to completion. IT project managers deal with the challenges of interdependent integrations, rapid technology upgrades, and version changes that can occur throughout the project timeline (Figure 2.1).

Figure 2.1: IT Project Management



2.1.2 Project Management Framework

- **Initiation:**

Project Managers work with other concerned stakeholders to evaluate and determine the values and feasibility of the project

- **Planning:**

Create a project plan: team, timeline, activities, and resources budgeting. Designing required solutions to solve the customer's problem. Communicate project plan to all stakeholders.

- **Execution:**

Team delivery of project plan. Test project implementation to ensure successful project integration. Monitor all aspect of project plan to ensure quality and delivery time. Establish training and modes of continues support for customers

- **Performance Control:**

Project execution process gets evaluated in this phase by means of KPIs like project objectives, quality deliverables, cost monitoring, overall project performance, etc.

- **Closure:**

The promised project deliverables are handed over to the client for validation. A final closure meeting is also held to discuss the overall experience and to close all project accounts.

2.1.3 Responsibilities of an IT Project Manager

Main Responsibilities of an IT Project Manager:

Today's IT project managers (IT PM) must be able to juggle a wide range of tasks and responsibilities. They must be able to handle firmware and software integrations, website construction, database storage and management, and also build complex and geographically diverse infrastructures and networks, all while planning for potential security and data risks.

Throughout their projects, IT PM is responsible for setting goals, communicating and motivating team members and stakeholders, identifying the right resources for each task, researching, managing change, performing needs assessment, and properly sequencing tasks.

Additional responsibilities of the IT project manager include the following:

- Project planning and overall management
- Promoting and achieving project support
- Ensuring overall capability with existing technology
- Minimizing duplicate work
- Utilizing team member skills
- Controlling costs and maintaining budgets

Challenges Faced by IT Project Managers:

The complexities and interdependencies of large-scale, long-term, diverse IT projects are among the most challenging issues of IT projects. Here are a few more top challenges faced by IT project managers:

- Making multiple assumptions when integrating different hardware, networks, and software to the existing system.
- Unclear expectations from the business, end-users, and stakeholders.
- Rapidly changing technology, leading to necessary mid-project upgrades that can affect timelines.
- Geographically diverse offices and remote work associated.

2.1.4 System view of a project

A system view of Project is to take a look into the scope of the project and to know how does that fit into the organization by analyzing the project using the following interrelated elements,

Programmed Objectives

- Rules, Regulations, Constraints and Policy Restrictions
- Management Control
- Inputs
- Implementation Process
- Output
- Outcome
- Impact
- Feedback

So by analyzing the above elements of the project, a project can be planned and managed.

2.1.5 Stakeholder Management

Who are Stakeholders?

Stakeholders are individuals who get impacted by the project. A Stakeholder can be a supporter and a resistor.

Project Stakeholder Management involves identification of stakeholders, analysis of their expectations and influences, development of appropriate strategies to work with the stakeholders and executing the process. Frequent communication is required with the stakeholders. Needs and expectations of the stakeholders to be understood. Managing conflicting interest and involving stakeholders in key project decisions and activities are also crucial. All of this forms a part of the stakeholder management process. The project manager is expected to possess the ability to identify the needs and influences of the stakeholders to manage them effectively.

Identify Stakeholders:

The process of identifying individuals who are impacted by the project is known as Identify Stakeholders Process. The project manager will be able to identify the appropriate focus of each stakeholder as an outcome of Identify Stakeholders process. Stakeholders can include the customers, sponsors, employees, management, government, and society as well. These stakeholders have a potential to exert positive or negative influence on the project deliverables.

Stakeholder needs are to be identified at an early stage of the project to ensure that all their requirements and voices are considered. The stakeholders can be classified on the basis of their interest in the project, the level of influence on the project outcome and their involvement. For the success of the project, the project manager needs to have a relationship that is cordial and extremely success oriented.

Stakeholders Process can receive inputs from:

Project Charter: Internal and external parties related to the project are identified using the project charter

Procurement Documents: The parties involved in a procurement contract are key project stakeholders

Enterprise Environmental Factors: Organizational culture, its structure, governmental regulations, trends, practices or habits of individuals represent enterprise environmental factors.

Organizational Process Assets: Stakeholder registers from previous projects, lessons learned are important inputs for identifying stakeholders

Stakeholder Analysis

A qualitative and quantitative analysis is required to systematically determine the interest of stakeholders throughout the project. The benefits of this analysis are:

Stakeholder interests can be identified

Stakeholder expectations can be identified

Another benefit includes identification of stakeholder relationships that can be leveraged to build partnerships with stakeholders to increase the probability of project success

Steps involved in stakeholder analysis process are:

Identification of potential stakeholders including their roles, departments, interests, knowledge, expectations, and influence levels.

Identify and analyze the potential impact each stakeholder could generate

Classify the stakeholder's basis logical categories of potential impact

Determine the likely reaction of these stakeholders to respond in various situations

Plan the approach strategy to enhance their positive support and reduce negative influences

Multiple classification models are used for stakeholder analysis, but not limited to:

Power/Interest grid: Bifurcation of stakeholders based on their level of authority and their level of concern regarding project outcomes.

Power/Influence grid: Bifurcation of stakeholders based on their level of authority and their level of involvement in the project

Power/Impact grid: Bifurcation of stakeholders based on their level of authority and their level of impacting changes on project activities

Salience model: This model describes categories of stakeholders based on their power, urgency, and legitimacy.

Outputs of Identifying Stakeholders:

- Stakeholder register is updated with details such as:
- Stakeholder information
- Includes their name, organizational position, location, role in the project, business phone number, email address, etc
- Stakeholder requirements
- Key expectations, major requirements, involvement in the project etc
- Stakeholder Classification

2.1.6 Project Phases and the project life cycle

The Project Life Cycle (Phases):

The project manager and project team have one shared goal: to carry out the work of the project for the purpose of meeting the project's objectives. Every project has a beginning, a middle period during which activities move the project toward completion, and an ending (either successful or unsuccessful). A standard project typically has the following four major phases (each with its own agenda of tasks and issues): initiation, planning, implementation, and closure. Taken together, these phases represent the path a project takes from the beginning to its end and are generally referred to as the project "life cycle."

Initiation Phase:

During the first of these phases, the initiation phase, the project objective or need is identified; this can be a business problem or opportunity. An appropriate response to the need is documented in a business case with recommended solution options. A feasibility study is conducted to investigate whether each option addresses the project objective and a final recommended solution is determined. Issues of feasibility ("can we do the project?") and justification ("should we do the project?") are addressed.

Once the recommended solution is approved, a project is initiated to deliver the approved solution and a project manager is appointed. The major deliverables and the participating work groups are identified, and the project team begins to take shape. Approval is then sought by the project manager to move onto the detailed planning phase.

Planning Phase:

The next phase, the planning phase, is where the project solution is further developed in as much detail as possible and the steps necessary to meet the project's objective are planned. In this step, the team identifies all of the work to be done. The project's tasks and resource requirements are identified, along with the strategy for producing them. This is also referred to as "scope management." A project plan is created outlining the activities, tasks, dependencies, and timeframes. The project manager coordinates the preparation of a project budget by providing cost estimates for the labour, equipment, and materials costs. The budget is used to monitor and control cost expenditures during project implementation.

Once the project team has identified the work, prepared the schedule, and estimated the costs, the three fundamental components of the planning process are complete.

This is an excellent time to identify and try to deal with anything that might pose a threat to the successful completion of the project. This is called risk management. In risk management, “high-threat” potential problems are identified along with the action that is to be taken on each high-threat potential problem, either to reduce the probability that the problem will occur or to reduce the impact on the project if it does occur. This is also a good time to identify all project stakeholders and establish a communication plan describing the information needed and the delivery method to be used to keep the stakeholders informed.

Finally, you will want to document a quality plan, providing quality targets, assurance, and control measures, along with an acceptance plan, listing the criteria to be met to gain customer acceptance. At this point, the project would have been planned in detail and is ready to be executed.

Implementation (Execution) Phase:

During the third phase, the implementation phase, the project plan is put into motion and the work of the project is performed. It is important to maintain control and communicate as needed during implementation. Progress is continuously monitored and appropriate adjustments are made and recorded as variances from the original plan. In any project, a project manager spends most of the time in this step. During project implementation, people are carrying out the tasks, and progress information is being reported through regular team meetings. The project manager uses this information to maintain control over the direction of the project by comparing the progress reports with the project plan to measure the performance of the project activities and take corrective action as needed. The first course of action should always be to bring the project back on course (i.e., to return it to the original plan). If that cannot happen, the team should record variations from the original plan and record and publish modifications to the plan. Throughout this step, project sponsors and other key stakeholders should be kept informed of the project’s status according to the agreed-on frequency and format of communication. The plan should be updated and published on a regular basis.

Status reports should always emphasize the anticipated end point in terms of cost, schedule, and quality of deliverables. Each project deliverable produced should be reviewed for quality and measured against the acceptance criteria. Once all of the deliverables have been produced and the customer has accepted the final solution, the project is ready for closure.

Closing Phase:

During the final closure, or completion phase, the emphasis is on releasing the final deliverables to the customer, handing over project documentation to the business, terminating supplier contracts, releasing project resources, and communicating the closure of the project to all stakeholders. The last remaining step is to conduct lessons-learned studies to examine what went well and what didn't. Through this type of analysis, the wisdom of experience is transferred back to the project organization, which will help future project teams.

Example: Project Phases on a Large Multinational Project:

A U.S. construction company won a contract to design and build the first copper mine in northern Argentina. There was no existing infrastructure for either the mining industry or large construction projects in this part of South America. During the initiation phase of the project, the project manager focused on defining and finding a project leadership team with the knowledge, skills, and experience to manage a large complex project in a remote area of the globe. The project team set up three offices. One was in Chile, where large mining construction project infrastructure existed. The other two were in Argentina. One was in Buenos Aires to establish relationships and Argentinean expertise, and the second was in Catamarca—the largest town close to the mine site. With offices in place, the project start-up team began developing procedures for getting work done, acquiring the appropriate permits, and developing relationships with Chilean and Argentine partners.

During the planning phase, the project team developed an integrated project schedule that coordinated the activities of the design, procurement, and construction teams. The project controls team also developed a detailed budget that enabled the project team to track project expenditures against the expected expenses. The project design team built on the conceptual design and developed detailed drawings for use by the procurement team. The procurement team used the drawings to begin ordering equipment and materials for the construction team; develop labour projections; refine the construction schedule; and set up the construction site. Although planning is a never-ending process on a project, the planning phase focused on developing sufficient details to allow various parts of the project team to coordinate their work and allow the project management team to make priority decisions.

The implementation phase represents the work done to meet the requirements of the scope of work and fulfil the charter. During the implementation phase, the project team accomplished the work defined in the plan and made adjustments when the project factors changed. Equipment and materials were delivered to the work site,

labour was hired and trained, a construction site was built, and all the construction activities, from the arrival of the first dozer to the installation of the final light switch, were accomplished.

The closeout phase included turning over the newly constructed plant to the operations team of the client. A punch list of a few remaining construction items was developed and those items completed. The office in Catamarca was closed, the office in Buenos Aires archived all the project documents, and the Chilean office was already working on the next project. The accounting books were reconciled and closed, final reports written and distributed, and the project manager started on a new project.

2.2 Summary

- IT project managers deal with the challenges of interdependent integrations, rapid technology upgrades, and version changes that can occur throughout the project timeline.
- IT project manager is responsible for setting goals, communicating and motivating team members and stakeholders, identifying the right resources for each task, researching, managing change, performing needs assessment, and properly sequencing tasks.
- Stakeholders have a potential to exert positive or negative influence on the project deliverables.
- Project Life Cycle contains initiation, planning, implementation, and closure.



SOFTWARE PROCESS MODELS

Unit Structure

3.0 Objectives

3.1 Introduction

3.2 Software Process Models:

3.2.1 Waterfall Model

3.2.2 Evolutionary process model

3.2.2.1 Prototype Model

3.2.2.2 Spiral Model

3.2.3 Incremental Process Model

3.2.3.1 Iterative Approach

3.2.3.2 Rapid Application Development Model

3.2.3.3 Joint Application Development Model

3.2.3.4 Concurrent Development Model

3.2.4 Agile Development

3.2.4.1 Extreme Programming

3.2.4.2 Scrum

3.3 Summary

3.0 Objectives

After going through this unit, you will be able to learn:

- Different kind of approaches to built Software
- Software development approaches benefits and limitations.
- Need for Software Development Models

3.1 Introduction

Software Process Models were originally proposed to bring order to the chaos of software development. History has indicated that these conventional models have brought a certain amount of useful structure to software engineering work and have provided reasonably effective roadmap for software teams.

3.2 Software Process Models

A software process model (sometimes called a Software Development Life Cycle or SDLC model) is a simplified representation of a software process. Each process model represents a process from a particular perspective and thus only provides partial information about that process. For example, a process activity model shows the activities and their sequence but may not show the roles of the people involved in these activities. In this section, I introduce a number of very general process models (sometimes called process paradigms) and present these from an architectural perspective. That is, we see the framework of the process but not the details of process activities. These generic models are high-level, abstract descriptions of software processes that can be used to explain different approaches to software development. You can think of them as process frameworks that may be extended and adapted to create more specific software engineering processes.

3.2.1 Waterfall Model

Waterfall Model:

The Waterfall Model was the first Process Model to be introduced. It is also referred to as a linear-sequential life cycle model. It is very simple to understand and use. In a waterfall model, each phase must be completed before the next phase can begin and there is no overlapping in the phases.

The Waterfall model is the earliest SDLC approach that was used for software development.

The waterfall Model illustrates the software development process in a linear sequential flow. This means that any phase in the development process begins only if the previous phase is complete. In this waterfall model, the phases do not overlap.

Waterfall Model – Design:

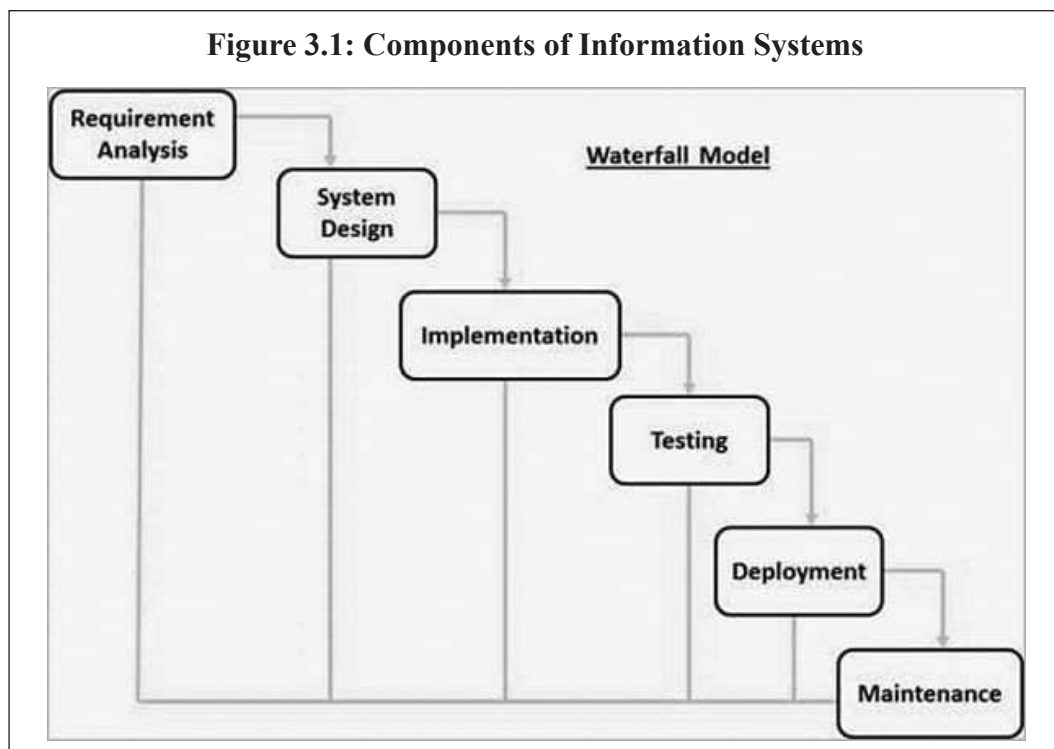
Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project. In “The Waterfall” approach, the whole process of software development is divided into separate phases. In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

The following illustration is a representation of the different phases of the Waterfall Model (Figure 3.1).

The sequential phases in Waterfall model are –

- Requirement Gathering and analysis – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- System Design – the requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- Implementation – with inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.

Figure 3.1: Components of Information Systems



- **Integration and Testing** – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
- **Deployment of system** – Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** – There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

All these phases are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases. The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name “Waterfall Model”. In this model, phases do not overlap.

Waterfall Model – Application:

Every software developed is different and requires a suitable SDLC approach to be followed based on the internal and external factors. Some situations where the use of Waterfall model is most appropriate are –

- Requirements are very well documented, clear and fixed.
- Product definition is stable.
- Technology is understood and is not dynamic.
- There are no ambiguous requirements.
- Ample resources with required expertise are available to support the product.
- The project is short.

Waterfall Model - Advantages

The advantages of waterfall development are that it allows for departmentalization and control. A schedule can be set with deadlines for each stage of development and a product can proceed through the development process model phases one by one.

Development moves from concept, through design, implementation, testing, installation, troubleshooting, and ends up at operation and maintenance. Each phase of development proceeds in strict order.

Some of the major advantages of the Waterfall Model are as follows –

- Simple and easy to understand and use
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

Waterfall Model – Disadvantages:

The disadvantage of waterfall development is that it does not allow much reflection or revision. Once an application is in the testing stage, it is very difficult to go back and change something that was not well-documented or thought upon in the concept stage.

The major disadvantages of the Waterfall Model are as follows –

- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Adjusting scope during the life cycle can end a project.
- Integration is done as a “big-bang. at the very end, which doesn’t allow identifying any technological or business bottleneck or challenges early.

3.2.2 Evolutionary process model:

Evolutionary model is a combination of Iterative and Incremental model of software development life cycle. Delivering your system in a big bang release, delivering it in incremental process over time is the action done in this model. Some initial requirements and architecture envisioning need to be done.

It is better for software products that have their feature sets redefined during

development because of user feedback and other factors. The Evolutionary development model divides the development cycle into smaller, incremental waterfall models in which users are able to get access to the product at the end of each cycle.

Feedback is provided by the users on the product for the planning stage of the next cycle and the development team responds, often by changing the product, plan or process. Therefore, the software product evolves with time.

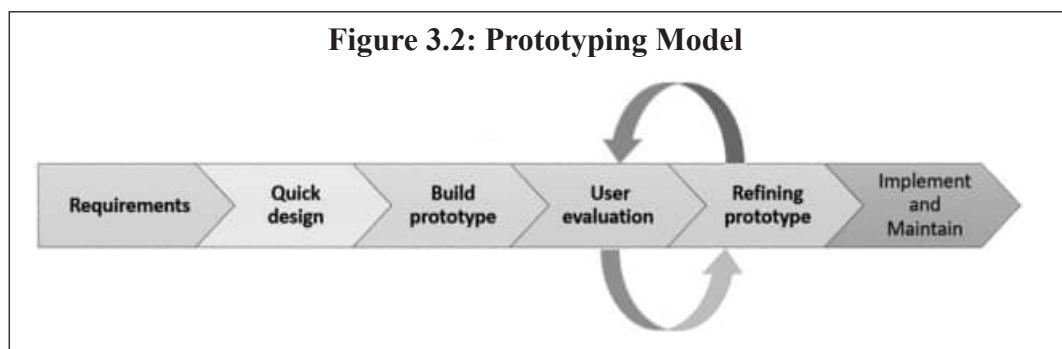
All the models have the disadvantage that the duration of time from start of the project to the delivery time of a solution is very high. Evolutionary model solves this problem in a different approach.

Evolutionary model suggests breaking down of work into smaller chunks, prioritizing them and then delivering those chunks to the customer one by one. The number of chunks is huge and is the number of deliveries made to the customer. The main advantage is that the customer's confidence increases as he constantly gets quantifiable goods or services from the beginning of the project to verify and validate his requirements. The model allows for changing requirements as well as all work in broken down into maintainable work chunks.

3.2.2.1 Prototype Model

Prototype methodology is defined as a Software Development model in which a prototype is built, tests, and then reworked when needed until an acceptable prototype is achieved. It also creates a base to produce the final system.

Software prototyping model works best in scenarios where the project's requirement are not known. It is an iterative, trial, and error method which take place between the developer and the client (Figure 3.2).



Prototyping Model Phases

Prototyping Model has following six SDLC phases as follow:

Step 1: Requirements gathering and analysis

A prototyping model starts with requirement analysis. In this phase, the requirements of the system are defined in detail. During the process, the users of the system are interviewed to know what their expectation from the system is.

Step 2: Quick design

The second phase is a preliminary design or a quick design. In this stage, a simple design of the system is created. However, it is not a complete design. It gives a brief idea of the system to the user. The quick design helps in developing the prototype.

Step 3: Build a Prototype

In this phase, an actual prototype is designed based on the information gathered from quick design. It is a small working model of the required system.

Step 4: Initial user evaluation

In this stage, the proposed system is presented to the client for an initial evaluation. It helps to find out the strength and weakness of the working model. Comment and suggestion are collected from the customer and provided to the developer.

Step 5: Refining prototype

If the user is not happy with the current prototype, you need to refine the prototype according to the user's feedback and suggestions.

This phase will not over until all the requirements specified by the user are met. Once the user is satisfied with the developed prototype, a final system is developed based on the approved final prototype.

Step 6: Implement Product and Maintain

Once the final system is developed based on the final prototype, it is thoroughly tested and deployed to production. The system undergoes routine maintenance for minimizing downtime and prevents large-scale failures.

Four types of Prototyping models are:

- Rapid Throwaway prototypes
- Evolutionary prototype
- Incremental prototype
- Extreme prototype

Rapid Throwaway Prototype:

Rapid throwaway is based on the preliminary requirement. It is quickly developed to show how the requirement will look visually. The customer's feedback helps drives changes to the requirement, and the prototype is again created until the requirement is base lined. In this method, a developed prototype will be discarded and will not be a part of the ultimately accepted prototype. This technique is useful for exploring ideas and getting instant feedback for customer requirements.

Evolutionary Prototyping:

Here, the prototype developed is incrementally refined based on customer's feedback until it is finally accepted. It helps you to save time as well as effort. That's because developing a prototype from scratch for every interaction of the process can sometimes be very frustrating.

This model is helpful for a project which uses a new technology that is not well understood. It is also used for a complex project where every functionality must be checked once. It is helpful when the requirement is not stable or not understood clearly at the initial stage.

Incremental Prototyping:

In incremental Prototyping, the final product is decimated into different small prototypes and developed individually. Eventually, the different prototypes are merged into a single product. This method is helpful to reduce the feedback time between the user and the application development team.

Extreme Prototyping:

Extreme prototyping method is mostly used for web development. It is consists of three sequential phases.

Basic prototype with the entire existing page is present in the HTML format. You can simulate data process using a prototype services layer. The services are implemented and integrated into the final prototype. Best practices of Prototyping. Here, are a few things which you should watch for during the prototyping process:

You should use Prototyping when the requirements are unclear. It is important to perform planned and controlled Prototyping. Regular meetings are vital to keep the project on time and avoid costly delays. The users and the designers should be aware of the prototyping issues and pitfalls. At a very early stage, you need to approve a prototype and only then allow the team to move to the next step.

In software prototyping method, you should never be afraid to change earlier decisions if new ideas need to be deployed. You should select the appropriate step

size for each version. Implement important features early on so that if you run out of the time, you still have a worthwhile system

Advantages of the Prototyping Model

Here, are important pros/benefits of using Prototyping models:

- Users are actively involved in development. Therefore, errors can be detected in the initial stage of the software development process.
- Missing functionality can be identified, which helps to reduce the risk of failure as Prototyping is also considered as a risk reduction activity.
- Helps team member to communicate effectively.
- Customer satisfaction exists because the customer can feel the product at a very early stage. There will be hardly any chance of software rejection.
- Quicker user feedback helps you to achieve better software development solutions. It allows the client to compare if the software code matches the software specification.
- It helps you to find out the missing functionality in the system. It also identifies the complex or difficult functions that encourage innovation and flexible designing.
- It is a straightforward model, so it is easy to understand. No need for specialized experts to build the model.
- The prototype serves as a basis for deriving a system specification.
- The prototype helps to gain a better understanding of the customer's needs. Prototypes can be changed and even discarded.
- Prototype also serves as the basis for operational specifications.
- Prototypes may offer early training for future users of the software system.

Disadvantages of the Prototyping Model:

- Prototyping is a slow and time taking process.
- The cost of developing a prototype is a total waste as the prototype is ultimately thrown away.
- Prototyping may encourage excessive change requests.
- Sometimes customers may not be willing to participate in the iteration cycle for the longer time duration.
- There may be far too many variations in software requirements when each time the prototype is evaluated by the customer.

- The poor documentation because the requirements of the customers are changing.
- It is very difficult for software developers to accommodate all the changes demanded by the clients.
- After seeing an early prototype model, the customers may think that the actual product will be delivered to him soon.
- The client may lose interest in the final product when he or she is not happy with the initial prototype.
- Developers who want to build prototypes quickly may end up building sub-standard development solutions.

3.2.2.2 Spiral Model

Spiral Model:

The spiral model combines the idea of iterative development with the systematic, controlled aspects of the waterfall model. This Spiral model is a combination of iterative development process model and sequential linear development model i.e. the waterfall model with a very high emphasis on risk analysis. It allows incremental releases of the product or incremental refinement through each iteration around the spiral.

Spiral Model – Design:

The spiral model has four phases. A software project repeatedly passes through these phases in iterations called Spirals.

Identification:

This phase starts with gathering the business requirements in the baseline spiral. In the subsequent spirals as the product matures, identification of system requirements, subsystem requirements and unit requirements are all done in this phase. This phase also includes understanding the system requirements by continuous communication between the customer and the system analyst. At the end of the spiral, the product is deployed in the identified market.

Design:

The Design phase starts with the conceptual design in the baseline spiral and involves architectural design, logical design of modules, physical product design and the final design in the subsequent spirals.

Construct or Build:

The Construct phase refers to production of the actual software product at every spiral. In the baseline spiral, when the product is just thought of and the design is being developed a POC (Proof of Concept) is developed in this phase to get customer feedback. Then in the subsequent spirals with higher clarity on requirements and design details a working model of the software called build is produced with a version number. These builds are sent to the customer for feedback.

Evaluation and Risk Analysis:

Risk Analysis includes identifying, estimating and monitoring the technical feasibility and management risks, such as schedule slippage and cost overrun. After testing the build, at the end of first iteration, the customer evaluates the software and provides feedback. The following illustration is a representation of the Spiral Model, listing the activities in each phase. Based on the customer evaluation, the software development process enters the next iteration and subsequently follows the linear approach to implement the feedback suggested by the customer. The process of iterations along the spiral continues throughout the life of the software.

Spiral Model Application:

The Spiral Model is widely used in the software industry as it is in sync with the natural development process of any product, i.e. learning with maturity which involves minimum risk for the customer as well as the development firms.

The following pointers explain the typical uses of a Spiral Model –

- When there is a budget constraint and risk evaluation is important.
- For medium to high-risk projects.
- Long-term project commitment because of potential changes to economic priorities as the requirements change with time.
- Customer is not sure of their requirements which are usually the case.
- Requirements are complex and need evaluation to get clarity.
- New product line which should be released in phases to get enough customer feedback.
- Significant changes are expected in the product during the development cycle.

Spiral Model - Pros and Cons:

The advantage of spiral lifecycle model is that it allows elements of the product to be added in, when they become available or known. This assures that there is no conflict with previous requirements and design.

This method is consistent with approaches that have multiple software builds and releases which allows making an orderly transition to a maintenance activity. Another positive aspect of this method is that the spiral model forces an early user involvement in the system development effort.

On the other side, it takes a very strict management to complete such products and there is a risk of running the spiral in an indefinite loop. So, the discipline of change and the extent of taking change requests is very important to develop and deploy the product successfully.

The advantages of the Spiral SDLC Model are as follows –

- Changing requirements can be accommodated.
- Allows extensive use of prototypes.
- Requirements can be captured more accurately.
- Users see the system early.
- Development can be divided into smaller parts and the risky parts can be developed earlier which helps in better risk management.

The disadvantages of the Spiral SDLC Model are as follows –

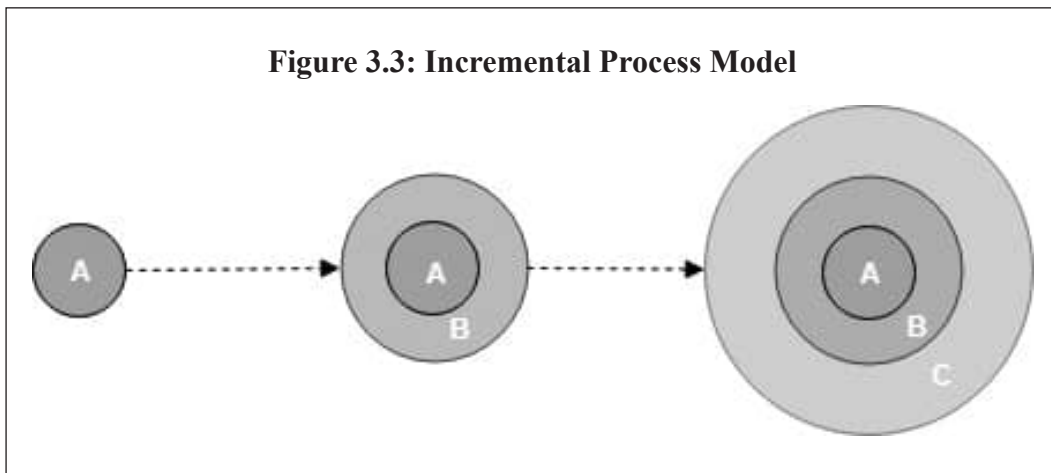
- Management is more complex.
- End of the project may not be known early.
- Not suitable for small or low risk projects and could be expensive for small projects.
- Process is complex
- Spiral may go on indefinitely.
- Large number of intermediate stages requires excessive documentation.

3.2.3 Incremental Process Model

Incremental process model is also known as successive version model.

First, a simple working system implementing only a few basic features is built and then that is delivered to the customer. Then thereafter many successive iterations/versions are implemented and delivered to the customer until the desired system is released (Figure 3.3).

A, B, C are modules of Software Product that are incrementally developed and delivered.

Figure 3.3: Incremental Process Model**Life cycle activities –**

Requirements of Software are first broken down into several modules that can be incrementally constructed and delivered. At any time, the plan is made just for the next increment and not for any kind of long term plans. Therefore, it is easier to modify the version as per the need of the customer. Development Team first undertakes to develop core features (these do not need services from other features) of the system.

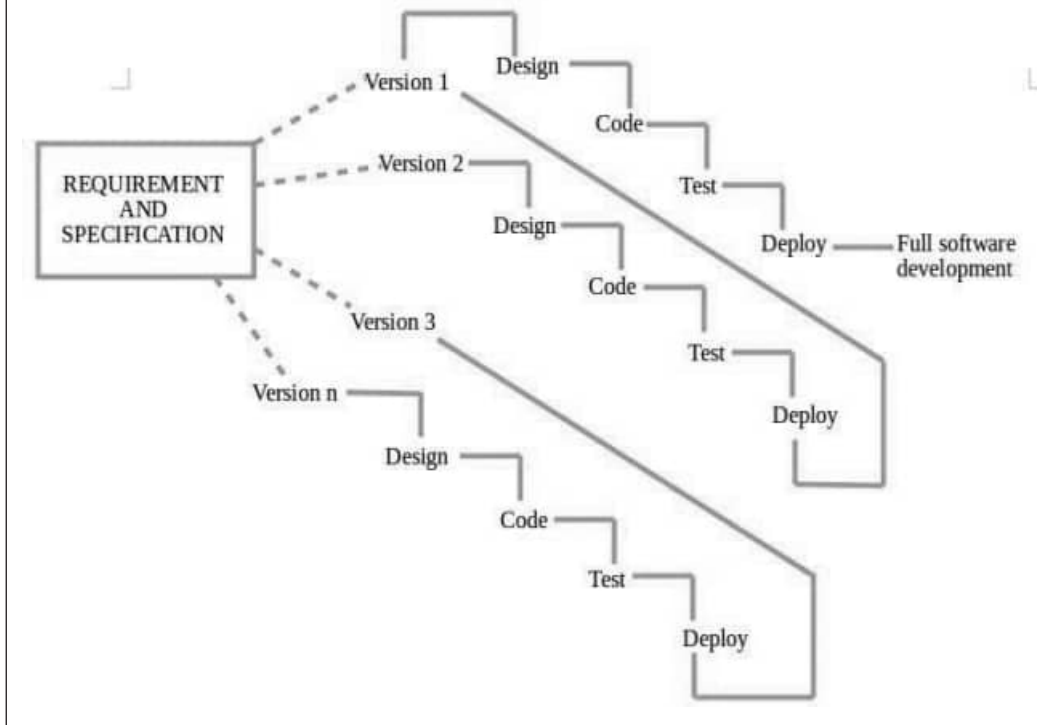
Once the core features are fully developed, then these are refined to increase levels of capabilities by adding new functions in Successive versions. Each incremental version is usually developed using an iterative waterfall model of development.

As each successive version of the software is constructed and delivered, now the feedback of the Customer is to be taken and these were then incorporated in the next version. Each version of the software has more additional features over the previous ones (Figure 3.4).

After Requirements gathering and specification, requirements are then spitted into several different versions starting with version-1, in each successive increment, next version is constructed and then deployed at the customer site. After the last version (version n), it is now deployed at the client site.

Types of Incremental model –

1. Staged Delivery Model – Construction of only one part of the project at a time.
2. Parallel Development Model – Different subsystems are developed at the same time. It can decrease the calendar time needed for the development, i.e. TTM (Time to Market), if enough Resources are available.

Figure 3.4: Requirements**When to use this –**

1. Funding Schedule, Risk, Program Complexity, or need for early realization of benefits.
2. When Requirements are known up-front.
3. When Projects having lengthy developments schedules.
4. Projects with new Technology.

Advantages –

- Error Reduction (core modules are used by the customer from the beginning of the phase and then these are tested thoroughly)
- Uses divide and conquer for breakdown of tasks.
- Lowers initial delivery cost.
- Incremental Resource Deployment.

Disadvantages –

- Requires good planning and design.
- Total cost is not lower.
- Well defined module interfaces are required.

3.2.3.1 Iterative Approach

In an Iterative Incremental model, initially, a partial implementation of a total system is constructed so that it will be in a deliverable state. Increased functionality is added. Defects, if any, from the prior delivery are fixed and the working product is delivered. The process is repeated until the entire product development is completed. The repetitions of these processes are called iterations. At the end of every iteration, a product increment is delivered.

Iterative Incremental Model – Strengths

The advantages or strengths of Iterative Incremental model are –

- You can develop prioritized requirements first.
- Initial product delivery is faster.
- Customers gets important functionality early.
- Lowers initial delivery cost.
- Each release is a product increment, so that the customer will have a working product at hand all the time.
- Customer can provide feedback to each product increment, thus avoiding surprises at the end of development.
- Requirements changes can be easily accommodated.

Iterative Incremental Model – Weaknesses

The disadvantages of the Iterative Incremental model are –

- Requires effective planning of iterations.
- Requires efficient design to ensure inclusion of the required functionality and provision for changes later.
- Requires early definition of a complete and fully functional system to allow the definition of increments.
- Well-defined module interfaces are required, as some are developed long before others are developed.
- Total cost of the complete system is not lower.

When to Use Iterative Incremental Model?

- Iterative Incremental model can be used when –
- Most of the requirements are known up-front but are expected to evolve over time.

- The requirements are prioritized.
- There is a need to get the basic functionality delivered fast.
- A project has lengthy development schedules.
- A project has new technology.
- The domain is new to the team.

3.2.3.2 Rapid Application Development

RAD (Rapid Application Development) model:

The RAD (Rapid Application Development) model is based on prototyping and iterative development with no specific planning involved. The process of writing the software itself involves the planning required for developing the product.

Rapid Application Development focuses on gathering customer requirements through workshops or focus groups, early testing of the prototypes by the customer using iterative concept, reuse of the existing prototypes (components), continuous integration and rapid delivery.

What is RAD?

Rapid application development is a software development methodology that uses minimal planning in favour of rapid prototyping. A prototype is a working model that is functionally equivalent to a component of the product. In the RAD model, the functional modules are developed in parallel as prototypes and are integrated to make the complete product for faster product delivery. Since there is no detailed preplanning, it makes it easier to incorporate the changes within the development process. RAD projects follow iterative and incremental model and have small teams comprising of developers, domain experts, customer representatives and other IT resources working progressively on their component or prototype. The most important aspect for this model to be successful is to make sure that the prototypes developed are reusable.

RAD Model Design

RAD model distributes the analysis, design, build and test phases into a series of short, iterative development cycles.

Following are the various phases of the RAD Model –**Business Modelling:**

The business model for the product under development is designed in terms of flow of information and the distribution of information between various business channels. A complete business analysis is performed to find the vital information for business, how it can be obtained, how and when is the information processed and what are the factors driving successful flow of information.

Data Modelling:

The information gathered in the Business Modeling phase is reviewed and analyzed to form sets of data objects vital for the business. The attributes of all data sets is identified and defined. The relation between these data objects are established and defined in detail in relevance to the business model.

Process Modelling:

The data object sets defined in the Data Modeling phase are converted to establish the business information flow needed to achieve specific business objectives as per the business model. The process model for any changes or enhancements to the data object sets is defined in this phase. Process descriptions for adding, deleting, retrieving or modifying a data object are given.

Application Generation:

The actual system is built and coding is done by using automation tools to convert process and data models into actual prototypes.

Testing and Turnover:

The overall testing time is reduced in the RAD model as the prototypes are independently tested during every iteration. However, the data flow and the interfaces between all the components need to be thoroughly tested with complete test coverage. Since most of the programming components have already been tested, it reduces the risk of any major issues.

The following illustration describes the RAD Model in detail.

RAD Model Vs Traditional SDLC:

The traditional SDLC follows a rigid process models with high emphasis on requirement analysis and gathering before the coding starts. It puts pressure on the

customer to sign off the requirements before the project starts and the customer doesn't get the feel of the product as there is no working build available for a long time. The customer may need some changes after he gets to see the software. However, the change process is quite rigid and it may not be feasible to incorporate major changes in the product in the traditional SDLC. The RAD model focuses on iterative and incremental delivery of working models to the customer. This results in rapid delivery to the customer and customer involvement during the complete development cycle of product reducing the risk of non-conformance with the actual user requirements.

RAD Model – Application:

RAD model can be applied successfully to the projects in which clear modularization is possible. If the project cannot be broken into modules, RAD may fail.

The following pointers describe the typical scenarios where RAD can be used –

- RAD should be used only when a system can be modularized to be delivered in an incremental manner.
- It should be used if there is a high availability of designers for modeling.
- It should be used only if the budget permits use of automated code generating tools.
- RAD SDLC model should be chosen only if domain experts are available with relevant business knowledge.
- Should be used where the requirements change during the project and working prototypes are to be presented to customer in small iterations of 2-3 months.

RAD Model - Pros and Cons

RAD model enables rapid delivery as it reduces the overall development time due to the reusability of the components and parallel development. RAD works well only if high skilled engineers are available and the customer is also committed to achieve the targeted prototype in the given time frame. If there is commitment lacking on either side the model may fail.

The advantages of the RAD Model are as follows –

- Changing requirements can be accommodated.
- Progress can be measured.
- Iteration time can be short with use of powerful RAD tools.

- Productivity with fewer people in a short time.
- Reduced development time.
- Increases reusability of components.
- Quick initial reviews occur.
- Encourages customer feedback.
- Integration from very beginning solves a lot of integration issues.

The disadvantages of the RAD Model are as follows –

- Dependency on technically strong team members for identifying business requirements.
- Only system that can be modularized can be built using RAD.
- Requires highly skilled developers/designers.
- High dependency on modelling skills.
- Inapplicable to cheaper projects as cost of modelling and automated code generation is very high.
- Management complexity is more.
- Suitable for systems that are component based and scalable.
- Requires user involvement throughout the life cycle.
- Suitable for project requiring shorter development times.

3.2.3.3 Joint Application Development Model

Collaboration and then building software is the key power which drives technology and its innovation. JAD is a model for software development that augments the stakeholders' association in cycles of software development. Its life cycle has been adopted for areas of dynamic software development method. It collects business and system requirements while building a new information system for any organization or enterprise. In this chapter, you will learn about the JAD model in detail.

JAD (Joint Application Development) Approach:

JAD (Joint Application Development) is a software development approach which engages the client and/or the end users for designing and developing the system. This model was designed and put forward by Dr. Chuck Morris and Dr. Tony Crawford of IBM, who propose this model in the late 1970s. As compared to other primitive SDLC model, Joint Application Development model leads to faster progression of the system development which has better client approval.

This model furthermore, is vast when it comes to agile delivery wherein the software products need to be developed as well as shipped in short iterations depending on agreements among the industrial as well as industry stakeholders which are termed as Minimum Viable Product (MVP).

Phases of JAD Model: Since you have become familiar with the JAD concept, it is time to know about its phases and how the model's design and development approach works:

Define Specific Objectives: The facilitator, in partnership with stakeholders, set all the objectives as well as a list of items which is then distributed to other developers and participants to understand and review. This objective contains elements like the scope of this projected system, its potential outcome, technical specification required, etc.

Session Preparation: The facilitator is solely responsible for this preparation where all relevant data is collected and sent to other members before time. For better insight, research carried out to know about the system requirement better and gather all the necessary information for development.

Session Conduct: Here the facilitator is accountable to identify those issues which have to be working out for making the system error-free. Here the facilitator will serve as a participant but will not have a say regarding any information.

Documentation: After the product is developed, the records and published documents are put forward into the meeting so that the stakeholders and consumers can approve it through the meeting.

Benefits of using JAD Model:

- **Improved Delivery Time:** The time required for developing a product using JAD model is lesser and efficient than that of other traditional models.
- **Cost Reduction:** Efficiently analyzing the requirements and facts with business executives and stakeholders will make less effort to develop the system and hence less cost will be required for the entire development process.
- **Better Understanding:** Since the entire requirement is analyzed by business executives, followed by a cautious choice of developers and team member who can professionally interact with each other better usually helps in understanding the product development better.

- **Improved Quality:** Since all the key decision makers and stakeholders of the project are involved in the development of the project so there is the least chance of error and hence the product quality becomes better and more accurate.

3.2.3.4 Concurrent Development Model

` The concurrent development model

- The concurrent development model is called as concurrent model.
- The communication activity has completed in the first iteration and exits in the awaiting changes state.
- The modelling activity completed its initial communication and then goes to the underdevelopment state.
- If the customer specifies the change in the requirement, then the modelling activity moves from the under development state into the awaiting change state.
- The concurrent process model activities moving from one state to another state

Advantages of the concurrent development model

- This model is applicable to all types of software development processes.
- It is easy to understand and use.
- It gives immediate feedback from testing.
- It provides an accurate picture of the current state of a project.

Disadvantages of the concurrent development model

- It needs better communication between the team members. This may not be achieved all the time.
- It requires remembering the status of the different activities.

3.2.4 Agile Development

Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver all at once.

Why Agile?

Technology in this current era is progressing faster than ever, enforcing the global software companies to work in a fast-paced changing environment. Because these businesses are operating in an ever-changing environment, it is impossible to gather

a complete and exhaustive set of software requirements. Without these requirements, it becomes practically hard for any conventional software model to work.

The conventional software models such as Waterfall Model that depends on completely specifying the requirements, designing, and testing the system are not geared towards rapid software development. As a consequence, a conventional software development model fails to deliver the required product.

This is where agile software development comes to the rescue. It was specially designed to curate the needs of the rapidly changing environment by embracing the idea of incremental development and developing the actual final product.

Let's now read about the on which the Agile has laid its foundation:

Principles:

1. Highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. It welcomes changing requirements, even late in development.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shortest timescale.
4. Build projects around motivated individuals. Give them the environment and the support they need, and trust them to get the job done.
5. Working software is the primary measure of progress.
6. Simplicity the art of maximizing the amount of work not done is essential.
7. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Development in Agile: Let's see a brief overview of how development occurs in agile philosophy.

- In Agile development, Design and Implementation are considered to be the central activities in the software process.
- Design and Implementation phase also incorporate other activities such as requirements elicitation and testing into it.
- In an agile approach, iteration occurs across activities. Therefore, the requirements and the design are developed together, rather than separately.
- The allocation of requirements and the design planning and development as executed in a series of increments. In contrast with the conventional model, where requirements gathering needs to be completed in order to proceed to

the design and development phase, it gives Agile development an extra level of flexibility.

- An agile process focuses more on code development rather than documentation.

Example: Let's go through an example to understand clearly about how agile actually works.

A Software company named ABC wants to make a new web browser for the latest release of its operating system. The deadline for the task is 10 months. The company's head assigned two teams named Team A and Team B for this task. In order to motivate the teams, the company head says that the first team to develop the browser would be given a salary hike and a one week full sponsored travel plan. With the dreams of their wild travel fantasies, the two teams set out on the journey of the web browser. The team A decided to play by the book and decided to choose the Waterfall model for the development. Team B after a heavy discussion decided to take a leap of faith and choose Agile as their development model.

The Development plan of the Team A is as follows:

- Requirement analysis and Gathering – 1.5 Months
- Design of System – 2 Months
- Coding phase – 4 Months
- System Integration and Testing – 2 Months
- User Acceptance Testing – 5 Weeks

The Development plan for the Team B is as follows:

- Since this was an Agile, the project was broken up into several iterations.
- The iterations are all of the same time duration.
- At the end of each iteration, a working product with a new feature has to be delivered.
- Instead of Spending 1.5 months on requirements gathering, They will decide the core features that are required in the product and decide which of these features can be developed in the first iteration.
- Any remaining features that cannot be delivered in the first iteration will be delivered in the next subsequent iteration, based in the priority

At the end of the first iterations, the team will deliver working software with the core basic features.

Both the team have put their best efforts to get the product to a complete stage. But then out of blue due to the rapidly changing environment, the company's head come up with an entirely new set of features and want to be implemented as quickly as possible and wanted to push out a working model in 2 days. Team A was now in a fix, they were still in their design phase and did not yet started coding and they had no working model to display. And moreover, it was practically impossible for them to implement new features since waterfall model there is not reverting back to the old phase once you proceed to the next stage that means they would have to start from the square one again. That would incur them heavy cost and a lot of overtime. Team B was ahead of Team A in a lot of aspects, all thanks to Agile Development. They also had the working product with most of the core requirement since the first increment. And it was a piece of cake for them to add the new requirements. All they had to do is schedule these requirements for the next increment and then implement them.

Advantages:

- Deployment of software is quicker and thus helps in increasing the trust of the customer.
- Can better adapt to rapidly changing requirements and respond faster.
- Helps in getting immediate feedback which can be used to improve the software in the next increment.
- People – Not Process. People and interactions are given a higher priority rather than process and tools.
- Continuous attention to technical excellence and good design.

Disadvantages:

- In case of large software projects, it is difficult to assess the effort required at the initial stages of the software development life cycle.
- The Agile Development is more code focused and produces less documentation.
- Agile development is heavily depended on the inputs of the customer. If the customer has ambiguity in his vision of the final outcome, it is highly likely for the project to get off track.
- Face to Face communication is harder in large-scale organizations.
- Only senior programmers are capable of taking the kind of decisions required during the development process. Hence it's a difficult situation for new programmers to adapt to the environment.

Agile is a framework which defines how the software development needs to be carried on. Agile is not a single method, it represents the various collection of methods and practices that follow the value statements provided in the manifesto. Agile methods and practices do not promise to solve every problem present in the software industry (No Software model ever can). But they sure help to establish a culture and environment where solutions emerge.

3.2.4.1 Extreme Programming

Extreme programming (XP) is one of the most important software development framework of Agile models. It is used to improve software quality and responsive to customer requirements. The extreme programming model recommends taking the best practices that have worked well in the past in program development projects to extreme levels.

Good practices needs to practiced extreme programming: Some of the good practices that have been recognized in the extreme programming model and suggested to maximize their use are given below:

- **Code Review:** Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their works between them every hour.
- **Testing:** Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach test cases are written even before any code is written.
- **Incremental development:** Incremental development is very good because customer feedback is gained and based on this development team come up with new increments every few days after each iteration.
- **Simplicity:** Simplicity makes it easier to develop good quality code as well as to test and debug it.
- **Design:** Good quality design is important to develop a good quality software. So, everybody should design daily.
- **Integration testing:** It helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

Basic principles of Extreme programming: XP is based on the frequent iteration through which the developers implement User Stories. User stories are simple and informal statements of the customer about the functionalities needed. A User story

is a conventional description by the user about a feature of the required system. It does not mention finer details such as the different scenarios that can occur. On the basis of User stories, the project team proposes Metaphors. Metaphors are a common vision of how the system would work. The development team may decide to build a Spike for some feature. A Spike is a very simple program that is constructed to explore the suitability of a solution being proposed. It can be considered similar to a prototype. Some of the basic activities that are followed during software development by using XP model are given below:

- **Coding:** The concept of coding which is used in XP model is slightly different from traditional coding. Here, coding activity includes drawing diagrams (modeling) that will be transformed into code, scripting a web-based system and choosing among several alternative solutions.
- **Testing:** XP model gives high importance on testing and considers it be the primary factor to develop a fault-free software.
- **Listening:** The developers need to carefully listen to the customers if they have to develop a good quality software. Sometimes programmers may not have the depth knowledge of the system to be developed. So, it is desirable for the programmers to understand properly the functionality of the system and they have to listen to the customers.
- **Designing:** Without a proper design, a system implementation becomes too complex and very difficult to understand the solution, thus it makes maintenance expensive. A good design results in elimination of complex dependencies within a system. So, effective use of suitable design is emphasized.
- **Feedback:** One of the most important aspects of the XP model is to gain feedback to understand the exact customer needs. Frequent contact with the customer makes the development effective.
- **Simplicity:** The main principle of the XP model is to develop a simple system that will work efficiently in present time, rather than trying to build something that would take time and it may never be used. It focuses on some specific features that are immediately needed, rather than engaging time and effort on speculations of future requirements.

Applications of Extreme Programming (XP): Some of the projects that are suitable to develop using XP model are given below:

- **Small projects:** XP model is very useful in small projects consisting of small teams as face to face meeting is easier to achieve.

- **Projects involving new technology or Research projects:** This type of projects faces changing of requirements rapidly and technical problems. So XP model is used to complete this type of projects.

3.2.4.2 Scrum

What is Scrum Project Management?

Scrum is an agile project management methodology or framework used primarily for software development projects with the goal of delivering new software capability every 2-4 weeks. It is one of the approaches that influenced the Agile Manifesto, which articulates a set of values and principles to guide decisions on how to develop higher-quality software faster.

Who Uses Agile Scrum Methodology?

Scrum is widely used by software development teams. In fact it's the most popular agile methodology. According to the 12th annual State of Agile report, 70% of software teams use Scrum or a Scrum hybrid. However, Scrum has spread to other business functions including IT and marketing where there are projects that must move forward in the presence of complexity and ambiguity. Leadership teams are also basing their agile management practices on Scrum, often combining it with lean and Kanban practices (subgroups of agile project management).

What is Scrum in Relation to Agile Project Management?

Scrum is a sub-group of agile:

- Agile is a set of values and principles that describe a group's day-to-day interactions and activities. Agile itself is not prescriptive or specific.
- The Scrum methodology follows the values and principles of agile, but includes further definitions and specifications, especially regarding certain software development practices.

Although developed for agile software development, agile Scrum became the preferred framework for agile project management in general and is sometimes simply referred to as Scrum project management or Scrum development.

What is the Benefits Received from the Scrum Methodology?

Organizations that have adopted agile Scrum have experienced:

- Higher productivity
- Better-quality products
- Reduced time to market
- Improved stakeholder satisfaction
- Better team dynamics
- Happier employees

3.3 Summary

- Software development using Waterfall Model approach.
- Evolutionary Process Model allows for changing requirements as well as all work is broken down into maintainable work chunks.
- In Prototype Model, prototype is built, tested, and then reworked when needed until an acceptable prototype is achieved.
- Spiral Model allows incremental releases of the product or incremental refinement through each iteration around the spiral.
- Incremental process model is a successive version model.
- Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver all at once.



SOFTWARE REQUIREMENT ANALYSIS AND SPECIFICATION

Unit Structure

- 4.0 Objectives
- 4.1 Introduction
- 4.2 Types of Requirement
 - 4.2.1 Functional Requirement
 - 4.2.2 Non-Functional Requirement
- 4.3 Requirement Engineering Process
- 4.4 Requirement Analysis and Design
 - 4.4.1 Data Flow Diagram
 - 4.4.2 Data Dictionary
 - 4.4.3 HIPO Chart
 - 4.4.4 Warnier Orr Diagram
- 4.5 Requirement Elicitation
 - 4.5.1 Interviews
 - 4.5.2 Questionnaire
 - 4.5.3 Brainstorming
 - 4.5.4 Facilitated Application Specification Technique(FAST)
 - 4.5.5 Use Case Approach
- 4.6 Summary

4.0 Objectives

The objective of this chapter is to introduce software requirements and to explain the processes involved in discovering and documenting these requirements. When you have read the chapter, you will:

- Understand the concepts of user and system requirements and why these requirements should be written in different ways;
- Understand the differences between functional and non-functional software requirements;
- Understand the main requirements engineering activities of elicitation, analysis, and validation, and the relationships between these activities;
- Understand why requirements management is necessary and how it supports other requirements engineering activities.

4.1 Introduction

The requirements for a system are the descriptions of the services that a system should provide and the constraints on its operation. These requirements reflect the needs of customers for a system that serves a certain purpose such as controlling a device, placing an order, or finding information. The process of finding out, analyzing, documenting and checking these services and constraints is called requirements engineering (RE).

The term requirement is not used consistently in the software industry. In some cases, a requirement is simply a high-level, abstract statement of a service that a system should provide or a constraint on a system. At the other extreme, it is a detailed, formal definition of a system function.

4.2 Types of Requirement

Software system requirements are often classified as functional or non-functional requirements:

1. Functional requirements:

These are statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations. In some cases, the functional requirements may also explicitly state what the system should not do.

2. Non-functional requirements:

These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards. Non-functional requirements often apply to the system as a whole rather than individual system features or services.

In reality, the distinction between different types of requirements is not as clear-cut as these simple definitions suggest. A user requirement concerned with security, such as a statement limiting access to authorized users, may appear to be a non-functional requirement. However, when developed in more detail, this requirement may generate other requirements that are clearly functional, such as the need to include user authentication facilities in the system. This shows that requirements are not independent and that one requirement often generates or constrains other requirements. The system requirements therefore do not just specify the services or the features of the system that are required; they also specify the necessary functionality to ensure that these services/features are delivered effectively.

4.2.1 Functional Requirement

The functional requirements for a system describe what the system should do. These requirements depend on the type of software being developed, the expected users of the software, and the general approach taken by the organization when writing requirements. When expressed as user requirements, functional requirements should be written in natural language so that system users and managers can understand them. Functional system requirements expand the user requirements and are written for system developers. They should describe the system functions, their inputs and outputs, and exceptions in detail.

Functional system requirements vary from general requirements covering what the system should do to very specific requirements reflecting local ways of working or an organization's existing systems. For example, here are examples of functional requirements for the Mentcare system, used to maintain information about patients receiving treatment for mental health problems:

1. A user shall be able to search the appointments lists for all clinics.
2. The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
3. Each staff member using the system shall be uniquely identified by his or her eight-digit employee number.

These user requirements define specific functionality that should be included in the system. The requirements show that functional requirements may be written at different levels of detail (contrast requirements 1 and 3).

Functional requirements, as the name suggests, have traditionally focused on what the system should do. However, if an organization decides that an existing off-the-shelf system software product can meet its needs, then there is very little point in developing a detailed functional specification. In such cases, the focus should be on the development of information requirements that specify the information needed for people to do their work. Information requirements specify the information needed and how it is to be delivered and organized. Therefore, an information requirement for the Mentcare system might specify what information is to be included in the list of patients expected for appointments that day.

Imprecision in the requirements specification can lead to disputes between customers and software developers. It is natural for a system developer to interpret an ambiguous requirement in a way that simplifies its implementation. Often, however, this is not what the customer wants. New requirements have to be established and changes made to the system. Of course, this delays system delivery and increases costs.

For example, the first Mentcare system requirement in the above list states that a user shall be able to search the appointments lists for all clinics. The rationale for this requirement is that patients with mental health problems are sometimes confused. They may have an appointment at one clinic but actually go to a different clinic. If they have an appointment, they will be recorded as having attended, regardless of the clinic.

A medical staff member specifying a search requirement may expect “search” to mean that, given a patient name, the system looks for that name in all appointments at all clinics. However, this is not explicit in the requirement. System developers may interpret the requirement so that it is easier to implement. Their search function may require the user to choose a clinic and then carry out the search of the patients who attended that clinic. This involves more user input and so takes longer to complete the search.

Ideally, the functional requirements specification of a system should be both complete and consistent. Completeness means that all services and information required by the user should be defined. Consistency means that requirements should not be contradictory.

In practice, it is only possible to achieve requirements consistency and completeness for very small software systems. One reason is that it is easy to make mistakes and omissions when writing specifications for large, complex systems. Another reason is that large systems have many stakeholders, with different backgrounds and expectations. Stakeholders are likely to have different—and often inconsistent—needs. These inconsistencies may not be obvious when the requirements are originally specified, and the inconsistent requirements may only be discovered after deeper analysis or during system development.

4.2.2 Non-Functional Requirement

Non-functional requirements, as the name suggests, are requirements that are not directly concerned with the specific services delivered by the system to its users. These non-functional requirements usually specify or constrain characteristics of the system as a whole. They may relate to emergent system properties such as reliability, response time, and memory use. Alternatively, they may define constraints on the system implementation, such as the capabilities of I/O devices or the data representations used in interfaces with other systems.

Non-functional requirements are often more critical than individual functional requirements. System users can usually find ways to work around a system function that doesn't really meet their needs. However, failing to meet a non-functional requirement can mean that the whole system is unusable. For example, if an aircraft system does not meet its reliability requirements, it will not be certified as safe for operation; if an embedded control system fails to meet its performance requirements, The control functions will not operate correctly.

While it is often possible to identify which system components implement specific functional requirements (e.g., there may be formatting components that implement reporting requirements), this is often more difficult with non-functional requirements. The implementation of these requirements may be spread throughout the system, for two reasons:

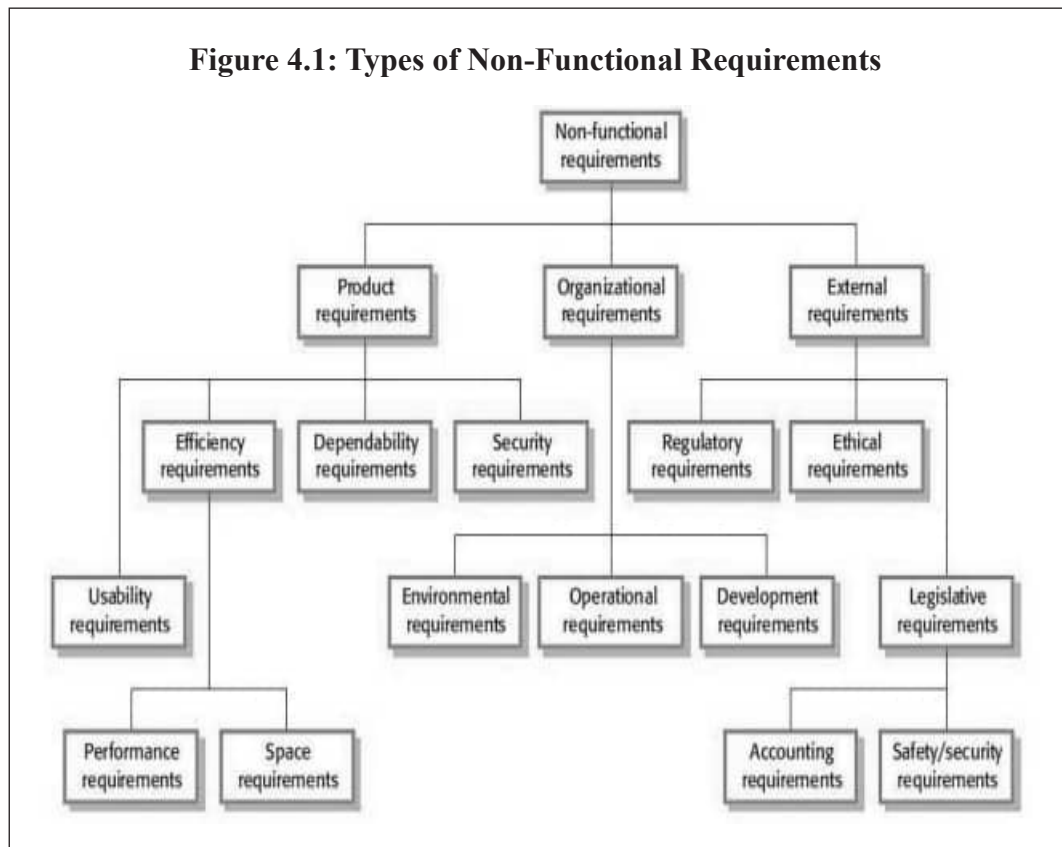
1. Non-functional requirements may affect the overall architecture of a system rather than the individual components. For example, to ensure that performance requirements are met in an embedded system, you may have to organize the system to minimize communications between components.

2. An individual non-functional requirement, such as a security requirement, may generate several, related functional requirements that define new system services that are required if the non-functional requirement is to be implemented. In addition, it may also generate requirements that constrain existing requirements; for example, it may limit access to information in the system (Figure4.1).

Non-functional requirements arise through user needs because of budget constraints, organizational policies, the need for interoperability with other software or hardware systems, or external factors such as safety regulations or privacy legislation. Above figure is a classification of non-functional requirements. You can see from this diagram that the non-functional requirements may come from required characteristics of the software (product requirements), the organization developing the software (organizational requirements), or external sources:

1. **Product requirements:** These requirements specify or constrain the runtime behaviour of the software. Examples include performance requirements for how fast the system must execute and how much memory it requires; reliability requirements that set out the acceptable failure rate; security requirements; and usability requirements.

Figure 4.1: Types of Non-Functional Requirements



2. Organizational requirements: These requirements are broad system requirements derived from policies and procedures in the customer's and developer's organizations. Examples include operational process requirements that define how the system will be used; development process requirements that specify the programming language; the development environment or process standards to be used; and environmental requirements that specify the operating environment of the system.

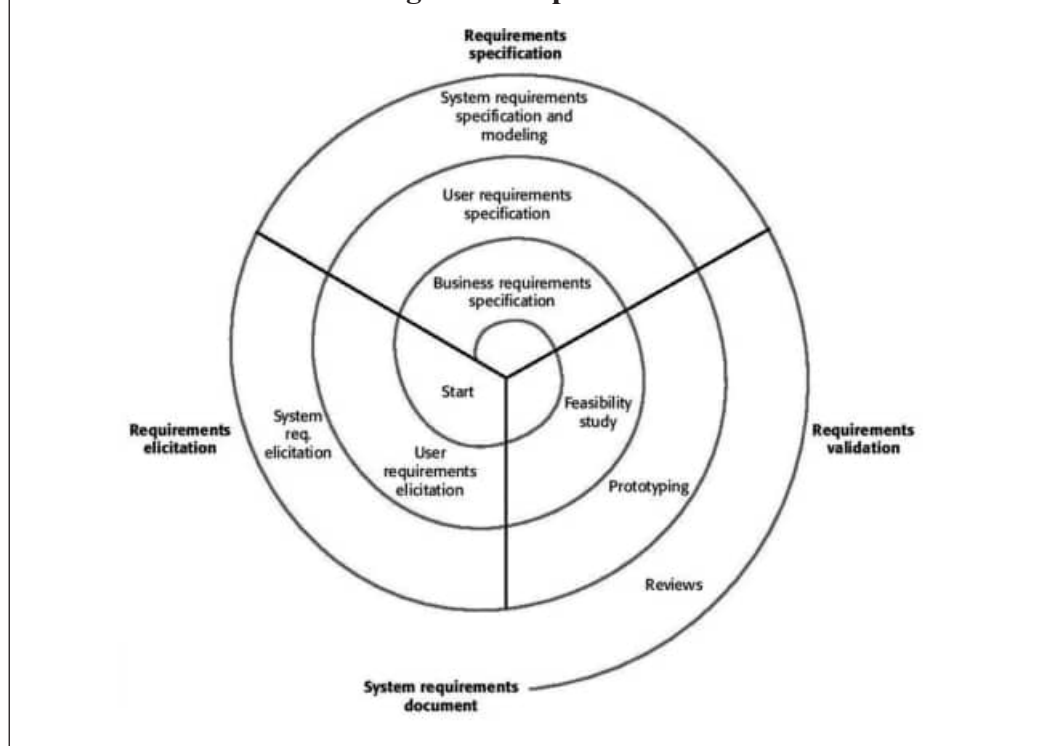
3. External requirements: This broad heading covers all requirements that are derived from factors external to the system and its development process. These may include regulatory requirements that set out what must be done for the system to be approved for use by a regulator, such as a nuclear safety authority; legislative requirements that must be followed to ensure that the system operates within the law; and ethical requirements that ensure that the system will be acceptable to its users and the general public.

4.3. Requirement Engineering Process

Requirements engineering involves three key activities. These are discovering requirements by interacting with stakeholders (elicitation and analysis); converting these requirements into a standard form (specification); and checking that the requirements actually define the system that the customer wants (validation). Requirements engineering is an iterative process in which the activities are interleaved.

The activities are organized as an iterative process around a spiral. The output of the RE process is a system requirements document. The amount of time and effort devoted to each activity in iteration depends on the stage of the overall process, the type of system being developed, and the budget that is available.

Early in the process, most effort will be spent on understanding high-level business and non-functional requirements, and the user requirements for the system. Later in the process, in the outer rings of the spiral, more effort will be devoted to eliciting and understanding the non-functional requirements and more detailed system requirements.

Figure 4.2: Spiral View

This spiral model accommodates approaches to development where the requirements are developed to different levels of detail. The number of iterations around the spiral can vary so that the spiral can be exited after some or all of the user requirements have been elicited. Agile development can be used instead of prototyping so that the requirements and the system implementation are developed together (Figure 4.2).

In virtually all systems, requirements change. The people involved develop a better understanding of what they want the software to do; the organization buying the system changes; and modifications are made to the system's hardware, software, and organizational environment. Changes have to be managed to understand the impact on other requirements and the cost and system implications of making the change.

4.4. Requirement Analysis and Design

Some of the problems that arise during the requirements engineering process are a result of failing to make a clear separation between these different levels of description. I distinguish between them by using the term user requirements to mean the high-level abstract requirements and system requirements to mean the detailed description of what the system should do.

User requirements and system requirements may be defined as follows:

1. User requirements are statements, in a natural language plus diagrams, of what services the system is expected to provide to system users and the constraints under which it must operate. The user requirements may vary from broad statements of the system features required to detailed, precise descriptions of the system functionality.
2. System requirements are more detailed descriptions of the software system's functions, services, and operational constraints. The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented. It may be part of the contract between the system buyer and the software developers.

Different kinds of requirement are needed to communicate information about a system to different types of reader. You need to write requirements at different levels of detail because different types of readers use them in different ways. The readers of the user requirements are not usually concerned with how the system will be implemented and may be managers who are not interested in the detailed facilities of the system. The readers of the system requirements need to know more precisely what the system will do because they are concerned with how it will support the business processes or because they are involved in the system implementation. System stakeholders include anyone who is affected by the system in some way and so anyone who has a legitimate interest in it. Stakeholders range from end-users of a system through managers to external stakeholders such as regulators who certify the acceptability of the system.

4.4.1 Data Flow Diagram

Introduction to data-flow diagrams

What are data-flow diagrams?

Data-flow diagrams (DFDs) model a perspective of the system that is most readily understood by users – The flow of information through the system and the activities that process this information.

Data-flow diagrams provide a graphical representation of the system that aims to be accessible to computer specialist and non-specialist users alike. The models enable software engineers, customers and users to work together effectively during the analysis and specification of requirements. Although this means that our customers are required to understand the modelling techniques and constructs, in data-flow

modelling only a limited set of constructs are used, and the rules applied are designed to be simple and easy to follow. These same rules and constructs apply to all data-flow diagrams (i.e., for each of the different software process activities in which DFDs can be used).

The benefits of data-flow diagrams

Data-flow diagrams provide a very important tool for software engineering, for a number of reasons:

- The system scope and boundaries are clearly indicated on the diagrams (more will be described about the boundaries of systems and each DFD later in this chapter).
- The technique of decomposition of high level data-flow diagrams to a set of more detailed diagrams provides an overall view of the complete system, as well as a more detailed breakdown and description of individual activities, where this is appropriate, for clarification and understanding.

The different kinds (and levels) of data-flow diagrams:

Although all data-flow diagrams are composed of the same types of symbols, and the validation rules are the same for all DFDs, there are three main types of data-flow diagram:

Data-Flow Diagrams (Figure 4.3)

- **Context diagrams** — context diagram DFDs are diagrams that present an overview of the system and its interaction with the rest of the “world”.

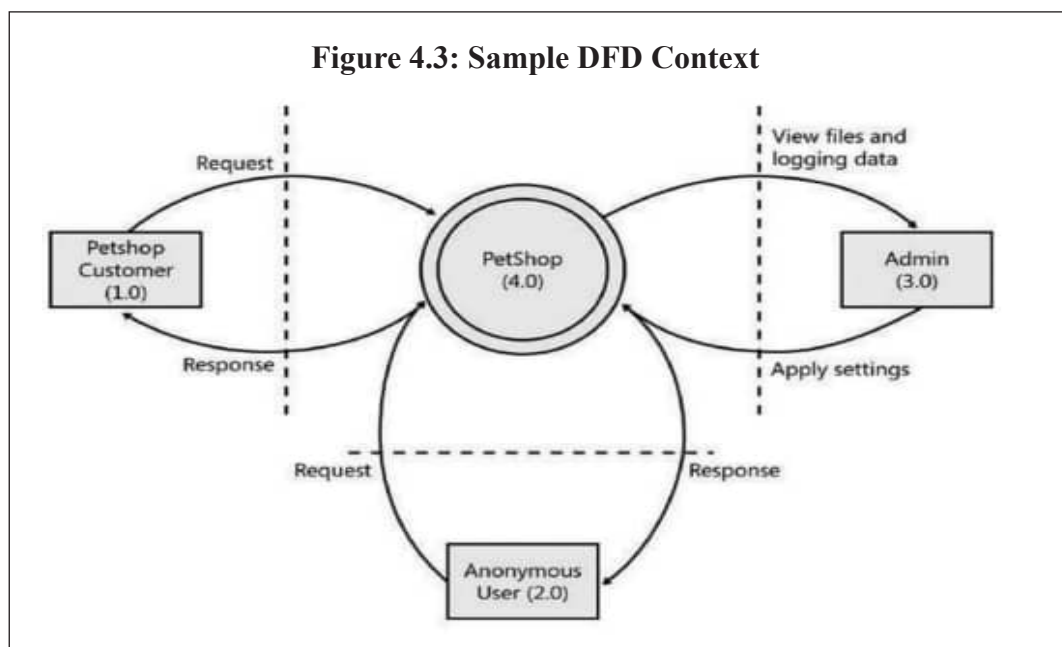
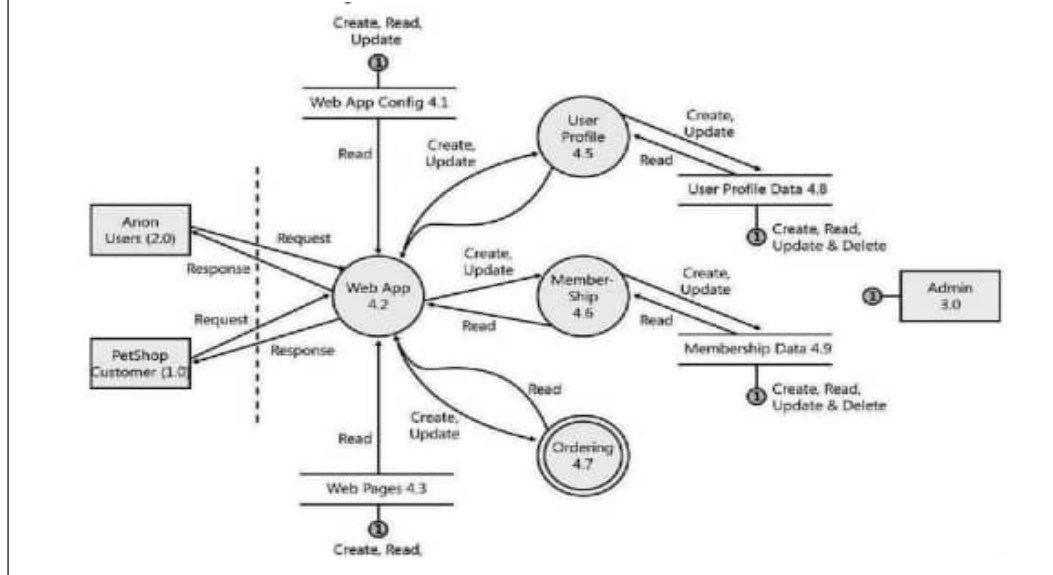
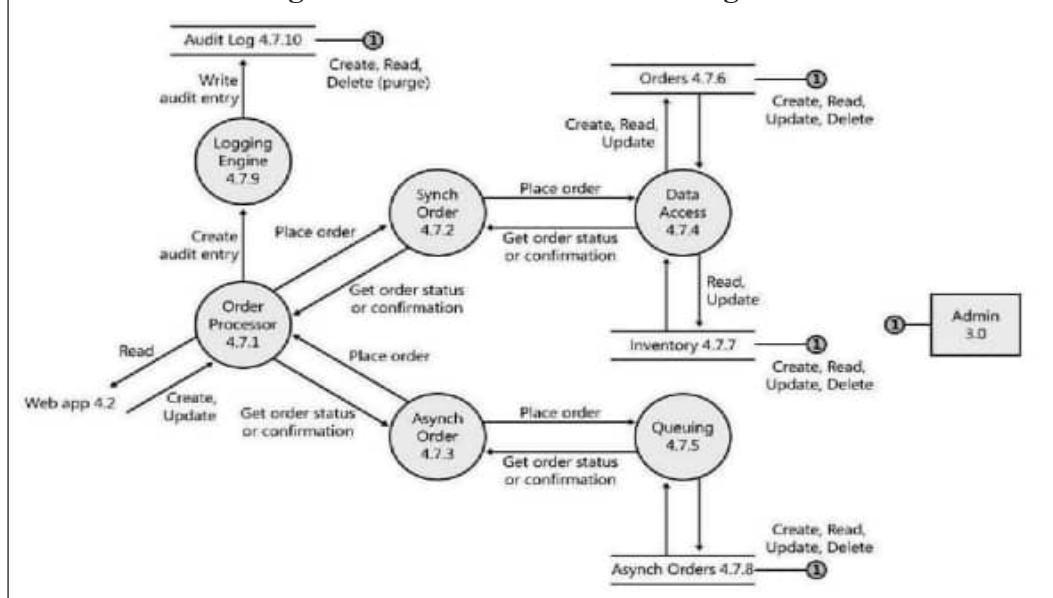


Figure 4.4: Level 1 Data-Flow Diagrams



- **Level 1 data-flow diagrams** — Level 1 DFDs present a more detailed view of the system than context diagrams, by showing the main sub-processes and stores of data that make up the system as a whole (Figure 4.4)
- **Level 2 (and lower) data-flow diagrams** — a major advantage of the data-flow modelling technique is that, through a technique called “levelling”, the detailed complexity of real world systems can be managed and modelled in a hierarchy of abstractions. Certain elements of any dataflow diagram can be decomposed (“exploded”) into a more detailed model a level lower in the hierarchy (Figure 4.5).

Figure 4.5: Level 2 Data-Flow Diagrams



During this unit we shall investigate each of the three types of diagram in the sequence they are described above. This is both a sequence of increasing complexity and sophistication, and also the sequence of DFDs that is usually followed when modelling systems.

For each type of diagram we shall first investigate what the features of the diagram are, and then we shall investigate how to create that type of diagram. However, before looking at particular kinds of dataflow diagrams, we shall briefly examine each of the symbols from which DFDs are composed.

4.4.2 Data Dictionary

There is a wide range of definitions for Data Dictionary /Directory Systems (DD/DS). Due to the increasing interest and the rapidly evolving nature of this field in recent years, terminology is somewhat confusing. One author speaks of a Data Dictionary System (DDS) or System Resources Dictionary, while another refers to DD/DS or Data Element Dictionary/Directory System, (DED/DS). Characteristic definitions include those of:

Leong-Hong and Marron (1977):

The DED/DS is a software tool that provides the means for defining and describing the characteristics of a database, as opposed to the contents of a database.

National Bureau of Standards (NBS) (1978):

The DED/DS is considered as a resource manager. It is an integrated repository that provides data necessary for managing data. Data management includes the planning, control, direction, and organization of data.

Allen, Loomis and Hannino (1982):

a DD/DS is an automated information system. It helps to achieve control of the data resource, by providing an inventory of that resource. It helps to control the cost of developing and maintaining applications. Finally it can provide for independence of metadata across computing environments, improving resiliency to the effects of hardware and software changes

Data has positive or negative value. The value of the data derives from the fact that the entire enterprise depends on its availability for the proper management of all other resources. Thus, it must be treated as a resource. The management and control of data resources begins with a proper definition and description of data. A DD/DS is a tool for the control and management of data as a resource.

To manage data as a resource it is basic that data about data must be clearly specified, easily accessible and well controlled. These data are called metadata. These are data objects, that in a data processing environment are represented in the form of elements, records, files or databases. Metadata is not user data, but identifies, defines and describes the characteristics of the latter. It describes the data resources of an organization. A DD/DS contains two types of metadata: Dictionary and Directory metadata. Dictionary metadata describes the data, and defines their meaning and structure. Directory metadata describes where the data is stored, and how internally represented and accessed.

A collection of related metadata comprises the metadata database. It consists of a database that contains descriptive and definitional information about the user database. It has basically the same characteristics of a user database. To achieve the goals of managing data as a resource it requires proper management. That is, planning for the design, implementation, maintenance, utilization and control. This implies that established lines of responsibility and authority; formal rules and detailed procedures to guide metadata- related activities; common procedures for collection, update, and maintenance; and common procedures for access control to the metadata must be developed. The DD/DS is the basic tool for managing the metadata database.

The DD/DS is divided into three categories, based on the scope of control exercised through metadata management: Active, Potentially Active and Passive. A DD/DS is said to be active with respect to a program or process if that program or process is fully dependent on the DD/DS for its metadata. A DD/DS is said to be passive if it does not generate metadata and does not have control over where and how a user or processing component obtains the required metadata. A potentially active DD/DS provides the capability of producing the metadata for a given program or process. A potentially active DD/DS can be extended to active through supportive administrative procedures. Many of the currently commercially available DD/DSs are of this type. In practice, the concept of active/passive DD/DS refers to interfaces that it provides to other software packages. A DD/DS with active interfaces can better serve the goals of managing data as a resource.

4.4.3 HIPO Chart

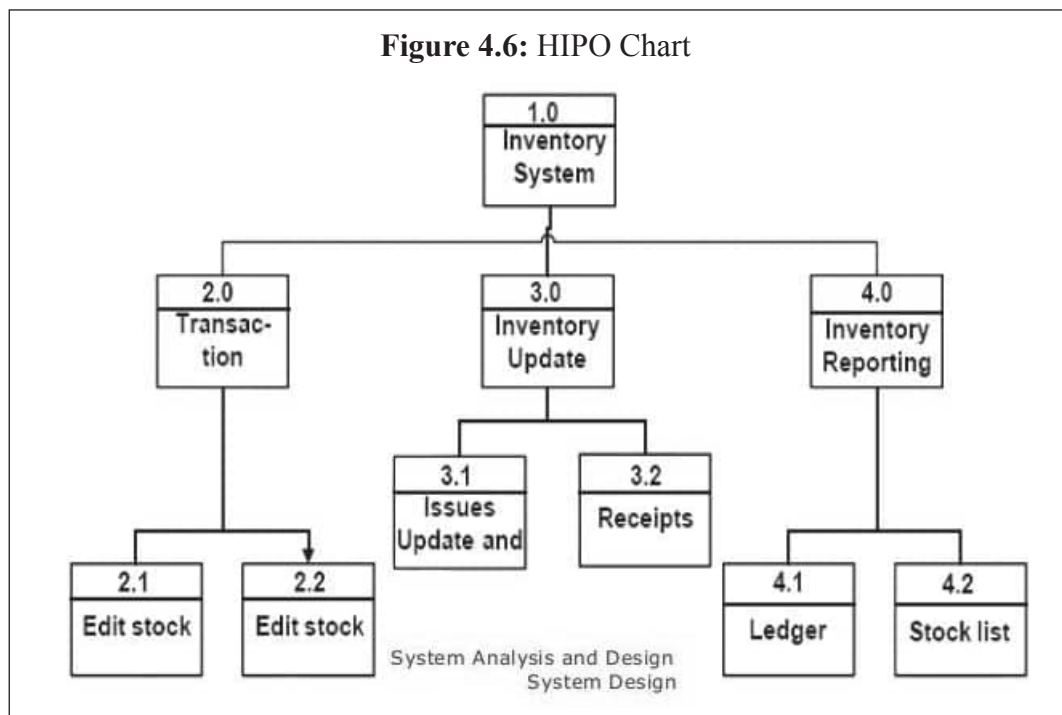
Hierarchy-Input-Process-Output (HIPO)

Hierarchy-Input-Process-Output (HIPO) is a documentation technique developed by IBM and used in a number of different situations. It is useful for design documentation as well as stating requirements and specifications before beginning

design. It is based on two concepts: The input-process-output model of information processing and functional hierarchies. HIPO is basically a graphical notation, consisting of hierarchy charts and process charts. The hierarchy chart is similar to a structure chart. Each box in the chart can represent a system, subsystem, program, or program module. Its purpose is to show the overall functional components and to refer to overview and detail HIPO diagrams. It does not show the data flow between functional components or any control information. It does not show the arrows with open or filled circles. Also, it does not give any information about the data components of the system or program.

The second part of the HIPO notation is process charts. Process charts have two levels: overview diagram and detail diagrams. They are used to describe a function in terms of its inputs, the processing to be done, and the outputs produced. Any of the information, especially the process part, may be described in more detail by accompanying textual material. The process charts show the flow of data through processes; however, they are more difficult to draw than data flow diagrams. HIPO diagrams often require more verbiage and symbols to give the same information as a comparable data flow diagram (Figure 4.6).

HIPO has been used in a number of DP (Data Processing) situations and some more complex specification tasks. Its drawbacks, especially with respect to interfaces and the description of data, were limited to its acceptance. HIPO does not include any guidelines, strategies, or procedures to guide the analyst in building a functional



specification or the designer in building a system or program design.

At the general level, a structure chart is usually preferred over a hierarchy chart. At the detailed level, pseudocode is often preferred over HIPO diagrams because it provides more information in a more compact form. HIPO diagrams have no symbols for representing detailed program structures such as conditions, case structures, and loops. HIPO diagrams cannot represent data structures or the linkage to data models.

4.4.4 Warnier Orr Diagram

A graphical representation of a horizontal hierarchy with brackets isolating the levels is utilized to plan or document a data structure, a set of detailed logic, a program, or a system.

Strengths, weaknesses, and limitations:

Warnier-Orr diagrams are excellent tools for describing, planning, or documenting data structures. They can show a data structure or a logical structure at a glance. Because only a limited number of symbols are required, specialized software is unnecessary and diagrams can be created quickly by hand. The basic elements of the technique are easy to learn and easy to explain. Warnier-Orr diagrams map well to structured code.

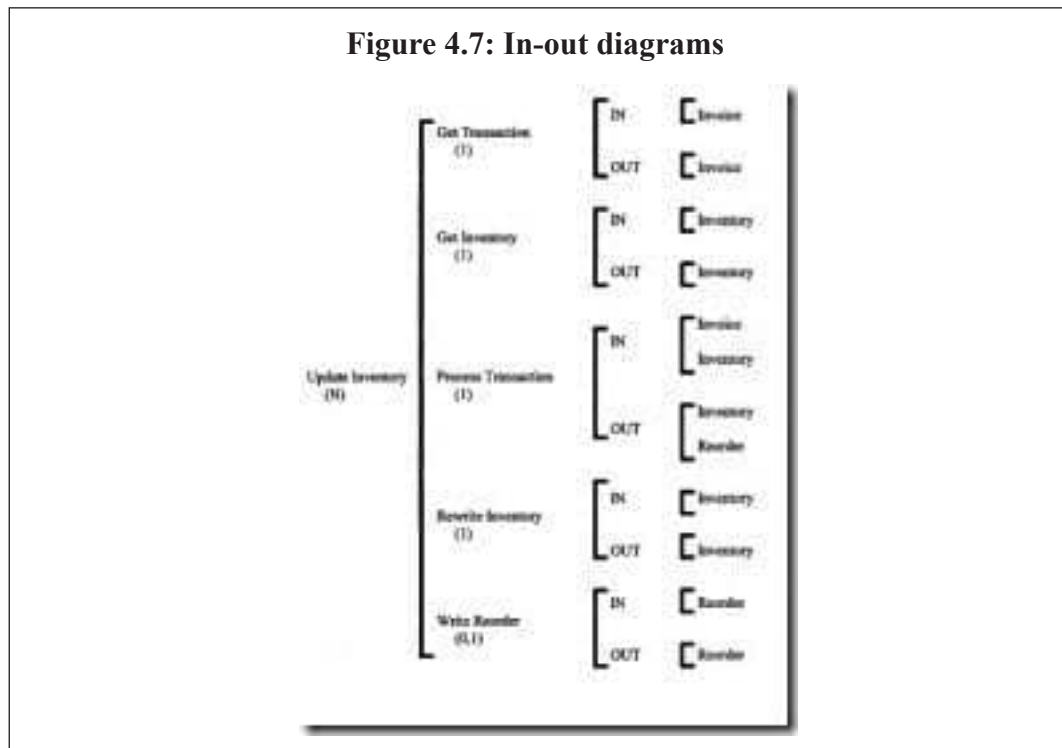
Concepts:

The Warnier-Orr design methodology, also known as the structured requirements definition methodology was developed in the early 1970s by Warnier and extended to system design by Orr. The first step in the methodology is to create entity diagrams for each major user. The entity diagrams are then merged to create a system entity diagram, and the major tasks that must be performed are derived from the system's data requirements.

In-out diagrams:

A Warnier-Orr diagram shows a data structure or a logical structure as a horizontal hierarchy with brackets separating the levels. Once the major tasks are identified, the systems analyst or information system consultant prepares an in-out Warnier-Orr diagram to document the application's primary inputs and outputs.

For example, Figure shows an in-out diagram for a batch inventory update application. Start at the left (the top of the hierarchy). The large bracket shows that the program, Update Inventory, performs five primary processes ranging from Get Transaction at the top to Write Reorder at the bottom. The letter N in parentheses

Figure 4.7: In-out diagrams

under Update Inventory means that the program is repeated many (1 or more) times. The digit 1 in parentheses under Get Transaction (and the next three processes) means the process is performed once. The (0, 1) under Write Reorder means the process is repeated 0 or 1 times, depending on a run-time condition. (Stock may or may not be reordered as a result of any given transaction.) (Figure 4.7)

Data flow into and out from every process. The process inputs and outputs are identified to the right of the in-out diagram. For example, the Get Transaction process reads an Invoice and passes it to a subsequent process. The last column is a list of the program's primary input and output data structures. Note how the brackets indicate the hierarchical levels.

4.5 Requirement Elicitation

The aims of the requirements elicitation process are to understand the work that Stakeholders do and how they might use a new system to help support that work.

During requirements elicitation, software engineers work with stakeholders to find out about the application domain, work activities, the services and system features that stakeholders want, the required performance of the system, hardware constraints, and so on. Eliciting and understanding requirements from system stakeholders is a difficult process for several reasons:

1. Stakeholders often don't know what they want from a computer system except in the most general terms; they may find it difficult to articulate what they want the system to do; they may make unrealistic demands because they don't know what is and isn't feasible.
2. Stakeholders in a system naturally express requirements in their own terms and with implicit knowledge of their own work. Requirements engineers, without experience in the customer's domain, may not understand these requirements.
3. Different stakeholders, with diverse requirements, may express their requirements in different ways. Requirements engineers have to discover all potential sources of requirements and discover commonalities and conflict.
4. Political factors may influence the requirements of a system. Managers may demand specific system requirements because these will allow them to increase their influence in the organization.
5. The economic and business environment in which the analysis takes place is dynamic. It inevitably changes during the analysis process. The importance of particular requirements may change. New requirements may emerge from new stakeholders who were not originally consulted.

4.5.1 Interviewing

Formal or informal interviews with system stakeholders are part of most requirements Engineering processes. In these interviews, the requirements engineering team puts questions to stakeholders about the system that they currently use and the system to be developed. Requirements are derived from the answers to these questions.

Interviews may be of two types:

1. Closed interviews, where the stakeholder answers a predefined set of questions.
2. Open interviews, in which there is no predefined agenda. The requirements engineering team explores a range of issues with system stakeholders and hence develops a better understanding of their needs.

In practice, interviews with stakeholders are normally a mixture of both of these. You may have to obtain the answer to certain questions, but these usually lead to other issues that are discussed in a less structured way. Completely open-ended discussions rarely work well. You usually have to ask some questions to get started and to keep the interview focused on the system to be developed.

Interviews are good for getting an overall understanding of what stakeholders do, how they might interact with the new system, and the difficulties that they face with current systems. People like talking about their work, and so they are usually happy to get involved in interviews. However, unless you have a system prototype to demonstrate, you should not expect stakeholders to suggest specific and detailed requirements. Everyone finds it difficult to visualize what a system might be like. You need to analyze the information collected and to generate the requirements from this.

Eliciting domain knowledge through interviews can be difficult, for two reasons:

1. All application specialists use jargon specific to their area of work. It is impossible for them to discuss domain requirements without using this terminology. They normally use words in a precise and subtle way that requirements engineers may misunderstand.
2. Some domain knowledge is so familiar to stakeholders that they either find it difficult to explain or they think it is so fundamental that it isn't worth mentioning. For example, for a librarian, it goes without saying that all acquisitions are catalogued before they are added to the library. However, this may not be obvious to the interviewer, and so it isn't taken into account in the requirements.

Interviews are not an effective technique for eliciting knowledge about organizational requirements and constraints because there are subtle power relationships between the different people in the organization. Published organizational structures rarely match the reality of decision making in an organization, but interviewees may not wish to reveal the actual rather than the theoretical structure to a stranger. In general, most people are generally reluctant to discuss political and organizational issues that may affect the requirements.

To be an effective interviewer, you should bear two things in mind:

1. You should be open-minded, avoid preconceived ideas about the requirements, and willing to listen to stakeholders. If the stakeholder comes up with surprising requirements, then you should be willing to change your mind about the system.
2. You should prompt the interviewee to get discussions going by using a springboard question or a requirements proposal or by working together on a prototype system. Saying to people "tell me what you want" is unlikely to result in useful information. They find it much easier to talk in a defined context rather than in general terms.

Information from interviews is used along with other information about the system from documentation describing business processes or existing systems, user observations, and developer experience. Sometimes, apart from the information in the system documents, the interview information may be the only source of information about the system requirements. However, interviewing on its own is liable to miss essential information, and so it should be used in conjunction with other requirements elicitation techniques.

4.5.2 Questionnaire

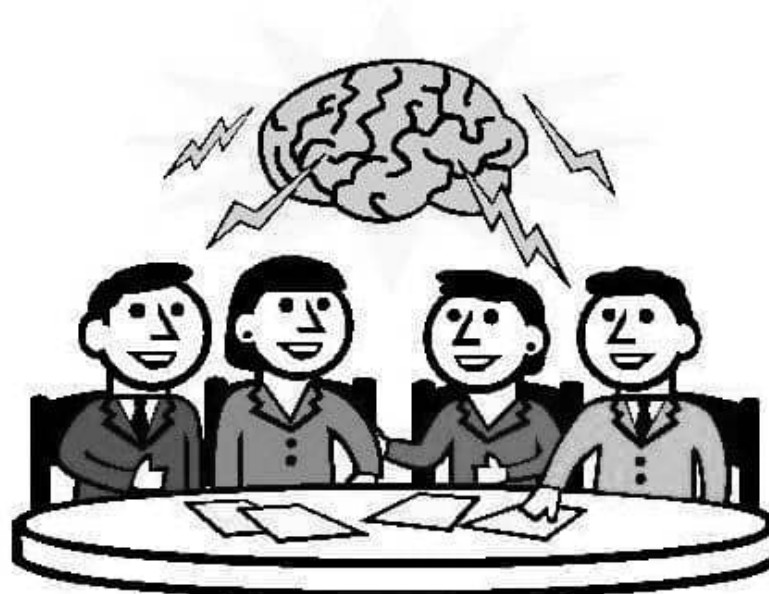
Interviews – Start Up Questions

- Context-free questions to narrow the scope a bit (Weinberg)
- Identify customers, goals, and benefits
- Who is (really) behind the request for the system?
- Who will use the system? Willingly?
- Are there several types of users?
- What is the potential economic benefit from a successful solution?
- Is a (pre-existing) solution available from another source?
- When do you need it by?
- Can you prioritize your needs?
- What are your constraints?
- Time
- Budget
- Resources (human or otherwise)
- Expected milestones (deliverables and dates)?
- Try to characterize the problem and its solution
- What would be a “good” solution to the problem?
- What problems is the system trying to address?
- In what environment will the system be used?
- Any special performance issues?
- Other special constraints?
- What is (un)likely to change?
- Future evolution?

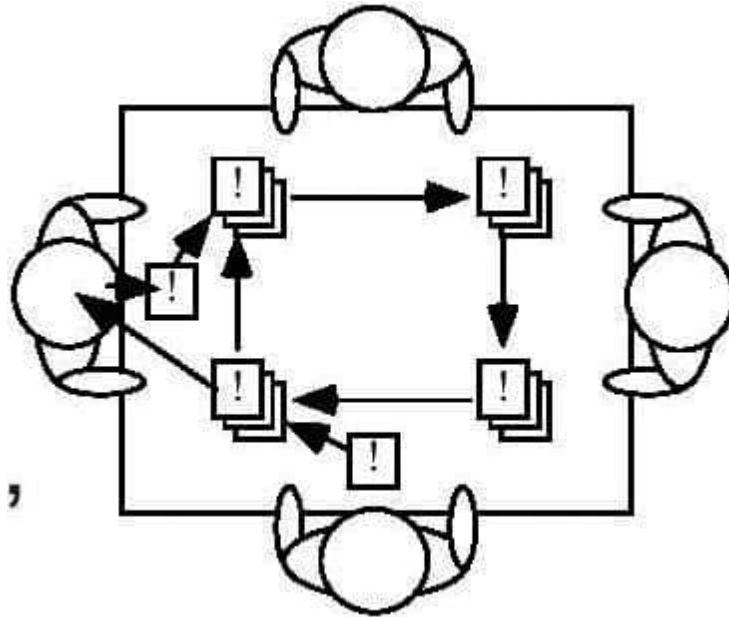
- What needs to be flexible (vs. quick & dirty)?
- Questions that cannot be asked directly (ask indirectly)
- Are you opposed to the system?
- Are you trying to obstruct/delay the system?
- Are you trying to create a more important role for yourself?
- Do you feel threatened by the proposed system?
- Are you trying to protect your job?
- Is your job threatened by the new system?
- Is anyone else's?

4.5.3 Brainstorming

- To invent new way of doing things or when much is unknown
- When there are few or too many ideas
- Early on in a project particularly when:
- Terrain is uncertain
- There is little expertise for the type of applications
- Innovation is important (e.g., novel system)



- Two main activities:
 - The Storm: Generating as many ideas as possible (quantity, not quality) – wild is good!
 - The Calm: Filtering out of ideas (combine, clarify, prioritize, improve...) to keep the best one(s) – may require some voting strategy
- Roles: scribe, moderator (may also provoke), Participants



Brainstorming – Objectives

- Hear ideas from everyone, especially unconventional ideas
- Keep the tone informal and non-judgemental
- Keep the number of participants “reasonable” – if too many, consider a “playoff”-type filtering and invite back the most creative to multiple sessions
- Encourage creativity
- Choose good, provocative project name.
- Choose good, provocative problem statement
- Get a room without distractions, but with good acoustics, whiteboards, coloured pens, provide coffee/donuts/pizza/beer
- Provide appropriate props/mock-ups

Brainstorming – Roles

- Scribe
- Write down all ideas (may also contribute)
- May ask clarifying questions during first phase but without criticizing
- Moderator/Leader
- Cannot be the scribe
- Two schools of thought: traffic cop or agent provocateur
- Traffic cop – enforces “rules of order”, but does not throw his/her weight around otherwise
- Agent provocateur – traffic cop plus more of a leadership role, comes prepared with wild ideas and throws them out as discussion wanes
- May also explicitly look for variations and combinations of other suggestions

Brainstorming – Participants

- Virtually any stakeholder, e.g.
- Developers
- Domain experts
- End-users
- Clients
- “Ideas-people” – a company may have a special team of people
- Chair or participate in brainstorming sessions
- Not necessarily further involved with the project

Brainstorming – The Storm

- Goal is to generate as many ideas as possible
- Quantity, not quality, is the goal at this stage
- Look to combine or vary ideas already suggested
- No criticism or debate is permitted – do not want to inhibit participants
- Participants understand nothing they say will be held against them later on
- Scribe writes down all ideas where everyone can see
- E.g., whiteboard, paper taped to wall
- Ideas do not leave the room

- Wild is good
- Feel free to be gloriously wrong
- Participants should NOT censor themselves or take too long to consider whether an idea is practical or not – let you go!

Brainstorming – The Calm

- Go over the list of ideas and explain them more clearly
- Categorize into “maybe” and “no” by pre-agreed consensus method
- Informal consensus
- 50% + 1 vote vs. “clear majority”
- Does anyone have veto power?
- Be careful about time and people
- Meetings (especially if creative or technical in nature) tend to lose focus after 90 to 120 minutes – take breaks or reconvene later
- Be careful not to offend participants
- Review, consolidate, combine, clarify and improve.
- Rank the list by priority somehow
- Choose the winning idea(s)

Brainstorming – Tool Support

- With many good ideas, some outrageous and even farfetched, brainstorming can be really fun!
- Creates a great environment that stimulates people and motivates them to perform well!
- Can be done by email, but a good moderator/leader is needed to
- Prevent flamers to come into play
- Prevent race conditions due to the asynchronous communication medium
- Be careful not to go into too much detail
- Collaboration tools are also possible
- TWiki and many other more appropriate tools such as BrainStorm and IdeaFisher

4.5.4 Facilitated Application Specification Technique

Its objective is to bridge the expectation gap – difference between what the developers think they are supposed to build and what customers think they are going to get.

A team oriented approach is developed for requirements gathering.

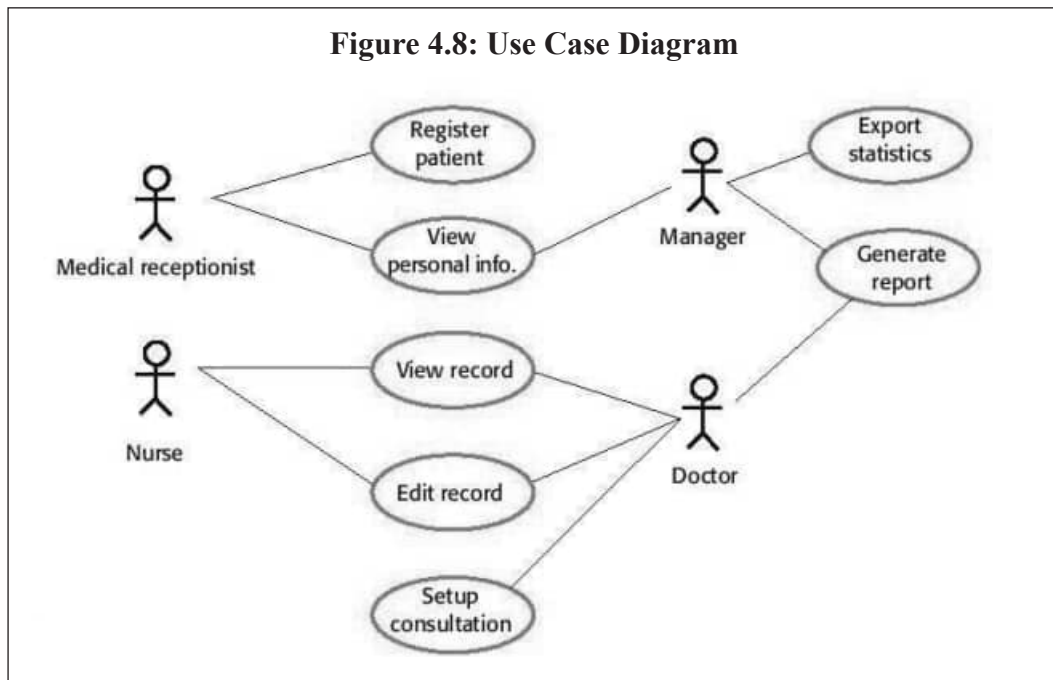
Each attendee is asked to make a list of objects that are-

1. Part of the environment that surrounds the system
2. Produced by the system
3. Used by the system

Each participant prepares his/her list, different lists are then combined, redundant entries are eliminated, team is divided into smaller sub-teams to develop mini-specifications and finally a draft of specifications is written down using all the inputs from the meeting.

4.5.5 Use Case Approach

Use cases are a way of describing interactions between users and a system using a graphical model and structured text. They were first introduced in the Objectory method (Jacobsen et al. 1993) and have now become a fundamental feature of the Unified Modelling Language (UML). In their simplest form, a use case identifies the actors involved in an interaction and names the type of interaction. You then add additional information describing the interaction with the system. The additional information may be a textual description or one or more graphical models such as the UML sequence or state charts (see Chapter 5). Use cases are documented using a high-level use case diagram. The set of use cases represents all of the possible interactions that will be described in the system requirements. Actors in the process, who may be human or other systems, are represented as stick figures. Each class of interaction is represented as a named ellipse. Lines link the actors with the interaction. Optionally, arrowheads may be added to lines to show how the interaction is initiated. This is illustrated in Figure 4.15, which shows some of the use cases for the Mentcare system. Use cases identify the individual interactions between the system and its users or other systems. Each use case should be documented with a textual description. These can then be linked to other models in the UML that will develop the scenario in more detail. For example, a brief description of the Setup Consultation use case from.



Setup consultation allows two or more doctors, working in different offices, to view the same patient record at the same time. One doctor initiates the consultation by choosing the people involved from a dropdown menu of doctors who are online. The patient record is then displayed on their screens, but only the initiating doctor can edit the record. In addition, a text chat window is created to help coordinate actions. It is assumed that a phone call for voice communication can be separately arranged (Figure 4.8).

The UML is a standard for object-oriented modeling, so use cases and use case based elicitation are used in the requirements engineering process. However, my experience with use cases is that they are too fine-grained to be useful in discussing requirements. Stakeholders don't understand the term use case; they don't find the graphical model to be useful, and they are often not interested in a detailed description of each and every system interaction. Consequently, I find use cases to be more helpful in systems design than in requirements engineering. I discuss use cases further in Chapter 5, which shows how they are used alongside other system models to document a system design.

Some people think that each use case is a single, low-level interaction scenario. Others, such as Stevens and Pooley (Stevens and Pooley 2006), suggest that each use case includes a set of related, low-level scenarios. Each of these scenarios is a single thread through the use case. Therefore, there would be a scenario for the normal interaction plus scenarios for each possible exception. In practice, you can use them in either way.

4.6 Summary

- Requirements for a software system set out what the system should do and define constraints on its operation and implementation
- Functional requirements are statements of the services that the system must provide or are descriptions of how some computations must be carried out.
- Non-functional requirements often constrain the system being developed and the development process being used. These might be product requirements, organizational requirements, or external requirements. They often relate to the emergent properties of the system and therefore apply to the system as a whole.
- The requirements engineering process includes requirements elicitation, requirements specification, requirements validation, and requirements management
- Requirements elicitation is an iterative process that can be represented as a spiral of activities requirements discovery, requirements classification and organization, requirements negotiation, and requirements documentation.

Reference Books:

1. Software Engineering 6th edition Roger S. Pressman
2. Software Engineering Tenth Edition Ian Sommerville, Pearson.



SOFTWARE REQUIREMENT SPECIFICATION

Unit Structure

- 5.0 Introduction
- 5.1 Purpose
- 5.2 Specific Requirements
 - 5.2.1 External Interface Requirements
 - 5.2.2 Functional Requirements
 - 5.2.3 Detailed Non-Functional Requirements
- 5.3 Software Project Estimation
 - 5.3.1 Lines of Code (LOC)
 - 5.3.2 Number of entities in ER diagram
 - 5.3.3. Total number of processes in detailed data flow diagram
 - 5.3.4 Function Point Analysis
- 5.4 COCOMO Model
 - 5.4.1 Types of COCOMO model
 - 5.4.2 Numerical
 - 5.4.3 Advantages and Disadvantages of COCOMO Model
- 5.5 COCOMO II Model
 - 5.5.1 Sub-models in COCOMO II
- 5.6 COMPARISION of COCOMO and COCZOMO-II
- 5.7 Earned Value Management
- 5.8 Variance Analysis
- 5.9 Summary
- 5.10 Questions
- 5.11 Reference Books

5.0 Introduction

A software requirements specification (SRS) is a detailed description of the intended purpose and environment for software under development. The SRS fully describes what the software will do and how it will be expected to perform. The introduction of the Software Requirements Specification (SRS) provides an overview of the entire SRS with purpose, scope, functional and non-functional requirements of the SRS. The aim of this document is to gather and analyze and give an in-depth insight of the complete problem statement in detail.

5.1 Purpose

An SRS forms the basis of an organization's entire project. It sets out the framework that all the development teams will follow. It provides critical information to all the teams, including development, operations, quality assurance QA and maintenance, ensuring the teams are in agreement.

Using the SRS helps an enterprise confirm that the requirements are fulfilled and helps business leaders make decisions about the lifecycle of their product, such as when to retire a feature.

In addition, writing an SRS can help developers reduce the time and effort necessary to meet their goals as well as save money on the cost of development.

5.2 Specific Requirements

5.2.1 External Interface Requirements

The only link to an external system is the link to the Historical Society (HS) Database to verify the membership of a Reviewer. The Editor believes that a society member is much more likely to be an effective reviewer and has imposed a membership requirement for a Reviewer. The HS Database fields of interest to the Web Publishing Systems are member's name, membership (ID) number, and email address (an optional field for the HS Database).

The Assign Reviewer use case sends the Reviewer ID to the HS Database and a Boolean is returned denoting membership status. The Update Reviewer use case requests a list of member names, membership numbers and (optional) email addresses when adding a new Reviewer. It returns a Boolean for membership status when updating a Reviewer.

5.2.2 Functional Requirements

- Search Article

Use Case Name	Search Article
Trigger	The Reader assesses the Online Journal Website
Precondition	The Web is displayed with grids for searching
Basic Path	The Reader chooses how to search the Web site. The choices are by Author, by Category, and by Keyword. If the search is by Author, the system creates and presents an alphabetical list of all authors in the database. In the case of an article with multiple authors, each is contained in the list. The Reader selects an author. The system creates and presents a list of all articles by that author in the database. The Reader selects an article. The system displays the Abstract for the article. The Reader selects to download the article or to return to the article list or to the previous list.
Alternative Paths	In step 2, if the Reader selects to search by category, the system creates and presents a list of all categories in the database. The Reader selects a category. The system creates and presents a list of all articles in that category in the database. Return to step 5. In step 2, if the Reader selects to search by keyword, the system presents a dialog box to enter the keyword or phrase. The Reader enters a keyword or phrase. The system searches the Abstracts for all articles with that keyword or phrase and creates and presents a list of all such articles in the database. Return to step 5.
Postcondition	The selected article is downloaded to the client machine.
Exception Paths	The Reader may abandon the search at any time.
Other	The categories list is generated from the information provided when article are published and not predefined in the Online Journal database.

- Communicate

Use Case Name	Communicate
XRef	Section 2.2.2, Submit Article; Section 2.2.3, Submit Review SDD, Section 7.2
Trigger	The user selects a mailto link.
Precondition	The user is on the Communicate page linked from the Online Journal Main Page.
Basic Path	This use case uses the mailto HTML tag. This invokes the client email facility.
Alternative Paths	If the user prefers to use his or her own email directly, sufficient information will be contained on the Web page to do so.
Postcondition	The message is sent.
Exception Paths	The attempt may be abandoned at any time.
Other	None

- Add Author

Use Case Name	Add Author
XRef	Section 2.2.4, Update Author SDD, Section 7.3
Trigger	The Editor selects to add a new author to the database.
Precondition	The Editor has accessed the Article Manager main screen.
Basic Path	The system presents a blank grid to enter the author information. The Editor enters the information and submits the form. The system checks that the name and email address fields are not blank and updates the database.
Alternative Paths	If in step 2, either field is blank, the Editor is instructed to add an entry. No validation for correctness is made.
Postcondition	The Author has been added to the database.
Exception Paths	The Editor may abandon the operation at any time.
Other	The author information includes the name mailing address and email address.

- Add Reviewer

Use Case Name	Add Reviewer
XRef	Section 2.2.4, Update Reviewer SDD, Section 7.4

Trigger	The Editor selects to add a new reviewer to the database.
Precondition	The Editor has accessed the Article Manager main screen.
Basic Path	The system accesses the Historical Society (HS) database and presents an alphabetical list of the society members. The Editor selects a person. The system transfers the member information from the HS database to the Article Manager (AM) database. If there is no email address in the HS database, the editor is prompted for an entry in that field. The information is entered into the AM database.
Alternative Paths	In step 3, if there is no entry for the email address in the HS database or on this grid, the Editor will be reprompted for an entry. No validation for correctness is made.
Postcondition	The Reviewer has been added to the database.
Exception Paths	The Editor may abandon the operation at any time.
Other	The Reviewer information includes name, membership number, mailing address, categories of interest, and email address.

- Update Person

Use Case Name	Update Person
XRef	Sec 2.2.4 Update Author; Sec 2.2.4 Update Reviewer SDD, Section 7.5
Trigger	The Editor selects to update an author or reviewer and the person is already in the database.
Precondition	The Editor has accessed the Article Manager main screen.
Basic Path	The Editor selects Author or Reviewer. The system creates and presents an alphabetical list of people in the category. The Editor selects a person to update. The system presents the database information in grid form for modification. The Editor updates the information and submits the form. The system checks that required fields are not blank.
Alternative Paths	In step 5, if any required field is blank, the Editor is instructed to add an entry. No validation for correctness is made.
Postcondition	The database has been updated.

Exception Paths	If the person is not already in the database, the use case is abandoned. In addition, the Editor may abandon the operation at any time.
Other	This use case is not used when one of the other use cases is more appropriate, such as to add an article or a reviewer for an article.

- Update Article Status

Use Case Name	Update Article Status
XRef	Section 2.2.4, Update Article SDD, Section 7.6
Trigger	The Editor selects to update the status of an article in the database.
Precondition	The Editor has accessed the Article Manager main screen and the article is already in the database.
Basic Path	The system creates and presents an alphabetical list of all active articles. The Editor selects the article to update. The system presents the information about the article in grid format. The Editor updates the information and resubmits the form.
Alternative Paths	In step 4, the use case Enter Communication may be invoked.
Postcondition	The database has been updated.
Exception Paths	If the article is not already in the database, the use case is abandoned. In addition, the Editor may abandon the operation at any time.
Other	This use case can be used to add categories for an article, to correct typographical errors, or to remove a reviewer who has missed a deadline for returning a review. It may also be used to allow access to the named use case to enter an updated article or a review for an article.

- Enter Communication

Use Case Name	Enter Communication
XRef	Section 2.2.4, Receive Article; Section 2.2.4, Receive Review SDD, Section 7.7
Trigger	The Editor selects to add a document to the system.
Precondition	The Editor has accessed the Article Manager main screen and has the file of the item to be entered available.

Basic Path	The Editor selects the article using the 3.2.6, Update Article Status use case. The Editor attaches the file to the grid presented and updates the respective information about the article. When the Editor updates the article status to indicate that a review is returned, the respective entry in the Reviewer table is updated.
Alternative Paths	None
Postcondition	The article entry is updated in the database.
Exception Paths	The Editor may abandon the operation at any time.
Other	This use case extends 3.2.6, Update Article Status

- Assign Reviewer

Use Case Name	Assign Reviewer
XRef	Section 2.2.4, Assign Reviewer SDD, Section 7.8
Trigger	The Editor selects to assign a reviewer to an article.
Precondition	The Editor has accessed the Article Manager main screen and the article is already in the database. .
Basic Path	The Editor selects the article using the 3.2.6, Update Article Status use case. The system presents an alphabetical list of reviewers with their information. The Editor selects a reviewer for the article. The system updates the article database entry and emails the reviewer with the standard message and attaches the text of the article without author information. The Editor has the option of repeating this use case from step 2.
Alternative Paths	None.
Postcondition	At least one reviewer has been added to the article information and the appropriate communication has been sent.
Exception Paths	The Editor may abandon the operation at any time.
Other	This use case extends 3.2.6, Update Article Status. The Editor, prior to implementation of this use case, will provide the message text.

- Check Status

Use Case Name	Check Status
XRef	Section 2.2.4, Check Status SDD, Section 7.9
Trigger	The Editor has selected to check status of all active articles.
Precondition	The Editor has accessed the Article Manager main screen.
Basic Path	The system creates and presents a list of all active articles organized by their status. The Editor may request to see the full information about an article.
Alternative Paths	None.
Postcondition	The requested information has been displayed.
Exception Paths	The Editor may abandon the operation at any time.
Other	The editor may provide an enhanced list of status later. At present, the following categories must be provided: 1. Received but no further action taken Reviewers have been assigned but not all reviews are returned (include dates that reviewers were assigned and order by this criterion). Reviews returned but no further action taken. Recommendations for revision sent to Author but no response as of yet. Author has revised article but no action has been taken. Article has been accepted and copyright form has been sent. Copyright form has been returned but article is not yet published. A published article is automatically removed from the active article list.

- Send Communication

Use Case Name	Send Communication
XRef	Section 2.2.4, Send Response; Section 2.2.4, Send Copyright SDD, Section 7.10
Trigger	The editor selects to send a communication to an author.
Precondition	The Editor has accessed the Article Manager main screen.

Basic Path	The system presents an alphabetical list of authors. The Editor selects an author. The system invokes the Editor's email system entering the author's email address into the To: entry. The Editor uses the email facility.
Alternative Paths	None.
Postcondition	The communication has been sent.
Exception Paths	The Editor may abandon the operation at any time.
Other	The standard copyright form will be available in the Editor's directory for attaching to the email message, if desired.

- Publish Article

Use Case Name	Publish Article
XRef	Section 2.2.4, Publish Article SDD, Section 7.11
Trigger	The Editor selects to transfer an approved article to the Online Journal.
Precondition	The Editor has accessed the Article Manager main screen.
Basic Path	The system creates and presents an alphabetical list of the active articles that are flagged as having their copyright form returned. The Editor selects an article to publish. The system accesses the Online Database and transfers the article and its accompanying information to the Online Journal database. The article is removed from the active article database.
Alternative Paths	None.
Postcondition	The article is properly transferred.
Exception Paths	The Editor may abandon the operation at any time.
Other	Find out from the Editor to see if the article information should be archived somewhere.

- Remove Article

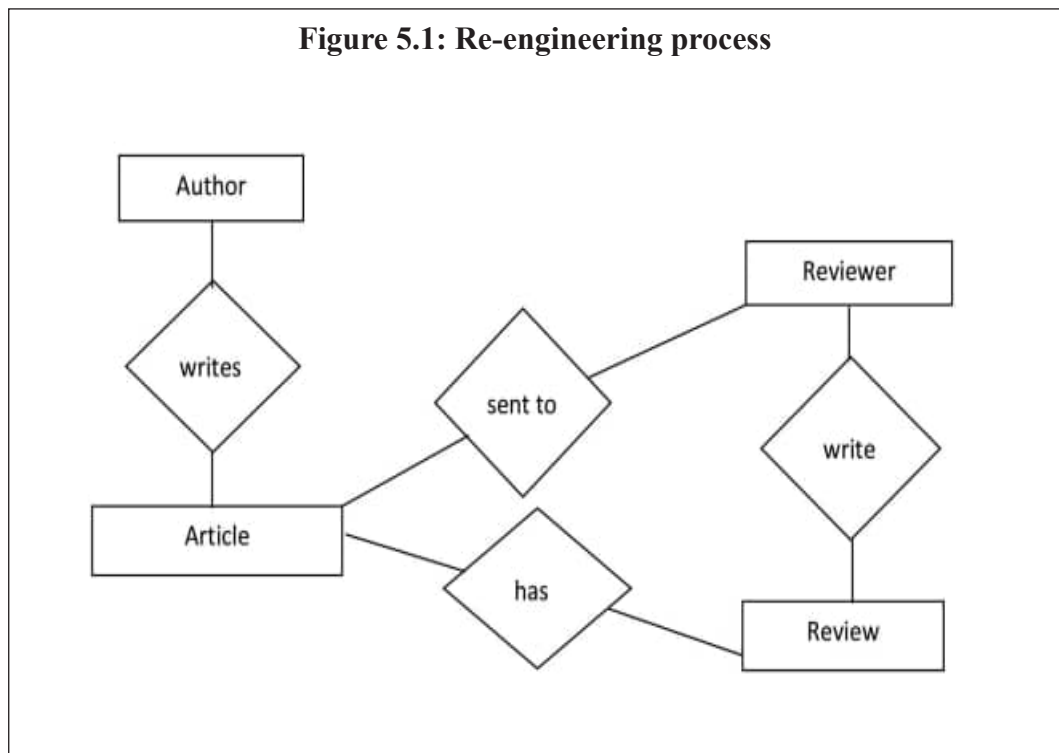
Use Case Name	Remove Article
XRef	Section 2.2.4, Remove Article SDD, Section 7.12

Trigger	The Editor selects to remove an article from the active article database.
Precondition	The Editor has accessed the Article Manager main screen.
Basic Path	The system provides an alphabetized list of all active articles. The editor selects an article. The system displays the information about the article and requires that the Editor confirm the deletion. The Editor confirms the deletion.
Alternative Paths	None.
Postcondition	The article is removed from the database.
Exception Paths	The Editor may abandon the operation at any time.
Other	Find out from the Editor to see if the article and its information should be archived somewhere.

i. Detailed Non-Functional Requirements

- Logical Structure of the Data

The logical structure of the data to be stored in the internal Article Manager database is given below (Figure 5.1).



Logical Structure of the Article Manager Data

The data descriptions of each of these data entities are as follows:

Author Data Entity

Data Item	Type	Description	Comment
Name	Text	Name of principle author	
Email Address	Text	Internet address	
Article	Pointer	Article entity	May be several

Reviewer Data Entity

Data Item	Type	Description	Comment
Name	Text	Name of principle author	
ID	Integer	ID number of Historical Society member	Used as key in Historical Society Database
Email Address	Text	Internet address	
Article	Pointer	Article entity of	May be several
Num Review	Integer	Review entity	Number of not returned reviews
History	Text	Comments on past performance	
Specialty	Category	Area of expertise	May be several

Review Data Entity

Data Item	Type	Description	Comment
Article	Pointer	Article entity	
Reviewer	Pointer	Reviewer entity	Single reviewer
Date Sent	Date	Date sent to reviewer	
Returned	Date	Date returned; null if not returned	
Contents	Text	Text of review	

Article Data Entity

Data Item	Type	Description	Comment
Name	Text	Name of Article	
Author	Pointer	Author entity	Name of principle author
Other Authors	Text	Other authors is any; else null	Not a pointer to an Author entity
Reviewer	Pointer	Reviewer entity	Will be several
Review	Pointer	Review entity	Set up when reviewer is set up
Contents	Text	Body of article	Contains Abstract as first paragraph.
Category	Text	Area of content	May be several
Accepted	Boolean	Article has been accepted for publication	Needs Copyright form returned
Copyright	Boolean	Copyright form has been returned	Not relevant unless Accepted is True.
Published	Boolean	Sent to Online Journal	Not relevant unless Accepted is True. Article is no longer active and does not appear in status checks.

The Logical Structure of the data to be stored in the Online Journal database on the server is as follows:

Published Article Entity

Data Item	Type	Description	Comment
Name	Text	Name of Article	
Author	Text	Name of one Author	May be several
Abstract	Text	Abstract of article	Used for keyword search
Content	Text	Body of article	
Category	Text	Area of content	May be several

Security

The server on which the Online Journal resides will have its own security to prevent unauthorized write/delete access. There is no restriction on read access. The use of email by an Author or Reviewer is on the client systems and thus is external to the system.

The PC on which the Article Manager resides will have its own security. Only the Editor will have physical access to the machine and the program on it. There is no special protection built into this system other than to provide the editor with *write* access to the Online Journal to publish an article.

5.3 Software Project Estimation

Estimation of the size of software is an essential part of Software Project Management. It helps the project manager to further predict the effort and time which will be needed to build the project. Various measures are used in project size estimation. Some of these are:

- Lines of Code
- Number of entities in ER diagram
- Total number of processes in detailed data flow diagram
- Function points

5.3.1 Lines of Code (LOC)

As the name suggest, LOC count the total number of lines of source code in a project. The units of LOC are:

- KLOC- Thousand lines of code
- NLOC- Non comment lines of code
- KDSI- Thousands of delivered source instruction

The size is estimated by comparing it with the existing systems of same kind. The experts use it to predict the required size of various components of software and then add them to get the total size.

Advantages:

- Universally accepted and is used in many models like COCOMO.
- Estimation is closer to developer's perspective.
- Simple to use.

Disadvantages:

- Different programming languages contains different number of lines.
- No proper industry standard exist for this technique.
- It is difficult to estimate the size using this technique in early stages of project.

5.3.2 Number of entities in ER diagram

ER model provides a static view of the project. It describes the entities and its relationships. The number of entities in ER model can be used to measure the estimation of size of project. Number of entities depends on the size of the project. This is because more entities needed more classes/structures thus leading to more coding.

Advantages:

- Size estimation can be done during initial stages of planning.
- Number of entities is independent of programming technologies used.

Disadvantages:

- No fixed standards exist. Some entities contribute more project size than others.
- Just like FPA, it is less used in cost estimation model. Hence, it must be converted to LOC.

5.3.3. Total number of processes in detailed data flow diagram

Data Flow Diagram (DFD) represents the functional view of a software. The model depicts the main processes/functions involved in software and flow of data between them. Utilization of number of functions in DFD to predict software size. Already existing processes of similar type are studied and used to estimate the size of the process. Sum of the estimated size of each process gives the final estimated size.

Advantages:

- It is independent of programming language.
- Each major process can be decomposed into smaller processes. This will increase the accuracy of estimation

Disadvantages:

- Studying similar kind of processes to estimate size takes additional time and effort.
- All software projects are not required to construction of DFD.

5.3.4 Function Point Analysis

In this method, the number and type of functions supported by the software are utilized to find FPC(function point count). The steps in function point analysis are:

- Count the number of functions of each proposed type.
- Compute the Unadjusted Function Points(UFP).
- Find Total Degree of Influence(TDI).
- Compute Value Adjustment Factor(VAF).
- Find the Function Point Count(FPC).

The explanation of above points given below:

- **Count the number of functions of each proposed type:** Find the number of functions belonging to the following types:
 - External Inputs: Functions related to data entering the system.
 - External outputs: Functions related to data exiting the system.
 - External Inquiries: They leads to data retrieval from system but don't change the system.
 - Internal Files: Logical files maintained within the system. Log files are not included here.
 - External interface Files: These are logical files for other applications which are used by our system.
- **Compute the Unadjusted Function Points(UFP):** Categorise each of the five function types as simple, average or complex based on their complexity. Multiply count of each function type with its weighting factor and find the weighted sum. The weighting factors for each type based on their complexity are as follows:

Function type	Simple	Average	Complex
External Inputs	3	4	6
External Output	4	5	7
External Inquiries	3	4	6
Internal Logical Files	7	10	15
External Interface Files	5	7	10

- **Find Total Degree of Influence:** Use the '14 general characteristics' of a system to find the degree of influence of each of them. The sum of all 14 degrees of influences will give the TDI. The range of TDI is 0 to 70. The 14 general characteristics are: Data Communications, Distributed Data Processing, Performance, Heavily Used Configuration, Transaction Rate, On-Line Data Entry, End-user Efficiency, Online Update, Complex Processing Reusability, Installation Ease, Operational Ease, Multiple Sites and Facilitate Change. Each of above characteristics is evaluated on a scale of 0-5.

- **Compute Value Adjustment Factor(VAF):**

Use the following formula to calculate VAF $VAF = (TDI * 0.01) + 0.65$

- **Find the Function Point Count:**

Use the following formula to calculate FPC $FPC = UFP * VAF$

Advantages:

- It can be easily used in the early stages of project planning.
- It is independent on the programming language.
- It can be used to compare different projects even if they use different technologies (database, language etc).

Disadvantages:

- It is not good for real time systems and embedded systems.
- Many cost estimation models like COCOMO uses LOC and hence FPC must be converted to LOC.

5.4 COCOMO Model

The Constructive Cost Model (COCOMO) is an algorithmic software cost estimation model developed by Barry Boehmin 1981. The model uses a basic regression formula, with parameters that are derived from historical project data and current project characteristics. It predicts the efforts, development time, average team size and effort required to develop a software project.

COCOMO applies to three classes of software projects:

- Organic projects - This suits small software team since it has a generally stable development environment. The problem is well understood and has been solved in the past. (In short, “small” teams with “good” experience working with “less than rigid” requirements.)

Example: Small data processing or Inventory management system.

- Semi-detached projects - The software project which requires more experience and better guidance and creativity. For example, Compiler or different Embedded System (In short, “medium” teams with mixed experience working with a mix of rigid and less than rigid requirements.)

Example: Database design or OS development.

- Embedded projects - The project with operating tight constraints and requirements is under this category. The developer requires high experiences and has to be creative to develop complex models. (In Short, developed within a set of “tight” constraints (hardware, software, operational,))

Example: Banking software or Traffic light control software.

5.4.1 Types of COCOMO model

COCOMO model allows software manager to decide how detailed they would like to conduct the cost estimation for their own project. COCOMO can unveil the efforts and schedule of a software product based on the size of the software in different levels. There are basic COCOMO, Intermediate COCOMO, and Detailed/Completed COCOMO.

- **Basic COCOMO model**

The basic COCOMO is used for rough calculation which limits accuracy in calculating software estimation. This is because the model solely considers based on lines of source code together with constant values obtained from software project types rather than other factors which have major influences to Software development process as a whole.

- **Intermediate COCOMO model**

Intermediate COCOMO model is an extension of the Basic COCOMO model which includes a set of cost drivers into account in order to enhance more accuracy to the cost estimation model as a result.

Classification of Cost Drivers and their attributes:

(i) Product attributes –

- Required software reliability extent
- Size of the application database
- The complexity of the product

(ii) Hardware attributes –

- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time

(iii) Personnel attributes –

- Analyst capability
- Software engineering capability
- Applications experience
- Virtual machine experience
- Programming language experience

(iv) Project attributes –

- Use of software tools
- Application of software engineering methods
- Required development schedule
- Complete/detailed COCOMO model

The model incorporates all qualities of both Basic COCOMO and Intermediate COCOMO strategies on each software engineering process. The model accounts for the influence of the individual development phase (analysis, design, etc.) of the project.

The effort is calculated as a function of program size and a set of cost drivers are given according to each phase of the software lifecycle.

For instance, detailed COCOMO will perform cost estimation in each development phase of Waterfall Model.

5.4.2 Numerical:

In COCOMO, the general calculation steps of COCOMO-based cost estimation are the following:

1. Get an initial estimate of the development effort from evaluation of thousands of delivered lines of source code KDLOC.
2. Determine a set of 15 cost factors from various attributes of the project.
3. Calculate the effort estimate by multiplying the initial estimate with all the multiplying factors i.e., multiply the values in previous steps.

Example:

1. Basic COCOMO model

Suppose the project was estimated to be 300 KDLOC, calculate the effort, development time, and time of each of the 3 modes

Note: the basic COCOMO is used in Organic mode by default.

Here, The estimated effort and scheduled time for the project are given by the relation:

$$\text{Effort (E)} = a * (\text{KLOC})^b \text{ MM}$$

$$\text{Scheduled Time (D)} = c * (E)^d \text{ Months(M)}$$

Where,

- **E** = Total effort required for the project in Man-Months (MM).
- **D** = Total time required for project development in Months (M).
- **KLOC** = the size of the code for the project in Kilo lines of code.
- **a, b, c, d** = The constant parameters for a software project.

PROJECT TYPE	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3	1.12	2.5	0.35
Embedded	3.6	1.2	2.5	0.32

Solution:

Given estimated size of project is: 300 KLOC

- For Organic

$$\text{Effort (E)} = a * (\text{KLOC})^b = 2.4 * (300)^{1.05} = 957.61 \text{ MM}$$

$$\text{Scheduled Time (D)} = c * (E)^d = 2.5 * (957.61)^{0.38} = 33.95 \text{ Months(M)}$$

$$\text{Avg. Resource Size} = E/D = 957.61/33.95 = 28.21 \text{ Mans}$$

$$\text{Productivity of Software} = \text{KLOC}/E = 300/957.61 = 0.3132 \text{ KLOC/MM} = 313 \text{ LOC/MM}$$

- For Semidetached

$$\text{Effort (E)} = a * (\text{KLOC})^b = 3.0 * (300)^{1.12} = 1784.42 \text{ MM}$$

$$\text{Scheduled Time (D)} = c * (E)^d = 2.5 * (1784.42)^{0.35} = 34.35 \text{ Months(M)}$$

- For Embedded

$$\begin{aligned} \text{Effort (E)} &= a * (\text{KLOC})^b = 3.6 * (300)^{1.2} = 3379.46 \text{ MM} \\ \text{Scheduled Time (D)} &= c * (\text{E})^d = 2.5 * (3379.46)^{0.32} = 33.66 \text{ Months(M)} \end{aligned}$$

2. Intermediate COCOMO model

The estimated effort and scheduled time are given by the relationship:

$$\begin{aligned} \text{Effort (E)} &= a * (\text{KLOC})^b * \text{EAF} \text{ MM} \\ \text{Scheduled Time (D)} &= c * (\text{E})^d \text{ Months(M)} \end{aligned}$$

Where,

- **E** = Total effort required for the project in Man-Months (MM).
- **D** = Total time required for project development in Months (M).
- **KLOC** = The size of the code for the project in Kilo lines of code.
- **a, b, c, d** = The constant parameters for the software project.

EAF = It is an Effort Adjustment Factor, which is calculated by multiplying the parameter values of different cost driver parameters. For ideal, the value is 1.

Suppose project was estimated with a size of 300 KLOC. Calculate the Effort, Scheduled time for development by considering developer having high application experience and very low experience in programming.

COST DRIVERS PARAMETERS	VERY LOW	LOW	NOMINAL	HIGH	VERY HIGH
Product Parameter					
Required Software	0.75	0.88	1	1.15	1.4
Size of Project Database	NA	0.94		1.08	1.16
Complexity of The Project	0.7	0.85		1.15	1.3
Hardware Parameter					
Performance Restriction	NA	NA	1	1.11	1.3
Memory Restriction	NA	NA		1.06	1.21
virtual Machine Environment	NA	0.87		1.15	1.3
Required Turnabout Time	NA	0.94		1.07	1.15

Personnel Parameter					
Analysis Capability	1.46	1.19	1	0.86	0.71
Application Experience	1.29	1.13		0.91	0.82
Software Engineer Capability	1.42	1.17		0.86	0.7
Virtual Machine Experience	1.21	1.1		0.9	NA
Programming Experience	1.14	1.07		0.95	NA
Project Parameter					
Software Engineering Methods	1.24	1.1	1	0.91	0.82
Use of Software Tools	1.24	1.1		0.91	0.83
Development Time	1.23	1.08		1.04	1.1

Solution:

Given the estimated size of the project is: 300 KLOC
 Developer having highly application experience: 0.82 (as per above table)
 Developer having very low experience in programming: 1.14(as per above table)

$$EAF = 0.82 * 1.14 = 0.9348$$

$$Effort (E) = a * (KLOC)^b * EAF = 3.0 * (300)^{1.12} * 0.9348 = 1668.07 MM$$

$$Scheduled Time (D) = c * (E)^d = 2.5 * (1668.07)^{0.35} = 33.55 Months(M)$$

3. The Detailed COCOMO

In detailed cocomo, the whole software is divided into different modules and then we apply COCOMO in different modules to estimate effort and then sum the effort.

The Six phases of detailed COCOMO are:

1. Planning and requirements
2. System design
3. Detailed design
4. Module code and test
5. Integration and test
6. Cost Constructive model

5.4.3 Advantages and Disadvantages of COCOMO Model

Following are some advantages and disadvantages of the COCOMO model.

Advantages

- Easy to estimate the total cost of the project.
- Easy to implement with various factors.
- Provide ideas about historical projects.

Disadvantages

- It ignores requirements, customer skills, and hardware issues.
- It limits the accuracy of the software costs.
- It mostly depends on time factors.

5.5 COCOMO II Model

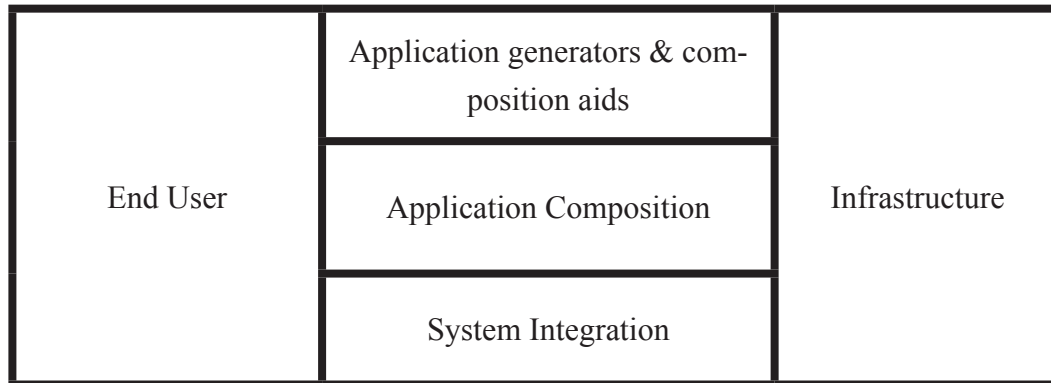
COCOMO-II is the revised version of the original Cocomo (Constructive Cost Model) and is developed at University of Southern California. COCOMO II takes into account different approaches to software development, reuse, etc. It is the model that allows one to estimate the cost, effort and schedule when planning a new software development activity.

5.5.1 Sub-models in COCOMO II

COCOMO II incorporates a variety of sub-models that generate increasingly detailed software estimates. The sub-models in COCOMO II are:

- Application composition model:** Used when software is composed from existing parts.
- Early design model:** Used when requirements are available but design has not yet started.
- Reuse model:** Used to compute the effort of integrating reusable components.
- Post architecture model:** Used once the system architecture has been designed and more information about the system is available.

COCOMO II Estimation

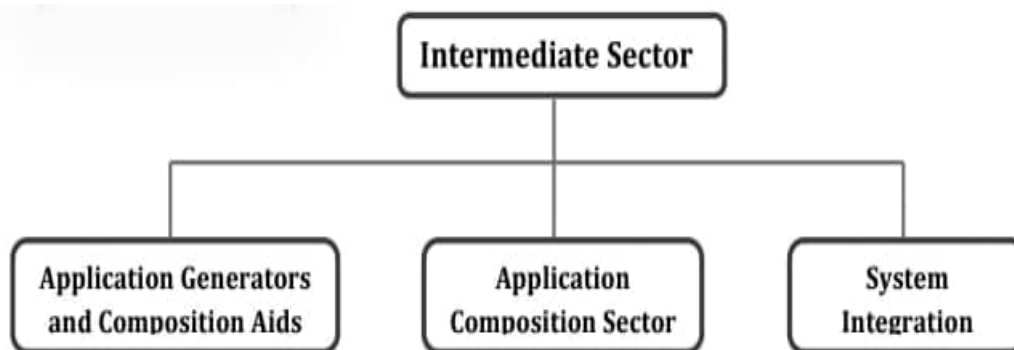


1. End User Programming:

Application generators are used in this sub-model. End user writes the code by using these application generators.

Example – Spreadsheets, report generator, etc.

2. Intermediate Sector:



a. Application Generators and Composition Aids:

This category will create largely prepackaged capabilities for user programming. Their product will have many reusable components. Typical firms operating in this sector are Microsoft, Lotus, Oracle, IBM, Borland, Novell.

b. Application Composition Sector:

This category is too diversified and to be handled by prepackaged solutions. It includes GUI, Databases, domain specific components such as financial, medical or industrial process control packages.

c. System Integration:

This category deals with large scale and highly embedded systems.

3. Infrastructure Sector:

This category provides infrastructure for the software development like Operating System, Database Management System, User Interface Management System, Networking System, etc.

- a. **Stage-I:** It supports estimation of prototyping. For this it uses Application Composition Estimation Model. This model is used for the prototyping stage of application generator and system integration.
- b. **Stage-II:** It supports estimation in the early design stage of the project, when we less know about it. For this it uses Early Design Estimation Model. This model is used in early design stage of application generators, infrastructure, and system integration.

- c. **Stage-III:**

It supports estimation in the post architecture stage of a project. For this it uses Post Architecture Estimation Model. This model is used after the completion of the detailed architecture of application generator, infrastructure, and system integration.

COCOMO II can be used for the following major decision situations

- Making investment or other financial decisions involving a software development effort
- Setting project budgets and schedules as a basis for planning and control
- Deciding on or negotiating tradeoffs among software cost, schedule, functionality, performance or quality factors
- Making software cost and schedule risk management decisions
- Deciding which parts of a software system to develop, reuse, lease, or purchase
- Making legacy software inventory decisions: what parts to modify, phase out, outsource, etc
- Setting mixed investment strategies to improve organization's software capability, via reuse, tools, process maturity, outsourcing, etc.

5.6 Comparison of COCOMO and COCZOMO-II

COCOMO I	COCOMO II
COCOMO I is convenient in the water-fall models of the software development cycle.	COCOMO II is helpful in non-sequential, rapid development and reuse models of software.
Effort equation's exponent is determined by 3 development modes.	Effort equation's exponent is determined by 5 scale factors.
This model is based upon the linear reuse formula.	This model is based upon the non-linear reuse formula
Development begins with the requirements assigned to the software.	It follows a spiral type of development.

5.7 Earned Value Management

Earned value management (EVM) is a project management methodology that integrates schedule, costs, and scope to measure project performance. Based on planned and actual values, EVM predicts the future and enables project managers to adjust accordingly. It is a technique that project managers use to track the performance of their project against project baselines. Often the progress of a project is thought of simply as being ahead or behind schedule and over or under budget.

This is where knowing the earned value helps. It can provide deeper information on your project. And when learning about earned value, it's important to remember that there are three terms associated with it, which are each slightly different.

1. **Earned Value Analysis (EVA):** This project management technique is quantitative. It evaluates project performance by figuring out the likely results of the project. It does this by comparing the progress and budget of work planned to the actual costs.
2. **Earned Value Management (EVM):** This methodology measures project performance with an integrated schedule and budget, which is based on the project work breakdown structure (WBS).
3. **Earned Valued Management System (EVMS):** This is the collection of tools, templates, processes and procedures that an organization uses to do EVM.

Calculations for Earned Value Management

There are calculations that can be done quickly and easily to execute EVM. To do so, you need to know the following:

c. Planned Value (PV)

Planned value is the budgeted cost for work scheduled (BCWS). PV varies based on the scope of work in consideration and the point where you're at in the overall schedule.

$PV = \text{Total project cost} * \% \text{ of planned work}$

For example, let's say, the PV for your 5-month project is \$25,000:

PV for the complete project = \$25,000

PV at 2 months = $\$25,000 * 40\% = \$10,000$

You can also calculate PV for a time period,
say, month 2 to month 4 = $\$25,000 * 60\% = \$15,000$.

d. Actual Costs (AC)

Actual costs, also referred to as actual cost of work performed (ACWP), is relatively straightforward. If you are using a robust project cost management software, tracking actual costs should not be a challenge. However, it's important to remember to include several hidden costs—material, resource, hardware, software licenses, overheads, etc.

You can look at AC cumulatively, accounting for all the activities done from the beginning of the project to date or over a specific time period.

In our example, let's assume, AC at the end of 2 months = \$15,000

e. Earned Value (EV)

Now, this is where EVM gets interesting. You've made a plan to finish some amount of work and budgeted accordingly. But, from experience, you know that there is bound to be some discrepancy from your estimate. At the end of 2 months, you may have planned to complete 40% of your work, but let's say you managed to just finish 30%.

The question, then, is, what's the budgeted cost for this work? EV, also called as budgeted cost for work performed (BCWP), gives you the answer.

In our example:

$EV = \text{Total project cost} * \% \text{ of actual work} = \$25,000 * 30\% = \$7,500$

5.8 Variance Analysis

Planned value, actual cost, and earned value numbers are fundamental to variance calculations. At this point, the project manager wants to know how far off we are from the project baseline. This can be determined through schedule and cost variance.

a. Schedule Variance (SV)

Schedule variance is a quantitative indicator of your divergence from the initial planned schedule. A negative SV indicates that we are behind schedule, a positive SV indicates that we are ahead of schedule and zero means that we are exactly on schedule.

$$SV = EV - PV$$

In our example, $SV \text{ at 2 months} = \$7,500 - \$10,000 = -\$2,500$
 $SV\% = (SV/PV) * 100 = (-\$2,500/\$10,000) * 100 = -25\%$

This implies that we are 25% behind schedule. It's interesting to note that we aim to understand schedule, a time component, from the perspective of costs. To arrive at these costs though, we needed to know the scope of work planned and completed. This is how the three pillars—scope, time and cost come together in EVM.

b. Cost Variance (CV)

Cost variance is a quantitative indicator of your divergence from the initial planned budget. A negative CV indicates that we are over budget, a positive CV indicates that we are under budget and zero means that we are exactly on budget.

$$CV = EV - AC$$

In our example, $CV \text{ at 2 months} = \$7,500 - \$15,000 = -\$7,500$
 $CV\% = (CV/EV) * 100 = (-\$7,500/\$7,500) * 100 = -100\%$

This implies that we are 100% over budget.

Again, this is an instance of how scope, time and cost come together to give you a clear picture of where you stand at the moment in your project.

5.9 Summary

This chapter based on requirement analysis which includes data requirement, data analysis, software requirements, system user requirements, project estimation with case study. Detailed COCOMO and COCOMO II model as well as difference between these two models are explained with numerical.

5.10 Questions

- Explain the software project requirement specification?
- State the software Project Estimation in detail.
- What is COCOMO? Explain types of COCOMO with example.
- Explain COCOMO II and its sub-model.

5.11 Reference books

- Information Technology Project Management by Jack T Marchewka Wiley India publication
- Software Engineering, by Roger S Pressman, McGraw Hill publication.
- Software Engineering by KK Agarwal , Yogesh Singh , New Age International publication.



SOFTWARE PROJECT PLANNING

Unit Structure

- 6.0 Objective
- 6.1 Introduction
- 6.2 The Business Case
 - 6.2.1 What Is A Business Case?
 - 6.2.2 Developing The Business Case
 - 6.2.3 Process For Developing A Business Case
- 6.3 Project Selection And Approval
 - 6.3.1 The IT Project Selection Process
 - 6.3.3 Balanced Scorecard Approach
 - 6.3.2 The Project Selection Decision
- 6.4 The Project Charter
 - 6.4.1 What Should Be In A Project Charter
- 6.5 Project Scope Definition
 - 6.5.1 Project- Oriented Scope
 - 6.5.2 Product – Oriented Scope
- 6.6 Project Scope Verification
- 6.7 Scope Change Control
- 6.8 Work Breakdown Structure
- 6.9 Summary

6.0 Objective

- Describe and be able to prepare a business case.
- Describe the project selection process as well as the Balanced Scored approach.
- Develop a project charter and describe its relationship to the project plan.
- Describe how project planning framework links the project's measurable organizational value (MOV) to the project's scope, schedule and budget.
- Describe the difference between product scope and project scope.
- Apply several tools and techniques for defining and managing the project's scope.
- Develop a work breakdown structure.

6.1 Introduction

A business case is a deliverable that documents the project's goal, as well as several alternative or options. The feasibility, costs, benefits, and risks for each alternative are analyzed and compared, and a recommendation to approve and fund one of the alternatives is made to senior management. The first phase of the IT project methodology, as in all of phases, ends with a review of the project by the client or sponsor.

The project charter and detailed project plan make up the project's tactical plan. The project charter defines the project infrastructure and identifies the project manager, the project team, the stakeholders, and the roles each will play within the project. In addition, the project charter formalizes the project's MOV, scope, supporting processes and controls, required resources, risks, and assumptions. This project infrastructure provides the foundation for developing a detailed project plan that answers four major questions. How much will the project cost? When will the project be finished? Who will be responsible for doing the work? And, what will we ultimately achieve at the end of the project?

The term scope is used to define the work boundaries and deliverables of the project so what needs to get done, gets done-and only what needs to get done, gets done. Therefore, it is important to define not only what is part of the project is considered to be outside of the project's scope.

A work breakdown structure (WBS) is discussed first. It provides a hierarchical structure that outlines the activities or work that needs to be done in order to complete the project scope. The WBS also provides a bridge or link between the project's scope and the detailed project plan that will be entered into a project management software package.

Today, most project management software packages are relatively inexpensive and rich in features. It is almost unthinkable that anyone would plan and manage a project without such a tool. Project success, however, will not be determined by one's familiarity with a project management software package or the ability to produce nice looking reports and graphs. It is the thought process that must be followed before using the tool that counts. Thinking carefully through the activities and their estimated durations first will make the use of a project management software package much more effective. You can still create nice looking reports and graphs, but you'll have more confidence in what those reports and graphs say.

6.2 The Business Case

6.2.1 What is a Business Case?

A business case provides the first deliverable in the IT project life cycle. It provides an analysis of the organizational value, feasibility, costs, benefits, and risks of several proposed alternatives or options. However, a business case is not a budget or the project plan. The purpose of a business case is to provide senior management with all the information needed to make an informed decision as to whether a specific project should be funded.

For large projects, a business case may be a large, formal document. Even for smaller projects, however, the process of thinking through why a particular project is being taken on and how it might bring value to an organization is still useful.

Because assumptions and new information are sometimes used to make subjective judgments, a business case must also document the methods and rationale used for quantifying the costs and benefits. Different people who work independently to develop a business case can use the information, tools, and methods, but still come up with different recommendations.

One can also think of a business case as an investment proposal or a legal case. The business case developer has a large degree of latitude to structure arguments, select or ignore evidence, and deliver the final presentations. The outcome depends

largely on the ability to use compelling facts and logic in order to influence an individual or group with decision - making authority.

A good IT business case should be

- (1) through in detailing all possible impacts, costs, and benefits;
- (2) clear and logical in comparing the costs/ benefits impact of each alternative;
- (3) objective though including all pertinent information; and
- (4) systematic in terms of summarizing the findings.

6.2.2 Developing the Business Case

The purpose of a business case is to show how an IT solution can create business value. For example, an IT project may be undertaken to:

- Reduce costs
- Create a new product or service
- Improve customer service
- Improve communication
- Improve decision making
- Create or strengthen relationships with suppliers, customers, or partners
- Improve processes
- Improve reporting capabilities
- Support new legal requirements

6.2.3 Process For Developing A Business Case

Step 1: Select the Core Team

The core team should include managers, business specialists, and users who understand the requirements to be met, as well as IT specialists who understand the opportunities, limitations, and risks associated with IT. There are several advantages for having a core team develop the business case

Credibility - A team made up of individuals from various organizational areas or departments can provide access to critical expertise and information that may not be readily accessible to others outside that particular area.

Alignment with organizational goals – Higher level managers can help connect the business case with organization's long term strategic plan and mission. This alignment may be beneficial in understanding and presenting how the expected business value of the IT project will support the overall goals and mission of the organization.

Access to the real costs – Core members with certain expertise or access to important information can help build more realistic estimates in areas such as salaries, overhead, accounting and reporting practices, training requirements, union rules and regulations, and hiring practices.

Step 2: Define Measurable Organizational Value (MOV)

The core team's objective should be to define the problem or opportunity and then identify several alternatives that will provide direct and measurable value to the organization. To provide real value to an organization, however, IT projects must align with and support the organization's goals, mission, and objectives.

- Be measurable – Measurement provides focus for the project team in terms of its actions. Instead of implementing an information system, the project team attempts to achieve a specific performance target.
- Provide value to organization – Resources and time should not be devoted to a project unless they provide some kind of value to the organization.
- Be agreed on – A clear and agreed on MOV sets expectations for the project stakeholders. It is important that all project stakeholders understand and agree to the project's MOV.
- Be verifiable – At the end of project, the MOV must be verified to determine if the project was a success.

The MOV guides all the decisions and processes for managing the IT project and serves as a basis for evaluating the project's achievements. In other words, a project cannot be properly planned or evaluated unless the goal of the project is clearly defined and understood.

Step 3: Identify Alternatives - Since no single solution generally exists for most organizational problems, it is imperative to identify several alternatives before dealing directly with a given business opportunity. The alternatives, or options, identified in the business case should be strategies for achieving the MOV.

The other options may provide the best solution. Consider a spectrum of choices that include:

Changing the existing business processes without investing in IT

Adopting or adapting an application developed by different area or department within the organization

Reengineering the existing system

Purchasing an off- the shelf application package from a software vendor.

Custom building a new application using internal resources or outsourcing the development to another company.

Step 4 : Define feasibility and Assess Risk –

Feasibility should focus on whether a particular alternative is worth doing. Risk – It focus on what can go wrong and what must go right.

Feasibility may be viewed in terms of:

Economic feasibility – An organization may evaluate an alternative in terms of whether funds and resources exist to support the project.

Technical feasibility – It focuses on the existing technical infrastructure needed to support the IT solution.

Organizational feasibility - It focuses on how people within the organization will adapt to this planned organizational change. How will people and way they do their jobs be impacted?

Other feasibilities – Depending on the situation and the organization, a business case may include other issues, such as legal and ethical feasibility.

Risk should focus on:

Identification – What can go wrong? What must be right ?

Assessment – What is impact of each risk?

Response - How can the organization avoid or minimize the risk ?

Step 5: Define Total cost of Ownership

It includes

- Direct or up-front costs – Initial purchase price of all hardware, software, and telecommunications equipment, all development or installation costs, outside consultant fees etc.
- Ongoing costs – Salaries, training, upgrades, supplies, maintenance etc.
- Indirect costs – initial loss of productivity, time lost by users when the system is down, the cost of auditing equipment, quality assurance, and post implementation reviews.

Step 6: Define Total Benefits of Ownership

It must include all of direct, ongoing, and indirect benefits associated with each proposed alternative.

- Increasing high -value work: - For example, a salesperson may spend less time on paper work and more time calling on customers.
- Improving accuracy and efficiency: - For example, reducing errors, duplication, or the number of steps in a process.
- Improving decision making: - For example, providing timely and accurate information.
- Improving customer service: - For example, new products or services, faster or more reliable service, convenience, and so on.

Step 7: Analyze Alternatives

Once costs and benefits have been identified, it is important that all alternatives be compared with each other consistently. There are several ways to analyze the proposed alternatives. The most common are financial models and scoring models.

Financial models – It focus on either profitability and/ or cash flows. Cash flow models focus on the net cash , may be positive or negative, and are calculated by subtracting the cash outflows from cash inflows. In general one could view the benefit associated with a particular alternative as a source of cash inflow and the costs as the source of outflows.

The most commonly used cash flow models include payback, breakeven, return on investment, net present value, and scoring.

Scoring models provide a method for comparing alternatives or projects based on a weighted score. Scoring model also allow for quantifying intangible benefits or for different alternatives using multiple criteria. Using percentage weights, one can assign values of importance to the different criteria. The weights must sum to 100 percent, and when multiplied by a score assigned to each criterion they allow a composite score that is weighted average.

Step 8 : Propose and Support the Recommendation- Once the alternatives have been identified and analyzed, the last step is to recommend one of the options. It is important to remember that a proposed recommendation must be supported. The business case should be formalized in a professional-looking report. Remember that the quality and accuracy of your work will be a reflection on you and your organization.

6.3 Project Selection and Approval

The objective of the business case is to obtain approval and funding for a proposed alternative. However, a proposed project may have to compete against several others.

The criteria for selecting a project portfolio, a set of projects that an organization may fund, are very similar to the analysis and subsequent selection of the proposed project alternatives. An IT project portfolio mainly composed of projects with low risk or those that do not attempt to take advantage of new technology may lead to stagnation. The organization may not move ahead strategically and the IT employees may fail to grow professionally due to lack of challenge. On other hand, an organization that focuses too heavily on risky projects employing cutting-edge technology may end up in a precarious position if the IT projects experience serious problems and failures. Learning from mistakes can be useful, unless the same mistakes are repeated over and over. Thus , an organization should attempt to balance its IT project portfolio with projects that have varying degrees of risk ,cutting -edge technologies and structure.

6.3.1 The IT Project Selection Process

The selection process determines which projects will be funded in a given period . This period can be a quarter, year, or a time frame used by organization. In order to weed out projects that have little chance of being approved , many organizations use an initial screening process in which business cases submitted for review are compared with a set of organizational standards that outline minimum requirements.

Projects that meet the minimum requirements are then forwarded to a decision-making committee of senior managers who have the authority to approve and provide the resources needed to support the project.

Projects selected should then be assigned to a project manager who selects the project team and then develops a project charter and detailed plan.

6.3.2 The Project Selection Decision

Even though each project proposal should be evaluated in terms of its value to the organization, it is important to reiterate that projects should not be undertaken for technology's sake. The decision to approve a project requires a number of conditions be met:

- The project must map directly to the organization's strategies and goals.

- The project must provide measurable organizational value that can be verified at the completion of the project.
- The selection of a project should be based upon diversity of measures that include :
 - i. Tangible costs and benefits.
 - ii. Intangible costs and benefits.
 - iii. Various levels throughout the organization (e.g. individual, process, department, and enterprise).

6.3.3 Balanced Scorecard approach

It helps balance traditional financial measures with operational metrics across four different perspective: finance, customer satisfaction, internal business processes, and the organization's ability to innovate and learn.

Financial perspective :-

How do we look to our shareholders or to those who provide funding to us ?

Customer perspective :-

How do we look to our customers and other stakeholders ?

Internal Processes Perspective :-

What internal processes must we excel at in order to attract and retain our customers and other key stakeholders ?

Innovation and Learning Perspective :-

How do we keep getting better ?

6.4 The Project Charter

The project charter and baseline project plan provide governance framework for carrying out or executing the IT project.

It serves as an agreement or contract between the sponsor and project team- documenting the project's MOV ,defining its infrastructure ,summarizing the project plan details ,defining roles and responsibilities ,showing project commitments, and explaining project control mechanisms.

- (I) **Documenting the project's MOV:-** Although the project's MOV was included in the business case, it is important that MOV be clearly defined and agreed upon before developing or executing the project plan.

- (II) **Defining the projects infrastructure :-**The project charter defines all of the people,resources,technology, methods, project management processes, and knowledge areas that are required to support the project. In short ,the project charter will detail everything needed to carry out the project.
- (III) **Summarizing the details of the project plan :-** The project charter should summarize the scope, schedule, budget, quality objectives, deliverables ,and milestones of the project.
- (IV) **Defining roles and responsibilities :-**The project charter should not only identify the project sponsor, project manager, and project team, but also when and how they will be involved throughout the project life cycle. In addition ,the project charter should specify the line of reporting and who will be responsible for specific decisions.
- (V) **Showing explicit commitment to the project :-** In addition to defining the roles and responsibilities of the various stakeholders , the project charter should detail to resources to be provided by project sponsor and specify clearly who will take ownership of the project's product once project is completed.
- (VI) **Setting out project control mechanisms :-** Changes to project's scope, schedule ,and budget will undoubtedly be required over the course of the project. But, the project manager can lose control and the project team can lose its focus if these changes are not managed properly.

6.4.1 What Should be in A Project Charter

Project Identification: - It is common for all projects to have a unique name or way to identify them .It is especially necessary if an organization has several projects underway at once. Naming a project can also give the project team and stakeholders a sense of identify and ownership.

Project Stakeholders: - It is important that the project charter specifically name the project sponsor and the project manager. This reduces the likelihood of confusion when determining who will take ownership of the project's product and who will be leader of the project. In addition ,the project team should be named along with their titles or roles in the project ,their phone numbers , and e-mail addresses. This section should describe who will be involved in the project, how will be involved , and when will be involved.

Project Description: - The project charter should be a single source of information. Therefore, it may be useful to include a description of the project to help someone unfamiliar with the project understand not only the details, but the larger picture as well. This may include a brief overview or background of the project as to the problem or opportunity that became a catalyst for the project and the reason or purpose for taking on the project.

Measurable Organizational Value (MOV):- The MOV should be clear, concise, agreed on, and made explicit to the entire project stakeholders. Therefore, the project's MOV should be highlighted and easily identifiable in the project charter.

Project Scope:- The project's scope is the work to be completed. A specific section of the project charter should clarify not only what will be produced or delivered by the project team, but what will not be part of the project's scope.

The scope defined in the project charter can take the form of a narrative description of the products or services produced by the project. This narrative is often called the statement of work (SOW).

Project Schedule: - Although the details of project's schedule will be in the project plan, it is important to summarize the detail of the plan with respect to the expected start and completion dates. In addition, expected dates for major deliverables, milestones, and phases should be highlighted and summarized at a very high level.

Project Budget:- A section of the project charter should highlight the total cost of the project. The total cost of the project should be summarized directly from the project plan.

Quality Standard: - Although a quality management plan should be in place to support the project, a section that identifies any known or required quality standards should be made explicit in the project charter.

Resources: - Because the project charter acts as an agreement or contract, it may be useful to specify the resources required and who is responsible for providing those resources. Resources may include people, technology, or facilities to support the project team.

Assumptions and Risks: - Any risks or assumptions should be documented in the project charter. Assumptions may include things that may go right, such as a particular team member being available for the project, or specific criteria used in developing the project plan estimates. Risks on the other hand, may be thought of as anything that can go wrong or things that may impact the success of the project.

Project Charter should be summarize the following potential impacts

- **Key situations or events that could significantly impact the project's scope, schedule, or budget** - These risks, their likelihood and the strategy to overcome or minimize their impact should be detailed in the project's risk plan.
- **Any known constraints that may be imposed by the organization or project environment** -Known constraints may include such 0things as imposed deadlines, budgets, or required technology tools or platforms.
- **Dependencies on other projects internal or external to the organization**- In most cases, an IT project is one of several being undertaken by an organization .Dependencies between projects may exists, especially if different application systems or technology platforms must be integrated.
- **Impacts on different areas of the organization** – IT projects operate in a broader environment than the project itself. As a result, the development and implementation of an IT solution will have impact on the organization. It is important to describe how the project will impact the organization in terms of disruption, downtime, or loss of productivity.
- **Any outstanding issues**– It is important to highlight any outstanding issues that need further resolution. These may be issues identified by project sponsor, the project manager, or the project team that must be addressed and agreed upon at some point during the project. They may include such things as resources to be provided or decisions regarding the features or functionality of the system.

Project Administration: - Project Administration focuses on knowledge areas, processes, and controls that will support the project. These are actually separate sub plans or strategies that make up the project management plan.

- A communication plan that outlines how the project's status or progress will be reported to various stakeholders. This plan also includes a process for reporting and resolving significant issues or problems as they arise.
- A scope management plan that describes how changes to the project's scope will be submitted logged and reviewed.
- A quality management plans that details how quality planning, assurance, and control will be supported throughout project life cycle.

- A change management and implementation plan that will specify how the project's product will be integrated into the organizational environment.
- A human resource plan for staff acquisition and team development.

Acceptance and Approval: - Since the project charter serves as an agreement or contract between the project sponsor and project team, it may be necessary to have key stakeholders sign off on the project charter. By signing the document, the project stakeholder show formal acceptance of the project and therefore, gives the project manager and team the authority to carry out the project plan.

References In developing the project charter and plan, the project manager may use a number of references. It is important to document these references in order to add credibility to the project charter and plan , as well as to provide a basis for supporting certain processes ,practices, or estimates.

6.5 Project Scope Definition

Developing a scope statement is a useful first step for defining the scope of project and setting a boundary. A project's scope, however, should also be defined in terms of the deliverables that the team must provide. These deliverables can be divided into project –oriented deliverables and product oriented deliverables. This separation gives the team a clearer definition of the work to be accomplished and improves the likelihood of accurately assigning resources and estimating the time and cost of completing the work.

6.5.1 Project- Oriented Scope

Project – oriented deliverables, or **scope**, support the project management and IT development processes that are defined in the information technology project methodology.

Project scope includes such things as the business case, project charter, and project plan and defines the work products of various ITPM phases.

Project – oriented deliverables also include specific deliverables such as a current systems study, requirements definition, and the documented design of the information system.

Project – oriented deliverables requires time and resources and therefore, must be part of the overall project schedule and budget. Their role is to ensure that project processes are being completed so that the project's product achieves the project's MOV and objectives. Project- oriented deliverables also provide tangible evidence of the project's progress.

6.5.2 Product – Oriented Scope

Product scope therefore focuses on identifying the features and functionality of the information system to be implemented. A useful tool for refining the scope boundary and defining what the system must do is a modeling tool called a context level **data flow diagram (DFD)**.

A **context level DFD**, presents a high level representation of the system that has one process and depicts all the inflows and outflows of data and information between the system and its external entities.

Another useful tool is **Use Case Diagram**, which has been used in object – oriented world as part of **Unified Modeling Language (UML)**.

6.6 Project Scope Verification

The project's scope must be verified and formally accepted by the project sponsor and other appropriate stakeholders. It is scope management process that provides a mechanism for ensuring that the project deliverables are completed.

- **MOV** – Is project's MOV clearly defined and agreed upon? Failure to define and agree upon the MOV could result in scope changes later in project, which can lead to added work impacting the project's schedule and budget.
- **Deliverables** – Are the deliverables tangible and verifiable? Do they support the project's MOV?
- **Quality Standards** - Are controls in place to ensure that the work was not only completed, but completed to meet specific standards?
- **Milestones** – Milestones tell us that a deliverable was not only completed, but reviewed and accepted.
- **Review and acceptance**- Are both sides clear in their expectations? The project's scope must be reviewed and accepted by the project stakeholders. The project sponsor must formally accept the boundary, product to be produced, and the project –related deliverables. The project team must be clear on what it must deliver. In both cases, expectations must be realistic and agreed upon.

6.7 Scope Change Control

It is concerned with managing actual changes to the project's scope as when they occur, to ensure that any changes to the project's scope will be beneficial.

Scope grove- It describes a project team's inability to define the project's scope. This situation is common early in a project when the project team and sponsor have trouble understanding what the project is supposed to accomplish. Scope grove can be minimized by having a clearly defined MOV and following or applying the processes, concepts, and tools.

Scope creep- It is adding features to the system once the scope of the project has been approved. It does not always come from the project sponsor side. The project team itself may come across interesting or novel ideas as the project work progresses. It must be identified and controlled throughout the project because it lengthen the project schedule and, in turn, lead to cost overruns.

Scope leap- It can occur as a result of changes in environment , the business ,and the competitive makeup of the industry .Scope leap entails changing the MOV and therefore, requires that the organization rethink the value of the current project. If this change is critical, the organization may be better off pulling the plug on the current project and starting over by conceptualizing and initiating a new project.

6.8 Work Breakdown Structure

The WBS should Be Deliverable Oriented:-

The focus of a project should be to produce something, not merely on completing a specified number of activates. Although the WBS does not provide for any explicit looping, some activities may have to be repeated until the milestone is achieved. For example, software testing may uncover a number of problems or bugs that make the software system unacceptable to the client.

The WBS Should Support the Project's MOV:-

The WBS should include only tasks or activities that allow for the delivery of the project's deliverables. Before continuing with the development of the project plan, project team should ensure that the WBS allows for the delivery of the entire project's deliverables as defined in the scope. In turn, this will ensure that the project is more likely to achieve its MOV.

-0.0 EC Bank Project

+1.0 Conceptualize and initialize project

+2.0 Develop charter and plan

+3.0 Analysis

+4.0 Design

+5.0 Construction

Figure 6.1 - Work Breakdown Structure

-6.0 Testing

+6.1 Test Plan

-6.2 Test results report

6.2.1 Review test plan with client

6.2.2 Carry out test plan

6.2.3 Analyze results

6.2.4 Prepare test results report and presentation

6.2.5 Present test results to client

6.2.6 Address any software issues or problems

6.2.7 Milestone: Client signs off on test results

+6.3 Milestone: Testing completed

+7.0 Implementation

+8.0 Close project

+9.0 Evaluate project success

The Level of Detail Should Support Planning and Control:-

The WBS provides a bridge between the project's scope and project plan- that is, the schedule and budget. Therefore, the level of detail should support not only the development of the project plan but allow the project manager and project team to monitor and compare the project's actual progress to the original plan's schedule and budget.

Developing the WBS Should Involve the People Who Will Be Doing the Work:-

One way to ensure that the WBS has the appropriate level of detail to ensure that the people who do the work are involved in its development. A person who has experience and expertise in a particular area probably has a better feel for what activities need to be performed in order to produce a particular project deliverable.

Learning Cycles and Lessons Learned can Support the Development of a WBS:-

By using the concept of learning cycles, the project team can focus on what they know (the facts) ,what they think they know (assumption) , and what they need to find out (research) in order to develop a more useful WBS. Lessons learned from previous projects can be helpful in ensuring that the WBS and subsequent project plan are realistic and complete.

6.9 Summary

The business case is defines the problem or opportunity, MOV, feasibility, costs , and benefits of several alternatives that an organization may choose in order to achieve its goals and strategies. Based on the analysis of the alternatives identified , a recommendation is made to approve and fund a specific project.

The business case is formalized in a report to senior management who may review several proposed projects. The decision to fund a particular project depends on resources available and the value of the project to the organization. The balanced scorecard approach focuses on four perspectives – financial, customer, internal processes and innovation and learning . An organization should make the project selection decision based on a diverse set of measures and in terms of how well the project supports the goal and strategies of the organization.

The project charter documents the project's MOV and describes the infrastructure needed to support the project. In addition, the project charter should provide a consolidated source of information about the project and reduce confusion and misunderstanding. The project charter and project plan should be developed together – the details of project plan need to be summarized in the project charter, and the infrastructure outlined in the project charter will influence the estimates used to develop the project plan.

Product-oriented deliverables focus on the high level features and functionality of the application system – the project's product. On the other hand, project-oriented deliverables focus on the project's processes as defined in the IT project methodology.

Scope grope is a common occurrence in the early stages of the project. Often the project team struggle to define what the project is all about and what work must be done.

Scope creep, on the other hand, is a common occurrence in many projects. It entails adding additional features or functions to the scope once the scope has been set and approved. This phenomenon can increase the schedule and budget, causing the project to miss its deadline and budget targets.

Sample Questions

1. What is a business case?
2. Why should an organization develop a business case?
3. Describe the balanced scorecard approach.
4. What is the purpose of a project charter
5. How does the project charter support the project plan?
6. What is a project's scope?
7. Describe the scope definition process.
8. Describe the scope verification process.
9. Describe the scope change control process.
10. What is a WBS? What purpose does it serve?

Reference Books:

1. Software Engineering, 5th and 7th edition, by Roger S Pressman, McGraw Hill publication.
2. Managing Information Technology Project 6th edition , by Kathy Schwalbe , Cengage Learning publication.
3. Software Engineering 3rd edition by KK Agarwal , Yogesh Singh , New Age International publication.
4. Information Technology Project Management by Jack T Marchewka Wiley India publication.
5. Software Engineering for students: A Programming Approach by Douglas Bell, Pearson publication.
6. Software Engineering Project Management by Richard H. Thayer Wiley India Publication.



PROJECT SCHEDULING AND PROCUREMENT MANAGEMENT

Unit Structure

- 7.0 Objectives
- 7.1 Introduction
- 7.2 Overview
 - 7.1.1 Scheduling Principals
 - 7.1.2 Relationship between People and Effort
 - 7.2.3 Project Effort Distribution
- 7.3 Scheduling
 - 7.3.1 Tracking Project Schedules
 - 7.3.2 Tracking Increment Progress for OO Projects
 - 7.3.3 Webapp Project Scheduling
 - 7.3.4 Earned Value Analysis
- 7.4 Project Procurement Management
 - 7.4.1 Process of Project Procurement Management
 - 7.4.2 Eight Steps for a Procurement Management Plan
- 7.5 Degree of Rigor
 - 7.5.1 Four various degrees of rigor
 - 7.5.2 Defining a Task Set for the Software Project
- 7.6 Critical Path Method (CPM)
 - 7.6.1 Critical Path Method helps in:
 - 7.6.2 Technique for CPM
 - 7.6.3 Steps for CPM
 - 7.6.4 Examples
- 7.7 Summary
- 7.8 Questions
- 7.9 Reference books

7.0 Objectives of Project Scheduling

To arrange the manufacturing activities in such a way that the cost of production is minimized and the goods produced are delivered on due dates is the main objective of Project Scheduling. In order to meet the delivery dates the sequence of operations is properly planned. To have minimum total time of production by having better resources utilisation we use scheduling. We can also make use of this for having maximum capacity utilization and reducing the labour cost by minimization of idleness of machines and manpower. It also helps avoid unbalanced allocation of work among the various departments and workstations.

7.1 Introduction

Project Scheduling in a project is a roadmap of all activities to be done with specified order and within time slot allotted to each activity. Project managers tend to define various tasks, and project milestones and they arrange them keeping various factors in mind. They look for tasks lie in critical path in the schedule, which are necessary to complete in specific manner (because of task interdependency) and strictly within the time allocated. Arrangement of all tasks, which lies out of critical path are less likely to impact overall schedule of the project.

7.2 Overview

Scheduling in project management is the listing of activities, deliverables, and milestones within a project. A schedule also usually includes the planned start and finish date, duration, and resources assigned to each activity. Effective project scheduling is a critical component of successful time management.

For scheduling a project, following is necessary to -

- Break down the project tasks into smaller, manageable form
- Find out various tasks and correlate them
- Estimate time frame required for each task
- Divide time into work-units
- Assign adequate number of work-units for each task
- Calculate total time required for the project from start to finish

In fact, when people discuss the processes for building a schedule, they are usually referring to the first six processes of time management:

1. Plan schedule management.
2. Define project activities.
3. Sequence activities.
4. Estimate resources.
5. Estimate durations.
6. Develop the project schedule

7.2.1 Scheduling Principles

- Compartmentalization - the product and process must be decomposed into a manageable number of activities and tasks
- Interdependency - tasks that can be completed in parallel must be separated from those that must be completed serially
- Time allocation - every task has start and completion dates that take the task interdependencies into account
- Effort validation - project manager must ensure that on any given day there are enough staff members assigned to complete the tasks within the time estimated in the project plan
- Defined Responsibilities - every scheduled task needs to be assigned to a specific team member
- Defined outcomes - every task in the schedule needs to have a defined outcome (usually a work product or deliverable)
- Defined milestones - a milestone is accomplished when one or more work products from an engineering task have passed quality review

7.2.2 Relationship between People and Effort

- Adding people to a project after it is behind schedule often causes the schedule to slip further
- The relationship between the number of people on a project and overall productivity is not linear (e.g. 3 people do not produce 3 times the work of 1 person, if the people have to work in cooperation with one another)
- The main reasons for using more than 1 person on a project are to get the job done more rapidly and to improve software quality.

7.2.3 Project Effort Distribution

- The 40-20-40 rule:
 - 40% front-end analysis and design
 - 20% coding
 - 40% back-end testing
- Generally accepted guidelines are:
 - 02-03 % planning
 - 10-25 % requirements analysis
 - 20-25 % design
 - 15-20 % coding
 - 30-40 % testing and debugging

7.3 Scheduling

- Task networks (activity networks) are graphic representations can be of the task interdependencies and can help define a rough schedule for particular project
- Scheduling tools should be used to schedule any non-trivial project.
- Program evaluation and review technique (PERT) and critical path method (CPM) are quantitative techniques that allow software planners to identify the chain of dependent tasks in the project work breakdown structure (WBS) that determine the project duration time.
- Timeline (Gantt) charts enable software planners to determine what tasks will be need to be conducted at a given point in time (based on estimates for effort, start time, and duration for each task).
- The best indicator of progress is the completion and successful review of a defined software work product.
- Time-boxing is the practice of deciding a priori the fixed amount of time that can be spent on each task. When the task's time limit is exceeded, development moves on to the next task (with the hope that a majority of the critical work was completed before time ran out).

7.3.1 Tracking Project Schedules

- Periodic project status meetings with each team member reporting progress and problems
- Evaluation of results of all work product reviews
- Comparing actual milestone completion dates to scheduled dates
- Comparing actual project task start-dates to scheduled start-dates
- Informal meeting with practitioners to have them assess subjectively progress to date and future problems
- Use earned value analysis to assess progress quantitatively

7.3.2 Tracking Increment Progress for OO Projects

- **Technical milestone:** OO analysis complete
 - All hierarchy classes defined and reviewed
 - Class attributes and operations are defined and reviewed
 - Class relationships defined and reviewed
 - Behavioural model defined and reviewed
 - Reusable classes identified
- **Technical milestone:** OO design complete
 - Subsystems defined and reviewed
 - Classes allocated to subsystems and reviewed
 - Task allocation has been established and reviewed
 - Responsibilities and collaborations have been identified
 - Attributes and operations have been designed and reviewed
 - Communication model has been created and reviewed
- **Technical milestone:** OO programming complete
 - Each new design model class has been implemented
 - Classes extracted from the reuse library have been implemented
 - Prototype or increment has been built

- **Technical milestone:** OO testing complete
 - The correctness and completeness of the OOA and OOD models has been reviewed
 - Class-responsibility-collaboration network has been developed and reviewed
 - Test cases are designed and class-level tests have been conducted for each class
 - Test cases are designed, cluster testing is completed, and classes have been integrated
 - System level tests are complete

7.3.3 Webapp Project Scheduling

- During the first iteration a macroscopic is developed by allocating effort to specific tasks (it is understood that this is changeable schedule)
- The project is broken up into increments and the increments are refined in to detailed schedules as each is begun (some increments may be developed in parallel)
- Each task on the task list for each increment is adapted in one of four ways as its detailed schedule is created
 - task is applied as is
 - task is eliminated because it is not necessary for the increment
 - new (custom) task is added
 - task is refined (elaborated) into a number of named subtasks that each becomes part of the schedule
- This process continues until each planned increment is completed

7.3.4 Earned Value Analysis

- Earned value is a quantitative measure given to each task as a percent of project completed so far.
 1. The total hours to complete each project task are estimated (BCWS - budgeted cost of work scheduled)

2. The effort to complete the project is computed by summing the effort to complete each task (BAC - budget at completion)
 3. Each task is given an earned value based on its estimated percentage contribution to the total (BCWP - budgeted cost of work completed).
- It is compute the actual cost of work performed (ACWP) at any point in the project and to compute progress indicators for both schedule and costs based on these measures

7.4 Project Procurement Management

In project Management, Procurement is when you need to purchase, rent or contract with some external resource to meet your project goal. These relationships and any process in the project need management.

Managing these relationships means getting the best quality from the outside vendors employed by the company to assist in its doing business. There are constraints in a relationship with vendors that revolve around cost and time. Procurement management is a way to more efficiently and productively handle the process of sourcing, requisitioning, ordering, expediting, inspecting and reconciliation of procurement.

7.4.1 Process of Project Procurement Management

Project management for procurement is usually divided into four major processes: Planning, Selection, Administering and Closing procurements.

1. Planning Process:

Planning, involves the creation of the official procurement management plan. The decisions involve which items will be internally procured and which items will be externally outsourced. For every external contractor, there needs to be a statement of work (SOW) to serve as a document outlining the work being contracted. This information, will heavily impact the project's budget and financial scope.

Sample procurement documents will be prepared and criteria frameworks will be developed to create a selection of potential vendors. This selection matrix is based on the project's scope, schedule, and requirements. Risk factors and budgetary constraints are also considered.

This process is collected in the procurement management plan, which includes requirement documents, risk register, activity resource requirements, project schedule, activity cost estimates and more.

2. Selection Process:

The selection process involves comparing and contrasting vendors' advantages, disadvantages, and contractual offerings. Standard tools and techniques are used to select procurements, such as video conferences with bidders that allow them to understand the project requirements and ask questions.

Procurement contracts are decided and awarded through collaborations between various managers. Resource calendars are then created that detail when, where and how resources will be used and managed. The corresponding project management plan is adjusted according to resource calendar updates. Proposals are carefully evaluated and if no satisfactory bids are available, the project management team may utilize online ads to solicit new bidders.

3. Administration Process:

The third major step is administration, which refers to the tools and processes used to manage relationships with vendors. The administration phase results in the continual creation of procurement documents and spreadsheets that may drive project changes. A centralized system of contract change monitoring and control will be used to evaluate and determine whether potential changes to contracts are needed.

Once the contracts are signed, the management of those contractors must be folded into the overall management responsibilities. Contractors can have negative impact on budgets and schedules, which can lead to a project going off-track or worse.

It's best to contract a change control system and have regular procurement performance reviews, including inspections and audits to make sure the work is going right. Performance reporting helps keep managers informed, too. A payment system needs to be in place as well as a claims administration and a records management system.

There are formal physical inspections, internal audits and reviews of procurement operations in order to generate synthesized performance reports that provide real-time feedback. The administration process is extremely important, so it's usually managed through supply chain or project management software.

4. Closing Process:

Just as there is a process to start the procurement, there needs one in place to finalize it. What constitutes completed work should be detailed in the initial agreement with the contractor, so there is no confusion of either's part as to when the work is done.

The closing process isn't just about ending procurement contracts; it's about noting weaknesses, documenting successful processes and summarizing the project for future needs. Some companies prefer to conduct simple audits using performance matrices in order to grade the overall project.

Documentation is important for future projects, which may involve entirely different teams in new locations. During the closing process, negotiations may be necessary to resolve contract disputes. Ideally, potential issues will be noted during the administration process in order to begin the mediation process early.

When it comes to project procurement management, there are standard features and functions. For example, most companies prefer to use a smaller number of suppliers with long-term relationships instead of using a group of suppliers to outbid each other for the lowest price. Establishing and nurturing relationships with suppliers is important because this enables various supply chain partners and shareholders to work closely together on improvement and coordination activities.

7.4.2 Eight Steps for a Procurement Management Plan

The project manager is the project team member responsible for overseeing the procurement management plan, but it's not a one-person job. Since the procurements will be project-wide, it's important that everyone is on board with the process. Everyone should have some involvement in approving and even managing contracts.

The procurement management plan can be broken down into these eight steps.

1. **Define Terms:** To begin, start by defining the procurement terms. This means listing what you need to procure in detail: how many, what size, for how long, etc. Then you want to know what service is provided to the project and why this is important. Now add a date of use to each of these procurements and who on the project team is authorized to make these purchases.
2. **Outline Type of Agreement:** The contract is how everyone agrees on the terms of service. There are different types of contracts, for instance a fixed price and cost reimbursement are two. Therefore, the type of agreement must be decided on and how it will be managed.

3. **Identify and Mitigate Risks:** Risks are inherent in every part of a project process, and so they lie dormant in procurement until they show themselves. It is now time to figure out what those risks might be and list them. Once a thorough list has been collected, each must have a way to resolve them. It's also good to assign a team member with the task of mitigating those risks, so they have ownership to follow through on closing them.
4. **Define Costs:** What are the costs involved with the project procurements? Once those have been figured out, it is likely that a request for proposal will be issued, with the needs outlined and requesting bids from suppliers. Be thorough and note everything required. The suppliers will come back with their cost for products or services.
5. **Identify Constraints:** It helps to try and identify any project constraints before starting the project to avoid getting blindsided by unforeseen limitations during execution. Once this list is complete it can be looked at throughout the project phases. Constraints related to procurements include cost, scope, limited resources and technical specifications.
6. **Get Contract Approved:** Review the bids and do a service and cost analysis. Then have a list of who the decision makers are in the project group and pass the bids on to them for review. This process makes sure that everyone who needs to oversee the contract approval is involved and can provide input.
7. **Make Decision Criteria:** You have a workflow, but now you need criteria by which to decide on which bid to go into contract with. Every person who reviews the bid should have these criteria at hand to measure their response.
8. **Create a Vendor Management Plan:** Once a contract is signed, the procurement management plan will segue into a vendor management plan. The terms of the contract must be met. And, to make sure that happens, a management plan surrounding the suppliers will help ensure that goods and services are delivered as specified and on time. It is a good idea to add a performance metric to rate how well each supplier does their job, so you can improve relations on the next project and know who is worth contracting with again.

7.5 Degree of Rigor

For a project of a particular kind the degree of rigor with which the software procedure is applied may vary significantly. The degree of rigor is function of several project characteristics. For an example non-business, small, critical projects can commonly be addressed with somewhat less rigor which is large complex baseline critical applications. It should be noted however, in which all projects must be conducted in a manner which results in timely high quality deliverables.

5.5.1 Four various degrees of rigor

Four various degrees of rigor can be defined that are:

1. **Casual:** In general umbrella tasks will be documentation and minimized needs will be reduced all basic principles of software engineering are yet applicable. All procedural frame work activities are applied but only a minimum task group is needed.
2. **Structured:** Framework related tasks and activities appropriate to the project type will be applied and umbrella activities necessary to ensure high quality will be applied. The process framework will be applied for this project. SQA, SCM measurement and documentation tasks will be conducted in a streamlined manner.
3. **Strict:** - The full process will apply for this project with a degree of discipline which will ensure high quality. Robust documentation will be produced and all umbrella activities will be applied.
4. **Quick Reaction:** Framework of process will be applied for this project but because of an emergency condition only those tasks essential to maintaining good quality will be applied Back-filling example for developing a complete set of documentation conducting additional reviews will be accomplished after the application or product is delivered to the customer.

The project manager must have to develop a systematic approach for selecting the degree of rigor which is appropriate for a special project. To accomplish this project adaptation criteria are describe and a task set selector value is computed.

5.5.2 Defining a Task Set for the Software Project

Task set - collection of software engineering work tasks, milestones, and deliverables that must be accomplished to complete a particular project.

Task sets are designed to accommodate different types of projects and different degrees of rigor.

- determine type of project
- assess the degree of rigor required
- identify adaptation criteria

Select appropriate software engineering tasks

Five Most software organizations encounter the following projects:

1. **Concept development projects** - initiated to explore some new business concept or application of some new technology.
2. **New application development projects** - undertaken as a consequence of a specific customer request.
3. **Application enhancement projects** - occur when existing software undergoes major modifications to function, performance, or interfaces that are observable by the end-user.
4. **Application maintenance projects** - correct, adapt, or extend existing software in ways that may not be immediately obvious to the end-user.
5. **Reengineering projects** - undertaken with the intent of rebuilding an existing (legacy) system in whole or in part.

7.6 Critical Path Method (CPM)

Critical Path Method (CPM) is a project schedule modeling technique. Mr. Morgan R. Walker and James E. Kelly developed this technique in the late 1950s.

Project planners use this method to develop schedules for many kinds of projects including IT, research, and construction.

Critical Path Method is a lengthy and complex concept. Please follow each step in this blog post and don't move on until you understand the previous steps. If you follow this advice and complete the blog post, you won't have any problems solving the questions on Critical Path Method.

A network diagram has many paths originating from one point and ending at another point. Every path has a duration and the one with the longest duration is the critical path.

You can define a critical path as:

- The longest path in the network diagram, or
- The shortest duration to complete the project.

7.6.1 Critical Path Method helps in:

1. Determining the minimum time in which the project can be completed
2. Determining the sequence of activities which must be completed on time in order to complete the project in time
3. Determining which all tasks can be delayed without delaying the project completion time
4. Determining the Early and Late Start of tasks
5. Tracking project progress with regards to agreed timeline and taking proactive corrective action if the project seems to be getting delayed

7.6.2 Technique for CPM

The technique for find out the critical path in your project can be derived as follows.

Break Down the Project: List all the tasks needed to complete the project. You can use a work breakdown structure, which is a hierarchical decomposition of the project, which includes every deliverable.

Estimate Task Duration: Now comes the tricky part, you want to know how long each task will take. If possible, get advice from others who have, so you can have the most accurate estimation of the duration of the various tasks possible.

Determine Task Dependencies: If there are any task dependencies, you want to note them, too. A task dependency is when one task cannot start until another one has been finished. It's a key element of good task management.

Add Milestones: What are the milestones in your project? Having milestones helps to keep you on track, so you can make sure you're meeting your baseline schedule.

When you have this data collected, you're able to calculate the longest path your planned tasks will take to reach the end of the project, as well as the earliest and latest that each task can start and finish without impacting the project schedule.

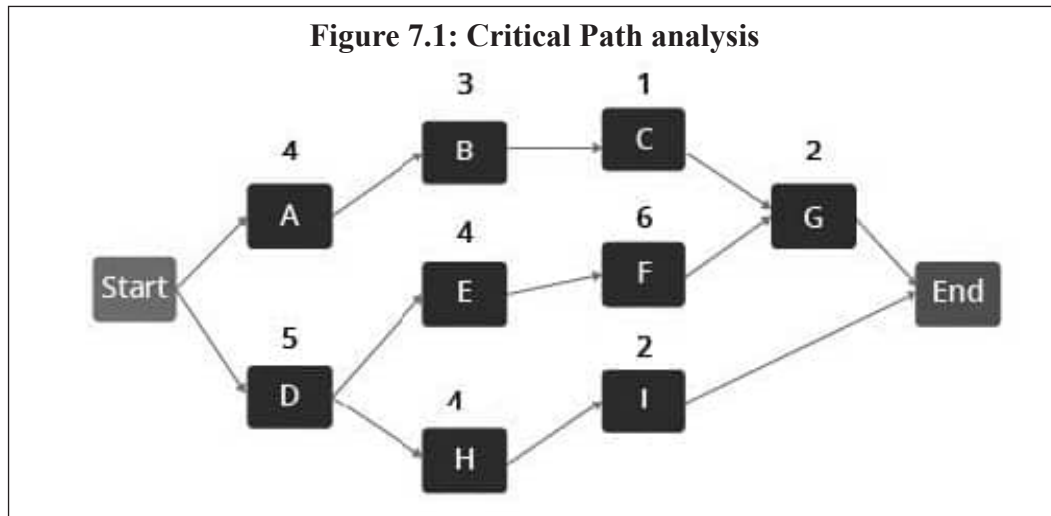
7.6.3 Steps for CPM

The typical steps involved to develop a project schedule using CPM method are as below:

1. Identify the activities for all the work packages from the project's Work Breakdown Structure (WBS)
2. Sequence all the activities by identifying all the dependencies between the activities.
3. Develop a Schedule Network Diagram involving all the project activities ensuring that each activity has at least one predecessor and one successor except the first activity which will not have a predecessor and last activity which will not have a successor.
4. Estimating the duration of each activity in the schedule network diagram.
5. Carrying out the process of "Forward Pass" where in the "Early Start" (ES) and "Early Finish" (EF) for each activity are calculated starting from the beginning of the network diagram.
6. Carrying out the process of "Backward Pass" where in the "Late Finish" (LF) and "Late Start" (LS) for each activity are calculated starting from the end (Finish) of the network diagram.
7. Identifying the "Path" which has the longest duration in the Network Diagram. The longest path will also have the ES and LS and EF and LF of all the activities as same.
8. The longest path is termed as the "Critical Path". The duration of this path will determine the shortest time taken to complete the project. Any delay on this path delays the project completion time. Hence they are critical from project's schedule constraint point of view.
9. The "Non-Critical-Path" path duration will be shorter than the "Critical Path" and hence those paths will have flexibility to delay the start of the tasks on them.
10. The amount of time a task can be delayed on a "non-critical path" is known as "float" or "slack", which is calculated by taking the difference between "LS-ES" or "LF-EF".
11. The float on critical path will be Zero to start with and not-critical paths will have a positive float time.
12. There may be more than one critical path in a network. But having more than one critical path increases the risk of falling behind the schedule as there are more number of tasks which if they get delayed, the project will get delayed.

7.6.4 Examples

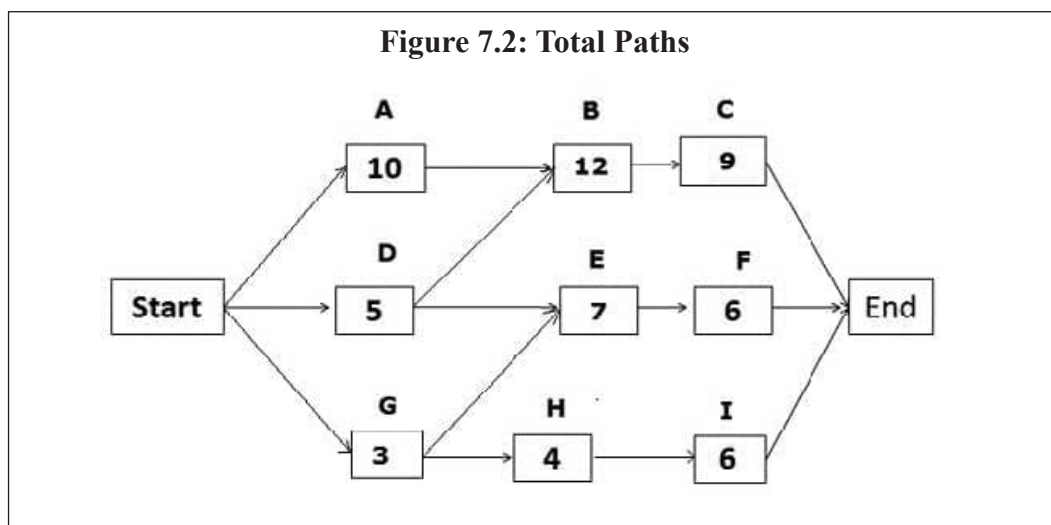
1. An example of Critical Path analysis is as below (Figure 7.1).



In the above diagram, the path with longest duration is Start-D-E-F-G-End is the critical path with duration of 17. The other 2 non-critical paths *Start-A-B-C-G-End* has duration of 10 and hence has a slack or float of 7 and other non-critical path *Start-D-H-I-End* has a duration of 11 and has a float of 6 days.

Critical Path once identified, the team can further explore if the duration of critical path can be compressed if the need be. Techniques such as crashing (*applying more resources on critical path*) or Fast Tracking (*doing tasks in parallel*) are applied. Compressing the critical path helps in compressing the overall project duration, thereby helping to meet the required deadline.

- Based on the network diagram below (Figure 7.2), identify the total paths, critical path, and float for each path.



The above network diagram has five paths. The paths and their durations are as follows:

1. Start -> A -> B -> C -> End {duration: 31 days.}
2. Start -> D -> E -> F -> End {duration: 18 days.}
3. Start -> D -> B -> C -> End {duration: 26 days.}
4. Start -> G -> H -> I -> End {duration: 13 days.}
5. Start -> G -> E -> F -> End {duration: 16 days.}

Since the duration of the first path is the longest, it is the critical path. The float on the critical path is zero.

The float for the second path “Start -> D -> E -> F -> End” = duration of the critical path – duration of the path “Start -> D -> E -> F -> End” = $31 - 18 = 13$

Hence, the float for the second path is 13 days.

Using the same process, we can calculate the float for other paths as well.

Float for the third path = $31 - 26 = 5$ days.

Float for the fourth path = $31 - 13 = 18$ days.

Float for the fifth path = $31 - 16 = 15$ days.

Calculate Early Start, Early Finish, Late Start, and Late Finish

We have identified the critical path and the duration of the other paths. Now it's time to move on to more advanced calculations: Early Start, Early Finish, Late Start and Late Finish.

Calculating Early Start (ES) and Early Finish (EF)

To calculate the Early Start and Early Finish dates, we use the forward pass; we will start from the beginning and proceed to the end.

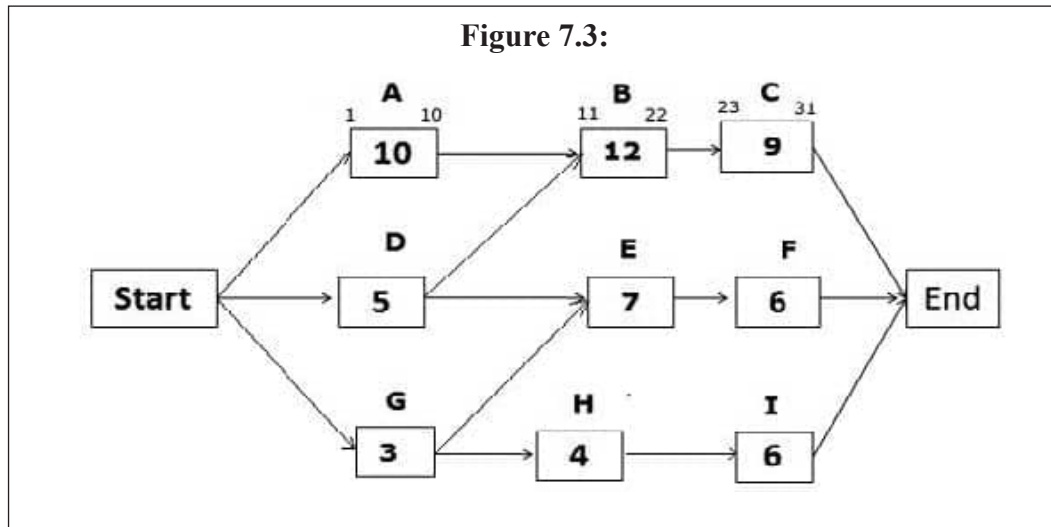
The Early Start (ES) for the first activity on any path will be 1 because you cannot start an activity before the first day of your project.

The starting point for any activity is the endpoint of the predecessor activity on the same path (plus one).

The formula used for calculating Early Start and Early Finish dates:

- Early Start of the activity = Early Finish of predecessor activity + 1
- Early Finish of the activity = Activity duration + Early Start of activity – 1

Early Start and Early Finish Dates for the path Start -> A -> B -> C -> End
(Figure 7.3)



Early Start of activity A = 1 (Since this is the first activity of the path)

Early Finish of activity A = ES of activity A + activity duration – 1 = 1 + 10 – 1 = 10

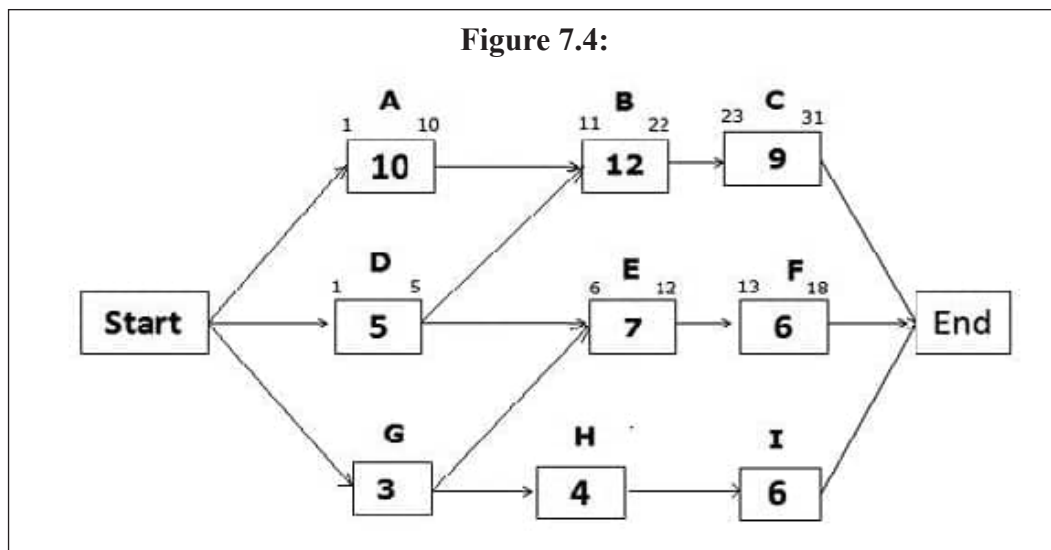
Early Start of activity B = EF of predecessor activity + 1 = 10 + 1 = 11

Early Finish of activity B = ES of activity B + activity duration – 1 = 11 + 12 – 1 = 22

Early Start of activity C = EF of predecessor activity + 1 = 22 + 1 = 23

Early Finish of activity C = ES of activity C + activity duration – 1 = 23 + 9 – 1 = 31

Early Start and Early Finish Dates for the path Start -> D -> E -> F -> End
(Figure 7.4)



Early Start of activity D = 1 (Since this is the first activity of the path)

Early Finish of activity D = $1 + 5 - 1 = 5$

Early Start of activity E = EF of predecessor activity + 1

Since activity E has two predecessor activities, which one will you select? The answer is the activity with the greater Early Finish date. The Early Finish of activity D is 5, and the Early Finish of activity G is 3 (we will calculate it later).

Therefore, we will select the Early Finish of activity D to find the Early Start of activity E.

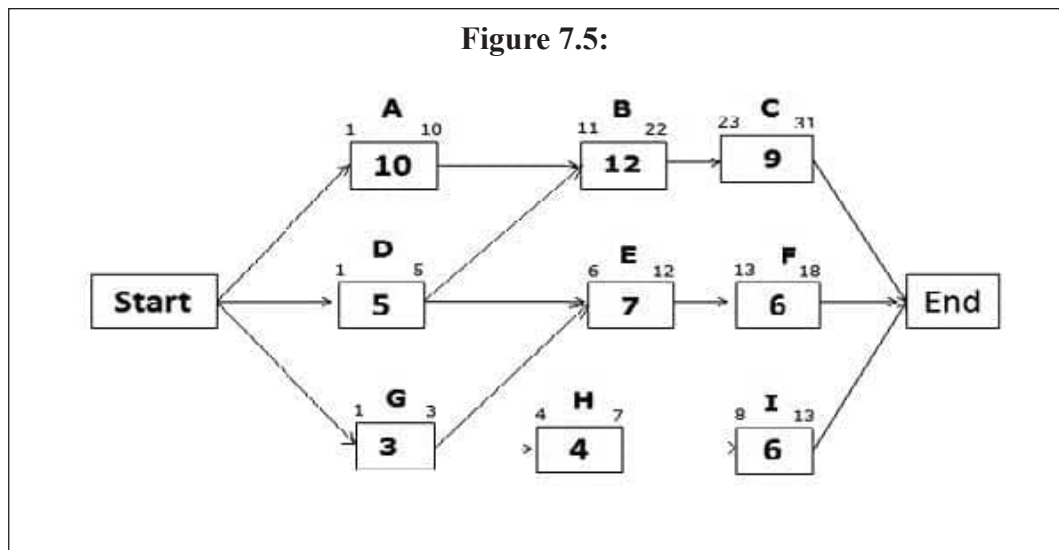
Early Start of activity E = EF of predecessor activity + 1 = $5 + 1 = 6$

Early Finish of activity E = $6 + 7 - 1 = 12$

Early Start of activity F = $12 + 1 = 13$

Early Finish of activity F = $13 + 6 - 1 = 18$

Early Start and Early Finish Dates for the path Start -> G -> H -> I -> End
(Figure 7.5)



Early Start of activity G = 1 (Since this is the first activity of the path)

Early Finish of activity G = $1 + 3 - 1 = 3$

Early Start of activity H = $3 + 1 = 4$

Early Finish of activity H = $4 + 4 - 1 = 7$

Early Start of activity I = $7 + 1 = 8$

Early Finish of activity I = $8 + 6 - 1 = 13$

Calculating Late Start (LS) and Late Finish (LF) We have calculated the Early Start and Early Finish dates of all activities. Now it is time to calculate the Late Start and Late Finish dates.

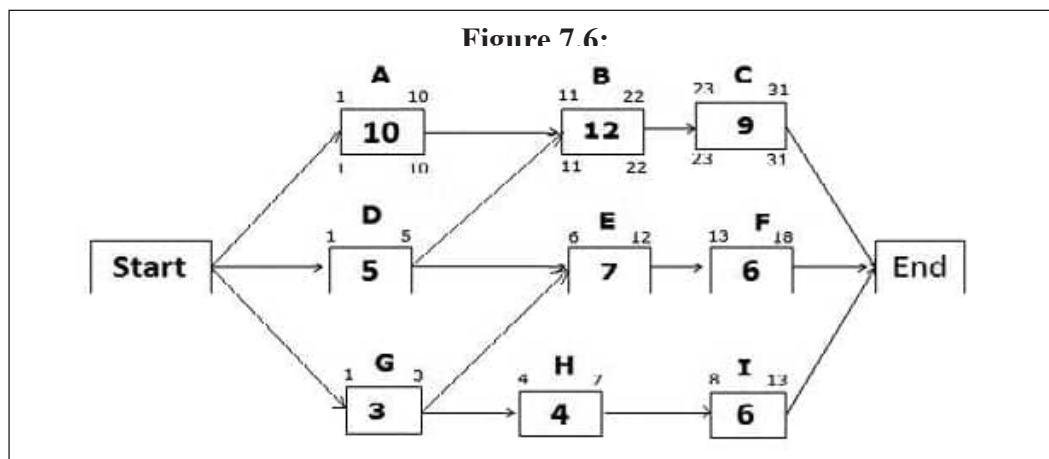
The Late Finish date of the last activity on all paths will be the same because no activities can continue once the project is completed.

The formula used for Late Start and Late Finish dates:

- Late Start of Activity = Late Finish of activity – activity duration + 1
- Late Finish of Activity = Late Start of successor activity – 1

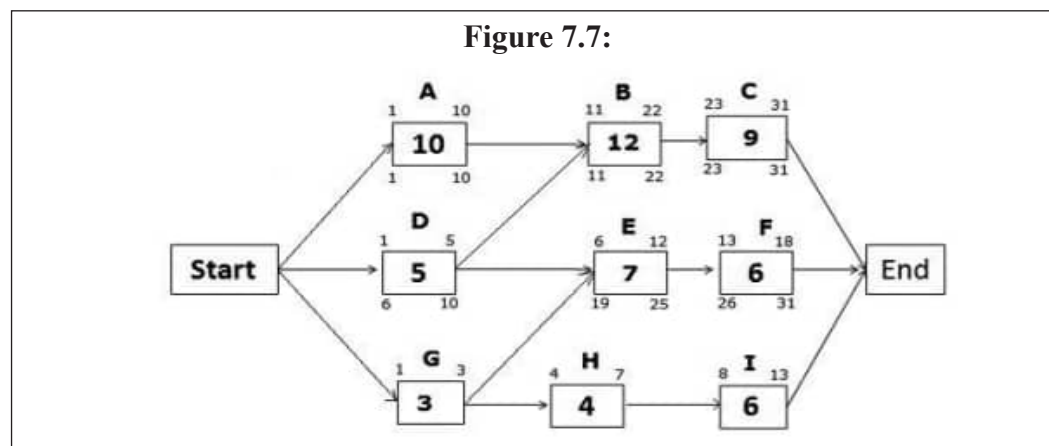
To calculate the Late Start and Late Finish, we use the backward pass; i.e. we will start from the last activity and move back towards the first activity.

Late Start and Late Finish Dates for the path Start -> A -> B -> C -> End
(Figure 7.6)



On a critical path, the Late Start, and Late Finish dates will be the same as the Early Start and Early Finish dates

Late Start and Late Finish Dates for the path Start -> D -> E -> F -> End
(Figure 7.7)



Late Finish of activity F = 31 (because you cannot allow any activity to pass the project completion date)

Late Start of activity F = LF of activity F – activity duration + 1 = $31 - 6 + 1 = 26$

Late Finish of Activity E = LS of successor activity – 1 = LS of Activity F – 1 = $26 - 1 = 25$

Late Start of Activity E = LF of activity E – activity duration + 1 = $25 - 7 + 1 = 19$

Late Finish of activity D = LS of successor activity – 1

If you look at the network diagram, you will notice that activity D has two successor activities, B and E. So, which activity would you select?

You will select the activity with the earlier (least) Late Start date. Here, the Late Start of activity B is 11, and the Late Start of activity E is 19.

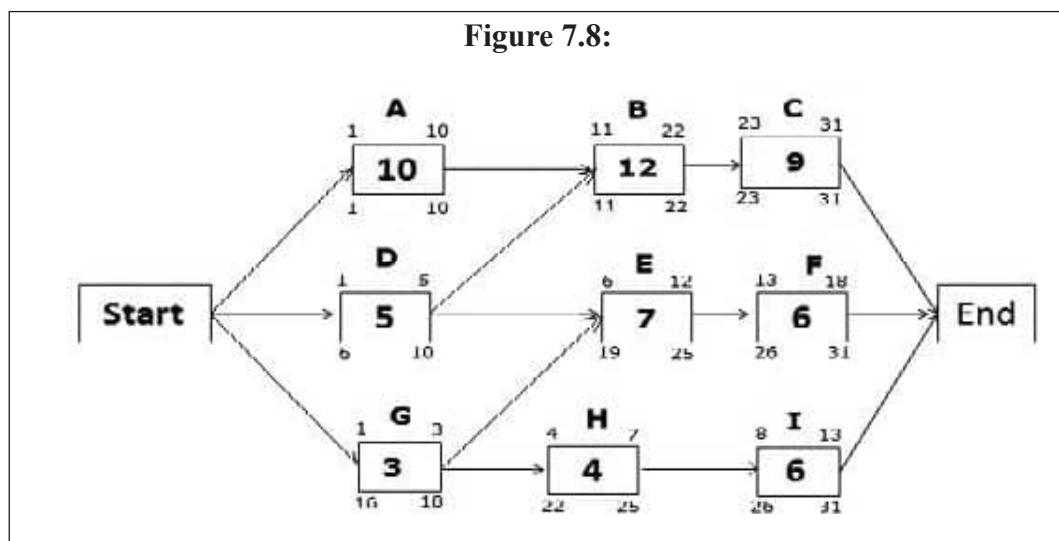
Therefore, you will select activity B, which has the earlier Late Start date.

Hence,

Late Finish of activity D = LS of activity B – 1 = $11 - 1 = 10$

Late Start of Activity D = LF of activity D – activity duration + 1 = $10 - 5 + 1 = 6$

Late Start and Late Finish Dates for the path Start -> G -> H -> I -> End
(Figure 7.8)



Late Finish of activity I = 31 (because you cannot allow any activity to pass the project completion date)

Late Start of activity I = $31 - 6 + 1 = 26$

Late Finish of activity H = $26 - 1 = 25$

Late Start of activity H = $25 - 4 + 1 = 22$

Late Finish of Activity G = $19 - 1 = 18$ (we will choose the late start of activity E, not activity H because the Late Start of activity E is earlier than the Late Start of activity H).

Late Start of activity G = $18 - 3 + 1 = 16$

7.7 Summary

This Chapter includes scheduling principals and relationship between people and efforts. It describes tracking of project schedule with earned value analysis. This chapter covered the process of procurement management with some plans. After this various Degree of rigor are listed. The main part of critical path method is explained step by step with numerical.

7.8 Questions

- Explain Project scheduling with the relationship between people and efforts?
- State the Project Effort Distribution
- Describe the Tracking Project Schedules
- Explain the process of project procurement Management
- What is degree of rigor?
- State the Technique for CPM with example.

7.9 Reference books

- Information Technology Project Management by Jack T Marchewka Wiley India publication
- Software Engineering, by Roger S Pressman, McGraw Hill publication.
- Software Engineering by KK Agarwal , Yogesh Singh , New Age International publication.



PROJECT PROCUREMENT MANAGEMENT

Unit Structure

- 8.0 Objective
- 8.1 Introduction
- 8.2 Basic Planning Purchases and Acquisitions
 - 8.2.1 Evaluating the Market Conditions
 - 8.2.2 Referring to the Scope Baseline
 - 8.2.3 Relying on the Project Management Plan
 - 8.2.4 Teaming with other organizations
 - 8.2.5 Planning for the Project Requirements
- 8.3 Completing procurement Planning
 - 8.3.1 Determining to Make or Buy
 - 8.3.2 Using Expert Judgment
- 8.4 Determining the Contract Type
 - 8.4.1 Fixed –Price Contracts
 - 8.4.1.1 Firm fixed price contract
 - 8.4.1.2 Fixed –price incentive fee contract
 - 8.4.1.3 Fixed price with economic price adjustment contract
 - 8.4.2 Cost- Reimbursable Contracts
 - 8.4.2.1 Cost plus fixed fee
 - 8.4.2.2 Cost plus incentive fee
 - 8.4.2.3 Cost plus award fee
 - 8.4.2.4 Cost plus percentage of costs
 - 8.4.3 Time and Materials Contracts
- 8.5 Planning Contracting (Preparing for Contracting)

8.5.1 Organizing Contracting Materials

8.5.2 Creating the Procurement Documents

8.5.3 Determining the Source Selection Criteria

8.5.4 Updating the Procurement Statement of Work

8.5.5 Completing Procurement Purchasing

8.6 Requesting Seller Response

8.6.1 Examining the Results of Contracting

8.7 Selecting The Seller

8.7.1 Preparing for Source Selection

8.7.2 Completing the Seller Selection Process

8.7.3 Examining the results of Seller Selection

8.8 Summary

8.0 Objective

- To understand the importance of project procurement Management
- To describe project procurement management processes
- Procurement Planning
- Planning Contracting
- Requesting Seller Responses

8.1 Introduction

Project routinely requires procurements. The need materials, equipment, consultants, training, and other goods and services. Project procurement management is the process of purchasing the products necessary for meeting the needs of the project scope. It involves planning, acquiring the products or services from sources, choosing a source, administering the contract, and closing out the contract. Procurement management, as far as the PMP exam is considered, focuses on the practices from the point of view of the buyer, not the seller (for example, contractor, subcontractor, vendor, or supplier).

When buying anything from a vendor, the buyer needs a contract, which becomes a

key input to many of the processes that occur within the project. The contract, more than anything else, specifies the rules and agreements for the project.

Here's neat twist: When the seller is completing their obligations to supply a product, PMI treats those obligations as a project itself. In other words, if ABC Electricians were wiring a building for your company, ABC Electricians would be performing organization completing its own project. Your company becomes the customer of their project – and is, of course, a stakeholder in their project.

When the vendor is completing work for a portion of your project, the close contract activities don't wait until the end of the project- they happen as needed.

In the scenarios described in this chapter, the seller will be outside of the performing organization. The buyer will be managing a project and procuring resources from a vendor. However, all of the details in this in this chapter can be applied to internal work orders, formal agreements, and contracts between organizational units within a single entity.

For the PMI examination, you'll need to be familiar with four processes dealing with project procurement management. First, like all of the project management knowledge areas, is the associated planning. Once the procurement management plan has been created, you'll actually do the procurement. You'll control the procurement process to ensure that all parties are meeting the terms of the agreement. Finally, you'll close out the procurement process.

8.2 Basic Planning Purchases and Acquisitions

Procurement planning is the process of identifying which part of the project should be procured from resources outside of the organization. Generally, procurement decisions are made early on in the planning processes. Procurement planning centres on the four elements:

- Whether procurement is needed
- What to procure
- How much to procure
- When to procure

When the project manager begins the procurement process, she'll rely on the usual enterprise environmental factors and organizational process assets. For example, the project manager has to consider the marketplace conditions, the availability of

the needed items or services in the marketplace, and how the procurement process works within the performing organization. If the project manager's organization has forms, policies, and management guidelines that direct the procurement process, she must follow those established processes.

Often, an organization will have resources for managing the procurement process, including contracting and negotiating on behalf of the project. If, however, the performing organization has no such resources for the project manager to rely upon, it is up to the project manager to supply the procurement management resources, including capabilities for negotiating and for obtaining in a fiscally responsible way the right products or services for fair price on behalf of the performing organization.

8.2.1 Evaluating the Market Conditions

Part of procurement management involves determining what sources are available to provide the required products or services for the project. An evaluation of the marketplace is needed to determine what products and services are available and from whom and on what terms and conditions they are available.

Although in most free-market enterprise societies multiple vendors offer comparable products, there may be times when your choices of vendors are limited. The following are three specific terms to know for the PMP exam that you may encounter:

- **Sole source** Only one qualified seller exists in the marketplace.
- **Single source** The performing organization prefers to contract with a specific seller.
- **Oligopoly** There are very few sellers, and the actions of one seller will have a direct effect on the other sellers' prices and the overall market condition.

8.2.2 Referring to the Scope Baseline

The project's scope statement, work breakdown structure (WBS), and WBS dictionary all serve as input in making procurement decisions, although the PMBOK lumps these into the project management plan. Because the project scope baseline defines the project work, and only the required work, to complete the project, it also defines the limitations of the project. Knowing the limits of what the project includes can help the project manager, the contract specialists, and other procurement professionals determine what needs to be purchased and what does not.

The WBS and the WBS dictionary define the details and requirements for acceptance

of the project. This information also serves valuable input regarding what needs to be procured and what does not. The WBS defines what the end result of the project will be. When dealing with vendors to procure a portion of the project, you must ensure that the work to be procured must support the requirements of the project customer.

A statement of work (SOW) may define the work to be accomplished within the project, but it generally does not define the product description as a whole. However, when an entire project is to be procured from a vendor, the SOW, you may need to reference the requirements documentation to ensure that the procurement planning process defines exactly what's needed and adheres to any relevant laws, regulations, and standards.

8.2.3 Relying on the Project Management Plan

The project management plan is also needed during the procurement planning processes because it will guide how the project should progress, and each subsidiary plan may need to be referenced for procurement guidelines. For example, the cost management plan, the scope management plan, quality management plan, and the staffing management plan may all be needed for effective procurement planning.

One of the biggest things you'll need to consider during procurement management is the reliance on the risk response transference. Recall that transference is the contractors for dangerous work are two common examples of transference. The risk register will help identify the costs associated with the identified risks, and the contractual agreements for transference will be reference as part of the project costs.

The project management plan also includes the project schedule-and the project manager needs to consider procurement leads, fulfilment time for vendors, and when resources are needed to keep the project moving along. Couple the project schedule with the activity cost estimate and the cost performance baseline, and the project manager can do cash flow forecasting, communicate with management about upcoming expenses, and ensure that vendors are paid on time.

8.2.4 Teaming with other organizations

If your organization and another organization partner on an opportunity, it's called a teaming agreement. It's a legally binding venture between two or more organizations that plan to complete a defined set of work or to seize an opportunity or some other venture that the parties involved couldn't complete necessarily on their own. Teaming agreements end when the venture ends. You might have heard these

agreements loosely defined as “coop-etition”-a fun way to describe cooperating with the competition, although teaming doesn’t have to be just a partnership with the competition.

Teaming agreements should define in a contract the roles, buyer and seller relationships, and how the teaming agreement ends. The parties must all be in agreement with one another as to who does what, how communication occurs, and decisions in the partnership will be made.

8.2.5 Planning for the Project Requirements

When it comes to project procurements, you specifically need the project requirements to determine what you’ll purchase from vendors. The requirements are also needed to determine what you’ll purchase from vendors. The requirements are also needed to determine whether any aspects may affect the contracting process, such as licenses, permits, intellectual property rights, nondisclosure agreements, and insurance. Basically, anything that affects the project’s performance, the project environment, or the people involved in the project should be considered when it comes to project procurement and the project requirements.

You’ll also need to consider the risks within the project, the project schedule, the costs of the activities within the project, and the overall project budget. The project schedule must be considered because the vendor can affect the project schedule if they don’t complete their contractual obligations on time.

The project budget is needed because you’ll need to determine how much you can afford to spend on the vendor –and whether your internal estimates are accurate or way, way off base.

8.3 Completing procurement Planning

Procurement planning should occur early in the planning processes, with certain exceptions. As needs arise, as project conditions change, or as other circumstances demand, procurement planning may be required throughout the project. Whenever procurement planning happens early in the project, as preferred, or later in the project, as needed, a logical approach to securing the proper resources is necessitated.

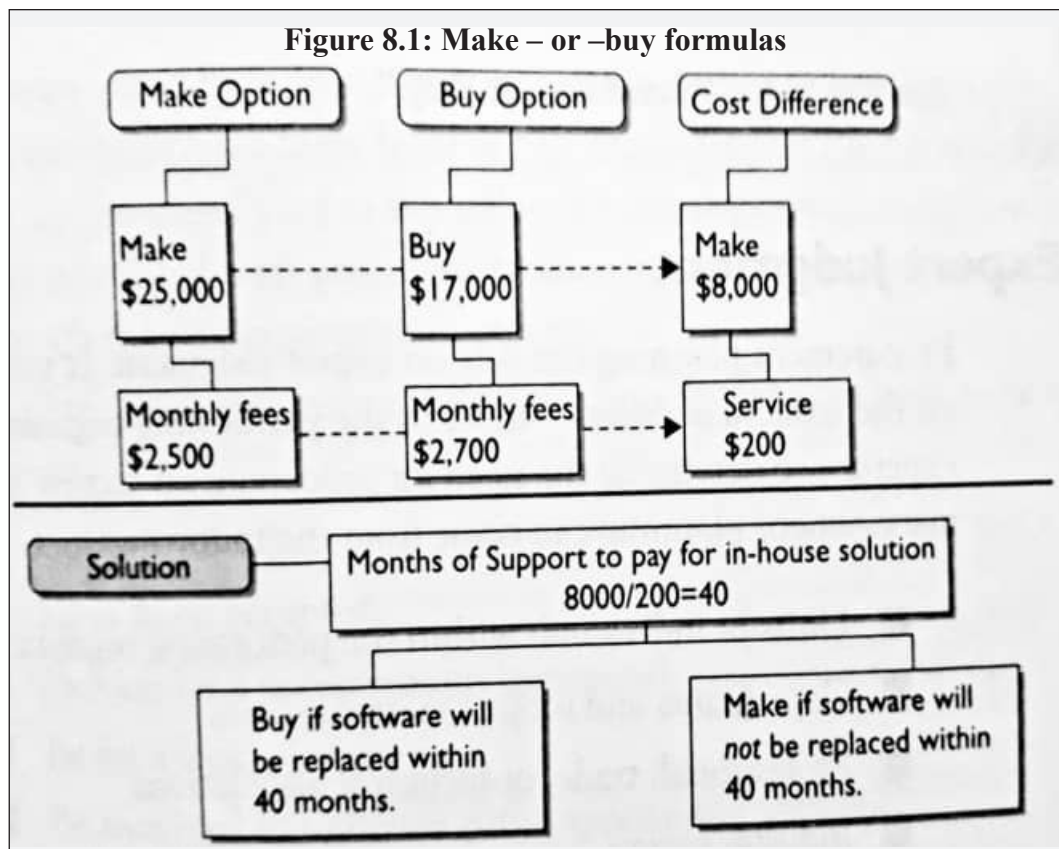
8.3.1 Determining to Make or Buy

The decision to make or buy a product is a fundamental aspect of management. Under some conditions, it is more cost-effective to buy; in other cases, it makes more sense to create an in-house solution. The make or buy analysis should occur in the initial scope definition to determine whether the entire project should be completed in-house or procured. As the project evolves, additional make or buy decisions are needed.

The initial costs of the solution for the in-house or procured product must be considered, but so, too, must the ongoing expenses of the solutions. For example, a company may elect to lease a piece of equipment. The ongoing expenses of leasing the piece of equipment should be weighed against the expected ongoing expenses of purchasing the equipment and the monthly costs to maintain, insure, and manage the equipment.

Here's an example: (Figure 8.1) shows the mathematical approach for determining whether it is better to create a software program in-house or buy one from a software company. The in-house solution will cost your company \$25,000 to create your own software package and (based on historical information) another \$2,500 per month to maintain the software.

The development company has a solution that will cost your company \$17,000



to purchase, but the development company requires a maintenance plan for each software program installed, which will cost your company \$2700 per month. The initial difference between making the software and buying the software is \$8000. The difference between supporting the software the organization has made and allowing the external company to support their software the organization has made and allowing the external company to support their software is only \$200 per month.

The \$200 per month is divided into the difference between creating the software internally and buying the software—which is \$8000 divided by \$200 –which equates to 40 months. If the software is to replace within 40 months, the company should buy the software. If the software will not be replaces within 40 months, it should build the software.

An organization may choose to make or buy for multiple reasons. The following table shows some common examples or reasons for making and buying:

Reasons to Make	Reasons to Buy
Less costly	Less costly
Use in-house skills	In-house skills aren't available or don't exist
Control of work	Small volume of work
Control of intellectual property	More efficient
Learn new skills	Transfer risks
Available staff	Available vendor
Focus on core project work	Allows project team to focus on other work items

8.3.2 Using Expert Judgment

Procurement planning can rely on expert judgement. It may be beneficial to rely on the wisdom of others –those in the performing organization or subject matter experts- to determine the need for procurement. Expert judgement for procurement management planning can come from the following:

- Units or individuals within the performing organization
- Consultants and subject matter experts
- Professional, trade, or technical associations
- Industry group

8.4 Determining the Contract Type

There are multiple types of contracts when it comes to procurement. The project work, the market, and the nature of the purchase determine the contract type. The following are some general rules that PMP exam candidates, and project managers, should know:

- A contract is a formal agreement between the buyer and the seller .Contracts can be oral or written – though written is preferred.
- The United States and most developed countries back all contracts through the court system.
- Contracts should clearly state all requirements for product acceptance.
- Any changes to the contract must be formally approved, controlled, and documented.
- A contract is not fulfilled until all of the requirements of the contract are met.
- Contracts can be used as a risk mitigation tool, as in transferring the risk. All contracts have some level of risk; depending on the contract type, the risk can be transferred to the seller. If a risk response strategy is to transfer to the seller. If a risk response strategy is to transfer, risks associated with procurement are considered secondary risks and must go through the risk management process.
- Legal requirements govern contracts .In order for a contract to be valid, it must
 - Contain an offer
 - Have been accepted
 - Provide for a consideration (payment)
 - Be executed by someone with capacity and authority
 - Be for a legal purpose
- The terms and conditions of the contract should define breaches, copyrights, intellectual rights, and force majeure.

Force majeure is French for superior force- sometimes called “acts of God”. You might know these as tornados, earthquakes, and hurricanes.

8.4.1 Fixed –Price Contracts

Fixed –price contracts (also known as firm fixed- price and lump sum contracts)

are agreements that define a total price for the product the seller is to provide. These contracts must clearly define the requirements the vendor is to provide. These contracts may also provide incentives for meeting or exceeding contract requirements such as meeting deadlines – and require the seller to assume the risk of cost overruns, as figure 8.2 demonstrate

You should know three types of fixed- priced contracts:

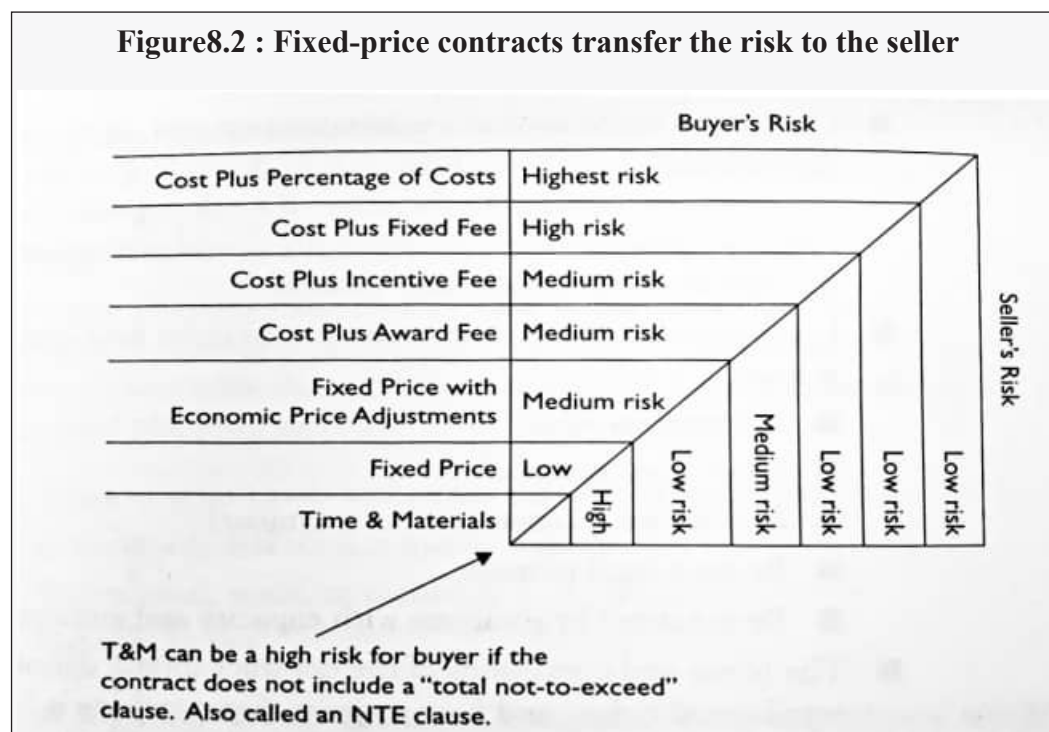
8.4.1.1 Firm fixed price contract

This contract type defines the exact amount for the goods or services provided by the vendor. It's the most common and most preferred contract type for organizations, as the risk for the buyer is relatively low.

For the seller, however, the risk is that if their cost of material, doing business, or completing the work defined in the contract increases, they cannot pass on the cost to the customer. A firm fixed- price contract is also known as a lump- sum contract.

8.4.1.2 Fixed –price incentive fee contract

This contract type is similar to the firm fixed-price contract in that a lump sum amount is agreed upon between the buyer and seller for the work to be performed. However, this contract type allows the contract to include incentives for the project, such as bonus for completing the project work early, savings costs in the project, or other performance objectives. This contract can also have penalties for the vendor if they're late on project work or their performance is less than it should be.



8.4.1.3 Fixed price with economic price adjustment contract

This contract type is for long-term projects that may span years to complete the project work. The contract does define a fixed price, with caveats for special categories of price fluctuations over the life of the project, including inflation, electricity , shipping , labour costs , cost of material , or other resources that could affect the feasibility of the vendor completing the work. The contract must define the financial indexes that will be used to determine the fluctuation in the identified cost categories.

8.4.2 Cost- Reimbursable Contracts

These contract types pay the seller for the product. In the payment to the seller, there is a profit margin- the difference between the actual costs of the product and the sales amount. Cost-reimbursable contracts require the buyer to assume the risk of cost overruns. There are four types of cost-reimbursable contracts:

8.4.2.1 Cost plus fixed fee

The buyer is responsible for all costs the contracted work incurs plus a predetermined fee for vendor to manage and complete the contracted work. The fee for the work is usually tied to a percentage of the estimated project costs, but not always. For example, I'll remodel your condo, but you have to pay for all the materials and labour I'll need, which should be close to \$80,000. In addition , you'll have to pay me 15 percent of the costs, which will be \$12,000. You'll pay the costs as the project progresses and pay my fee based on milestones completed in the project. If I don't finish the project, you don't finish the project, you don't finish paying me. You do have some risks , though –if I waste materials , you have to buy more and you'll still have to pay my fee for the work.

8.4.2.2 Cost plus incentive fee

This contract type requires that the buyer pay for all the preapproved costs for materials and labour in the project plus an incentive fee for completing the project early, saving on project costs, managing certain risks, or meeting other performance objectives. The contract will define how the incentives are determined. One popular method attaches dollar amounts to completed milestones and dates. If the vendor delivers the milestones ahead of the promised dates on a consistent basis, the value of the work increases and so will the incentive fee for the vendor. If the contract is based on the cost savings for the project and early completions are cost savings, the contract must define how the cost savings are split between the buyer and the

seller. Usually, the seller receives 20 percent of the cost savings in what's called an "80/20 split".

8.4.2.3 Cost plus award fee

This contract requires the buyer to pay for all the project costs and gives the seller an award fee based on the project performance, certain project criteria, or other goals established by the buyer.

The award fee can be tied to any factor the buyer determines, and the factor doesn't have to be exact.

For example, the buyer can set an award fee off up to \$100,000 for a \$1 million project based on the technical ingenuity of the project solution, the quality of the work, or the actual cost savings the solution creates for the organization.

8.4.2.4 Cost plus percentage of costs

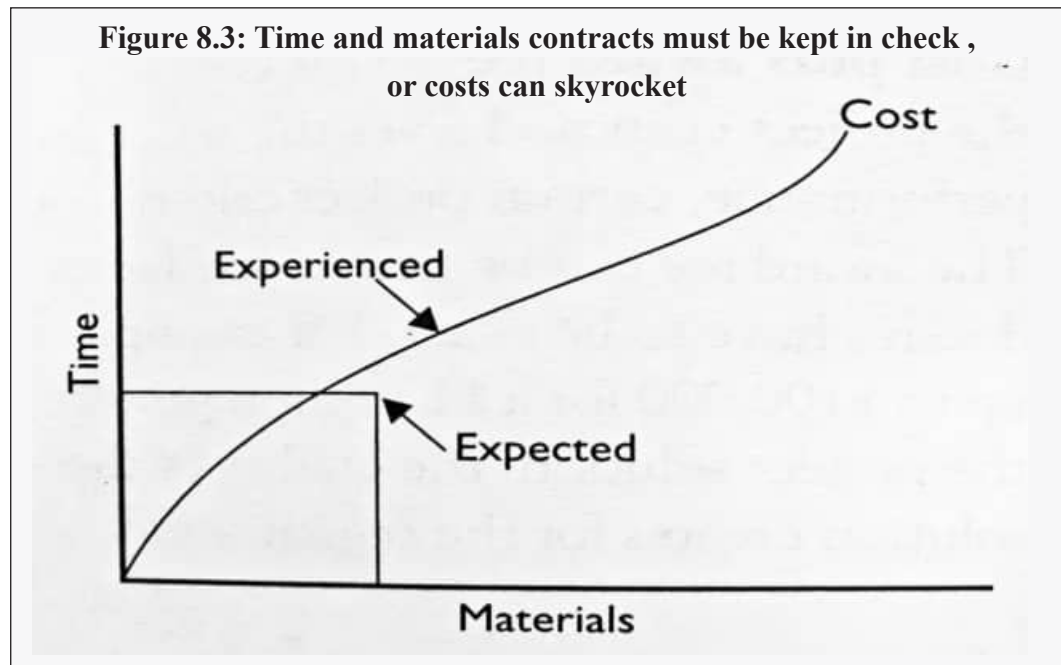
This contract type is the absolute pits for the buyer, and most organizations won't participate in these contracts. In this case, the buyer has to pay for all of the costs of the materials. The obvious risk is that the vendor can waste materials and buyer will have to buy new materials and pay the percentage of costs for the material again. It's easy for the vendor to run up the total project costs just by wasting materials.

The only time it might be appropriate, and I stress the word might, to use a cost plus percentage of costs contract is when the vendor is working with a highly specialized material and type of work.

For example, imagine an artist who's sculpting a marble statue for the lobby of a building or a scientist who's working with a highly complex chemical. The nature of this type of work is so specialized that the artist and the scientist are unlikely to waste materials on purpose just to crank up their project costs. Having said that, I doubt you will see this on the PMP exam. As a general rule, avoid the cost plus percentage of costs contract.

8.4.3 Time and Materials Contracts

Time and materials (T&M) contracts are sometimes called unit price contracts. They are ideal when an organization contracts out a small project when smaller amounts of work within a larger project are to be completed by a vendor. T&M contracts, however, can grow dangerously out of control as more work is assigned to the seller. T&M contracts should have a not to extend clause (NTE clause) to put a ceiling on the procured work. Figure 8.3 shows an example of how T&M contracts can pose a risk for the buyer.



8.5 Planning Contracting (Preparing for Contracting)

Contracting planning is the process of preparing to acquire sellers to provide products that the project needs. It's a pretty straightforward business. Three inputs are used for contracting planning:

- **The Procurement Management plan:** This subsidiary plan sets out the methodologies and expectations of procurement within the performing organization.

- **The statement of work:**

The contracts SOW provides detailed information about what the seller will be providing for the performing organization. Recall that this document allows the seller to determine whether it can provide the product and meet the requirements of the project team.

- **Other Planning outputs:**

Other details within the project plan, such as the schedules, estimates, constraints, and assumptions, are referenced, since their values may have a direct influence on the contracting process.

8.5.1 Organizing Contracting Materials

Contracting planning relies on the outputs of procurement planning. The procurement management plan will guide the process as the project team has planned, as the

performing organization requires, or under the guidance of a procurement office within the performing organization.

Two primary tools are used for contracting planning:

- **Standard forms:** Within the performing organization, there may be many different standardized forms for contracts, descriptions of procurement items, bid documents, and other procurement related documents.
- **Expert judgement**
Expert judgement may be needed to review and help the project manager select the best source for the procured product.

8.5.2 Creating the Procurement Documents

The primary outputs of contracting planning are the procurement documents. These documents guide the relationship between the buyer and the seller. Communication between the buyer and the seller should always be specific as to the requirements and expectations of the seller. In initial communications, especially when requesting a price or proposal, the buyer should include the SOW, relevant specifications, and, if necessary, any non disclosure agreements (NDAs). Requests from buyers to sellers should be specific enough to allow the seller to provide viable alternatives.

Following are some specific terms the project manager – and the PMP candidate should be familiar with:

Document	Purpose
Bid	From seller to buyer. Price is the determining factor in the decision making process
Quotation	From seller to buyer. Price is the determining factor in the decision making process
Proposal	From seller to buyer. Other factors –such as skill sets, reputation, and ideas for the project solution –may be used in the decision making process
Invitation for bid(IFB)	From buyer to seller. Requests the seller to provide a price for the procured product or service.
Request for quote (RFQ)	From buyer to seller. Requests the seller to provide a price for the procured product or service.
Request for proposal (RFP)	From buyer to seller. Requests the seller to provide a proposal to complete procured work or to provide the procured product.

8.5.3 Determining the Source Selection Criteria

Another output of contracting planning is the evaluation criteria to determine which source the organization will purchase from. The evaluation criteria is used to rate and score proposal from the sellers. In some instances, such as a bid or quote, the evaluation criterion is focused just on the price the seller offers. In other instances, such as proposal, the evaluation criteria can be multiple values: experience, references, certifications, and more.

It's essential for the project manager and the project team to create selection criteria that will guide their decision making later in the project. Common questions that should be considered prior to vendor selection include the following:

- Does the vendor understand the project need?
- What is the overall project and / or life- cycle cost?
- What is the vendor's technical capability?
- What is the vendor's management and technical approach to the project work?
- What is their financial capacity to complete the project work?
- What is risk is associated with the work and how will the vendor manage the risk?
- Does the vendor qualify in areas that may help in rewarding the contract (such as a small business, disadvantages business, or a woman-owned business)?
- What are the proprietary rights and intellectual property rights associated with the project work?
- Will the vendor provide a warranty for the work they complete?

8.5.4 Updating the Procurement Statement of Work

The final outputs of contracting planning are updates to the procurement statement of work. As the project team creates the requirements from the sellers during invitations for bids, requests for quotes, or requests for proposals, they may discover other needed elements in the SOW. In addition, it is possible that the bids, quotes, and proposals may offer alternatives the project team has not considered – and a new SOW can then be created.

Changes to the SOW should be updated, documented, and recorded to reflect the logic and reason behind the change.

Changes should also be managed as part of the project's integrated change control process and documented in the project's change log.

8.5.5 Completing Procurement Purchasing

Once the contracting planning has been completed, the actual process of contracting can begin. Fortunately, the seller, not the buyers perform most of the activity in solicitations- usually at no additional cost to the project. The sellers are busy trying to win the business. The process of conducting procurements has eight inputs:

- Procurement management plan
- Procurement documents
- Source selection criteria
- Proposals from sellers
- Project documentation
- Outcomes of make or buy decisions
- Statement of work
- Organizational process assets

8.6 Requesting seller response

Requesting seller response is the process of inviting sellers to acquire the business of the performing organization. Three primary tools are needed to complete this process:

- **Bidder conferences**

A bidder conference, also called a contractor conference or vendor conference, is a meeting with prospective sellers to ensure that all sellers have a clear understanding of the product or service to be procured and are on equal footing. Bidder conferences allow sellers to query the buyer on the details of the product to help ensure that the seller's proposal is adequate and appropriate for the proposed agreement. At this point of the process, all sellers are considered equal.

- **Advertising**

In most circumstances, advertisements inviting bidders are expected. These advertisements can run in newspapers or trade journals specific to the industry of the organization. Some government agencies require advertisements inviting sellers to acquire the project work, attend a bidder conference, or present a proposal for the described work.

- **Developing a qualified sellers list**

Many organizations use a qualified sellers list to guide their procurement decisions. The project team may elect to create their own qualified sellers list through the Internet or other third- party resources.

A standard of procurement is that bids and quotes are looking for sellers to provide a price. Proposals are asking the sellers to provide solutions.

8.6.1 Examining the Results of Contracting

The end result of contracting, as expected, is a collection of proposals, bids , and quotations. These documents indicate the seller's ability and preparedness to complete the project work. The proposals should be in alignment with the buyer's stated expectations, and they may be presented orally, electronically, or in hard-copy format.. Of course, the relationship between the buyer and the seller- and the type of information being shared- will determine which modality is the best choice for communication.

8.7 Selecting the seller

Once the seller have presented their proposals, bids,or quotes (depending on what the buyer requested of them), their documents are examined so that the project manager can select which seller are the best choice for the project work. In many instances, price may be the predominant factor for choosing a particular seller-but not always. Factors other than price may also be taken into consideration:

- The cost of an item may not reflect the true cost to the performing organization if the item cannot be delivered in a timely manner. If a seller promises to have a product on site by a specific date and fails to do so, the project can be delayed, costing the organization thousands- or more—in losses.
- Proposals can be separated into two categories: technical and commercial. The technical category describes the approach and methodology to complete the project work, and the commercial category delves into consideration both categories to determine the best choice for the project.
- Critical, high-priority projects may rely on multiple sellers to complete the project work. This redundancy can balance risk, cost, and opportunity among multiple vendors.

8.7.1 Preparing for Source Selection

Source selection weighs and evaluates the proposals, bids, and quotes for the procured portions of the project and then makes a determination as to which seller is the best for the project work. The source selection decision-making process involves three inputs:

- **Proposals** The proposals, bids, and quotations provided by the sellers are key inputs. These are the documents the performing organization will evaluate to determine which seller is the best provider for the project.
- **Evaluation criteria** The evaluation criteria, such as referrals, samples of previous work, references, are considered. The evaluation criteria are evidence of the quality, depth, and experience of work the seller has performed in the past and, hopefully, is capable of performing on the current project. Evaluation criteria are developed in contracting planning and are applied in source selection.
- **Organizational policies** The performing organization likely has procurement policies and procedures that the project manager is expected to follow in regard to source selection. The organizational policies should be known before starting the source selection process to avoid any discrepancies, conflicts of interest, or other breaches of policies. For example, some organizations' procurement policies do not allow project managers to accept any gifts worth more than \$25 in value.

8.7.2 Completing the Seller Selection Process

For the performing organization to finalize the process of seller selection, there must first be eligible sellers. Assuming more than one seller can satisfy the demands of the project, the manager can rely on several tools and techniques when making a selection:

- **Weighting system** A weighting system takes out the personal preferences of the decision-maker in the organization to ensure that the best seller is awarded the contract. A weighting system creates a matrix, as shown in Figure Weights are assigned to the values of the proposal, and each proposal is scored. Because the weights are determined before the decision-maker reviews the proposals, the process is guaranteed to be free of personal preferences and bias. The seller with the highest score is awarded the contract.

Possible Score	20	20	15	10	10	5	20	100
Value	Expe- rience	Certi- fica- tions	Level IV Engi- neers	Secu- rity Clear- ance	Start date	Waste Re- moval	Price	Total score
ABC construction	15	20	7	10	10	5	12	79
Allen Builders	12	20	12	10	10	0	10	74
FRJ Construction	18	20	11	0	10	5	18	82
Howo & who Construction	18	15	5	0	5	5	15	73
Martin & Martin	9	20	13	10	5	0	18	65
Ralph Engineer	15	8	8	0	10	5	17	73

Figure : Weighting systems remove personal preference from the selection process

- **Independent estimates:-**These estimates are often referred to as “should-cost” estimates. They are created by the performing organization or outside experts to predict what the cost of the procured product should be. If there is a significant difference between what the organization has predicated and what the sellers have proposed, the statement of work was inadequate, the sellers misunderstood the requirements, or the prices provided by the sellers is too high.
- **Screening system:-** A screening system is a method to remove sellers from consideration if they do not meet given conditions. For example, screening could require that sellers be certified by a specific organization, have prior experience with the project technology, or meet other values. Sellers that don't meet the requirements are removed from the selection process and their proposals are not considered.
- **Contract negotiation** The performing organization creates an offer, and the seller considers it. The contract negotiation process is an activity to create a fair

price for the work the seller is to complete. The performing organization and the seller must be in agreement on the expectations, requirements, authorities, terms technical and business management approaches, price, and any other pertinent factors covered within, and by, the contract prior to signing it.

- **Seller rating systems** How the vendor has performed in the past may guide current and future project procurement decisions. Consider a vendor that has offered poor performance in quality, delivery, and contractual compliance versus a vendor that has scored high marks in quality, delivery, and contract compliance ; which should the project manager choose? That's the goal of the seller rating system: to collect and disseminate information on the performance of sellers in order to guide project decisions.
- **Expert Judgement:** - Often, the project manager and the project team may not be knowledgeable in the discipline the vendor is offering and that the project requires. In these instances , the project manager can rely on expert judgement to help make the best decisions regarding the project's welfare.
- **Proposal evaluation techniques :-** There are many different approaches to evaluating vendors proposal from weighting systems to screening systems- but all will rely on expert judgement and some sort of evaluation criteria.

8.7.3 Examining the results of Seller Selection

The one output of seller selection is a contract between the buyer and the seller.

A contract is a legally binding agreement between the buyer and the seller in which the seller provides the described product and the seller pays for the product.

Contracts are known by many names:

- Agreement
- Subcontract
- Purchase order
- Memorandum of understanding

Contracts have to be signed by a person with the power to authorize the requirements and payment specified in the contract. This role is called the delegation of procurement authority. Whether or not this person is the project manager depends on the procurement policies of the performing organization.

In some organizations, all contracting requires all contracts flow through centralized contracting. Centralized contracting requires all contracts for all projects to be approved through a central unit within the performing organization. Other

organizations use a decentralized contracting approach, which assigns a contract administrator or contract officer to the project.

8.8 Summary

Project procurement management involves acquiring goods and services for a project from outside the performing organization. Procurement purchasing or outsourcing is acquiring goods or services from outside sources. Organizations outsource to reduce costs, focus on their core business access skills and technologies, provide flexibility and increase accountability. In this chapter we could discuss the various steps involved in the procurement management process needed for a project.

Processes include:

- Planning purchases and acquisitions
- Planning contracting
- Requesting seller responses
- Selecting sellers

Questions:

1. What is involved in the process of requesting seller responses? How do organizations decide whom to send RFPs or RFQs?
2. Explain the make or buy decision process and describe how to perform the financial calculations involved in the lease or buy example.

Reference:

- 1) Flemming, Quentin: Project procurement management contracting, subcontracting, teaming.
- 2) Kathy Schwable: Information Technology project management
- 3) Jack T. Marchewka: Information Technology project management
www.Informationweek.com
www.cio.com



OUT SOURCING

Unit Structure

- 9.0 Objective
- 9.1 Introduction
- 9.2 Outsourcing
- 9.3 The Beginning of the Outsourcing phenomenon
- 9.4 Types of Outsourcing Relationships
- 9.5 The Realities of Outsourcing
- 9.6 Managing the Outsourcing Relationship
- 9.7 Summary

9.0 Objective

- Define outsourcing, business process outsourcing and offshoring.
- Describe the reasons why organizations outsource projects and project components.
- Describe the advantages and disadvantages of outsourcing.
- Describe several ways to improve the likelihood of outsourcing success.

9.1 Introduction

Physical items are not the only things a project may acquire from external sources. Often, services and components of the project scope are subcontracted to another firm. More specifically, the project's scope can be broken up into a number of subprojects. This idea is not new, since construction contractors often subcontract specific components of a building to other subcontractors such as framers, electricians, or plumbers. Today, however, the term “subcontracting” has been often substituted with the term “outsourcing”.

Outsourcing has become a strategic initiative for many organizations worldwide. It also has become controversial topic, especially when organizations look to fulfill their outsourcing needs overseas.

Outsourcing is a broad term that has different implications. For example, organizations, can outsource entire functions and business processes (e.g., data centers, customer support, accounting, human resources) . In this case, the outsourcing activities associated with selecting, negotiating, and transferring that particular function over to a third party would be a project in and of itself. On the other hand, a company may outsource a project (e.g. building an application system) or components of projects (e.g., programming,testing,training) to another organization, such as a consulting firm. While outsourcing can make goodbusiness sense, it can be a complicatedundertaking that requires a project management approach. For example, asurvey of organizations that have contracted outsourcing services reports that 53 percent of the respondents have encountered outsourcing challenges because their organizations lack project management skills

It is important for you to understand both sides of the procurement / outsourcing equation because it is very likely that you will be both a buyer and seller of outsourced products and services. For example, if you work as a consultant or for a software development house, you may not only be the seller of outsourcing services, but you might be a buyer of these services if you subcontract out any project components.

9.2 Outsourcing

Outsourcing can be defined as the procurement of products or service from an external vendor, supplier, or manufacturer. In this respect, outsourcing is analogous to procurement management. Perhaps one way to make a distinction is to view outsourcing as a strategic approach and regard the processes that make up project procurement management as a more tactical approach.

9.3 The Beginning of the Outsourcing phenomenon

In 1989, the Eastman Kodak Company in Rochester, New York was a leader in the Photography industry. With annual revenues of \$18.4 billion, Kodak was doing well as a company. However, Kodak was spending \$250 million a year on information technology, and management questioned why so much was being spent on IT when the company's mission was to be a leader in photography.

Kodak explored other options and eventually signed a 10-year, \$250-million deal to outsource its entire IT function to IBM Corp., Digital Equipment Corporation (DEC), and Business land, Inc. As part of the arrangement, DEC took over telecommunications, and Business land handled all PC purchases and maintenance. IBM received the biggest share of the deal and assumed responsibility for data center operations. As part of the arrangement, 300 Kodak employees transferred to IBM, and 400 transferred to DEC and Business land. Within the first year of the deal, Kodak's capital costs decreased almost 95 percent, while PC support costs dropped to about 5 to 10 percent. Mainframe costs also were reduced by 10 to 15 percent.

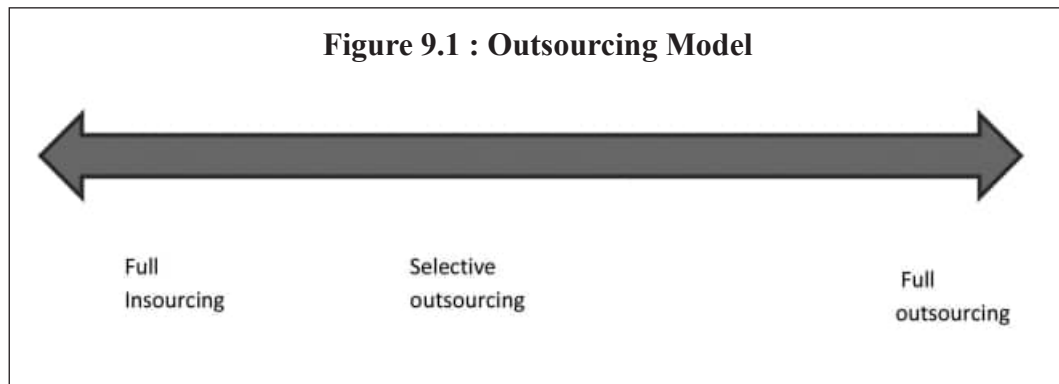
Kodak was not the first, or the largest, organization to turn to outsourcing, but it was the first well known and successful company to outsource an entire IT function. As a result, the perception that an organization had to provide its own IT support changed. Senior management began to talk about core competencies, cost saving and strategic partnerships with IT vendors (Field 1999).

By the year 2000, the field of IT had come to a critical crossroads, when more than 54 percent of IT services purchased in north America were outsourced. Even today, the momentum for outsourcing has grown and it appears that Europe now exceeds the United States in terms of the value of major outsourcing deals (Pruitt 2005b).

9.4 Types of Outsourcing Relationships

Today, outsourcing has expanded to include business process outsourcing in which an organization turns over processes other than just IT (i.e., accounting, human resources management, research and development) to another organization that specializes in that process (Brown and Wilson 2005). In recent years, a great deal of attention has been given to the outsourcing of jobs overseas. This type of outsourcing, or **offshoring**, allows an organization to take advantage of labor arbitrage (i.e., cheap labor) by procuring a product or service from a supplier that operates in another country.

Outsourcing can be an organizational-level decision or a project-level decision. Just as an organization can pursue outsourcing as a strategic approach, so too can a project teams have the opportunity to follow different approaches to outsourcing. This idea is illustrated in Figure 9.1, which shows that a continuum of outsourcing relationships can exist. For example, an organization or project could follow a **full-insourcing approach** in which all products and services would be retained internally.



For a project, this would mean that the project team is responsible for all the project's processes and scope. On the other hand, a **full-outsourcing approach** would be followed if an organization or project acquires all products or services from external sources. In this case, we would have a virtual organization or project. However, the best approach for organizations and projects probably would be selective outsourcing. More specifically, **selective outsourcing** provides greater flexibility to choose which project process deliverables should be outsourced and which should be kept internal. Although low cost is one advantage for outsourcing and offshoring, the objective should be to increase flexibility and quality (Lacity, Willcocks, & Feeny 1995).

9.5 The Realities of Outsourcing

Before the Kodak deal, outsourcing did not have many negative connotations, especially by IT professionals. Afterward, however, a minority understood the virtues of outsourcing, while a vocal minority despised it (Field 1999). Today this debate continues, as many organizations view offshore outsourcing as an important organizational or project strategy.

The controversy over outsourcing, especially offshoring, centers on the perception that jobs within one country are replaced by lower-wage jobs in another. Subsequently, domestic unemployment increases and thereby creates a negative impact on the nation's economy.

Although some people lose their jobs because of outsourcing, many new higher-paying jobs are often created. For example, Delta Airlines had over 6,000 representatives in 20 call centers worldwide. The agents in these call centers interacted with customers through voice, fax, and e-mail. Delta made the decision to offshore many of these activities and so 1,000 call center jobs were outsourced

to India. This resulted in \$2s million in saving that allowed Delta to add 1,200 reservation and sales positions in the United States (Robinson and Kalakota 2004).

Moreover, the McKinsey Global Institute estimates that the United states reaps between \$1.12 and \$1.14 for every dollar spent on outsourcing to India. In addition, Drezner also reports

That although 70,000 computer programmers lost their jobs between 1999 and 2003, more than 115000 software engineers found higher-paying jobs (Drezner 2004).

Organizational change management plays an important role for outsourcing successfully. A poll conducted in Europe examined the opinions of 200 employees in large organizations before and after their position were outsourced. Although this change can be unwelcome, the study found that, if done right, people may find that they have more opportunity to advance their careers and hone their skill (Pruit 2005a). However if the outsourcing decision is not handled properly, the remaining survivors can feel outrage or guilt, thus affecting the remaining employee's morale and motivation (Hamblen 2004). On the other hand, as peter F. Drucker (2002) points out, developing people is the most important task in business. The trend toward outsourcing and relying on nontraditional employees can reduce an organization's ability to gain competitive advantage in a knowledge economy.

9.6 Managing the Outsourcing Relationship

If outsourcing provides value to organizations and projects, how can we improve its likelihood of success? First, since outsourcing is a project, following a project management approach makes sense. However, Barthelemy (2003) provides some insight from a survey of just over 90 outsourcing efforts in Europe and the United states. These mistakes, termed the seven deadly sins of outsourcing, can be applied to the outsourcing of organizational activities and projects as well:

Outsourcing activities that should not be outsourced-Many believe that outsourcing results in an automatic reduction of costs and an increase in performance. However, this view is nonrealistic, and many organizations outsource to mimic their competitors or success stories in trade journals. For outsourcing to be successful, an organization must understand where it gets its competitive advantage so that a decision can be made to determine what activities should be performed externally. This may not be that easy. For example, a car rental company outsourced its IT department to reduce costs, but found that applications development and maintenance should

have remained in-house because these activities were important to its core business. A loss of control and the risk of the vendor going out of business can have grave consequences for the company.

Selecting the wrong vendor-Although selecting a good vendor seems like common sense, vendors can be selected for the wrong reasons. It is important to note that not all organizations outsource to reduce costs. Some outsource because a vendor can do something better or faster. An organization looking to outsource should verify a prospective vendor's qualifications as well as their experience and financial strength. There should also be a good fit between the two organizations' cultures, as well as a commitment to continuous improvement, flexibility, and a long-term relationship.

Writing a poor contract-A good contract must be in place to establish a balance of power between client and vendor. Time must be invested to carefully negotiate the terms of the contract because this not only sets expectations, but also creates a safety net in case the relationship breaks down. A well-written contract should be precise, be complete, provide incentives for the right behavior, be balanced so that it does not become one-sided or in favor of one party, and be flexible so that changes in business conditions can allow for changes in the contract.

Overlooking personnel issues-Outsourcing, or even a rumor that an organization is contemplating Outsourcing, can have a negative impact on employee loyalty and sense of job security. In turn, this may lead to reduced productivity, dysfunctional behaviors, or a mass exodus of employees before the Outsourcing decision is even made. Organizations must retain and motivate key employees since not all employees will be let go or transferred to the vendor. On the other hand, employees who will be transferred to the vendor to the vendor. On the other hand, employees who will be transferred to the vendor may have concerns about their job security, pay, and benefits. Therefore, the retention of organizational-specific knowledge is important for those employees who will be retained and for those who will be transferred.

Losing control over the Outsourced activity-An organization that outsources an activity can lose control if it does not manage the vendor actively. Often managers are tempted to Outsource an activity that is performing poorly or is not well understood. Outsourcing does not mean full abdication of that activity. Even when an activity is Outsourced, an individual or small group must still manage the vendor. A good contract is important, but good governance is essential.

Overlooking the hidden costs of outsourcing-Clients must take into account a number of hidden costs before they can be confident that an Outsourced activity results in a cost savings. The main types of hidden costs include searching for vendors, negotiating and writing the contract, and managing the vendor relationship. These costs are an important consideration since they can turn a potentially attractive Outsourcing arrangement into one that challenges the rationale for Outsourcing.

Failing to plan an exit strategy-Some Outsourcing relationships should be short term and others more long term. All Outsourcing relationship should include an exit strategy that allows the client organization a means to switch vendors or reintegrate the Outsourced activity later on. If an Outsourced activity with a particular vendor is working, then the contract can be resigned with little renegotiation. However, an organization that Outsources selectively should have options in the contract to buy premises and equipment, or hire employees back from the vendor.

9.7 Summary

Outsourcing has received a great deal of attention since the late 1980s, and the growth of outsourcing is expected to continue. Outsourcing is defined as the procurement of products or services from an external vendor, supplier, or manufacturer and thus is analogous to procurement management. However, outsourcing has more of a strategic focus, while project procurement management may be viewed as a more tactical – level approach.

Business process outsourcing occurs when an organization turns over processes of functions such as IT, accounting, human resources, and so forth.

Offshoring is a type of outsourcing when an organization takes advantage of lower wages in another country. Outsourcing decisions can be made at an organizational level, such as the outsourcing of a business process or function. In addition, an organization could make a decision to outsource the development of an application system or the implementation of a software package .In turn, a project manager could outsource specific project components such as programming, testing, or training as well. Subsequently, an organization or project could follow different approaches to outsourcing, such as full insourcing, full outsourcing, or something in between called selective outsourcing. Selective outsourcing may allow the most flexibility since some activities would remain in –house while others would be outsourced or subcontracted to external parties.

Offshoring or the outsourcing of organizational or project activities overseas has become a controversial topic. Some critics argue that IT work transferred overseas to lower wage countries has led to higher domestic unemployment and, in turn, a negative impact on the economy . Others have argued that while off-shoring has negatively impacted some individuals, it has increased the number of higher value-adding and paying jobs. Regardless of which side of the debate you support, it appears that outsourcing and offshore outsourcing will remain viable options for both organizations and projects. However, for outsourcing to be successful, project management processes must be followed to ensure that effective decisions are made that the outsourcing relationship is managed and controlled properly.

Sample Questions

1. What is outsourcing? How does it relate to project procurement management?
2. What is business process outsourcing?
3. What is offshoring or offshore outsourcing?
4. What is meant by full insourcing?
5. What is meant by full outsourcing?
6. What is meant by selective outsourcing? Why might selective outsourcing be a better approach than full insourcing or full outsourcing?

Reference Books :

7. Software Engineering, 5th and 7th edition, by Roger S Pressman, McGraw Hill publication.
8. Managing Information Technology Project 6th edition, by Kathy Schwalbe, Cengage Learning publication.
9. Software Engineering 3rd edition by KK Agarwal, Yogesh Singh, New Age International publication.
10. Information Technology Project Management by Jack T Marchewka Wiley India publication.



SOFTWARE AND SYSTEM QUALITY MANAGEMENT

Unit Structure

- 10.0 Introduction
- 10.1 Overview of ISO 9001
- 10.2 SEI Capability Maturity Model
- 10.3 McCalls Quality Model
- 10.4 Six Sigma
- 10.5 Formal Technical Reviews
- 10.6 Tools and Techniques for Quality Control
 - 10.6.1 Pareto Analysis
 - 10.6.2 Statistical Sampling
 - 10.6.3 Quality Control Charts
 - 10.6.4 The Seven Run Rule

10.0 Introduction

Definition by ISTQB,

Quality: The degree to which a component, system or process meets specified requirements and/or user/customer needs and expectations.

software quality: The totality of functionality and features of a software product that bear on its ability to satisfy stated or implied needs.

Software quality management (SQM) is a management process that aims to develop and manage the quality of software in such a way so as the best ensure the product meets the quality standards expected by the customer while also meeting any necessary regulatory and developer requirements, if any.

10.1 Overview of ISO 9001

- ISO 9000-3, the Guidelines offered by the International Organization for Standardization (ISO), represent implementation of the general methodology of quality management ISO 9000 Standards to the special case of software development and maintenance.
- Both ISO 9001 and ISO 9000-3 are reviewed and updated once every 5–8 years, with each treated separately.
- As ISO 9000-3 adaptations are based on those introduced to ISO 9001, publication of the revised Guidelines follows publication of the revised Standard by a few years.
- At the time of writing, the 2000 edition of ISO 9001 (ISO, 2000a) has been issued, but only the final just-completed draft of ISO 9000-3 (ISO/IEC, 2001) is awaiting approval.
- From the 1997 edition on, the ISO 9000-3 will represent the stand-alone ISO standard for the software industry.
- The 2000 edition of ISO 9001 as well as the new edition of ISO 9000-3 are supported by two additional conceptual standards:
 1. ISO 9000 (ISO, 2000b), which deals with fundamental concepts and terminology, and
 2. ISO 9004 (ISO, 2000c), which provides guidelines for performance improvement.

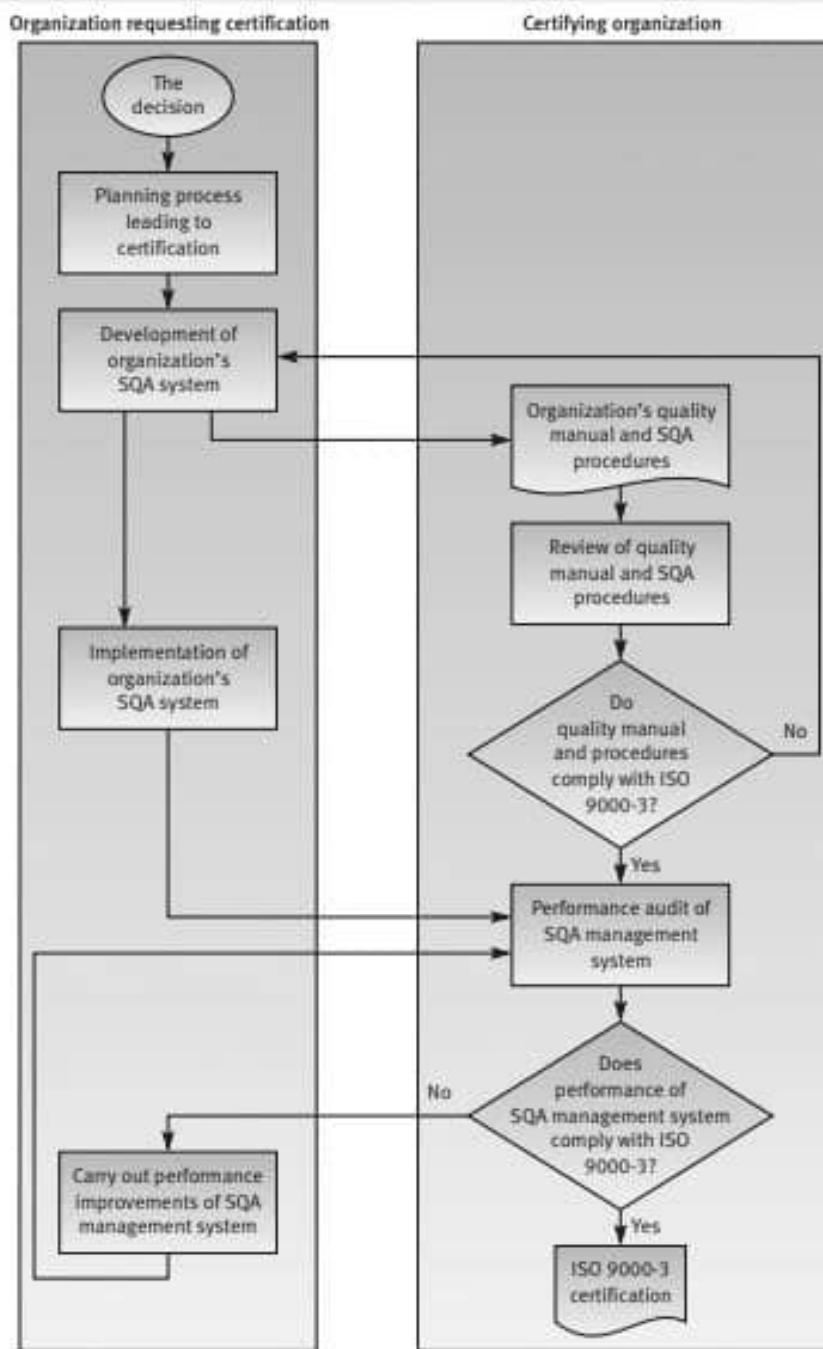
ISO 9001 — application to software: the TickIT initiative :

TickIT was launched in the late 1980s by the UK software industry in cooperation with the UK Department for Trade and Industry to promote development of a methodology for adapting ISO 9001 to the characteristics of the software industry known as the TickIT initiative. TickIT is currently authorized to accredit other organizations as certification bodies for the software industry in the UK (Figure 10.1).

TickIT activities include:

- Publication of the TickIT Guide, that supports the software industry's efforts to spread ISO 9001 certification. The current guide (edition 5.0, TickIT, 2001), which includes references to ISO/IEC 12207 and ISO/IEC 15504, is distributed to all TickIT customers.

- Performance of audit-based assessments of software quality systems and consultation to organizations on improvement of software development and maintenance processes in addition to their management.
- Conduct of ISO 9000 certification audits.

Figure 10.1:**Figure 23.1: The ISO 9000-3 certification process**

10.2 SEI Capability Maturity Model

Carnegie Mellon University's Software Engineering Institute (SEI) took the initial steps toward development of what is termed a capability maturity model (CMM) in 1986.

The initial version of the CMM was released in 1992, mainly for receipt of feedback from the software community.

The first version for public use was released in 1993 (Paulk et al., 1993, 1995; Felschow, 1999).

The principles of CMM

Following are the concepts and principles of CMM assessment:

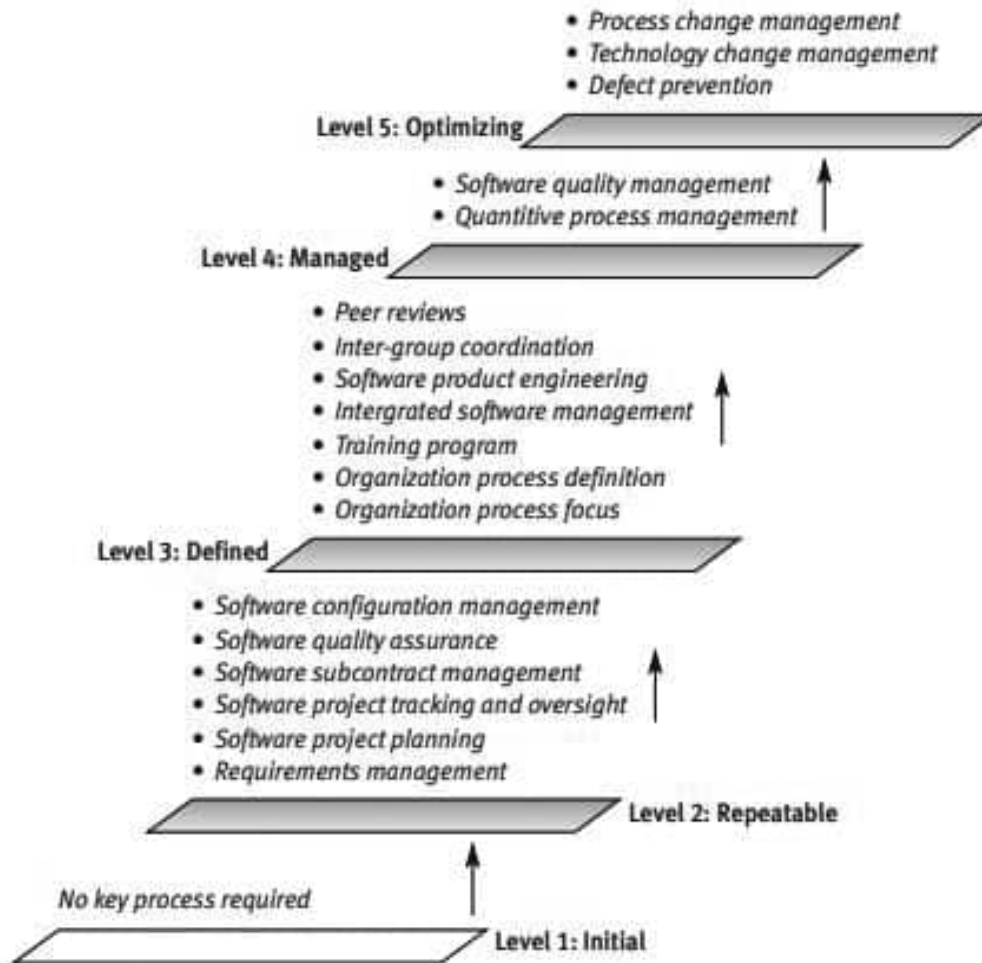
- Application of more elaborate management methods based on quantitative approaches increases the organization's capability to control the quality and improve the productivity of the software development process.
- The vehicle for enhancement of software development is composed of the five-level capability maturity model. The model enables an organization to evaluate its achievements and determine the efforts needed to reach the next capability level by locating the process areas requiring improvement.
- Process areas are generic; they define the "what", not the "how". This approach enables the model to be applied to a wide range of implementation organizations because:
 - It allows use of any life cycle model
 - It allows use of any design methodology, software development tool and programming language
 - It does not specify any particular documentation standard.

The CMM and its key process areas (KPAs) are presented in following (Figure 10.2)

Variety of specialized capability maturity models are as following:

- **System Engineering CMM (SE-CMM)**
 - It focuses on system engineering practices related to product-oriented customer requirements.
 - It deals with product development: analysis of requirements, design of product systems, management and coordination of the product systems and their integration.
 - In addition, it deals with the production of the developed product: planning production lines and their operation.

Figure 10.2: The CMM and its key process areas (KPAs)



- **Trusted CMM (T-CMM)**

It was developed to serve sensitive and classified software systems that require enhanced software quality assurance.

- **System Security Engineering CMM (SSE-CMM)**

It focuses on security aspects of software engineering and deals with secured product development processes, including security of development team members.

- **People CMM (P-CMM)**

It deals with human resource development in software organizations: improvement of professional capacities, motivation, organizational structure, etc.

- **Software Acquisition CMM (SA-CMM)**

It focuses on special aspects of software acquisition by treating issues – contract tracking, acquisition risk management, quantitative acquisition management, contract performance management, etc. – that touch on software purchased from external organizations.

- **Integrated Product Development CMM (IPD-CMM)**

It serves as a framework for integration of development efforts related to every aspect of the product throughout the product life cycle as invested by each department.

10.3 McCalls Quality Model

The classic model of software quality factors, suggested by McCall, consists of 11 factors (McCall et al., 1977). The McCall factor model provides a practical, up-to-date method for classifying software requirements (Pressman, 2000).

McCall's Factor Model

The 11 factors are grouped into three categories – product operation, product revision, and product transition factors.

- **Product operation factors –**
Product operation category includes five software quality factors, which deal with the requirements that directly affect the daily operation of the software. They are as follows – **Correctness, Reliability, Efficiency, Integrity, Usability.**
- **Product revision factors –**
According to McCall's model, three software quality factors are included in the product revision category. These factors are as follows – **Maintainability, Flexibility, Testability.**
- **Product transition factors –**
According to McCall's model, three software quality factors are included in the product transition category that deals with the adaptation of software to other environments and its interaction with other software systems. These factors are as follows – **Portability, Reusability, Interoperability.**

1	Correctness: These requirements deal with the correctness of the output of the software system. They include – • Output mission • The required accuracy of output that can be negatively affected by inaccurate data or inaccurate calculations. • The completeness of the output information, which can be affected by incomplete data. • The up-to-dateness of the information defined as the time between the event and the response by the software system. • The availability of the information. • The standards for coding and documenting the software system.
2	Reliability: Reliability requirements deal with service failure. They determine the maximum allowed failure rate of the software system, and can refer to the entire system or to one or more of its separate functions.
3	Efficiency: It deals with the hardware resources needed to perform the different functions of the software system. It includes processing capabilities (given in MHz), its storage capacity (given in MB or GB) and the data communication capability (given in MBPS or GBPS). It also deals with the time between recharging of the system's portable units, such as, information system units located in portable computers, or meteorological units placed outdoors.
4	Integrity: This factor deals with the software system security, that is, to prevent access to unauthorized persons, also to distinguish between the group of people to be given read as well as write permit.
5	Usability: Usability requirements deal with the staff resources needed to train a new employee and to operate the software system.
6	Maintainability: This factor considers the efforts that will be needed by users and maintenance personnel to identify the reasons for software failures, to correct the failures, and to verify the success of the corrections.

7	Flexibility: This factor deals with the capabilities and efforts required to support adaptive maintenance activities of the software. These include adapting the current software to additional circumstances and customers without changing the software. This factor's requirements also support perfective maintenance activities, such as changes and additions to the software in order to improve its service and to adapt it to changes in the firm's technical or commercial environment.
8	Testability: Testability requirements deal with the testing of the software system as well as with its operation. It includes predefined intermediate results, log files, and also the automatic diagnostics performed by the software system prior to starting the system, to find out whether all components of the system are in working order and to obtain a report about the detected faults. Another type of these requirements deals with automatic diagnostic checks applied by the maintenance technicians to detect the causes of software failures.
9	Portability: Portability requirements tend to the adaptation of a software system to other environments consisting of different hardware, different operating systems, and so forth. The software should be possible to continue using the same basic software in diverse situations.
10	Reusability: This factor deals with the use of software modules originally designed for one project in a new software project currently being developed. They may also enable future projects to make use of a given module or a group of modules of the currently developed software. The reuse of software is expected to save development resources, shorten the development period, and provide higher quality modules.
11	Interoperability: Interoperability requirements focus on creating interfaces with other software systems or with other equipment firmware. For example, the firmware of the production machinery and testing equipment interfaces with the production control software.

10.4 Six Sigma

- The process of producing high and improved quality output is known as Six Sigma.
- This can be done in two phases – identification and elimination.
- The cause of defects is identified and appropriate elimination is done which reduces variation in whole processes.
- Six Sigma's aim is to eliminate waste and inefficiency and increase customer satisfaction by delivering what the customer is expecting.
- It follows a structured methodology, and has defined roles for the participants.
- It is a data driven methodology, and requires accurate data collection for the processes being analyzed.
- It is about putting results on Financial Statements.

Following are few characteristics of Six Sigma:

The Characteristics of Six Sigma are as follows:

- **Statistical Quality Control:**
Six Sigma denotes Standard Deviation in statistics. Standard Deviation is used for measuring the quality of output.
- **Methodical Approach:**
The Six Sigma is a systematic approach of application in DMAIC (Design-Measure- Analyze-Improve-Control) and DMADV (Design-Measure-Analyze-Design-Verify) which can be used to improve the quality of production.
- **Fact and Data-Based Approach:** The statistical and methodical method shows the scientific basis of the technique.
- **Project and Objective-Based Focus:**
The Six Sigma process is implemented to focus on the requirements and conditions.
- **Customer Focus:**
The customer focus is fundamental to the Six Sigma approach. The quality improvement and control standards are based on specific customer requirements.
- **Teamwork Approach to Quality Management:**
The Six Sigma process requires organizations to get organized for improving quality.

10.5 Formal Technical Reviews

*“A formal review produces a **report** that identifies the material, the reviewers, and the review team’s judgment as to whether the product is acceptable. The **principal deliverable is a summary of the defects found and the issues raised**. The members of a formal review **team share responsibility for the quality of the review**, although authors are ultimately responsible for the quality of the products they create (Freedman and Weinberg 1990).”*

- The purpose of an Formal Technical Review (FTR) is to identify errors in function, logic and implementation of software.
- It is used to verify that the software under review meets its requirements.
- It ensures that the software has been represented according to predefined standards.
- It also helps to achieve software that is developed in a uniform manner and to make projects more manageable.

The FTR is actually a class of reviews that includes walkthroughs and inspections.

- A **review** is ‘a process or meeting during which a software product is presented to project personnel, managers, users, customers, user representatives, or other interested parties for comment or approval’
- An **inspection** is ‘a visual examination of a software product to detect and identify software anomalies, including errors and deviations from standards and specifications’
- A **walkthrough** is ‘a static analysis technique in which a designer or programmer leads members of the development team and other interested parties through a software product, and the participants ask questions and make comments about possible errors, violation of development standards, and other problems’

Characteristics of FTR are:

- Well-defined process: Phases (orientation, etc.) Procedures (checklists, etc.)
- Well-defined roles: Moderator, Reviewer, Scribe, Author, etc.
- Well-defined objectives: Defect removal, requirements elicitation, etc.
- Well-defined measurements: Forms, consistent data collection, etc.

10.6 Tools and Techniques for Quality Control

- The control quality process is defined as the “process of monitoring and recording the results of executing the quality activities to assess performance and recommend necessary changes.”
- In other words, quality control focuses on project results ensuring that they comply with the quality standards defined for the project and eliminating any causes of unsatisfactory performance.
- This process measures the details of the product results, such as deliverables or defects, and also of the project management results, such as schedule.
- Many of the techniques under the control quality process assume a working knowledge of statistical quality control, in particular the concepts of sampling and probability.
- The distinctions between attribute and variable sampling, precision and accuracy, and tolerance and control limits are fundamental components of a working knowledge of statistical quality control:

Prevention aids in identifying and avoiding potential problems so that they never enter or impact the process.

Inspection helps to identify and eliminate or correct errors so that they are not delivered to the customer.

Tolerance is a range of acceptable performance or results.

There are seven basic quality tools identified as appropriate for use in both the quality management plan and control quality processes. They are known as Ishikawa’s seven basic tools of quality:

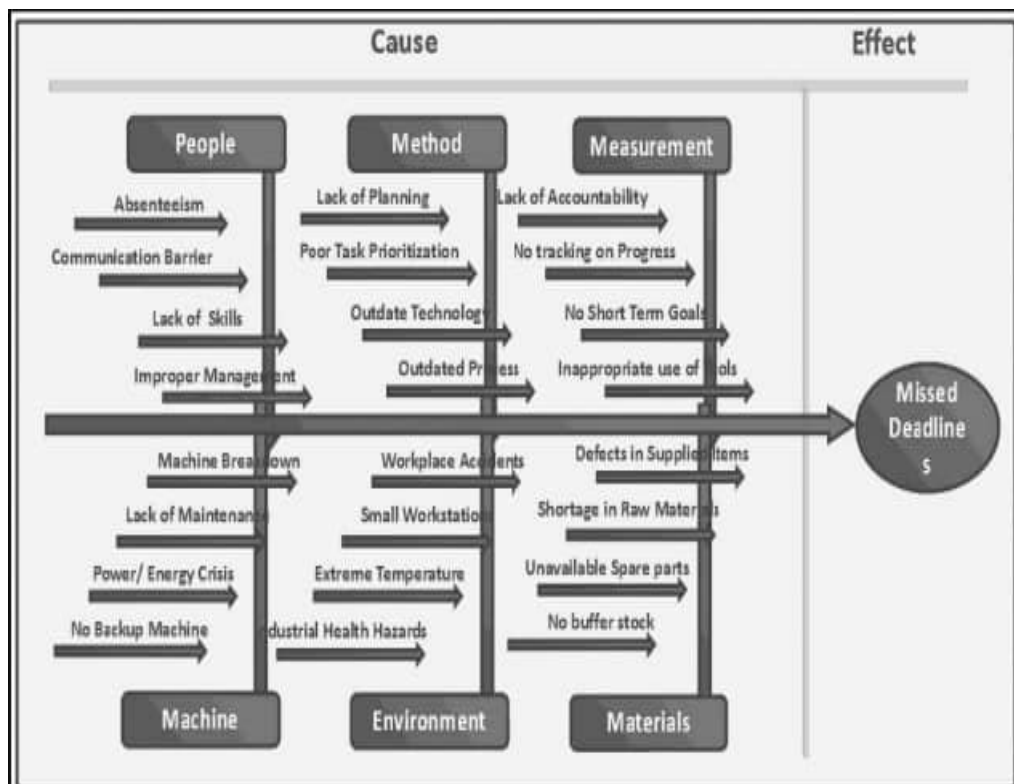
1. cause-and-effect diagrams,
2. flowcharting,
3. check sheets,
4. Pareto diagrams,
5. control charts,
6. histograms and
7. scatter diagrams.

Ishikawa's seven basic tools are also referred to as the 7QC tools.

Cause and Effect Diagrams (Figure 10.3)

- Cause-and-effect diagrams, or Ishikawa diagrams, were developed by Kaoru Ishikawa to illustrate and help determine how various factors relate to potential problems.
- Also called fishbone diagrams because they resemble the skeleton of a fish.
- The head of the fish is the effect and each bone of the fish is a cause that leads to that effect.
- The bones can branch off into smaller bones as you determine the lowerlevel cause-effect relationships.
- When all the bones are filled in, the diagram lets you look at all the possible causes (individual or combinations) of the effect (or problem) so that you can develop a solution to mitigate that effect.
- The diagram allows organized thought and encourages orderly consideration of the factors that result in a certain outcome.

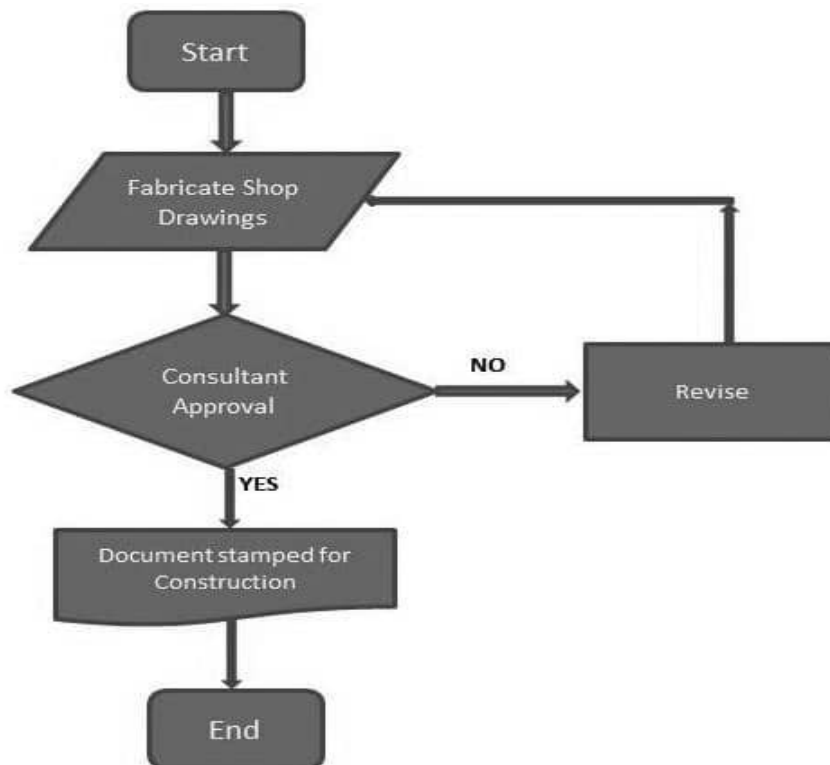
Figure 10.3: The CMM and its key process areas (KPsAs)



Flowcharts

- Flowcharts show the logical steps in a process and how various elements within a system are related.
- They can be used to determine and analyze potential problems in quality planning and quality control.
- Flowchart outlines the logical steps to complete a process.
- By documenting these logical steps, the team can identify where quality problems might occur and then develop approaches to proactively manage them.
- Flowcharting also helps create a process that is repeatable (Figure 10.4).

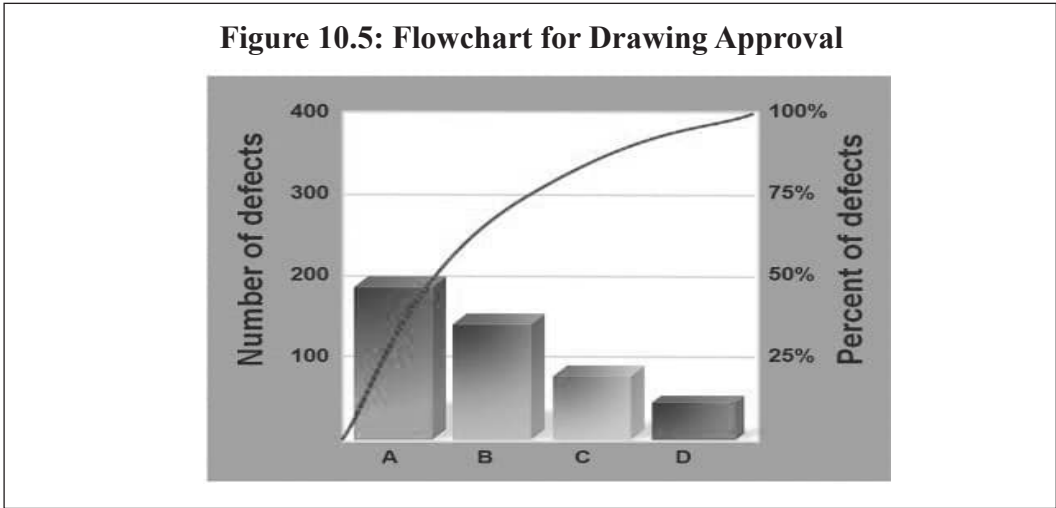
Figure 10.4: Flowchart for Drawing Approval



Check Sheets

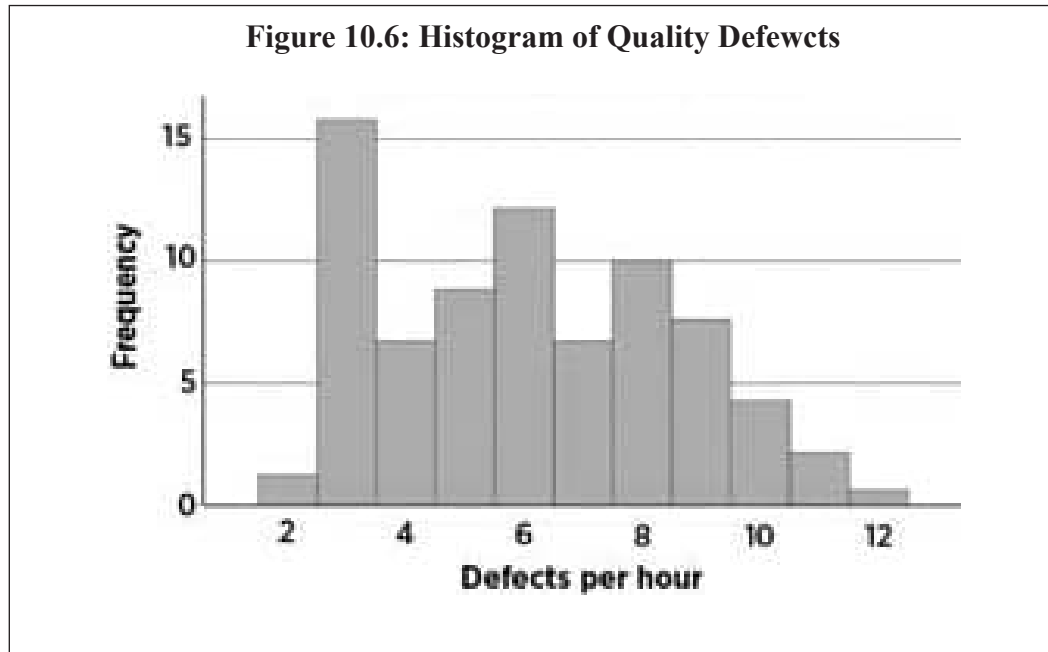
- Check sheets are used to organize information in order to facilitate data gathering.
- Check sheets are particularly effective for doing inspections, enabling focus on the particular attributes that may be contributing to potential or identified quality problems.

- A Pareto chart or diagram, is a specific type of histogram that is based on Pareto's principle, which states that a large number of defects or problems are caused by a small number of causes.
- Pareto's principle, frequently referred to as the 80/20 rule or 80/20 principle.
- Which means that eighty percent of the cost of defects are caused by twenty percent of the problems.
- A Pareto diagram is an ordered bar graph showing the number of defects and their causes ranked by frequency.
- The bars on the diagram graphically show the number and percentage of causes individually and the line shows the cumulative value.
- Pareto charts help focus attention on the most critical issues to get the most benefit.



Histograms

- A histogram is a vertical bar graph that represents the frequency of each measured category (known as bins) of variable.
- The histogram is particularly useful for identifying common causes.
- The histogram can be ordered, similar to a Pareto chart, or unordered.

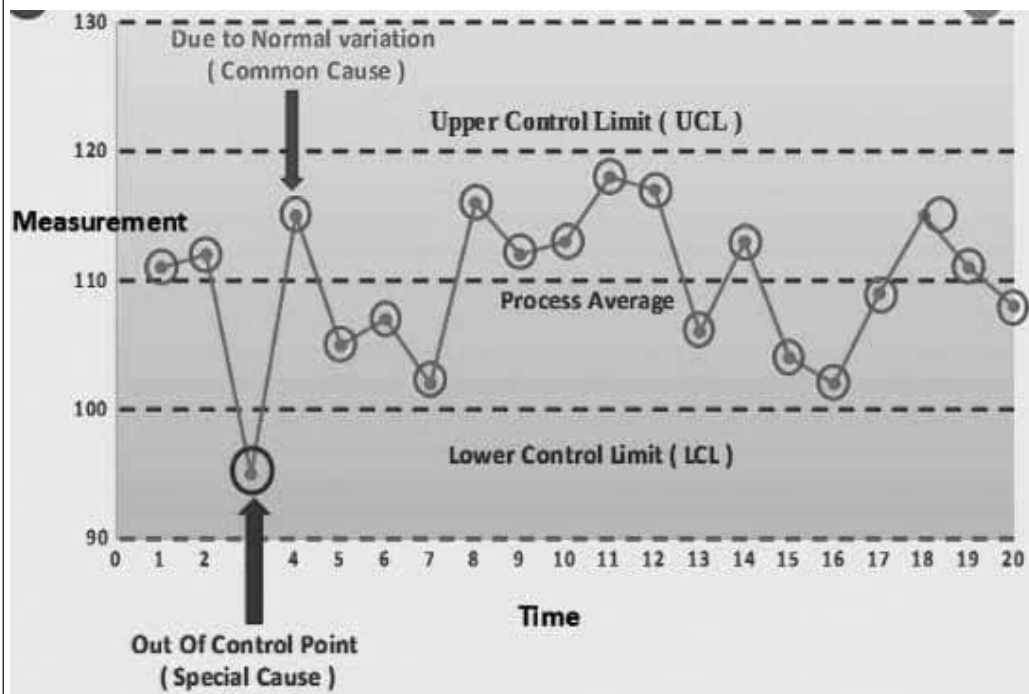


Control Charts

- Control charts are used to determine if processes are in or out of statistical control. Most processes experience a degree of normal variation i.e. most processes do not achieve target performance all the time.
- Control charts provide a mechanism for establishing a statistically objective range of acceptable variation around the target performance, thereby enabling attention to be focused on special cause variations (those that fall outside of the established performance limits).
- Control chart limits are established on the basis of standard deviations from the mean (target) performance. The upper control limit (UCL) and lower control limit (LCL) are established so that 99.73 percent (three standard deviations above and below the mean) of the measured data points fall within the range.

- The following ones are more common:
- Rule of Seven, or Seven Run Rule: Seven data points in a row are above or below the mean.
- Trend of Seven Rule: Seven data points in a row follow a trend up or down.
- Rule of One: Any single data point is outside of the control limits (upper or lower).

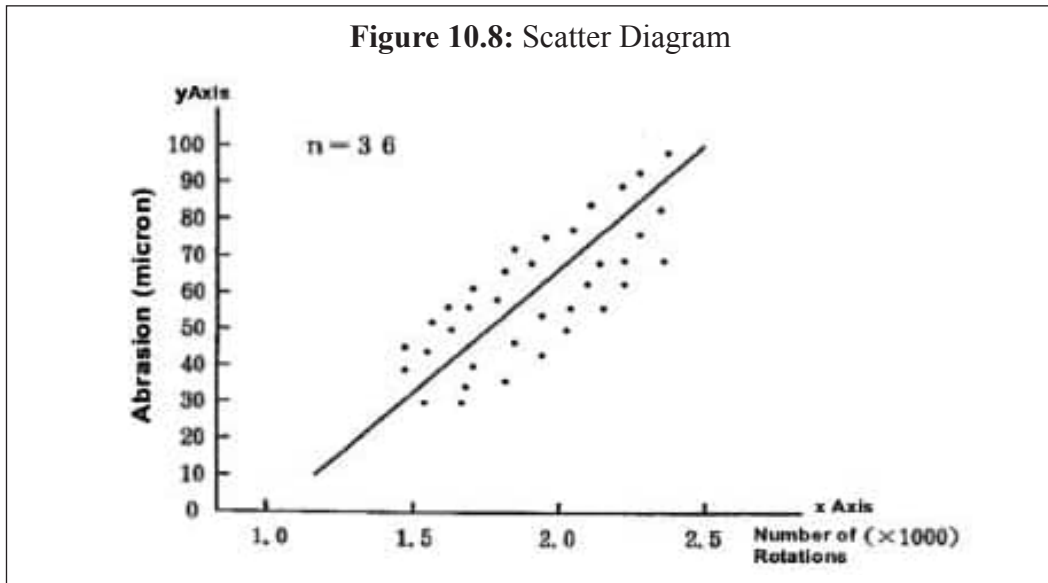
Figure 10.7: Rule of Seven



Scatter Diagrams

- Scatter diagrams plot two variables, the independent variable and the dependent variable, to graphically show the relationship between them.
- The X-axis in the diagram represents one characteristic (usually the independent variable), and the Y-axis measures the other.
- To interpret the diagram, look at two characteristics of the clustering:
- Tightness: The closer the cluster is to a diagonal line drawn through the graph, the more the two variables are likely to be linearly correlated. High correlation between the characteristics means that a change in one characteristic will be accompanied by a change in the other.

- Direction: If the correlation is positive, then as one variable increases so does the other, and the line will have a positive slope (from lower left to upper right). On the other hand, if the correlation is negative, it implies that as one characteristic increases, the other decreases, and the line will have a negative slope (from lower right to upper left).



Answer the following :

1. Explain the benefits of the use of SQA standards.
 - The ability to make use of the most sophisticated and comprehensive professional methodologies and procedures
 - Better understanding and cooperation between users of the same standards:
 - Between team members and between project teams
 - Between software developers and external participants in the project
 - Between suppliers and customers.
2. Describe the contributions made by the use of standards.
 - Provision of superior professional methodologies for use in the development process and for its management
 - Provision of SQA certification services based on independent professional quality audits
 - Provision of tools for “self-assessment” of achievements in planning and operating an organization’s SQA system.

3. Describe the general principles underlying quality management according to ISO 9000-3.
 - Customer focus – understanding a customer’s current and future needs
 - Leadership exercised in the creation and maintenance of a positive internal environment in order to achieve the organization’s objectives
 - Involvement of people at all levels to further organizational goals
 - Process approach – activities and related resources perceived and managed as a process
 - Systems approach to management – managing processes as a system
 - Continual improvement of the organization’s overall performance
 - Factual approach to decision-making – decisions based on the analysis of data and information
 - Mutually beneficial supplier relationships – emphasis on coordination and cooperation
4. Describe the principles embodied in the Capability Maturity Model (CMM).
 - Application of more highly elaborated software quality management methods increases the organization’s capability to control quality and improve software process productivity
 - Application of the five levels of the CMM enables the organization to evaluate its achievements and determine what additional efforts are needed to reach the next capability level
 - Process areas are generic, with the model defining “what” and leaving the “how” to the implementing organizations, i.e., the choice of life cycle model, design methodology, software development tool, programming language and documentation standard.



MODERN QUALITY MANAGEMENT

Unit Structure

11.0 Modern Quality Management

11.1 Juran

11.2 the importance of Top management

11.3 Commitment to Quality

11.4 Crosby and Striving for Zero defects

11.5 Ishikawa and the Fishbone Diagram

11.0 Modern Quality Management

To become a successful organization a study says that a Total Quality Management system is required. Modern quality management entails customer satisfaction, it prefers prevention against inspection and it recognizes the managerial team's responsibility for quality.

Several remarkable persons in the management field, like W. Edwards Deming, Joseph M. Juran, Philip B. Crosby, Kaoru Ishikawa, Genichi Taguchi and Arnold V. Feigenbaum, brought excellent contributions to the development of modern quality management.

Following are different 7 Management Tools For Quality Control :

1. Flowchart
2. Check Sheet
3. Cause and Effect (fish bone) Diagram.
4. Pareto Chart.
5. Control Charts.
6. Histograms.
7. Scatter Diagrams.

11.1 Juran and the importance of Top management

Besides Deming, Joseph Juran settled the basics of benchmarking with his contribution to modern quality management. Joseph M. Juran has brought significant input in regards to productivity issues. In 1984, he wrote the first edition of the “Quality Control Handbook,” which stresses the importance, for management, to be engaged in the continuous development of products.

He developed 10 steps for quality improvement:

1. Assess needs and improvement opportunities;
2. Set goals for improvement;
3. Become an organization focused on achieving goals (establishment of a Quality Council, identification of problems, selection of projects, establishment of teams, facilitators);
4. Facilitate trainings;
5. Initiate projects to solve issues;
6. Report progress;
7. Recognize merits;
8. Communicate results;
9. Track scores;
10. Focus on annual improvements so that they become usual processes within the organization.

Juran’s management theory continued to develop throughout his lifetime. He died at the age of 103 in 2008. Juran’s theories of quality management are part of other quality management theories such as Six Sigma and lean manufacturing.

11.2 Importance of Top Management

Many quality assurance managerial tasks are shared by managers of the same level or of more than one level.

Each manager takes on the responsibilities suitable to his or her level of authority and expertise.

Following are the responsibilities of the top management in ensuring Software Quality –

- Assure the quality of the company's software products and software maintenance services
- Communicate the importance of the product and service quality in addition to customer satisfaction to employees at all levels
- Assure satisfactory functioning and full compliance with customer requirements
- Ensure that quality objectives are established for the organization's SQA system and that its objectives are accomplished
- Initiate planning and oversee implementation of changes necessary to adapt the SQA system to major internal as well as external changes related to the organization's clientele, competition, and technology
- Intervene directly to support resolution of crisis situations and minimize damages
- Ensure the availability of resources required by SQA systems

The following steps can be taken by the top management to fulfill its responsibilities

- Establishing and updating the organization's software quality policy.
- Assigning one of the executives such as Vice President for SQA to be in charge of software quality issues
- Conducting regular management reviews of performance with respect to software quality issues

11.3 Commitment to Quality

- It is the policy of Carmacks to develop, produce and deliver products and services which consistently conform to customer requirements, and to pursue the goal of 100% error-free performance through product, process and management quality development.
- During certification audits, Auditors should look for evidence that Top Management has a 'hands-on' approach to the management of their QMS during interviews and auditing other requirements e.g. Context of the organization, policies and objectives, Management review minutes, Resources etc.

- Without solid management commitment, you will not have a successful quality management system.
- Communicating the importance of Leadership commitment and active involvement to quality is essential.
- It is the continuous and active demonstration to everyone in the organization that the need to meet customers' expectations is vital.
- Top Management commitment towards the QMS to demonstrate that they have a presence in the organization, are providing direction, are leading by example, are making informed decisions.
- **Taking accountability** for the effectiveness of the QMS e.g. established measures, system/process performance monitoring, management review, realization of planned activities, achievement of planned results and taking action when process performance is not meeting intended results
- **Establishing and maintaining the policy and objectives** aligned to the strategic direction e.g. context of your organization, external/issues
- **Integrated quality**, environmental and health and safety requirements into your organization's business processes e.g. system architecture, business model, process model
- **Promoting the process approach and risk-based thinking** e.g. process modelling, process mapping, inputs, outputs, activities, interactions, interfaces, resources, controls, risk management
- **Supporting process owners** in their process management activities e.g. deployment, governance, process evaluation, process improvement
- **Enabling resources**, including people, required for an effective QMS e.g. resource planning, workload, priorities, constraints, balance, organization flexibility, business benefits, organization growth
- **Communicating** the importance of conformity to the QMS and effective quality management e.g. meetings, briefs, e-mail, intranet, campaigns, roadshows, focused training, voice of the regulator or customer, consequence of non-conformity
- **Creating an environment** for continual improvement, e.g. proactive- product/ service/process implementation and improvement initiatives, improvement

projects, waste reduction, process re-engineering, cost reduction etc., and reactive - acting on process performance results, audit findings and complaints

- **Supporting other relevant management roles** e.g. organization hierarchy, trust, empowerment, responsible delegation, coaching, sharing knowledge, removing barriers, route to escalation.

11.4 Crosby and striving for zero defects

- Mr. Philip Crosby introduced the term Zero Defects in his book “Absolutes of Quality Management”.
- It has emerged as a popular and highly-regarded concept in quality management, so much so that Six Sigma is adopting it as one of its major theories.
- It means ensuring the highest quality standards in projects.
- Attaining zero defects is technically not possible in any sizable or complex manufacturing project.

According to the Six Sigma standard, the definition of zero defects is defined as 3.4 defects per million opportunities (DPMO), allowing for a 1.5-sigma process shift.

The zero defects concept should use for perfection in order to improve quality in the development or manufacturing process.

True perfection might not be achievable but at least the quest will push quality and improvements to a point that is acceptable under even the most stringent metrics.

Zero Defects – The Theory and Implementation

1. Zero defects theory ensures that there is no waste existing in a project.
2. Waste refers to all unproductive processes, tools, employees and so on.
3. Anything that is unproductive and does not add value to a project should be eliminated, called the process of elimination of waste.
4. Eliminating waste creates a process of improvement and correspondingly lowers costs.
5. Common with the zero defects theory is the concept of “doing it right the first time” to avoid costly and time-consuming fixes later in the project management process.

Zero defects theory is based on four elements for implementation in real projects.

1. **Quality** is a state of assurance to requirements. Therefore, zero defects in a project means fulfilling requirements at that point in time.
2. **Right the first time.** Quality should be integrated into the process from the beginning, rather than solving problems at a later stage.
3. **Quality is measured in financial terms.** One needs to judge waste, production and revenue in terms of budgetary impact.
4. **Performance** should be judged by the accepted standards, as close to perfection as possible.

Advantages of Zero Defects

1. achieving a zero defect level is waste and cost reduction when building products to customer specifications.
2. Zero defects means higher customer satisfaction and improved customer loyalty, which invariably leads to better sales and profits.

Disadvantages of Zero Defects

1. A zero defects goal could lead to a scenario where a team is striving for a perfect process that cannot realistically be met.
2. The time and resources dedicated to reaching zero defects may negatively impact performance and put a strain on employee morale and satisfaction.
3. There can also be negative implications when you consider the full supply chain with other manufacturers that might have a different definition of zero defects.

11.5 Ishikawa and the Fishbone Diagram.

This cause analysis tool is considered one of the seven basic quality tools.

The fishbone diagram identifies many possible causes for an effect or problem.

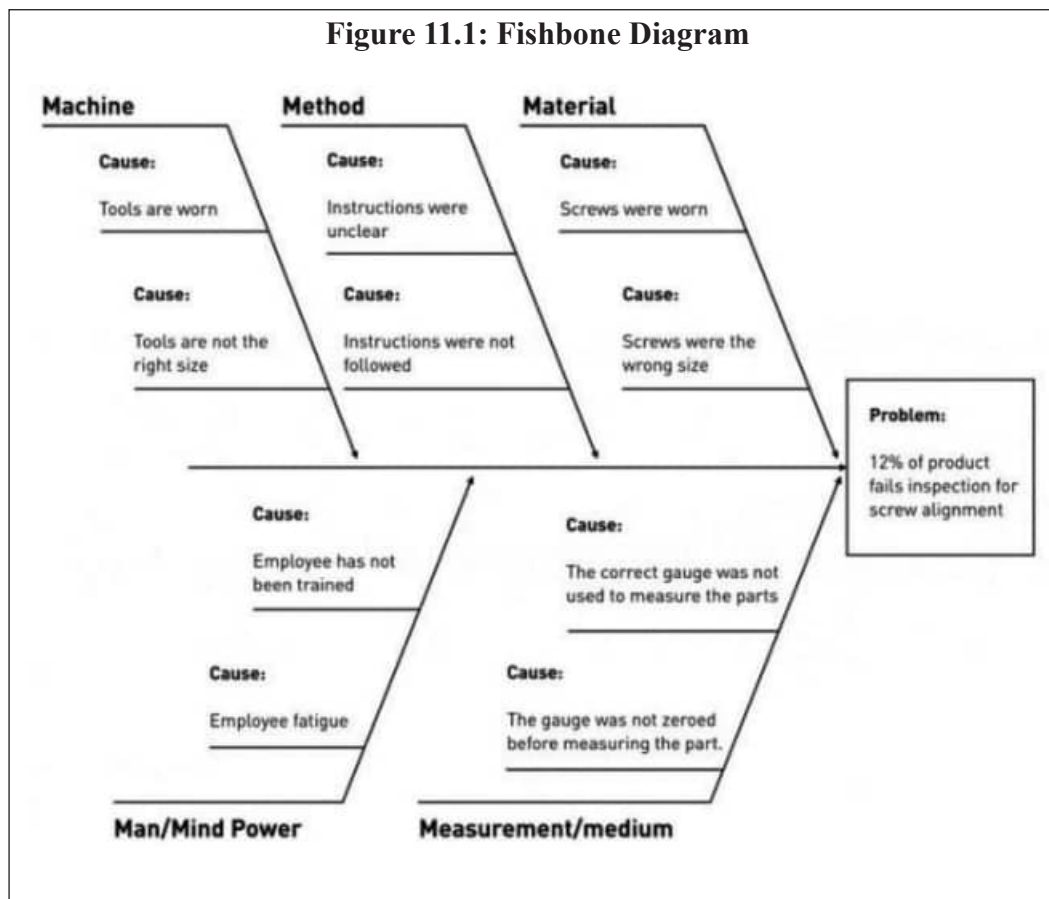
It can be used to structure a brainstorming session.

It immediately sorts ideas into useful categories.

We can use this when we need to identify possible causes for a problem and a team's thinking tends to fall into ruts.

Process to draw the Fishbone diagram:

- Agree on a problem statement (effect).
- Write it at the center right of the flipchart or whiteboard.
- Draw a box around it and draw a horizontal arrow running to it.
- Brainstorm the major categories of causes of the problem. If this is difficult use generic headings:
 - ☐ Methods
 - ☐ Machines (equipment)
 - ☐ People (manpower)
 - ☐ Materials
 - ☐ Measurement
 - ☐ Environment

Figure 11.1: Fishbone Diagram

- Write the categories of causes as branches from the main arrow.
- Brainstorm all the possible causes of the problem.
- Ask “Why does this happen?” As each idea is given, the facilitator writes it as a branch from the appropriate category.
- Causes can be written in several places if they relate to several categories.
- Again ask “Why does this happen?” about each cause.
- Write sub-causes branching off the causes.
- Continue to ask “Why?” and generate deeper levels of causes.
- Layers of branches indicate causal relationships.
- When the group runs out of ideas, focus attention on places on the chart where ideas are few.

Answer the following:

1. List the actors of a typical quality assurance organizational framework.
2. Describe top management’s responsibilities regarding software quality.
3. Describe the main objectives of management reviews.
4. List the SQA professional hands-on tasks required of project managers.
5. Explain fishbone diagram. Give example.



HUMAN RESOURCE MANAGEMENT

Unit Structure

- 12.0 Objectives
- 12.1 Introduction
- 12.2 What Is Project Human Resource Management
- 12.3 Keys To Managing People
 - 12.3.1 Motivation Theories
 - 12.3.2 Influence and Power
 - 12.3.3 Improving Effectiveness
- 12.4 Organizational Planning
- 12.5 Issues In Project Staff Acquisition And Team Development
 - 12.5.1 Staff Acquisition
- 12.6 Resource Loading and Leveling
 - 12.6.1 Resource Leveling
 - 12.6.2 Resource Loading
- 12.7 Team Development
 - 12.7.1 Training
 - 12.7.2 Team Building Activities
 - 12.7.3 Reward and Recognition Systems
 - 12.7.4 General Advice on Teams
- 12.8 Using Software To Assist In Human Resource Management
- 12.9 Summary

12.0 Objectives

When you have completed this chapter you should be able to:

1. Explain the importance of good human resource management on projects, especially on information technology projects for which experienced professionals in high demand.
2. Define the major processes involved in human resource management.
3. Summarize crucial theories of human resource management, including the contributions of Maslow and Herzberg on motivation, H.J. Thamhain and D.L. Wilemon on influencing workers, and Stephen Covey on how people and teams can become more effective.
4. Discuss organizational planning and be able to create a responsibility assignment matrix.
5. Understand key issues involved in projects staff acquisition and team development and explain the concepts of resource loading and resource levelling.
6. Describe how software can assist in project human resource management.

12.1 Introduction

Project human resource management is a vital component of project management, especially in the information technology field-in which qualified people are often hard to find and keep.

Beginning in the 1990s, there has been a growing shortage of personnel in information technology. In 1997, the information technology Association of America (ITAA), in cooperation with Virginia Polytechnic Institute and State University, conducted a survey of 1,500 information technology and non-information technology firms around the country. The results of the survey indicate that employers are having trouble finding, training, and retaining information technology workers. ITAA calculated that there were over 345,000 openings for programmers, systems analysts, computer scientists, and computer engineers in information technology and non-information technology companies with one hundred or more employees. These estimated job openings represent about ten percent of current employment in these three occupations. Many states or other geographic regions have conducted

their own surveys. The Minnesota Department of Economic Security Research and Statistics Office published a report stating that the 1,000 students graduating each year from information technology-related post-secondary programs in Minnesota were not nearly enough to fill the 8,800 positions projected to be open each year between 1998 and 2006. Coopers & Lybrand conducted a survey in 1997 and found that 70 percent of CEOs in high-tech firms listed the lack of highly skilled, trained workers as the number one barrier to growth. More than 80 percent of survey respondents planned to hire new staff in the coming year, adding a total of 19.8 percent to their total workforce.

These studies highlight what many people consider to be a serious national problem. High-tech firms, which add thousands of high-skilled, high-wage jobs to the U.S. economy every year, cannot find the workers they need. Many firms have turned to other countries, such as India, to find information technology workers. Some companies are offering interest-free loans to people seeking education and training in information technology. Colleges, universities, and private firms are expanding course offerings and programs in information technology.

It is crucial for organizations to practice what they preach about human resources. If people truly are their greatest asset, organizations must work to fulfill their human resource needs of individual people in their companies.

If organizations want to be successful at implementing information technology projects, they need to understand the importance of project human resource management and take actions to make effective use of people.

12.2 What is Project Human Resource Management

Project human resource management includes the processes required to make the most effective use of people involved with a project. Human resource management includes all project stakeholders: sponsors, customers, project team members, support staff, vendors supporting the project, and so on. The major processes involved in human resources management includes:

- **Organizational Planning**, which involves identifying, assigning, and documenting project roles, responsibilities, and reporting relationships.
Key outputs of this process include roles and responsibilities assignments, often shown in a matrix form, and an organizational chart for the projects.
- **Staff acquisition**, which involves getting the needed personnel assigned to and working on the project. Getting personnel is one of the crucial challenges of information technology projects.

- **Team Development**, which involves building individual and group skills to enhance project performance. Building individual and group skills is also a challenge for many information technology projects.

12.3 Keys To Managing People

Industrial – organizational psychologists and management theorists have devoted much research and thought to field of managing people at work.

Psychosocial issues that affect how people work and how well they work include motivation, influence and power, and effectiveness.

12.3.1 Motivation Theories:-

Maslow developed a **hierarchy of needs** – a pyramid structure illustrating Maslow’s theory that people’s behaviours are guided or motivated by a sequence of needs.

At bottom of the hierarchy are physiological needs. Once the physiological needs are satisfied, safety needs guide the behaviour. Once safety needs are satisfied, social needs come to forefront, and so on up the hierarchy.

The order of these needs and their relative sizes in pyramid are significant.

It suggests that each level of the hierarchy is a prerequisite for the level above.

It is not possible for a person to consider self- actualization if he or she has not addressed basic needs concerning security and safety.

A satisfied need is no longer a motivator



People in an emergency situation, such as a flood or hurricane , are not going to worry about personal growth. Personal survival will be their main motivation. Once a particular need is satisfied, however, it no longer serves as a potent motivator of behaviour.

Each layer in pyramid is smaller than the previous layer.

The issues in each level are of greater value than issues in the preceding level , which presumably have been satisfied .

Issues higher in the hierarchy that are associated with the needs that are currently not satisfied are more important than earlier issues, although , by definition , they are less urgent than issues lower in hierarchy.

The bottom four needs in Maslow's hierarchy – physiological, safety, social, and esteem needs are referred to as deficiency needs, and the highest level, self – actualization, is considered to be a growth needs.

Self – actualized people are characterized as problem – focused , having an appreciation for life, being concerned about personal growth, and having the ability to have peak experiences.

Most people working on an information technology project will probably have their basic physiological and safety needs met.

To motivate project team members, the project manager need to understand each person's motivation with regard to social, esteem, and self – actualization or growth needs. Team members new to a company and city might be motivated by social needs. To address social needs, some companies organize gatherings and social events for new workers in information technology.

Other project members may find these events to be an invasion of personal time they would rather spend with their friends and family or working on an advanced degree.

Maslow's hierarchy conveys a message of hope and growth. People can work to control their own destinies and naturally strive to achieve higher and higher needs in Maslow's hierarchy.

Project managers should know something about team members' professional and personal lives so they can provide motivational incentives that meet their team members' needs.

12.3.2 Influence and Power

Many people working on a project do not report directly to project managers, and project managers often do not have control over project staff who report to them.

For example, people are free to change jobs. If given assignments they do not like, many workers will simply quit or transfer to other departments or projects.

Wilemon investigated what approaches to dealing with workers project manager use and how those approaches relate to project success

1. Authority – the legitimate hierarchical right to issue orders.
2. Assignment – the project manager’s perceived ability to influence a worker’s later work assignment.
3. Budget – the project manager’s perceived ability to authorize other’s use of discretionary funds.
4. Promotion – the ability to improve a worker’s position.
5. Money – the ability to increase a worker’s pay and benefits.
6. Penalty – the project manager’s perceived ability to dispense or cause punishment.
7. Work challenge – the ability to assign work that capitalizes on a worker’s enjoyment of doing a particular task, which taps an intrinsic motivational factor.
8. Expertise – the project manager’s perceived special knowledge that others deem important.
9. Friendship – the ability to establish friendly personal relationships between the project manager and others.

Influence is related to power. Power is the potential ability to influence behaviour to get people to do things they would not otherwise do.

The five main types of power include

1. Coercive power, which involves using punishment threats, or other negative approaches to get people to do things they do not want to do.
2. Legitimate power, which is getting people to do things based on a position of authority.
3. Expert power, which involves using one’s personal knowledge and expertise to get people to change their behaviour. If people perceive that project managers are experts in certain situations, they will follow their suggestions.

4. Reward power , which involves using incentives to induce people to do things. Rewards can include money, status, recognition, promotions , special work assignments, or other means of rewarding someone for desired behaviour.
5. Referent power, which is based on an individual's personal charisma. People hold someone with referent power in very high regard and will do what they say based on their regard for the person.

12.3.3 Improving Effectiveness

Project managers can apply Covey's seven habits to improve effectiveness on projects.

1. **Be proactive:** - Project managers must be proactive and anticipate and plan for problems and inevitable changes on projects. They can also encourage their team members to be proactive in working on their project activities.
2. **Begin with the end in mind:** - Covey asks people to visualize their own funerals to help them focus on how they really want to be remembered in their lives.
3. **Put first things first:** - Project managers need to spend a lot of time working on important and not urgent activities , such as developing the project plan, building relationships with major project stakeholders, and mentoring project team members. They also need to avoid focusing only on important yet urgent activities.
4. **Think win / win:** - When you use a win / win paradigm , parties in potential conflict work together to develop new solutions that make them all winners.
5. **Seek first to understand, then to be understood:** - Empathic listening is listening with the intent to understand. To really understand other people , you must learn to focus on others first. Two-way communication is critical for project managers so they can really understand their stakeholders' needs and expectations.
6. **Synergize :-** A project team can synergize by collaborating to create products that are much better than a collection of individual efforts.
7. **Sharpen the saw :-** When you practice sharpening the saw , you take time to renew yourself physically , spiritually, mentally and socially. The practice of self- renewal helps people avoid burnout.

12.4 Organizational Planning

Organizational planning for a project involves identifying, documenting, and assigning project roles, responsibilities, and reporting relationships. This process generates an organizational chart for the project, roles, and responsibility assignments, often shown in a matrix form called a responsibility assignment matrix (RAM), and a staffing management plan.

Before creating an organizational chart for a project, senior management and the project manager must identify what types of people are really needed to ensure project success.

If the key to success lies in having the best Java programmers you can find, the organizational planning should reflect that need. If the real key to success is having top-notch project manager and team leaders whom people respect in the company, that need should drive organizational planning.

After identifying important skills and the types of people needed to staff a project, the project manager should work with senior management and project team members to create an organizational chart for the project.

The project personnel include a deputy project manager, subproject managers and teams.

Deputy project managers fill in for project managers in their absence and assist him or her as needed, similar to the role of a vice president.

Subproject managers are responsible for managing the subprojects into which large project might be divided.

This structure is typical for large projects, with many people working on a project, clearly defining and allocating project work is essential. Smaller information technology projects usually do not have deputy project managers or subproject managers.

The project manager might have just team leaders reporting directly to them.

Figure 12.2 : Sample organizational chart for a Large Information Technology Project

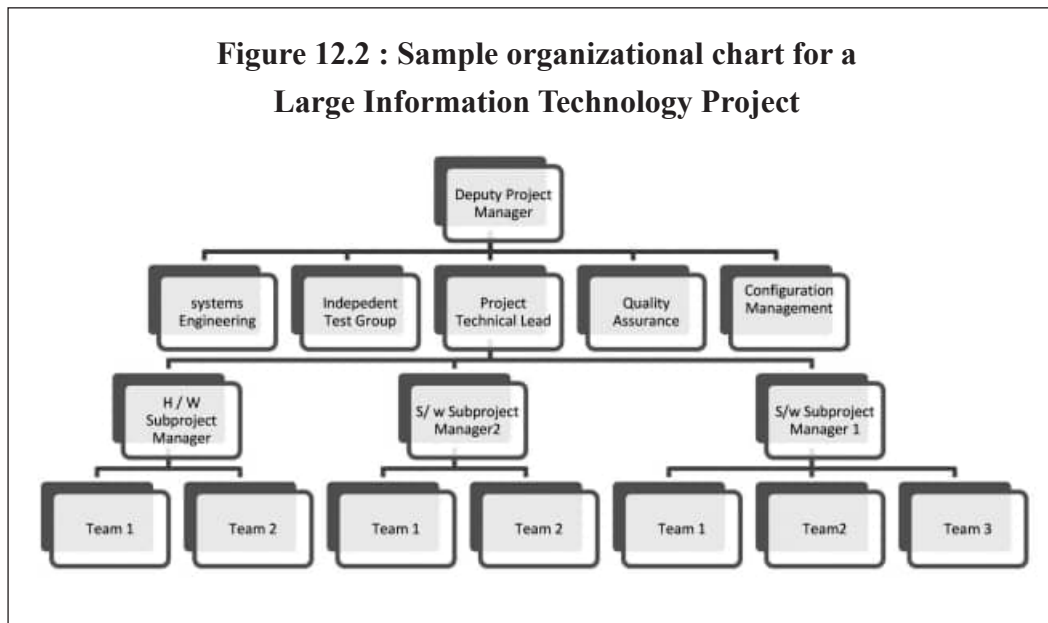


Figure provides a framework for defining and assigning work. This process consists of four steps :

1. Finalizing the project requirements
2. Defining how the work will be accomplished
3. Breaking down the work into manageable elements
4. Assigning work responsibilities

The work definition and assignment process is carried out during the proposal and start up phase of a project.

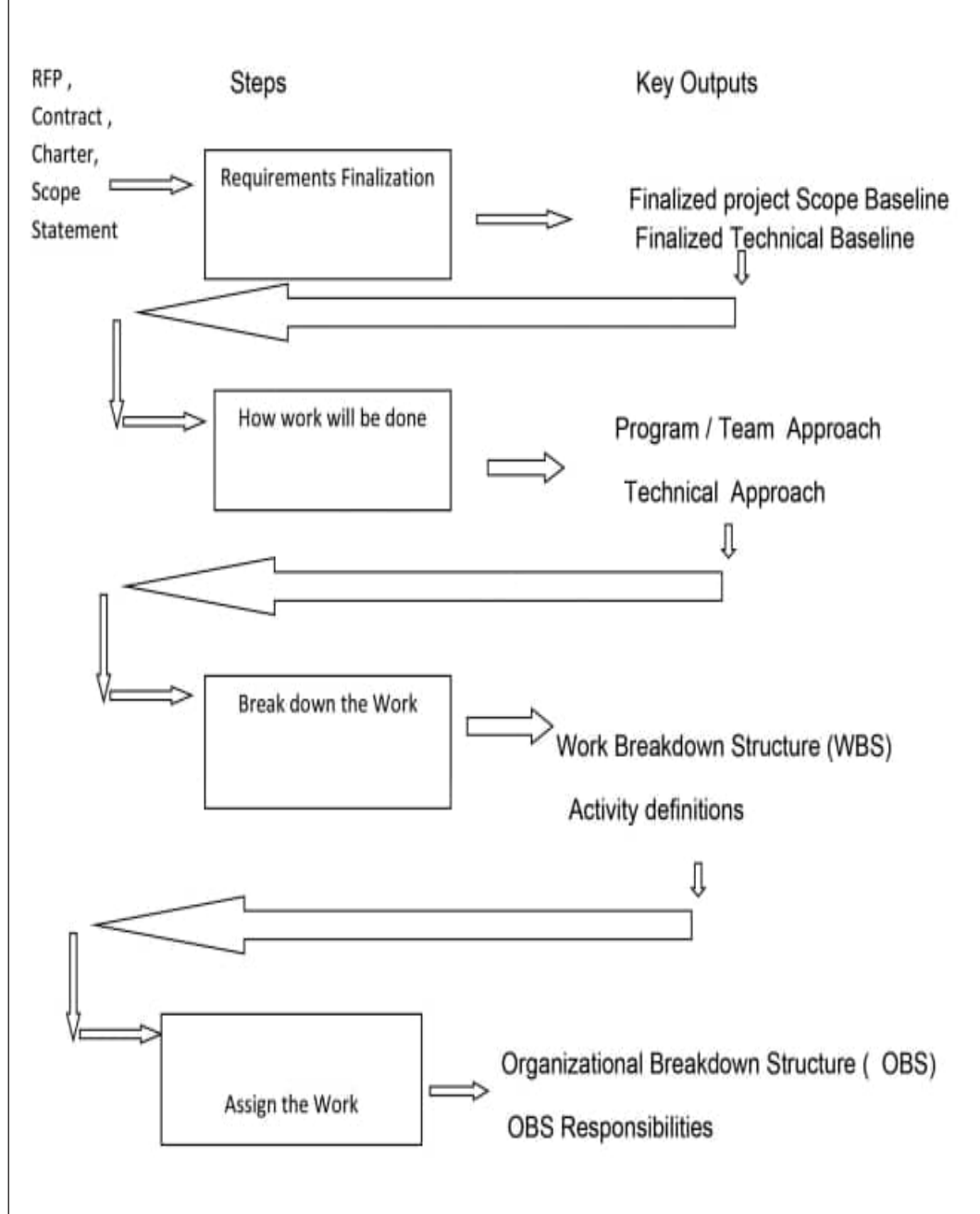
A Request for Proposal (RFP) or draft contract often provides the basis for defining and finalizing work requirements, which are then, documented in a final contract and technical baseline.

If there is not an RFP, then internal project charter and scope statement would provide the basis for defining and finalizing work requirements.

The project team leaders then decide on a technical approach for how to do the work.

Once the project team has decided on a technical approach , they develop a work breakdown structure (WBS) to establish manageable elements of work.

Once the project manager and project team have broken down the work into manageable elements, the project manager assigns work to organizational units.

Figure 12.3: Work Definition and Assignment Process

An **organizational breakdown structure (OBS)** is a specific type of organizational chart that shows which organizational units are responsible for which work items.

The OBS can be based on a general organizational chart and then broken down into more detail based on specific units within departments in the company or units in any subcontracted companies.

After developing an OBS, the project manager is in a position to develop a responsibility assignment matrix (RAM) .

Figure 12.4 :- Sample Responsibility Assignment Matrix (RAM)

WBS activities 									
OBS		1.1.1	1.1.2	1.1.3	1.1.4	1.1.5	1.1.6	1.1.7	1.1.8
Units 	System En- gineering	R	R P						
	Software De- velopment			R P					
	Hardware Development				R P				
	Test Engi- neering	P							
	Quality As- surance					R P			
	Configura- tion Management						R P		
	Integrated Logistics Support							P	
	Training								R P
R= Responsible organizational unit P= Performing organizational unit									

A responsibility assignment matrix (RAM) is a matrix that maps the work of the project as described in the OBS.

The RAM allocates work to responsible and performing organizations, teams, or individuals, depending on the desired level of detail.

For smaller projects, it would be best to assign individual people to WBS activities. For very large projects, it is more effective to assign the work to organizational units or teams.

In addition to using a RAM to assign detailed activities, you can also use a RAM to define general roles and responsibilities on projects. This type of RAM can include the stakeholders in the project.

RAM shows whether stakeholders are accountable or just participants in part of a project and whether they are required to provide input, review , or sign off on parts of a project.

This simple tool can be very effective way for project manager to communicate roles and expectations of important stakeholders on projects.

Figure 12.5: RAM Showing Stakeholder Roles					
Stakeholders					
Items	A	B	C	D	E
Unit Test	S	A	I	I	R
Integration Test	S	P	A	I	R
System Test	S	P	A	I	R
User Acceptance Test	S	P	I	A	R
A= Accountable; P= Participant; R= Review Required; I = Input Required S= Sign – off Required					

Another output of organization planning is a staffing management plan, it describes when and how people will be added to and taken off the project team.

It can be formal or informal plan, and the level of detail may vary based on type of project.

For example, if an information technology project is projected to need one hundred people on average over a year , the staffing management plan would describe types of people needed to work on project, such as Java programmers, business analysts, technical writers , and so on , the number of each type of person needed each month.

The staffing management plan often includes a resource histogram – a column chart that shows the number of resources assigned to a project over time.

The column represents the number of people needed in each area- Java programmers, business analysts, technical writers, managers, administrative staff, database analysts, and testing specialists.

After determining the project staffing needs, the next steps in project human resource management are to acquire the necessary staff and then develop the project team.

12.5 Issues In Project Staff Acquisition And Team Development

During the late 1990s, the information technology job market became extremely competitive. Job market projections indicate that this highly competitive job market is likely to continue well into 21st century. Thus, finding qualified information technology professional – staff acquisition – is critical.

It is important to assign the appropriate type and number of resources to work on projects at the appropriate times.

Resource loading and levelling is a project human resource management technique that addresses this issue.

Even if you can find great workers and assign them well, if those professionals cannot work together as a team, the project will not be successful.

Once professional have been hired to staff a project, team development becomes an important issue.

12.5.1 Staff Acquisition

After developing a staffing management plan, project managers must work with other people in their organizations to assign particular personnel to their projects or to acquire additional human resource needed to staff the project.

Project managers with strong influencing and negotiating skills are often good at getting internal people to work on their projects.

Organization must ensure that people are assigned to the projects that best fit their skills and the needs of the organization.

Organizations that do a good job of staff acquisition have good staffing plans. These plans describe the number and type of people who are currently in the organization and the number and type of people anticipated to be needed for the project based on current and upcoming activities.

An important component of staffing plans is maintaining a complete and accurate inventory of employees' skills.

If there is a mismatch between the current mix of people's skills and needs of organization, it is the project manager's job to work with senior management , human resource managers, and other people in the organization to address staffing and training needs.

It is also important to have good procedures in place for hiring subcontractors and recruiting new employees. Since the Human Resources Department is normally responsible for hiring people, project managers, must work with their human resources managers to address any problems recruiting appropriate people.

It is also priority to address retention issues, especially for information technology professionals.

For example, several consulting companies give their employees one dollar for every hour a new person they helped hire works. It provides an incentive for current employees to help attract new people and to keep both them and the person they recruited working at that company.

Another approach that several companies are taking to attract and retain information technology professionals is to provide benefits based on their personal needs.

For example, some people might want to work only four days a week or have the option of working a couple of days a week from home.

12.6 Resource Loading and Leveling

12.6.1 Resource loading refers to amount of individual resources an existing schedule requires during specific time periods. This helps project managers develop a general understanding of the demands a project will make on organization's as well as on individual, resources.

A histogram can be very helpful in determining staffing needs or in identifying staffing problems.

A resource loading histogram can show when work is being over- allocated to a certain person or group. Over- allocation means more resources than are available are assigned to perform work at a given time.

This histogram illustrates how much one individual is assigned to work on the software launch project each week.

The percentage on vertical axis represent the percentage of available time is allocated to work on the project.

Time is represented in weeks along the horizontal axis.

12.6.2 Resource leveling is a technique for resolving conflicts by delaying tasks. It is a form of network analysis in which resource management concerns drive scheduling decisions (start and finish dates).

The main purpose of resource levelling is to create a smoother distribution of resource usage. Project managers examine the network diagram for areas of slack or float and to identify resource conflicts.

Over- allocation is one type of resource conflict. If a certain resource is Over-allocated, the project manager can change the schedule to remove resource over-allocation. If a certain resource is under-allocated , the project manger can change the schedule to try to improve the use of the resource.

Resource levelling, therefore aims to minimize period– by- period variation in resource loading by shifting tasks within their slack allowances.

Resource levelling has several benefits:

First, when resources are used on a more constant basis, they require less management . For example, it is much easier to manage a part time project member who is scheduled to work twenty hours per week on a project for the next three months than it is to manage the same person who is schedule to work ten hours one week, forty the next, five the next, and so on.

Second, resource levelling may enable project managers to use a just-in-time inventory type of policy for using subcontractors or other expensive resources. For example, a project manager might want to level resources related to work that must be done by particular subcontractor like testing consultants. This levelling might allow the project to use four outside consultants full-time to do testing for four months instead of spreading the work out over more time or needing to use more than people. The latter approach is usually more expensive.

Third, resource levelling results in fewer problems for project personnel and Accounting Departments. Increasing and decreasing labor levels and particular a person with expertise in a particular area is only assigned to a project two project those same days, they cannot work well together. The Accounting Department might complain when subcontractors charge a higher rate possible.

Finally, resource levelling often improves morale. People like to have some stability in their jobs. It is very stressful for people to not know from week to week or even day to day what projects they will be working on and with whom they will be working.

Project management software can automatically level resources. However, the project manager must be careful in using the results without making adjustments. Automatic levelling often pushes out the project's completion date. Resources may also be reallocated to work at times that are inappropriate with other constrains. A wise project manager would have one of his or her team members who is proficient in using the project management software ensure that levelling is done appropriately.

12.7 Team Development

Even if you have successfully recruited enough skilled people to work on a project, you must ensure that people can work together as a team to achieve project, goals. Many information technology projects have had very talented individuals working on them. However, it takes teamwork to successfully complete most projects. The main goal of team development is to help people work together more effectively to improve project performance. There is an extensive body of literature on team development. This section will highlight a few important tools and techniques for team development including training, teambuilding activities, and reward and recognition systems. It will also provide general advice on the effective use of teams.

12.7.1 Training

Project managers often recommend that people take specific training course to improve individual and team development. For example, Sarah from the opening case had gone through training in dealing with difficult people. She was familiar with the mirroring technique and felt comfortable using that approach with Ben. Many other people would not have reacted so quickly and effectively in the same situation. If Ben and Sarah did reach agreement on what actions they could take to resolve the F-44 program's information technology problems, it might result in a new project to develop and deliver a new system for Ben's group. If Sarah became the project manager for this new project, she would understand the need for special training in interpersonal skill for specific people in her and Ben's departments. Individuals could take special training classes to improve their personal skills. If Sarah thought the whole project team could benefit from taking training together to learn to work as a team, she could arrange for a special team-building session for the entire project team and key stakeholders.

12.7.2 Team Building Activities

Many companies provide in-house team-building training activities, and many also use specialized services provided by external companies that specialize in this area. Two common approaches to team building activities include using physical challenges and psychological preference indicator tools.

Several organizations have teams of people go through certain physically challenging activities to be help them develop as a team. Military certain basic training or boot camps provide one example. Men and women who wish to join the military must first make it through basic training, which often involves several

strenuous physical activities such as rappelling off towers, running and marching in full military gear, going through obstacle courses, obstacle courses, passing marksmanship training, and mastering survival training. Many companies use a similar approach by sending teams of people to special locations where they work as a team to navigate white water rapids, climb mountains or rocks, participate in paintball exercises, and so on.

Even more companies have teams participate in more mental team-building activities where they learn more about themselves, each other, and how to most effectively work as a group. It is important for people to understand and value each other's differences in order to work effectively as a team.

A common exercise used in these team-building activities is the **MyersBriggs Type Indicator (MBTI)**- a popular tool for determining personality preferences. During World War II, Isabel B. Myers and Katherine C. Briggs developed the first version of the MBTI based on psychologist Carl Jung's theory of psychological type. The four dimensions of psychological type in MBTI include:

1. **Extrovert / Introvert (E / I)** : The dimension also signifies where people draw their energy – from other people (extroverts) or from inside themselves (introverts).
2. **Sensation / Intuition (S / N)** : The second dimension relates to the manner in which you gather information. Sensation type people take in facts, details and reality and describe themselves as practical. Intuitive type people are imaginative , ingenious, and attentive to hunches or intuition. They describe themselves as innovative and conceptual.
3. **Thinking /Feeling (T/F)**: The third dimension represents thinking judgment and feeling judgment. Thinking judgment is objective and logical, and feeling judgment is subjective and personal.
4. **Judgment/perception (J/P)**: The fourth dimension concerns people's attitude towards structure. Judgment type people like closure and task completion. They tend to establish deadlines and take them seriously, expecting others to do the same. Perceiving types prefer to keep things open and flexible. They regard deadlines more as a signal to start rather than complete a project and do not feel that work must be done before play or rest begins.

12.7.3 Reward and Recognition Systems

If management rewards teamwork, they will promote or reinforce people to work more effectively in teams. Some companies offer bonuses, trips, or other rewards to work groups that meet or exceed company or project goals.

In a project setting, project managers can recognize and reward people who willingly work overtime to meet an aggressive schedule objective or go out of their way to help a teammate.

Project managers should not reward people who work overtime just to get extra pay or as a result of their own poor work planning.

12.7.4 General Advice on Teams

Effective project managers must be good team builders. Suggestions for ensuring that teams are productive include the following:

- Be patient and kind with your team, and assume the best about people. Do not assume that your team members are lazy and careless.
- Fix the problem instead of blaming people. Help people work out problems by focusing on behaviours.
- Establish regular, effective meetings. Focus on meeting projects objectives and producing positive results.
- Limit the size of work team to three to seven members.
- Plan some social activities to help project team members and other stakeholders get to know each other better. Make the social events fun and not mandatory.
- Stress team identity. Create traditions that team members enjoy.
- Nurture team members and encourage them to help each other. Identify and provide training that will help individuals and the team as a whole become more effective.
- Acknowledge individual and group accomplishments.

12.8 Using Software To Assist In Human Resource Management

By defining and assigning resources in Project, you can:

- Keep track of the whereabouts of resources through stored information and reports on resource assignments.
- Identify potential resource shortages that could force a project to miss scheduled deadlines and possibly extend the duration of a project.
- Identify underutilized resources and reassign them, which may enable you to shorten a project's schedule and possibly reduce costs.
- Use automated levelling to make level resources easier to manage.

12.9 Summary

People are most important assets in organizations and on projects.

Psychosocial issues that affect how people work and how well they work include motivation, influence and power, and effectiveness.

The major processes involved in project human resource management include organizational planning, staff acquisition, and team development. Organizational planning involves identifying, assigning, and documenting project roles, responsibilities, and reporting relationships. A responsibility assignment matrix (RAM) is a key tool for defining roles and responsibilities on projects.

Resource loading shows the amount of individual resources an existing schedule requires during specific time frames. Histograms are often used to show resource loading and to identify the over-allocation of resources.

Resource levelling is a technique for resolving resource conflicts, such as over-allocated resources, by delaying tasks. Leveled resources require less management, lower costs, produce fewer personnel and accounting problems, and often improve morale.

One of the most important skills of a good project manager is team development. Teamwork helps people work more effectively to achieve project goals. Project managers can recommend individual training to improve skills related to teamwork, organize team-building activities for the entire project team and key stakeholders, and provide reward and recognition systems that encourage teamwork.

Sample Questions

1. Briefly summarize work done by Maslow, Wilemon. How do their theories relate to project management?
2. Give examples of different ways to have influence on projects. Which do you think would be most effective with you ?
3. Describe the work definition and assignment process.
4. Discuss key issues related to staff acquisition and team building.
5. How could you use resource loading and resource levelling to ensure that a project runs smoothly?

Reference Books:

1. Information Technology Project Management by Jack T Marchewka Wiley India publication.
2. Managing Information Technology Project 6th edition , by Kathy Schwalbe, Cengage Learning publication.
3. Software Engineering Project Management by Richard H. Thayer Wiley India Publication.



MANAGING CHANGE, RESISTANCE, AND CONFLICT

Unit Structure

13.0 Objective

13.1 Introduction

13.2 Developing The Core Team

13.3 The Nature Of Change

13.3.1 Change Has an Impact

13.3.2 Change Is a Process

13.3.3 Change Can Be Emotional

13.4 The Change Management Plan

13.4.1 Assess Willingness, Readiness and Ability to Change

13.5 Dealing With Resistance And Conflict

13.5.1 Resistance

13.5.2 Conflict

13.6 Ethics And Leadership

13.6.1 Ethics

13.6.2 Leadership

13.7 Ethical Leadership

13.8 Making Sound Ethical Decisions

13.9 Summary

13.0 Objective

- Describe the discipline of organizational change management and its role in assessing the organization's readiness and capability to support and assimilate a change initiative.
- Describe how change can be viewed as process and identify the emotional responses people might have when faced with change.
- Describe the framework for managing change that will be introduced.
- Apply the concepts and idea in order to develop a change management plan.
- Discuss the nature of resistance and conflict and apply several techniques for dealing with conflict and resistance in an efficient and effective way.

13.1 Introduction

IT projects are planned organizational change. Organizations are made up of people, and the implementation of the IT project's product can change the way people work, affect the way they share information, and alter their relationships. Whether you are outside consultant or work for an internal IS department within the organization, your presence will often be met with suspicion and hostility because you will be viewed as a person who has the potential to disrupt their stability. You are an agent of change.

Dealing with people issues, or soft side of technology, is an idea that most technical people do not enjoy. It is human nature to focus on what we can accomplish with minimal conflict or on what we can control. Implementing a network of computers that communicate with each other or getting a program to work properly may be much easier and less stressful than dealing with resistance and conflict during system development.

Although a system may include the required features and perform as intended; but it can lead to a system that is a technical success but an organizational failure.

Implementation of the new system is a technical challenge. The system must be moved from the development environment to a production environment and properly tested before going live. The people within the organization, however, must be prepared for the impact that the new system will have on them. It is easy to

underestimate this impact and given human nature, downplay the response people will have. Manager and technical people may be given to false beliefs:

- People want this change.
- We see the need for helping our people adjust, but we had to cut something.
- They have two choices: They can change or they can leave.

These statements reflect the view that it is easier to gain compliance than it is to gain acceptance. This supposition is faulty because it assumes that everyone will comply and that compliance will be long lasting .This results may be quite different:

- The change may not occur
- People will comply for a time and then do things to get around the change.
- Users will accept only a portion of the change.

The full benefits of the project are never realized or are realized only after a great deal of time and resources have been expended.

Acceptance by the users of the system is much more powerful and longer lasting than compliance which means we need to ensure that the people within the organization are prepared properly before the system is implemented. The change management is the area of IT project management that helps smooth the transition and implementation of the new IT solution.

The transforming of the organization so it is aligned with the execution of a chosen corporate business strategy. It is the management of the human element in a large-scale change project.

13.2 Developing The Core Team

Peeter knew that he had to work well with people on his management team and that they, in turn, had to work well with other project stakeholders. Peeter had several strategy meetings with Arvid Lee, the head of the Information Services people on the ResNet project , and Kathy (Krammer) Christenson , the head of software application development for the ResNet interface. They worked together to determine how to motivate different people involved in the project . Kathy came from marketing area, and she knew that the sales agents and other people in marketing were very excited by themes , office decorations, gifts, contents, and so on.

Arvid knew that he and most other members of the Information services Department would think those types of things were silly , but he fully supported the approach. Arvid motivated the information services staff by providing them with challenging work and keeping them informed of project progress.

Peeter was a hands- on manager and he felt that every single person involved in ResNet was important. He gave a lot of responsibility to Arvid and Kathy, keeping in constant communication with them and relying heavily on their judgements.

Peeter kept their project sponsor, Fay Beauchine, well informed of project's progress. He went out to talk to lower-level people Involved IN ResNet. He wanted to know what everyone was doing and how h or she thought things were going.

Peeter had an excellent memory and made a point of having focused discussions with all ResNet stakeholders.

Peeter provided resources to people as they were needed. In addition to hiring professional actors to help his team run better meetings , he sent people to technical training classes. He shared his experience and advice with others, and more importantly, he worked with them to develop critical thinking skills. Peeter knew that a key part of his job was supporting and developing his staff.

13.3 The Nature of Change

In order to effectively plan and manage organizational change, it is important to understand the impact of change, how change may be viewed as a process, and the emotional, behavioural patterns of change.

13.3.1 Change Has an Impact

At any given time, we must deal with changes that effect us. These changes may result from world or local events, the organizations we are part of, or personal decisions and relationships.

Whether we view change as positive (anticipation) or negative (dread), there is a certain amount of stress that accompanies each change.

For example, let's say that you will graduate this semester and start a new job that requires you to move to distinct city. Although you may be looking forward to leaving school and earning some real money, you may still feel some apprehension. After all, you will have to leave your circle of family and / or friends and the familiarity of your present environment. Once you arrive in your new city, you will need to find a new place to live, make new friends, and become familiar with

your new job, the company, and its people. Moving to a new city is relatively easy compared to the transition. The move itself is a change that will occur fairly quickly; the transition required to adjust to the change takes longer.

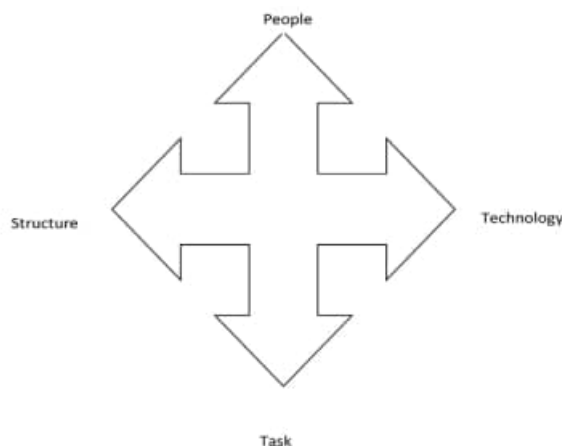
Assimilation is a process of adapting to change and determines our ability to handle current and future change. Problems occur when we cannot assimilate change fast enough. When an individual passes a certain threshold, he or she become stressed out and exhibit dysfunctional behaviours. Therefore, it is important to manage the assimilation of change to keep things below the change threshold.

Organizations are made up of people and these people have any number of personal changes going on in their lives. Changes proposed by an organization will certainly affect the way people work and the relationships that have become established. Although these organizational changes will have to be assimilated by each person, the organization must assimilate change similar to an individual. After all, organizations are made up of people. Therefore, each change adopted by an organization must be assimilated and managed within the change threshold. Just like people, organizations can exhibit dysfunctional behaviours.

Change within an organization can affect different things in different ways. Leavitt's model, suggests that changes in people, technology, task, or organizational structure can influence or impact the other areas.

These four components are interdependent, where a change in one can result in a change in the others. For example, a change in the organization's technology (e.g., implementing a new information system) can impact the people within the organization (e.g. new roles, responsibilities) as well as the tasks the individual's perform (i.e. the work they perform), and the organization's structure (i.e. , formal or informal).

Figure 13.1 : Leavitt's Model of organization change



13.3.2 Change Is a Process

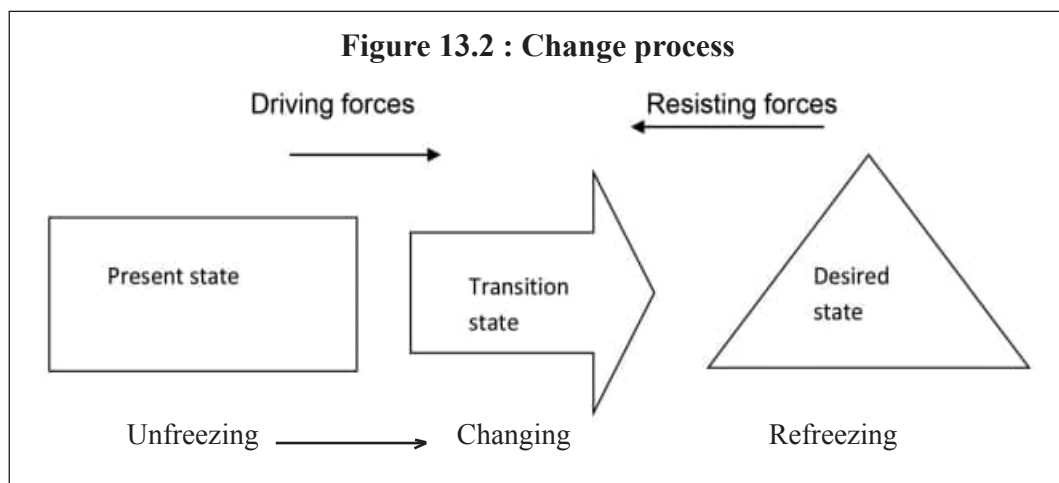
Lewin's basic model includes three concepts: unfreezing, changing and refreezing.

The present state represents an equilibrium or state quo. To change from current state, there must be driving forces both to initiate and to motivate the change. This requires an unfreezing, or an altering of the current state's habits, perceptions and stability.

Figure shows a transition from present state to the desired state. This state is sometimes referred to as the neutral zone and can be a limbo or emotional wilderness for many individuals.

Problems arise when managers do not understand, expect, or acknowledge the neutral zone. Those in the organization who act and support the driving forces for the change may be likely to rush individuals through the transition. This rushing often results in confusion on the part of those in the neutral zone, and the resisting forces (i.e. the emotional and psychological barriers) tend to push those individuals back to their present state. People do not like being caught in the neutral zone. They may try to revert to the original status quo or escape.

Escape may mean leaving the organization or resistance to the change initiative altogether. In addition, individuals who find themselves in the neutral zone too long may attempt to create a compromise in which only a portion of the change is implemented. This compromise will only result in missed opportunities and sets a bad precedence for the next change initiative.



People do not necessarily resist change; they resist losses and endings. Unfreezing, or moving from the current state, means letting go of something. Therefore, viewing change from Lewin's model suggests that beginning a change starts with an ending of the present state.

Transition through the neutral zone also means a loss of equilibrium until an individual or organization moves to the desired state. Once there, it is important that the attitudes, behaviours, and perceptions be refrozen so that the desired state becomes the new status quo and equilibrium for the individuals involved.

13.3.3 Change Can Be Emotional

An individual may have an emotional response to a change when the change is perceived as a significant loss or upsets a familiar or well-established equilibrium.

It still provides some valuable insight for understanding how people may react to significant changes that affect their lives. The five stages include:

Denial – The first stage is characterized by shock and denial. It is a common reaction when a person is given first notice of a change that will have significant impact. The initial news, however, provides a beginning for understanding the full impact of change that is about to take place.

Anger – Once a person gets over the initial shock of the announcement, he or she may become angry toward others, or even the messenger. The reaction is to blame whoever is responsible for creating the change. Although anger is more active emotional response, it can be a cathartic expression when people are allowed to vent their emotions. Keep in mind that there is a difference feeling anger and acting out in anger. While having feelings is always acceptable, the latter never is.

Bargaining – In the third stage, the person is no longer angry. In fact, he or she may be quite cooperative and may try to make deals to avoid the change. For example, the person who lost her job may begin making promises that she will “double my productivity” or “take a cut in pay” in order to avoid being let go. A person may look for ways to extend the status quo, or the present equilibrium, by trying to “work things out.”

Depression – Once a person admits that the change is inevitable, he or she may understand the full impact of the change and may enter the fourth stage – depression. This stage generally occurs when there is an overwhelming sense of the loss of the status quo. Although losing a job involves losing income, most people become depressed because they also lose the identity associated with their job.

Acceptance – The last stage is when a person comes to grips with the change. A person does not have to like the change in order to accept it. This fifth stage has more to do with one’s resolve that the change is inevitable and must be dealt with. Acceptance is an important part of ending the status quo and getting on with a new state.

13.4 The Change Management Plan

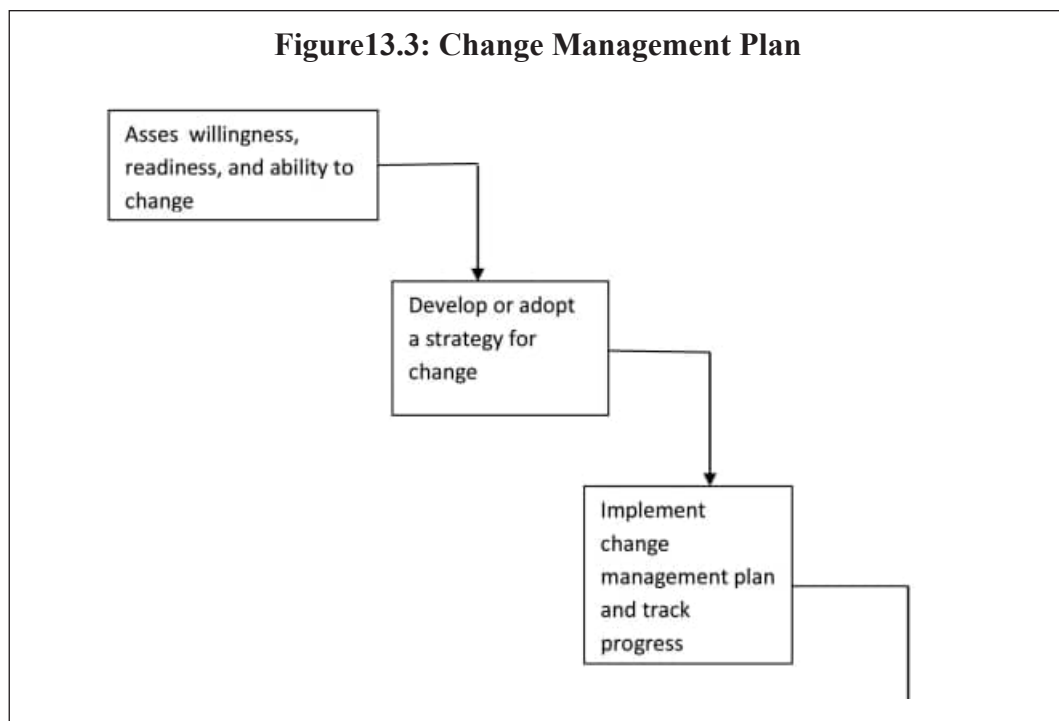
The key to any organizational change is to plan for and manage the change and the associated transition effectively. Depending on the size and impact of the change initiative, the change management plan can be an informal or formal document; however, the project team and sponsor should address and be clear on several important areas.

13.4.1 Assess Willingness, Readiness and Ability to Change

The first step to developing a change management plan is to assess the organization's willingness, readiness, and ability to change.

Conner defines several roles or players involved in a change initiative : sponsor ,change agent s , and targets.

Sponsor: - The sponsor can be an individual or group that has the willingness and power, in terms of authority and making resources available, to support the project. Although this person or group is often the project sponsor, an **initiating sponsor** may hand off the project to a **sustaining sponsor**. More specifically, after making the decision to fund and support the project, the initiating sponsor may become completely removed from the project. Without the support of a sustaining sponsor, the project will eventually lose steam and direction. Therefore, the sustaining sponsor must become the primary sponsor for the project.



A major portion of the organization's ability and willingness to support the change rests with the sponsor's commitment to the project and the associated change that will impact the organization. This commitment may be in terms of how they communicate with the rest of the organization, how they deal with challenges and issues, and the amount and quality of resources made available. In addition, sponsors must be effective leaders. If the project fails because the organization cannot adopt to change, the project's envisioned value to the organization is lost and the sponsor's credibility is diminished.

Change Agents – In the most basic terms, the change agents will be the project manager and team; however, others from inside or outside the organization may be involved as well. An agent may be individual or group responsible for making the change happen in order to achieve the project's goal and objectives. Change agents report directly to the sponsor and must be able to diagnose problems, plan to deal with these issues and challenges effectively, and act as a conduit of communication between sponsor and the targets of change. The ability to sustain the change associated with the IT project rests largely with the change agents. They must be ready and properly prepared to meet the challenges they face.

Targets - The target is individual or group that must change. In general, these may be the users of the new system or those who will use or be directly involved with the final product of the change effort and who play a critical role in the ultimate success of the project.

Although the project sponsors and change agents play important roles in supporting and carrying out the change effort, the dynamics associated with the targets of change become the most critical. Therefore, the willingness, ability and readiness to change also rest largely with the change targets.

It may require:

- 1) Clarifying the real impacts of the change
- 2) Understanding the breadth of change
- 3) Defining what's over and what's not and
- 4) Determining whether the rules for success have changed.

The project team and sponsor often do not think about how the planned change and transition will really affect people within the organization. Change often brings about endings and a sense of loss of control. The project team and sponsor should take time to think about what various individuals or groups stand to lose.

For example, perceptions of loss may include power, relationships with other people, stability, or even control. As a result, people may become confused and disoriented.

13.5 Dealing With Resistance And Conflict

13.5.1 Resistance:-

Resistance should be anticipated from the outset of the project. Rumours and gossip will add fuel to the fire, and the change effort can run out of steam if those affected by the change begin to resist. Resistance can be either overt, in the form of memos, meetings, and so on, or covert, in the form of sabotage, foot dragging, politicking etc. Once the change is compromised, management and the project team will lose credibility, and the organization may become resistant to all future changes.

Resistance can arise for many valid reasons. For example, someone may resist an information system because the response time is slow or because it does not provide the features or functionality that were originally specified as part of the requirements.

On the other hand, resistance due to cultural or behavioural reasons is harder to rationalize, but still can keep a project from reaching its intended goal. People may resist change even though they understand that the change will be beneficial.

For example:

- Some people perceive the change as requiring more time and energy than they are willing to invest.
- Sometimes, people feel that a change will mean giving up something that is familiar, comfortable, and predictable.
- People may be annoyed with the disruption caused by change, even if they know that it will be beneficial in long run.
- People may believe that the change is being imposed on them externally, and their egos will not tolerate being told what to do.
- In addition, people may resist because of the way the decision to change was announced or because it was forced on them.

Resistance is human nature and a natural part of any change process. Understanding what an individual or group perceives a loss is the first step to dealing with resistance effectively.

Because the project team and sponsor are agents of change, the project team and sponsor have had the luxury of knowing about the change early and, therefore, have had a time to become used to it. The rest of the organization, however, may

learn about the change much later and , therefore , may not be at the same place for digesting the change,

Subsequently, it is important that the project team and sponsor listen to what the rest of the organization is saying. Instead of arguing and trying to reason, it is better to allow people to vent their anger and frustration. Again, having defined a boundary of what is and what is not part of the change can help deal with stressful conflict situations. Keep in mind that empathizing or sympathizing with an individual is not same as agreeing with them.

13.5.2 Conflict

Closely associated with resistance is the concept of conflict. Conflicts arise when people perceive that their interests and values are challenged or not met. Conflict management focuses on preventing, managing, or resolving conflicts. Therefore, it is important to identify potential conflicts as early as possible so that the conflict can be addressed. Although conflict can be positive and help form new ideas and establish commitment , negative conflict left unresolved can lead to damaged relationships, mistrust, unresolved issues, continued stress, dysfunctional behaviour , and low productivity and morale.

There are three different views of conflict

Traditional view – It considers conflict in a negative light and feels that conflict should be avoided. Conflict, according to this view, leads to poor performance, aggression, and devastation if left to escalate. Therefore, it is important to manage conflict by suppressing it before it occurs or eliminating it as soon as possible. Harmony can be achieved through authoritarian means, but root causes of the conflict may not be adequately addressed.

Contemporary view - The contemporary view, on the other hand, suggests that conflict is inevitable and natural. Depending on how conflict is handled, conflict can be either positive or negative.

Positive conflict among people can stimulate ideas and creativity; however, negative conflict can have damaging effects if left unresolved. Therefore, positive conflict should be encouraged while keeping negative conflict in check.

Interactionist view - It holds that conflict is an important and necessary ingredient for performance. Although the contemporary view accepts conflict, if too harmonious or tranquil. Subsequently, the project manager should occasionally stir the pot in order to encourage conflict to an appropriate level so that people engage in positive conflict. This may, however, be a fine line to walk for many project

managers. Although someone who plays the role of the devil's advocate can be effective in many situations, people may become annoyed when it is used in every situation, or used ineffectively.

To better understand the nature of conflict, projects can fit one, or a combination, of three categories:

- Conflicts associated with the goals, objectives, or specifications of the project.
- Conflicts associated with the administration, management structures, or underlying philosophies of the project.
- Conflicts associated with the interpersonal relationships among people based on work ethics, styles, egos, or personalities.

Avoidance- Avoiding conflict focuses on retaining, withdrawing, or ignoring conflict. Sometimes, a cooling –off period may be a wise choice, especially when emotions and tempers are high. Avoidance may be appropriate when you can't win, the stakes are low, or gaining time is important. However, it may not be useful when the immediate, successful resolution of an issue is required.

Accommodation - Accommodation, or smoothing, is an approach for appeasing the various parties in conflict. This approach may be useful when trying to reach an overall goal when the goal is more important than the personal interests of the parties' involved. Smoothing may also be effective when dealing with an issue that has low risk and low return or when in a no-win situation. Because accommodation tends to work only in the short run, conflict may reappear in another form later on.

Forcing – When using this approach, a person uses his or her dominant authority to resolve the conflict. This approach often results in a one-sided or win- lose situation in which one party gains at the other's expense. This approach may be effective when no common ground exists, or when time is of the essence. Forcing resolution may, however, cause the conflict to redevelop later because people dislike having a decision or someone else's views imposed on them.

Compromise – Compromise includes aspects of both forcing and accommodation; it gives up more than forcing and less than accommodation. Compromise is essentially bargaining – one person or group gives up something in exchange for gaining something else. In this case, no party actually wins and none actually loses, so that some satisfaction is gained from resolution of the conflict. This approach may be useful when the risks and rewards are moderately high. Unfortunately, important aspects of a project may be compromised as a means of achieving short-term results- for example, quality standards may be compromised in order to meet

the project's schedule.

Collaboration - When the risks and benefits are high, collaboration may be the best approach for dealing with conflict. This approach requires confronting and attempting to solve the problem by incorporating different ideas, viewpoints, and perspectives. The focus of collaboration is learning from others and gaining commitment, trust, respect, and confidence from the various parties involved. Collaboration takes time and requires a sincere desire to work out a mutually acceptable solution. In addition, it requires a willingness to engage in a good- faith problem- solving process that facilitates open and honest communication.

Each conflict situation is unique and the choice of an approach to resolve conflict depends on:

Type of conflict and its relative importance to the project.

Position of power to resolve the conflict.

Whether the emphasis is on maintaining the goals or objectives of the project or maintaining relationships.

13.6 Ethics And Leadership

13.6.1 Ethics – The principles, norms, and standards of conduct governing an individual or group – focuses on conduct. We expect employer to establish guidelines for work- related conduct, including what time to arrive and leave the workplace, how customers are to be treated, and how quickly work should be done. Many employers spend a lot of time and money developing policies for a range of employee activities, from how to fill out expense reports to what kind of client gifts are acceptable, to what constitutes a conflict of interest or bribe.

13.6.2 Leadership – Leaders are often ordinary people who help guide others along journeys rather than follow well- worn paths.

The five practices of leadership to become a more effective and successful project leader.

Model the way – A leader's behaviour wins respect, not his or her title or position within the organization. You must find your own voice based on your personal values and beliefs. Your words and deeds must be consistent so that you convey the right message. People follow the person first, not the plan.

Inspire a shared vision – An exemplary leader has an exciting vision or dream that acts as a force for inventing the future. In turn, vision should inspire people so they become committed to a purpose. A leader must engage in dialogue, not monologue, to understand the hopes and dreams of others and gain their support.

Challenge the process - Exemplary leaders do not rely on fate or luck. They venture out and accept challenges. Leaders are pioneers who challenge the status quo by seeking out new opportunities to innovate, grow, and improve. However, most leaders do not create, develop, or come up with new products, services or processes. Leaders accept risk and failure, they minimize it by taking and encouraging others to make incremental steps. They strive for small wins to boost confidence, commitment, and learning.

Enable others to act - Leaders provide an environment that makes it possible for others to do good work. People should feel empowered and motivated to do their best, feel a sense of ownership, and take pride in what they do. A leader enables others to act by giving power away , not hanging onto it .People should be made to feel strong and capable; otherwise , they will not give their best or stay around very long.

Encourage the hearts – Exemplary leaders rally others to carry on by encouraging the heart. It can be simple gesture such as a thank –you note or something more dramatic like a marching band , the leader should show appreciation for people’s contributions and create a culture of celebration that recognizes those accomplishments.

13.7 Ethical Leadership

An ethical leader is someone who makes it clear that bottom-line results are important, but only if they can be achieved in an ethical manner. Moreover, research suggests that when a culture is viewed as being ethical , employees tend to engage in fewer unethical behaviours, are more committed to the organization, and more willing to report problems to management.

Some of common ethical situations are

Human resource situations - Project leaders must ensure that qualified team members are recruited and retained. Issues that can lead to ethical situations include discrimination, privacy, sexual, or other types of harassment, as well as appraisals, discipline, hiring, firing, and layoff policies and decisions.

Conflicts of interest – Many organizations have policies that define situations that are not acceptable. Conflict of interest issues can include such things as overt or subtle bribes or kickbacks, as well as relationships that could question your impartiality. Moreover, as project team member, you will gain access to confidential and private information that could be of value to you or someone else.

Confidence - Confidence issues can include a wide variety of issues such as product safety or reliability, truth in advertising, and special fiduciary responsibilities that require special commitments or obligations to the client or other project stakeholders; they can create special ethical considerations that can affect your relationship with project stakeholders. Trust can erode when your fairness, honesty, or respect come into question.

Corporate resources- As a project team member, your company may give you an e-mail address with their domain, business cards, and stationary with the corporate letterhead. Therefore, your e-mail and organizational stationery should only be used for business purposes. Make sure that your personal opinion and corporate opinions are not in conflict. In addition, company equipment and services should only be used for business purposes.

13.8 Making Sound Ethical Decisions

It includes the following considerations:

- **Gather the facts** – facts may not be clear or readily available.
- **Define the ethical issue**- A number of related and interwoven issues may complicate things once we begin to realize them.
- **Identify the affected stakeholders** - It involves identifying those who will be affected by the decision as well as any benefits or harm that will come to them .
- **Identify the consequences** – It may be impossible to identify every consequence, you can still get a good idea of whether the good of your decision or action outweighs the bad.
- **Identify the obligations** - It involves identifying the obligations you have to the affected stakeholders. It may help to think of obligations in terms of values, principles, character , and outcomes.

- **Consider your character and integrity** – It is to ask yourself , would you feel comfortable if your action or decision were to appear on the news. Do you want people to say that you were a person of integrity or not.
- **Think creatively about potential actions-** Explaining the impact and consequences of the action may be enough to change the supervisor's mind .If that does not work, other options might include making sure that someone else knows what the original (and correct) test results report .Each situation is unique and therefore requires a unique and sometimes creative approach . Talking to someone outside the situation can be a great help.
- **Check your gut-** Empathy should not be discarded over logic because it can raise a warning flag that someone might be harmed. Making a decision based solely on emotion is probably not a good idea either. Cheking your gut should be a final stage of process that provides confidence in action or decision.

13.9 Summary

Understanding organizational change is an important area for IT project management. IT professionals may concentrate exclusively on the technical or hard side of the project at the expense of the people, or soft side.

Lewin's model of change helps us to understanding that we must unfreeze the current status , or status quo, and then move through a transitional state until the new or desired state is reached. Then, these new behaviours must be refrozen so that they become ingrained as the new status quo.

Understanding the effects of change on the organization allows us to develop a change management plan. It focus on assessing the organization's willingness, readiness, and ability to change. This assessment should focus on change sponsor's commitment to supporting the change an associated transition and on the change agents ability to facilitate the change.

Both resistance and conflict are a natural part of change process and should be anticipated from the outset of the project. Resistance can arise for many reasons and take many forms. Although the traditional view of conflict suggests that all conflict is bad and should be avoided or resolved as soon as possible , the contemporary and interactionist views of conflict support the idea that positive conflict can stimulate new idea and improve creativity.

Sample Questions

1. What is the difference between a change and a transition?
2. Why is having a change management plan important?
3. Define assimilation and its importance to understanding how people deal with change.
4. Describe the five practices for exemplary leadership.
5. What is the definition of ethics ?

Reference Books:

4. Information Technology Project Management by Jack T Marchewka Wiley India publication.
5. Managing Information Technology Project 6th edition , by Kathy Schwalbe, Cengage Learning publication.
6. Software Engineering Project Management by Richard H. Thayer Wiley India Publication.



SOFTWARE RISK MANAGEMENT

Unit Structure

14.0 Objective

14.1 Introduction

14.2 Risk Analysis And Management

14.2.1 Steps of Risk Analysis and Management

14.3 Types Of IT Project Risk

14.4 Reactive Vs Proactive Risk Strategies

14.4.1 Reactive risk strategies

14.4.2 Proactive risk strategies

14.5 Risk Identification

14.5.1 Assessing Overall Project Risk

14.5.2 Risk Components and Drivers

14.6 Risk Projection

14.6.1 Developing a Risk Table

14.7 Risk Assessment

14.8 Risk Mitigation, Monitoring, And Management

14.9 The RMMM Plan

14.10 Summary

14.0 Objectives

- Define risk identification and causes , effects , and nature off project risks.
 - Apply several analysis techniques that can be used to prioritize and analyze various project risks.
 - Describe the various risk strategies
 - Describe the risk monitoring and control
 - Describe risk evaluation in terms of how the entire risk management process should be evaluated in order to identify best practices.
-

14.1 Introduction

Project risk management provides an early warning system for problems that need to be addressed or resolved .Although risk has a certain negative connotation , project stakeholders should be vigilant in identifying opportunities. Although many associate uncertainty with threats, it is important to keep in mind that there is uncertainty when pursuing opportunities, as well.

Plan risk management determines how to approach and plan the project risk management activities.an output of this process is the development of arisk management plan.

Deciding which risks impact the project. Risk identification generally includes many of the project stakeholders and requires an understanding of the project's goal , as well as the project's scope ,schedule,budget , and quality objectives.

Developing procedures and techniques to reduce the threats of risks, while enhancing the likelihood of opportunities.

14.2 Risk Analysis and Management

Risk analysis and management are series of steps that help a software team to understand and mange uncertainty.

Many problems can plague a software project .A risk is a potential problem –it might happen it might not.

14.2.1 Steps of Risk Analysis and Management

1. Recognizing what can go wrong is the first step , called “risk identification.”
 2. Each risk is analyzed to determine the likelihood that it will occur and the damage that it will do if it does occur.
-

3. Risks are ranked, by probability and impact.
4. Finally, a plan is developed to manage those risks with high probability and high impact.

In short, the four steps are

- Risk Identification
- Risk Projection
- Risk Assessment
- Risk Management

Risk always involves two characteristics a set of risk information sheets is produced.

- **Uncertainty** - the risk may or may not happen; that is, there are no 100 % probable risks.
- **Loss** – if the risk becomes a reality, unwanted consequences or losses will occur.

14.3 Types of it Project Risk

What types of risks are we likely to encounter as the software is built??

- **Project risks** threaten the project plan. That is, if project risks become real, it is likely that project schedule will slip and that costs will increase. Project risks identify potential budgetary, schedule, personnel (staffing and organization), resource, customer, and requirements problems and their impact on a software project.
- **Technical risks** threaten the quality and timeliness of the software to be produced. If a technical risk becomes a reality, implementation may become difficult or impossible.
- **Business risks** –threaten the viability of the software to be built. Business risks often jeopardize the project or the product. Candidate for top five business risks are
 - ☐ Market risk
 - ☐ Strategic risk
 - ☐ Management risk and
 - ☐ Budget risk

- **Known risks** are those that can be uncovered after careful evaluation of the project plan.
- **Predictable risks** are extrapolated from past project experience (e.g., staff turnover, poor communication with customer, dilution of staff effort as ongoing maintenance requests are serviced).
- **Unpredictable risks** – are the joker in the deck. They can do occur, but they are extremely difficult to identify in advance.

14.4 Reactive vs Proactive Risk Strategies

14.4.1 Reactive risk strategies

1. Reactive risk strategies follows that the risks have to be tackled at the time of their occurrence.
2. No precautions are to be taken as per this strategy.
3. They are meant for risks with relatively smaller impact.

14.4.2 Proactive risk strategies

1. Proactive risk strategies follows that the risks have to be identified before start of the project.
2. They have to be analyzed by assessing their probability of occurrence ,their impact after occurrence , steps to be followed for its precaution.
3. They are meant for risks with relatively higher impact.

14.5 Risk Identification

Risk identification is asystematicattemptto the project plan (estimates, schedule, resource loading, etc.) .By identifying known and predictable risks, the project manager takes a first step toward avoiding them when possible and controlling them when necessary.

One method for identifying risks is to create a risk item checklist. The checklistcan be used for risk identification and focuses on some subset of known and predictable risks a in the following generic categories:

- **Product size** – risks associated with overall size of the software to be built or modified.

- **Business impact** – risks associated with constraints imposed by management or the marketplace.
- **Customer characteristics**—risks associated with sophistication of the customer and developer’s ability to communicate with the customer in a timely manner.
- **Process definition** - risks associated with the degree to which the software process has been defined and is followed by the development organization.
- **Development environment** – risks associated with the availability and quality of the tools to be used to build the product.
- **Technology to be built** – risks associated with the complexity of the system to be built and the “newness” of the technology that is packaged by the system.
- **Staff size and experience** - risks associated with the overall technical and project experience of the software engineers who will do the work.

14.5.1 Assessing Overall Project Risk

Is the software project we are working on at serious risk?

The questions are ordered by their relative importance to success of a project.

1. Have top software and customer managers formally committed to support the project?
2. Are end –users enthusiastically committed to the project and the system/product to be built?
3. Are requirements fully understood by the software engineering team and their customers?
4. Have customers been involved fully in the definition of requirements?
5. Do end –users have realistic expectations?
6. Is the project scope stable?
7. Does the software engineering team have the right mix of skills?
8. Are project requirements stable?
9. Does the project team have experience with the technology to be implemented?
10. Is the number of people on the project team adequate to do the job?
11. Do all customer/ user constituencies agree on the importance of the project and on the requirements for the system/ product to be built?

If any one of these questions is answered negatively, mitigation, monitoring, and management steps should be instituted without fail.

14.5.2 Risk Components and Drivers

The risk components are defined in the following manner:

- **Performance risk**- the degree of uncertainty that the product will meet its requirements and fit for its intended use.
- **Cost risk** - the degree of uncertainty that the project budget will be maintained.
- **Support risk** – the degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance.
- **Schedule risk**- the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time.

The impact of each risk driver on the risk component is divided into one of four impact categories– negligible, marginal, critical, or catastrophic.

Components Category		Performance	Support	Cost	Schedule
Catastrophic	1	Failure to meet the requirement would result in mission failure		Failure results in increased costs and schedule delays with expected values in excess of \$ 500K	
	2	Significant degradation to non-achievement of technical performance	Nonresponsive or unsupportable software	Significant financial shortage budget overrun likely	Unachievable IOC
Critical	1	Failure to meet the requirement would degrade system performance to a point where mission success is questionable.		Failure results in operational delays and /or increased costs with expected value of \$ 100K to \$500K	
	2	Some reduction in technical performance	Minor delays in software modifications	Some shortage of financial resources ,possible overruns	Possible slippage in IOC

Marginal	1	Failure to meet the requirement would result in degradation of secondary mission.		Costs, impacts, and /or recoverable schedule slips with expected value of \$1K to \$100K	
	2	Minimal to small reduction in technical performance	Responsive software support	Sufficient financial resources	Realistic achievable schedule
Negligible	1	Failure to meet the requirement would create inconvenience or non-operational impact		Error results in minor cost and/ or schedule impact with expected value of less than \$ 1K	
	2	No reduction in technical performance	Easily supportable software	Possible budget under run	Early achievable IOC

- 1) The potential consequence of undetected software errors or faults.
- 2) The potential consequence if the desired outcome is not achieved.

14.6 Risk Projection

Risk projection , also called risk estimation, attempts to rate each risk in two ways – the likelihood or probability that the risk is real and the consequences of the problems associated with the risk , should it occur.

The project planner, along with other managers and technical staff , performs four risk projection activities.

1. Establishing a scale that reflects the perceived likelihood of a risk.
2. Delineating the consequences of the risk.
3. Estimating the impact of the risk of the project and the product.
4. Noting the overall accuracy of the risk projection so that there will be no misunderstandings.

14.6.1 Developing a Risk Table

Risks	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse the planned	PS	70%	2	
End –users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	

- i) A risk table provides a project manager with a simple technique for risk projection.
- ii) A sample risk table is illustrated in Figure. The risk table is sorted by probability and impact to rank risks.
- iii) A project team begins by listing all risks in the first column of the table. This can be accomplished with the help of risk item checklists referenced. Each risk is categorized in the second column (e.g. PS implies a project size risk, BU implies business risk).
- iv) The probability of occurrence of each risk is entered in the next column of the table. The probability value for each risk can be estimated by team members individually. Individual team members are polled in round –robin fashion until their assessment of risk probability begins to converge.
- v) Next, the impact of each risk is assessed.
- vi) The categories for each of four risk components – performance, support, cost, and schedule- are averaged to determine an overall impact value.

- vii) Once the first four columns of the risk table have been completed , the table is sorted by probability and by impact. High –probability, high –impact risks percolate to the top of the table, and low –probability risks drops drop to bottom.
- viii) This accomplishes first- order risk prioritization. The project manager studies the resultant sorted table and defines a cutoff line. The cutoff line implies that only risks that lie above the line will be given further attention.
- ix) Risks that fall below the line are re- evaluated to accomplish second- order prioritization.
- x) A risk factor that has a high impact but a very low probability of occurrence should not absorb a significant amount of management time.
- xi) All risks that lie above the cut off line must be managed.
- xii) The column labeled RMMM contains a pointer into a Risk Mitigation, Monitoring and Management Plan or alternatively, a collection of risk information sheets developed for all risks that lie above the cutoff.

How do we assess the consequences of a risk?

The following steps are recommended to determine the overall consequences of a risk.

- i) Determine the average probability of occurrence value for each risk component.
- ii) Using figure determine the impact for each component based on criteria shown.
- iii) Complete the risk table and analyze the results as described in the preceding sections.

The overall risk exposure RE, is determined using the following relationship

$$RE = P * C$$

Where P is probability of occurrence for a risk, and C is cost to the project should the risk occur.

14.7 Risk Assessment

At this point in the risk analysis process we have established a set of triples of the form:

$$[r_i, l_i, x_i]$$

Where r_i is risk, l_i is the likelihood (probability) of the risk, and x_i is the impact of the risk.

During risk assessment, we further examine the accuracy of the estimates that were made during risk projection, attempt to rank the risks that have been uncovered, and begin thinking about ways to control and / or avert risks that are likely to occur.

Therefore, during risk assessment we perform the following steps:

- i) Define the risk referent levels for the project.
- ii) Attempt to develop a relationship between each (r_i, l_i, x_i) and each of the referent levels.
- iii) Predict the set of referent points that define a region of termination, bounded by a curve or areas of uncertainty.
- iv) Try to predict how compound combinations of risks will affect a referent level.

14.8 Risk Mitigation, Monitoring, and Management

- An effective strategy must consider three issues:
 - Risk avoidance
 - Risk monitoring
 - Risk management and contingency planning
- High staff turnover in any organization will have a critical impact on project cost and schedule.
- To mitigate the risk, project management must develop a strategy for reducing turnover.

Among the possible steps to be taken are

- Meet with current staff to determine causes for turnover * Mitigate those causes that are under our control before the project starts.

- Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave.
- Organize project teams so that information about each development activity is widely dispersed.
- Define documentation standards and establish mechanisms to be sure that documents are developed in a timely manner.
- Conduct peer reviews of all work.
- Assign a backup staff member for every critical technologist.

14.9 The Rmmm Plan

1. A risk management strategy can be included in the software project plan or the risk management steps can be organized into a separate Risk Mitigation, Monitoring and Management Plan.
2. The RMMM plan documents all work performed as part of risk analysis and is used by the project manager as part of the overall project plan
3. Some software teams do not develop a formal RMMM document. Rather, each risk is documented individually using a risk information sheet(RIS).
4. Once RMMM has been documented and the project has begun, risk mitigation and monitoring steps commence.

Risk monitoring is a project tracking activity with three primary objectives:

- 1) To assess whether predicted risks do, in fact, occur
- 2) To ensure that risk aversion steps defined for the risk are being properly applied
- 3) To collect information that can be used for future risk analysis. In many cases, the problems that occur during a project.

Risk information sheet			
Risk ID : PO 2-4-32	Date: 5/9/02	Prob: 80%	Impact : high
Description : Only 70 percent of the software components scheduled for reuse will, in fact, be Integrated into the application. The remaining functionality will have to be custom developed.			
Refinement / context : Sub condition 1: certain reusable components were developed by a third party with no knowledge of internal design standards. Sub condition 2: The design standard for component interface has not been solidified and may not conform to certain existing reusable components. Sub condition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.			
Mitigation / monitoring: 1. Contact third party to determine conformance with design standards. 2. Press for interface standards completion; consider component structure when deciding on interface protocol. 3. Check to determine number of components in sub condition 3 category; check to determine if language support can be acquired.			
Management / contingency plan / trigger : RE computed to be \$ 20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly. Trigger : Mitigation steps unproductive as of 7/ 1 /02			
Current status : 5/12/02: Mitigation steps initiated.			
Originator: D. Gagne		Assigned : B.Laster	

14.10 Summary

Risk identification should include identifying both threats and opportunities. Since most risks are interrelated and can affect the project in different ways, the project stakeholders should take a broad view of project risks.

Risk assessment allows the project stakeholders to determine what risks require a response. The goal of project risk management is not to avoid each and every risk at all costs, but to make well- informed decisions as to which risks are worth taking

and which risks require a response. A well informed decision requires an analysis of the probability of a particular risk occurring and its likely impact.

Risk strategies define how the project stakeholders will respond to risk. It include (1) accepting or ignoring the risk (2) avoiding the risk (3) mitigating or reducing the likelihood and / or impact of the risks , and (4) transferring the risk to someone else.

Once the risk response plan has been completed and the project is underway, the various risks identified must be monitored and controlled. This process should include vigilance on the identified and unidentified threats and or opportunities. As these risks present themselves, project risk owners should make resources available and respond to risk in an appropriate manner, as outlined in the risk response plan.

Risk evaluation provides a key to learning and identifying best practices. A formal and documented evaluation of a risk response or episode can help an organization evaluate its entire risk management approach and provide insight for future project teams that may have to deal with a similar risk in the future.

Sample Questions:

- 1) What is project risk?
- 2) What is project risk management?
- 3) What is the purpose of risk analysis and assessment?
- 4) What is risk monitoring and control?
- 5) Why can identifying IT project risks be difficult?
- 6) Discuss the risk strategies?

Reference Books:

11. Software Engineering, 5th and 7th edition, by Roger S Pressman, McGraw Hill publication.
12. Managing Information Technology Project 6thedition , by Kathy Schwalbe , Cengage Learning publication.
13. Software Engineering 3rd edition by KK Agarwal , Yogesh Singh , New Age International publication.
14. Information Technology Project Management by Jack T Marchewka Wiley India publication.
15. Software Engineering for students: A Programming Approach by Douglas Bell, Pearson publication.
16. Software Engineering Project Management by Richard H. Thayer Wiley India Publication.



SOFTWARE RELIABILITY ISSUES

Unit Structure

- 15.0 Objective
- 15.1 Introduction
- 15.2 A Framework For Technical Software Metrics
- 15.3 The Attributes Of Effectives Software Metrics
- 15.4 Metrics For The Analysis Model
- 15.5 Metrics For The Design Model
- 15.6 Component-Level Design Metrics
- 15.7 Metrics For Source Code
- 15.8 Metrics For Testing
- 15.9 Metrics For Maintenance
- 15.10 Summary

15.0 Objective

- Software engineers use technical metrics to help them build higher-quality software.
- There will always be a qualitative element to the creation of computer software. The problem is that qualitative assessment may not be enough.
- A software engineer needs objective criteria to help guide the design of data, architecture, interfaces, and components.
- Technical metrics provide a basis from which analysis, design, coding, and testing can be conducted more objectively and assessed more quantitatively.

15.1 Introduction

The first step in the measurement process is to derive the software measures and metrics that are appropriate for the representation of the software that is being considered. Data required to derive the formulated metrics are collected. Once computed, appropriate metrics are analyzed based on pre- established guidelines and fast data. The results of the analysis are interpreted to gain insight into the quality of the software, and the results of the interpretation lead to modification of the work products arising out of analysis, design, and code or test.

Software quality factors focus on three important aspects of a software product :its operational characteristics ,its ability to undergo change, and its adaptability tonew environments.

Reliability refers to extent to which a program can be expected to perform its intended function with required precision.

Correctness refers to extent to which a program satisfies its specification and fulfils the customer's mission. Efforts are required to locate and fix an error in a program and modify an operational program and to test a program to ensure that it performs its intended functions.

15.2 A Framework for Technical Software Metrics

Technical metrics for the software

1. The danger of attempting to find measures which characterize so many different attributes is that inevitable the measures have to satisfy conflicting aims. This is counter to the representational theory of measurement.
2. In spite of the intuitive connections between the internal structure of software products [technical metrics] and its external product and process attributes, there have actually been very few scientific attempts to establish specific relationships. There are a number of reason why this is so; the most commonly cited is the impracticality of conducting relevant experiments.

Measurement Principles

Roche suggests a measurement process that can be characterized by five activities:

- **Formulation.** The derivation of software measure and metrics that are appropriate for the representation of the software that is being considered.

- **Collection.** The mechanism used to accumulate data required to derive the formulated metrics.
- **Analysis.** The computation of metrics and the application of mathematical tools.
- **Interpretation.** The evaluation of metrics results in an effort to gain insight into the quality of the representation.
- **Feedback.** Recommendations derived from the interpretation of technical metrics transmitted to the software team.

The principles that can be associated with the formulation of technical metrics are

- The objectives of measurement should be established before data collection begins.
- Each technical metric should be defined in an unambiguous manner.
- Metrics should be derived based on a theory that is valid for the domain of application (e.g., metrics for design should draw upon basic design concepts and principles and attempt to provide an indication of the presence of an attribute that is deemed desirable).
- Metrics should be tailored to best accommodate specific products and processes.

Although formulation is a critical starting point, collection and analysis are the activities that drive the measurement process.

Roche suggest the following principles for these activities:

- Whenever possible, data collection and analysis should be automated.
- Valid statistical techniques should be applied to establish relationships between internal product attributes and external quality characteristics (e.g. is the level of architectural complexity correlated with the number of defects reported in production use?).
- Interpretative guidelines and recommendations should be established for each metric.
- In addition to these principles, the success of a metrics activity is tied to management support. Funding, training, and promotion must all be considered if a technical measurement program is to be established and sustained.

15.3 The attributes of Effectives Software Metrics

Ejiogu defines a set of attributes that should be encompassed by effective software metrics. The derived metric and the measures that lead to it should be

- **Simple and computable.** It should be relatively easy to learn how to derive the metric, and its computation should not demand inordinate effort or time.
- **Empirically and intuitively persuasive.** The metric should satisfy the engineer's intuitive notions about the product attribute under consideration (e.g., a metric that measures module cohesion should increase in value as the level of cohesion increases).
- **Consistent and objective.** The metric should always yield results that are unambiguous. An independent third party should be able to drive the same metric value using the same information about the software.
- **Consistent in its use of units and dimensions.** The mathematical computation of the metric should use measures that do not lead to bizarre combinations of units. For example, multiplying people on the project teams by programming language variables in the program results in a suspicious in a suspicious mix of units that are not intuitively persuasive.
- **Programming language independent.** Metrics should be based on the analysis model, the design model, or the structure of the program itself. They should not be dependent on the vagaries of programming language syntax or semantics.
- **An effective mechanism for high-quality feedback.** That is, the metric should provide a software engineer with information that can lead to a higher quality end product.

15.4 Metrics for the Analysis Model

Technical work in software engineering begins with the creation of the analysis model. It is at this stage that requirements are derived and that a foundation for design is established. Therefore, technical metrics that provide insight into the quality of the analysis model are desirable.

Metrics for specification quality

Characteristics that can be used to assess the quality of the analysis model and the corresponding requirements specification: specificity (lack of ambiguity), completeness, correctness, understandability, verifiability, internal and external consistency, achievability, concision, traceability, modifiability, precision and reusability.

Davis suggests that each can be represented using one or more metrics. For example, we assume that there are nr requirements in a specification, such that

$$nr = nf + nnf$$

Where

nf is the number of functional requirements and

nnf is the number of non-functional (e.g., performance) requirements.

To determine the specificity (lack of ambiguity) of requirements, Davis suggests a metric that is based on the consistency of the reviewers' interpretation of each requirement:

$$Q1 = nui / nr$$

Where

nui is the number of requirements for which all reviewers had identical interpretations.

The closer the value of Q to 1, the lower is the ambiguity of the specification.

The completeness of functional requirements can be determined by computing the ratio

$$Q2 = nu / [ni_ns]$$

Where

nu is the number of unique function requirements,

ni is the number of inputs (stimuli) defined or implied by the specification, and

ns is the number of states specified.

The $Q2$ ratio measures the percentage of necessary functions that have been specified for system. However, it does not address non-functional requirements. To incorporate these into an overall metric for completeness, we must consider the degree to which requirements have been validated:

$$Q3 = nc / [nc + nnv]$$

Where

nc is the number of requirements that have been validated as correct and

nnv is the number of requirements that have not been validated.

15.5 Metrics for the Design Model

We examine some of the more common design metrics for computer software. Each can provide the designer with improved insight and all can help design to evolve to a higher level of quality.

Architectural Design Metrics

Architectural design metrics focus on characteristics of the program architecture with an emphasis on the architectural structure and the effectiveness of modules. These metrics are black box in the sense that they do not require any knowledge of the inner working of a particular software component.

The U.S. Air Force uses information obtained from data and architectural design to derive a design structure quality index (DSQI) that ranges from 0 to 1. The following values must be ascertained to compute the DSQI:

- **S1**=the total number of modules defined in the program architecture.
- **S2**=the number of modules whose correct function depends on the source of data input or that produce data to be used elsewhere (in general, control modules, among others, or that produce data to be used elsewhere (in general, control modules, among others, would not be counted as part of S2).
- **S3**=the number of modules whose correct function depends on prior processing.
- **S4**=the number of database items (includes data objects and all attributes that define objects).
- **S5**=the total number of unique database items.
- **S6**=the number of database segments (different records or individual objects).
- **S7**=the number of modules with a single entry and exit (exception processing is not considered to be a multiple exit).

Once values S1 through S7 are determined for a computer program, the following intermediate values can be computed:

Program structure: D1, where D1 is defined as follows: If the architectural design was developed using a distinct method (e.g., data flow-oriented design or object-oriented design), then D1=1, otherwise D1=0.

Module independence: $D2=1_ (S2 / S1)$

Modules not dependent on prior processing: $D3=1_ (S3 / S1)$

Database size: $D4=1_ (S5 / S4)$

Database compartmentalization: $D5=1_ (S6 / S4)$

Module entrance/exit characteristic: $D6=1_ (S7 / S1)$

With these intermediate values determined, the DSQI is computed in the following manner.

$$DSQI = \sum_{i=1}^6 w_i D_i \quad (19-5)$$

Where

$i=1$ to 6,

w_i is the relative weighting of the importance of each of the intermediate values, and $w_i=1$ (if all D_i are weighted equally, then $w_i=0.167$).

The value of DSQI for past designs can be determined and compared to a design that is currently under development. If the DSQI is significantly lower than average, further design work and review are indicated. Similarly, if major changes are to be made to an existing design, the effect of those changes on DSQI can be calculated.

15.6 Component-Level Design Metrics

Components-level design metrics focus on internal characteristics of a software engineer to judge the quality of a component-level design.

1. Cohesion metrics

Bieman and ott define a collection of metrics that provide an indication of the cohesiveness of a module. The metrics are defined in terms of five concepts and measures: Data slice. Stated simply, a data slice is a backward walk through a module that looks for data values that affect the module location at which the walk began. It should be noted that both program slices (which focus on statements and conditions) and data slices can be defined.

Data tokens. This variables defined for a module can be defined as data tokens for the module.

Glue tokens. This set of data tokens lies on one more data slice.

Superglue tokens. These data tokens are common to every data slice in a module.

Stickiness. The relative stickiness of a glue token is directly proportional to the number of data slices that it binds.

2. Coupling metrics.

Module coupling provides an indication of the “connectedness” of a module to other modules, global data, and the outside environment.

For **data and control flow coupling**,

di = number of input data parameters

ci = number of input control parameters

do = number of output data parameters

co = number of output control parameters

For **global coupling**,

gd= number of global variables used as data

gc=number of global variables used as control

For **environmental coupling**,

W= number of modules called (fan-out)

r = number of modules calling the module coupling indicator, **mc**, is defined in the following way:

$$mc = k / M$$

where **k=1**, a proportionality constant 8 and

$$M = di + (a_ci) + do + (b_co) + gd + (c_gc) + w + r$$

Where **a=b=c=2**.

The higher the value of **mc**, the lower is the overall module coupling.

3. Complexity metrics

Complexity metrics can be used to predict critical information about reliability and maintainability of software systems from automatic analysis of source code [or procedural design information]. Complexity metrics also provide feedback during the software project to help control the [design activity]. During testing and maintenance, they provide detailed information about software modules to help pinpoint area of potential instability.

The most widely used (and debated) complexity metric for computer software is cyclomatic complexity, originally developed by Thomas Mc Cabe and discussed earlier.

4. Interface Design Metrics

Sears suggests that layout appropriateness (LA) is a worthwhile design metric for human/computer interfaces. A typical GUI uses layout entities-graphic icons, text, menus, windows, and the like-to assist the user in completing tasks.

For a specific layout (i.e., a specific GUI design), cost can be assigned to each sequence of actions according to the following relationship:

$$\text{Cost} = [\text{frequency of transition (k)} * \text{cost of transition (k)}]$$

Where, k is a specific transition from one layout entity to the next as a specific task is accomplished. The summation occurs across all transitions for a particular task or set of tasks required to accomplish some application function. Cost may be characterized in terms of time, processing delay, or any other reasonable value, such as the distance that a mouse must travel between layout entities. Layout appropriateness is defined as

$$\text{LA} = 100 * [(\text{cost of LA-optimal layout}) / (\text{cost of proposed layout})]$$

Where LA= 100 for an optimal layout.

15.7 Metrics for Source Code

Software science assigns quantitative laws to the development of computer software, using a set of primitive measures that may be derived after code is generated or estimated once design is complete. These follow:

n1=the number of distinct operators that appear in a program.

n2=the number of distinct operands that appear in a program.

N1=the total number of operator occurrences.

N2=the total number of operand occurrences.

Halstead uses these primitive measures to develop expressions for the overall program length, potential minimum volume for an algorithm, the actual volume (number of bits required to specify a program), the program level (a measure of software complexity), the language level (a constant for a given language), and other features such as development effort, development time, and even the projected number of faults in the software.

Halstead shows that length N can be estimated

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

And program volume may be defined

$$V = N \log_2 (n_1 + n_2)$$

It should be noted that V will vary with programming language and represents the volume of information (in bits) required to specify a program. Theoretically, a minimum volume must exist for a particular algorithm. Halstead defines a volume ratio of volume of the most compact form of a program to the volume of the actual program. In actuality, L must always be less than 1.

In terms of primitive measures, the volume ratio may be expressed as

$$L = 2^{n_1/N} \cdot 2^{n_2/N}$$

15.8 Metrics for Testing

Function-based metrics can be used as a predictor for overall testing effort. Various project-level characteristics (e.g., testing effort. Various project-level characteristics (e.g., testing effort and time, errors uncovered, number of test cases produced) for past projects can be collected and correlated with the number of FP produced by a project team. The team can then project “expected values” of these characteristics for the current project.

The number of functional primitives (FuP), data elements (DE), objects (OB), relationships (RE), states (ST), and transition (TR), can be used to project the number and type of black-box and white-box tests for the software.

For example, the number of tests associated with the human/computer interface can be estimated by

Examining the number of transitions (TR) contained in the state transition representation of the HCI and evaluating the test required to exercise each transition;

Examining the number of data objects (OB) that move across the interface, and

The number of data elements that are input or output.

Architectural design metrics provide information on the ease or difficulty associated with integration testing (Chapter 18) and the need for specialized testing software (e.g., stubs and drivers). Cyclomatic complexity (a component –level design metric) lies at the core of basis path testing. In addition, cyclomatic complexity can

be used to target modules as candidates for extensive unit testing. Modules with high cyclomatic complexity are more likely to be error prone than modules whose cyclomatic complexity is lower. For this reason, the tester should expend above average effort to uncover errors in such modules before they are integrated in a system. Testing effort to uncover errors in such modules before they are integrated in a system. Testing effort can also be estimated using metrics

Derived from the definitions for program volume, V , and program level, PL , software science effort, e , can be computed as

$$PL = 1 / [(n1/2) (N2/n2)]$$

$$e = V / PL$$

The percentage of overall testing effort to be allocated to a module k can be estimated using the following relationship:

$$\text{Percentage of testing effort (k)} = e(k) / \sum e(i)$$

Where $e(k)$ is computed for module k and the summation in the denominator of Equation is the sum of software science effort across all modules of the system.

15.9 Metrics for Maintenance

All of the software metrics introduced in this chapter can be used for the development of new software and the maintenance of existing software. However, metrics designed explicitly for maintenance activities have been proposed.

IEEE Std. suggests a software maturity index (SMI) that provides an indication of the stability of a software product (based on changes that occur for each release of the product).

The following information is determined:

MT = the number of modules in the current release

F_c = the number of modules in the current release that have been changed

F_a = the number of modules in the current release that have been added

F_d = the number of modules from the preceding release that were deleted in the current release

The software maturity index is computed in the following manner:

$$SMI = [MT - (F_a + F_c + F_d)] / MT$$

As SMI approach 1.0, the product begins to stabilize. SMI may also be used as metric for planning software maintenance activities.

15.10 Summary

The first step in the measurement process is to derive the software measures and metrics that are appropriate for the representation of the software that is being considered. Data required to derive the formulated metrics are collected. Once computed, appropriate metrics are analyzed based on pre- established guidelines and fast data.

Reliability refers to extent to which a program can be expected to perform its intended function with required precision.

The objectives of measurement should be established before data collection begins. Each technical metric should be defined in an unambiguous manner. Metrics should be derived based on a theory that is valid for the domain of application (e.g., metrics for design should draw upon basic design concepts and principles and attempt to provide an indication of the presence of an attribute that is deemed desirable).

Technical work in software engineering begins with the creation of the analysis model. It is at this stage that requirements are derived and that a foundation for design is established. Therefore, technical metrics that provide insight into the quality of the analysis model are desirable.

Characteristics that can be used to assess the quality of the analysis model and the corresponding requirements specification: specificity (lack of ambiguity), completeness, correctness, understandability, verifiability, internal and external consistency, achievability, concision, traceability, modifiability, precision and reusability.

Architectural design metrics focus on characteristics of the program architecture with an emphasis on the architectural structure and the effectiveness of modules.

Components-level design metrics focus on internal characteristics of a software engineer to judge the quality of a component-level design.

Sample Questions:

1. Explain framework for technical software metrics?
2. What are the attributes of effective's software metrics?
3. Explain metrics for the analysis model
4. What are metrics for the design model?
5. Explain component-level design metrics?
6. What are metrics for source code?
7. Explain metrics for testing?
8. What are metrics for maintenance?

Reference Books:

7. Software Engineering, 5th and 7th edition, by Roger S Pressman ,McGraw Hill publication.
8. Software Engineering 3rd edition by KK Agarwal, Yogesh Singh,New Age International publication.
9. Software Engineering for students : A Programming by Douglas Bell, Pearson publication.
10. Information Technology Project Management by Jack T Marchewka Wiley India publication.
11. Managing Information Technology Project 6th edition , by Kathy Schwalbe, Cengage Learning publication.
12. Software Engineering Project Management by Richard H.Thayer Wiley India Publication.

